

基于 ALNS 算法的城市绿色物流配送调度优化研究

摘要

城市物流配送系统在满足客户时间窗约束的同时，需兼顾运输成本与碳排放控制。随着绿色配送区限行政策推行，燃油车在特定时段和区域内的通行受限，进一步增加了油电混合车队调度难度。本文针对含异构油电混合车队（燃油车 F1/F2/F3 与新能源车 E1/E2）、时变行驶速度、软时间窗及绿色配送区限行政策的带容量车辆路径问题（VRPTW），构建包含启动成本、载重相关能耗成本、碳排放成本、等待成本与晚到惩罚的综合成本模型，并设计三阶段递进式求解框架。

问题一在完成缺失数据补全后，建立无政策约束下的静态调度模型，采用自适应大邻域搜索算法 ALNS 进行求解。算法结合随机移除、最差移除、Shaw 相关性移除和路径移除等破坏算子，以及贪心插入、后悔-2 插入和后悔-3 插入等修复算子，并引入模拟退火接受准则、自适应权重更新和多趟合并机制。最优方案以 77 辆物理车完成 161 趟配送，总成本为 **74020.19 元**，其中能源成本占比约 46.0%，F1 车型平均重量装载率为 85.99%。

问题二在问题一基础上加入绿色配送区限行约束，即以市中心为圆心、半径 10km 区域内，08:00–16:00 禁止燃油车到达区内客户点（Rule A）并禁止燃油车行驶弧穿越绿色区域（Rule B）。针对该约束，本文设计政策感知 ALNS 算法，在搜索过程中嵌入合规性评估、违规惩罚、燃油转电动切换和返仓交接策略。最优方案以 108 辆物理车完成 168 趟配送，总成本为 **96143.62 元**，政策违约次数为 **0**；相较问题一，总成本上升 22123.43 元，增幅为 29.89%，碳排放量由 11325.49 kg CO₂ 增至 13307.36kgCO₂。绿色区服务记录中合规访问 37 次。

问题三构建事件驱动的滚动时域动态调度框架，处理订单取消、新增订单、地址变更和时间窗调整等突发事件。算法以服务完成时刻为依据冻结已承诺路线段，仅对未执行的柔性尾段进行事件定向修复与局部重优化，并结合 2-opt、relocate、政策违反修复及“柔性段重插入—燃油转电动—返仓交接—延迟过窗—应急拆分”五级兜底机制，将绿色配送区合规性转化为动态调度中的硬性保障。确定性场景下，经 4 次连续事件后，总成本由 Q2 基线 96143.62 元降至 95155.55 元，降低 988.06 元，降幅为 **1.03%**；实际物理车辆数由 **108 辆**降至 **107 辆**，启用趟次由 **168 趟**降至 **165 趟**，所有快照均保持零政策违约，单事件平均响应时延为 **142.8ms**。100 次蒙特卡洛随机事件流验证表明，终态政策合规率为 **100%**，新增订单插入失败率为 **0.0%**，平均成本变化为 **-1546.46 元 (-1.62%)**，94.0% 的试验成本优于静态基线，单试验最大响应时延 P95 为 **240.4ms**，验证了所提动态调度策略的鲁棒性与实时性。

关键词：车辆路径问题；自适应大邻域搜索；配送区限行政策；时变速度；动态调度；滚动时域优化；蒙特卡洛验证

目录

1	问题重述	1
1.1	问题背景	1
1.2	问题重述	1
2	问题分析	2
2.1	问题一分析	2
2.2	问题二分析	2
2.3	问题三分析	2
2.4	三问关系分析	2
3	模型假设与符号说明	3
3.1	模型假设	3
3.2	符号说明	3
4	问题一模型的建立与求解	4
4.1	数据预处理	4
4.2	综合成本目标函数	5
4.2.1	固定车辆成本	6
4.2.2	能源成本	6
4.2.3	碳排放成本	6
4.2.4	等待成本	7
4.2.5	晚到惩罚	7
4.2.6	总成本函数	7
4.3	约束条件	7
4.4	求解算法	7
4.5	求解结果	10
4.6	问题一结论	11
5	问题二模型的建立与求解	11
5.1	模型思路与分析	11
5.2	绿色配送区政策约束	12
5.2.1	绿色配送区定义	12
5.2.2	政策时间窗	12
5.2.3	点规则（绿色区客户访问约束）	12
5.2.4	弧规则（绿色区穿越约束）	12
5.3	综合优化模型	13
5.4	政策感知的 ALNS 求解算法	13
5.4.1	政策感知的初始解构造	13
5.4.2	政策感知的插入算子	13
5.4.3	政策修复机制	13
5.5	求解结果与分析	14
5.6	问题二结论	18

6	问题三模型的建立与求解	18
6.1	模型建立	19
6.1.1	动态事件假设	19
6.1.2	冻结-柔性划分模型	19
6.1.3	多种动态事件响应机制	20
6.1.4	政策合规性修复	21
6.2	模型求解	21
6.2.1	求解流程	22
6.3	结果分析	22
6.3.1	确定性场景结果分析	22
6.3.2	冻结-柔性空间演变分析	23
6.3.3	响应实时性分析	24
6.3.4	动态路径终态分析	25
6.3.5	蒙特卡洛鲁棒性检验	25
6.4	问题三结论	26
7	模型的评价	27
A	附录 A 代码文件与核心功能说明	28
B	附录 B 代码	28
B.1	公共模块代码	28
B.2	问题一代码	72
B.3	问题二代码	93
B.4	问题三代码	120

1 问题重述

1.1 问题背景

随着电子商务的快速发展与物流信息技术的普及，城市物流配送正面临需求动态化、规模扩大化与环保要求严格化的多重挑战。在“双碳”目标引领下，绿色物流成为城市可持续发展的关键环节。近年来，我国大力推进绿色物流示范城市建设，多个城市设立“绿色配送区”，对燃油车辆通行实施时段限制，这对物流企业的车辆调度提出了更高要求。

车辆路径规划问题（Vehicle Routing Problem, VRP）自 1959 年由 Dantzig 和 Ramser 提出以来 [1]，一直是运筹学领域的核心研究课题。带时间窗的车辆路径问题（VRPTW）作为重要扩展，受到学者广泛关注。曹庆奎等 [2] 学者在《基于交通流的多模糊时间窗车辆路径优化》一文中指出，传统研究大多假设车辆行驶速度为恒定值，但现实中受交通流影响，车速随时间动态变化，全天可划分为畅通、一般与拥堵三个时段。关于速度分布特征，研究表明畅通时段速度服从对数正态分布，一般与拥堵时段服从正态分布。此外，客户通常具有多个可接受配送的时间段，且时间要求往往具有模糊性而非完全刚性，多模糊时间窗模型比传统单时间窗模型更贴近实际。

在求解算法方面，针对传统启发式算法易陷入局部最优、收敛速度慢的问题，董文永等 [3] 首次提出了伊藤算法用于求解组合优化问题。曹庆奎等进一步改进了该算法：在路径选择策略中增加了目标节点对下一节点的影响，设计了新的路径权重更新规则，并引入柯西变异机制以避免算法陷入局部最优，成功将其应用于基于交通流的多模糊时间窗车辆路径问题的求解，取得了优于蚁群算法的效果。

在绿色物流背景下，混合车队（燃油车与新能源车）的协同调度进一步增加了问题的复杂性。研究表明，油耗与电耗随车速呈 U 型关系，满载能耗显著高于空载，燃油车满载能耗较空载高出约 40%，新能源车则高出约 35%。碳排放成本已逐渐内化为企业运营成本的重要组成部分。

本研究以某省会城市“绿色物流”股份有限公司的实际配送任务为对象，该公司每日需完成 98 个客户点、2169 个订单的配送，其中 30 个客户位于绿色配送区内，拥有三种燃油车与两种新能源车。研究内容涵盖无政策限制下的静态车辆调度、绿色配送区限行政策的影响分析，以及动态事件下的实时调度策略，旨在为城市绿色物流配送提供理论支持与实践参考。

1.2 问题重述

本文需要解决的问题如下：

问题一：在无政策限制条件下，建立考虑车辆类型、载重体积约束、时间窗约束与速度时变特性的车辆调度模型，以总配送成本最低为目标，设计调度方案，给出车辆使用方案、行驶路径、到达时间及成本构成。

问题二：在问题 1 的基础上，加入绿色配送区限行政策（8:00-16:00 禁止燃油车进入以市中心为圆心、半径 10 公里的圆形区域），重新规划调度路径，分析该政策对总成本、车辆使用结构与碳排放总量的影响。

问题三：针对订单取消、新增订单、地址变更或时间窗调整等突发事件，设计能实时响应并调整路径的动态调度策略，并给出一个或多个突发事件下的车辆调度策略。

2 问题分析

2.1 问题分析

问题一的核心是在无绿色配送区政策限制下，建立一个能够同时处理异构车型、双维容量、软时间窗和时变速度的静态车辆调度模型。建模时先把原始订单聚合到客户层面，再对超出单车容量的客户进行虚拟节点拆分，使每个待服务单元都能被某一趟路径独立承载。

该问题的求解重点不在于单纯缩短距离，而在于综合平衡车辆启动、行驶能耗、碳排放、等待和迟到五类成本。由于车速随时间段变化，同一路径在不同出发时刻会产生不同到达时间；同时，软时间窗允许早到等待和晚到惩罚，因此路径评价需要动态更新服务时刻和成本增量。

在求解思路上，本文采用自适应大邻域搜索算法 ALNS [4] 生成初始静态路线，并结合局部搜索改进访问顺序。随后利用“车辆返仓后可再次发车”的特点进行多趟合并，将单趟路径层面的优化结果转化为物理车辆层面的启用方案，从而形成问题二、问题三可比较的静态基准。

2.2 问题二分析

问题二在问题一框架上加入绿色配送区限行政策。政策约束不仅限制燃油车在指定时段访问绿色区客户，还限制燃油车在该时段穿越绿色配送区，因此需要同时处理点访问规则和弧穿越规则。相比问题一，问题二的可行路径空间被压缩，车型分配和服务时刻安排也必须随之调整。

该问题的关键是把政策要求嵌入路径构造和修复过程：绿色区客户优先尝试由新能源车服务；燃油车若服务相关客户或经过相关弧段，则必须避开限行时段；对搜索过程中产生的潜在违规路径，进一步通过重插入、车型替换和返仓交接等方式修复。

因此，问题二是在政策合规、车辆资源、时间窗满足和成本控制之间重新寻找平衡。通过与问题一结果对比，可以识别绿色配送区政策对车辆使用结构、成本构成和碳排放水平的影响。

2.3 问题三分析

问题三面向配送执行过程中的动态扰动。订单取消、新增订单、地址变更和时间窗调整会改变当前计划，但已完成服务或已经承诺前往的客户不能随意改变，因此不能简单重新求解全局静态模型，而应在保持既有执行计划稳定的基础上，对尚未执行部分进行滚动修复。

动态调度的核心是冻结-柔性划分。事件发生时，已服务或已承诺的路径前缀保持不变；尚未执行的后缀则作为柔性区域，允许删除、插入、重排或跨路径调整。不同事件对应不同修复方向：取消事件释放容量与时间，新增事件需要寻找低成本插入位置，地址变更影响距离和政策判定，时间窗调整影响等待与迟到成本。

由于绿色配送区政策在动态阶段仍然有效，每次事件处理后都必须重新进行合规校验。本文采用局部重插入、路径修复、多趟重合并和政策专项修复相结合的方式，使动态调度在实时响应、执行稳定和政策合规之间取得平衡。

2.4 三问关系分析

三个问题构成递进关系：问题一提供无政策限制下的静态成本模型和基准路径；问题二在此基础上加入绿色配送区访问与穿越约束，形成政策可行的静态方案；问题三再以问题二方案为初始计划，处理执行过程中的动态事件。三问共用同一套节点拆分、车型约束、时间窗评价和成本函数，使模型从静态优化、政策约束到动态调度保持一致。

3 模型假设与符号说明

3.1 模型假设

为简化问题，本文做出以下假设：

假设 1：一辆车送完一趟后可返回配送中心花 15 分钟装货，然后接着送下一趟，每辆车每天只产生一次启动费用；

假设 2：对于问题 2 中的“绿色配送区”，燃油车在 8:00-16:00 不得配送该区域内的客户点，该时段配送其他客户点的轨迹也不得与该区域相交（由于真实路径不可知，具体实现为：限行时段任意两停靠点连线线段不与绿色配送区相交）；

假设 3：当已上路的燃油车因动态事件无法完成部分客户送达时，可返回配送中心把货物转交给一辆新能源车，由新能源车接棒完成剩余配送。

3.2 符号说明

为避免三个问题中同一物理含义变量重复命名，本文统一采用表1所示符号。除特别说明外，问题一、问题二和问题三均沿用同一套节点、路径、车辆、成本和时间符号。

表 1: 主要符号说明

符号	含义
$\mathcal{V}$	调度节点集合，包含配送中心节点 0 与全部虚拟客户节点
$\mathcal{V}_c$	虚拟客户节点集合，即 $\mathcal{V} \setminus \{0\}$
i, j	虚拟节点索引；当 $i = 0$ 或 $j = 0$ 时表示配送中心
$o(i)$	虚拟节点 i 对应的原始客户编号
W_i, V_i	原始客户 i 的总重量需求与总体积需求
w_i, v_i	虚拟节点 i 的重量需求与体积需求
a_i, b_i	虚拟节点 i 的服务时间窗起点与终点
s_i	虚拟节点 i 的服务时长
(x_i, y_i)	节点 i 的平面坐标
d_{ij}	节点 i 与节点 j 之间的距离
A_i, D_i	车辆到达节点 i 的时刻与离开节点 i 的时刻
$\tau_{ij}(D_i)$	车辆在时刻 D_i 从节点 i 出发前往节点 j 的行驶时间
K	全部车型集合， $K = \{F1, F2, F3, E1, E2\}$
K_F, K_E	燃油车型集合与新能源车型集合
Q_k^w, Q_k^v	车型 k 的重量容量与体积容量
N_k, N_k^{used}	车型 k 的可用车辆数与实际启用车辆数
$\mathcal{R}$	配送路径集合，一条路径对应一趟配送任务
r	单条配送路径，形式为 $r = (0, i_1, i_2, \dots, i_m, 0)$
N^{phy}	实际启用的物理车辆数，同一物理车可执行多趟路径
x_{irk}	若虚拟节点 i 由车型 k 的路径 r 服务，则 $x_{irk} = 1$ ，否则为 0
ρ_{ij}^k	车型 k 在弧段 (i, j) 上的载重率，取重量利用率和体积利用率的较大值
C^{fixed}, C^{energy}	固定启动成本、能源成本
C^{carbon}, C^{wait}	碳排放成本、等待成本
C^{late}	迟到惩罚
Z, Z_1, Z_2, Z_t	综合配送成本；分别表示一般模型、问题一、问题二和时刻 t 动态快照的目标值
G	绿色配送区， $G = \{(x, y) \mid x^2 + y^2 \leq R^2\}$
R	绿色配送区半径，本文取 $R = 10$ km
$[T_s, T_e)$	绿色配送区限行时段，本文取 $[8, 16)$
V_t	动态事件发生后仍需服务的虚拟客户集合
$r^{fix}(t), r^{flex}(t)$	事件时刻 t 下路径 r 的冻结前缀与柔性后缀
V_t^{policy}	事件时刻 t 后方案中的政策违约次数
$e_m = (\tau_m, \kappa_m, \Omega_m)$	第 m 个动态事件，分别表示触发时刻、事件类型和事件信息

4 问题一模型的建立与求解

4.1 数据预处理

本研究基于附件 1 提供的四个数据文件构建预处理框架。

订单字段补全: 原始订单明细中存在少量重量或体积字段缺失。若直接将缺失值置为 0，会低估客户需求并影响后续容量约束、能耗和碳排放计算。本文采用基于货物密度的补全方法：先筛选重量和体积均完整的订单，计算单位体积重量密度 $\rho = \text{重量} / \text{体积}$ ；再以同一客户完整历史订单的密度中位数作为该客户的典型货物密度。对于缺失重量但体积已知的订单，按 $\text{重量} = \text{体积} \times \text{客户密度中位数}$ 估计；对于缺失体积但重量已知的订单，按 $\text{体积} = \text{重量} / \text{客户密度中位数}$ 估计。若某客户缺乏完整历史订单，则采用全样本密度中位数作为兜底。经统计，2169 条原始订单中共有 30 条存在重量或体积缺失，其中重量缺失 14 条、体积缺失 16 条，占全部订单的 1.38%。补全后总重量由 285122.65 kg 增至 286205.74 kg，增加 1083.09 kg；总体积由 772.431 m³ 增至 780.964 m³，增加 8.533 m³。该处理保证了重量与体积之间的物理一致性，并利用中位数降低异常值对补全结果的影响。缺失订单及补全结果见表 2。

表 2: 存在缺失字段的订单及补全结果

订单编号	客户编号	重量/kg	体积/m ³
61	4	36.800	0.08252
75	7	97.835	0.24310
143	8	58.217	0.16000
194	9	26.470	0.06060
223	12	47.672	0.12220
260	9	44.160	0.10110
302	76	31.752	0.08783
357	48	959.627	2.39798
464	61	146.317	0.39928
534	73	6.709	0.01714
563	55	433.747	1.19259
618	66	34.765	0.09520
683	41	33.789	0.09207
741	67	25.203	0.06740
883	42	27.524	0.06638
1038	59	23.639	0.06469
1149	50	28.757	0.07563
1226	74	692.076	1.77711
1337	62	13.150	0.03223
1474	54	13.466	0.03428
1539	42	127.674	0.30791
1625	47	18.913	0.04697
1677	52	25.730	0.06930
1721	65	1266.050	3.21478
1815	51	79.339	0.20290
1890	58	10.670	0.02774
1955	58	30.000	0.07799
2021	52	18.387	0.04952
2093	64	44.986	0.10940
2166	51	14.222	0.03637

订单聚合与客户筛选: 完成缺失字段补全后，按“目标客户编号”对订单进行聚合，得到每个客户的总重量 W_i 和总体积 V_i 。坐标表共登记 99 个节点（1 个配送中心 +98 个客户点），其中 10 个客户（ID: {1,14,15,17,18,20,21,22,23,96}）在订单表中无记录，不计入客户需求，仅保留 88 个有需求的客户进入后续计算。

时间窗数值化: 原始时间窗为 HH:MM 字符串格式, 无法直接参与连续时间运算。将其转换为小时浮点数: $\tau = h + m/60$, 统一量纲。

虚拟节点拆分: 部分客户单日总需求超过单辆车装载上限。以车队最小车型容量 $(Q_w^{\min}, Q_v^{\min}) = (1250\text{kg}, 6.5\text{m}^3)$ 为基准, 定义客户 i 的拆分份数:

$$k_i = \max \left(1, \left\lceil \frac{W_i}{Q_w^{\min}} \right\rceil, \left\lceil \frac{V_i}{Q_v^{\min}} \right\rceil \right)$$

将客户 i 均匀拆分为 k_i 个虚拟子节点, 每个子节点继承原客户的坐标 (x_i, y_i) 、时间窗 $[a_i, b_i]$ 和服务时长 s_i , 子需求为:

$$w_p = \frac{W_i}{k_i}, \quad v_p = \frac{V_i}{k_i}, \quad p \in \{1, \dots, k_i\}$$

88 个有需求的客户经拆分后, 共生成 284 个虚拟节点, 加上配送中心, 总节点数 $N = 285$ 。部分代表性客户的拆分结果如表3所示。

表 3: 部分客户虚拟节点拆分示例

客户 ID	订单数	总重 (kg)	总体 (m ³)	拆分 k_i	子节点重 (kg)	子节点体 (m ³)
4	11	1443.56	3.49	2	721.78	1.71
6	28	4473.64	11.98	4	1118.41	3.00
7	38	7463.23	18.48	6	1243.87	3.04
8	69	11749.55	36.03	10	1174.96	3.60
55	16	12197.65	30.80	10	1219.76	3.08

虚拟距离矩阵: 拆分后节点集从 99 个原始点膨胀到 285 个调度点。距离矩阵按如下规则填充:

$$d_{ij} = \begin{cases} 0, & i = j \\ 0, & o(i) = o(j) \text{ (同一原始客户的不同子点)} \\ D_{o(i), o(j)}^{\text{orig}}, & \text{其他} \end{cases}$$

其中 $o(i)$ 为虚拟节点 i 对应的原始客户编号, D^{orig} 为原始 99×99 路网距离矩阵。该规则确保同一客户的不同子点在同一路径中紧邻时不产生额外行驶距离。

时变速度参数: 城市交通速度随时间变化, 本文采用分段常数函数 $v(t)$:

$$v(t) = \begin{cases} 9.8, & t \in [8, 9) \cup [11.5, 13) \\ 35.4, & t \in [10, 11.5) \cup [15, 17) \\ 55.3, & t \in [9, 10) \cup [13, 15) \cup [17, 24] \end{cases}$$

车辆从节点 i 出发的时刻为 D_i , 行驶至节点 j (距离 d_{ij}) 的行驶时间 $\tau_{ij}(D_i)$ 满足:

$$\int_{D_i}^{D_i + \tau_{ij}(D_i)} v(s) ds = d_{ij}$$

该积分跨越多个时段时采用分段累加计算, 到达时刻为 $A_j = D_i + \tau_{ij}(D_i)$ 。

4.2 综合成本目标函数

总成本 Z_1 由五项构成, 各项均基于预处理后的参数定义。

符号说明：本节使用的主要符号已在表1中统一给出。为与后续问题保持一致，下文统一以 i, j 表示虚拟节点，以 $\mathcal{R}$ 表示路径集合，以 N^{phy} 表示实际启用的物理车辆数。

4.2.1 固定车辆成本

$$C^{\text{fixed}} = 400 \cdot N^{phy}$$

同一物理车辆执行多趟配送时只计一次启动成本。

4.2.2 能源成本

车辆在行驶过程中同时受重量和容积两个维度的容量约束。定义车型 k 在弧段 (i, j) 上的载重率 ρ_{ij}^k 为实际装载占额定容量的最大比例：

$$\rho_{ij}^k = \max \left\{ \frac{L_{ij}^{k,w}}{Q_k^w}, \frac{L_{ij}^{k,v}}{Q_k^v} \right\}, \quad \rho_{ij}^k \in [0, 1]$$

该值取重量利用率与容积利用率中的较大者，用于修正满载工况下的额外能耗。

燃油车（F1/F2/F3）的百公里油耗函数为：

$$\text{FPK}(v) = 0.0025v^2 - 0.2554v + 31.75 \quad (\text{L}/100\text{km})$$

弧段 (i, j) 上的燃油消耗量为：

$$F_{ij} = \frac{d_{ij}}{100} \cdot \text{FPK}(\bar{v}_{ij}) \cdot (1 + 0.40 \cdot \rho_{ij}^k)$$

燃油费用为：

$$C_{ij}^{\text{fuel}} = 7.61 \cdot F_{ij}$$

新能源车（E1/E2）的百公里电耗函数为：

$$\text{EPK}(v) = 0.0014v^2 - 0.12v + 36.19 \quad (\text{kWh}/100\text{km})$$

弧段 (i, j) 上的电耗量为：

$$P_{ij} = \frac{d_{ij}}{100} \cdot \text{EPK}(\bar{v}_{ij}) \cdot (1 + 0.35 \cdot \rho_{ij}^k)$$

电费为：

$$C_{ij}^{\text{elec}} = 1.64 \cdot P_{ij}$$

总能源成本为所有弧段费用之和：

$$C^{\text{energy}} = \sum_{r \in \mathcal{R}} \sum_{(i,j) \in r} C_{ij}^{\text{fuel}/\text{elec}}$$

4.2.3 碳排放成本

燃油车碳排放：

$$E_{ij}^{\text{fuel}} = 2.547 \cdot F_{ij}$$

新能源车碳排放:

$$E_{ij}^{\text{elec}} = 0.501 \cdot P_{ij}$$

总碳排放成本:

$$C^{\text{carbon}} = 0.65 \cdot \sum_{r \in \mathcal{R}} \sum_{(i,j) \in r} E_{ij}$$

4.2.4 等待成本

车辆到达虚拟节点 i 的时刻为 A_i , 时间窗为 $[a_i, b_i]$ 。早到时间:

$$e_i = \max(a_i - A_i, 0)$$

$$C^{\text{wait}} = 20 \cdot \sum_{i \in \mathcal{V}_c} e_i$$

4.2.5 晚到惩罚

晚到时间:

$$l_i = \max(A_i - b_i, 0)$$

$$C^{\text{late}} = 50 \cdot \sum_{i \in \mathcal{V}_c} l_i$$

4.2.6 总成本函数

$$Z_1 = C^{\text{fixed}} + C^{\text{energy}} + C^{\text{carbon}} + C^{\text{wait}} + C^{\text{late}}$$

4.3 约束条件

- (1) 服务约束: 每个虚拟节点被恰好服务一次
- (2) 容量约束: $\sum_{i \in r} w_i \leq Q_k^w, \sum_{i \in r} v_i \leq Q_k^v$
- (3) 车辆数量约束: $N_k^{\text{used}} \leq N_k$
- (4) 路径闭合: 每条路径从配送中心出发并返回
- (5) 返仓时间: $T_r^{\text{return}} \leq 24.00$

4.4 求解算法

接下来用自适应大领域搜索 (ALNS) 算法求解, 以预处理后的 284 个虚拟节点为决策单元, 配送中心作为固定起点和终点单独处理。

首先进行初始解构造, 将所有虚拟节点按时间窗起点 a_i 升序排序, 依次调用最优插入 (即贪心插入) 将节点插入当前成本增量最小的位置, 最终生成一个可行初始解。 **然后进行自适应迭代优化**, 每次迭代独立选择一种破坏算子和修复算子。其中破坏算子包括随机移除、最差移除、Shaw 移除、整路径移除和路径簇移除, 随机移除 $q \in [28, 85]$ 个虚拟节点; 修复算子包括贪心插入和 Regret-2/3 插入。破坏算子和修复算子的选择相互独立, 各自基于权重向量进行轮盘赌选择, 权重越大被选中概率越高。新解生成后, 采用模拟退火接受准则: 若新解优于当前解则直接接受, 否则以概率 $P = \exp(-(Z' - Z)/T)$ 接受较差解, 其中初始温度 $T_0 = 0.02Z_0$, 降温系数 $\alpha = 0.9975$ 。每 100 次迭代更新一次算子权重: 每个算子维护权重 w_i 和累计得分 s_i , 若当前使用的算子对帮助发现

了新全局最优解，则各得 +33 分；若改进了当前解（非全局最优），则各得 +9 分；若接受了一个较差解（通过模拟退火），则各得 +13 分；否则不得分。每 100 次迭代后按 $w_i \leftarrow (1 - \rho)w_i + \rho \cdot (s_i/u_i)$ 更新权重（ $\rho = 0.1$ 为反应系数， u_i 为被选中次数），使表现更好的算子获得更高的被选概率。

最后进行多趟合并输出，ALNS 主循环结束后，依次执行 **2-opt** 和 **Relocate** 局部搜索进行路径内和路径间优化，然后进行多趟合并：按出发时刻升序遍历所有路径，若两条路径车型相同且前一条路径的返仓时刻加上 0.25 小时不晚于后一条路径的发车时刻，则赋予相同的物理车辆编号 *vehicle_id*），视为同一辆物理车执行多趟配送。启动成本按实际物理车辆数计算（每辆 400 元），而非按路径趟数计算。最终输出物理车辆使用方案、配送路径详情、总成本构成（启动成本、能源成本、碳排放成本、等待成本、晚到惩罚）及各车型装载利用率分布。

图 1 展示了问题一的三维时空配送路径。图中每个点代表一次虚拟节点服务，点颜色表示同一原始客户的服务频次，点边框区分车型。从图中可以看出，配送路径以配送中心为核心向外辐射，时间维度上主要分布在日间配送时段内，体现了空间路径与时间窗调度的耦合关系；最新方案中，77 辆物理车完成 161 趟配送。

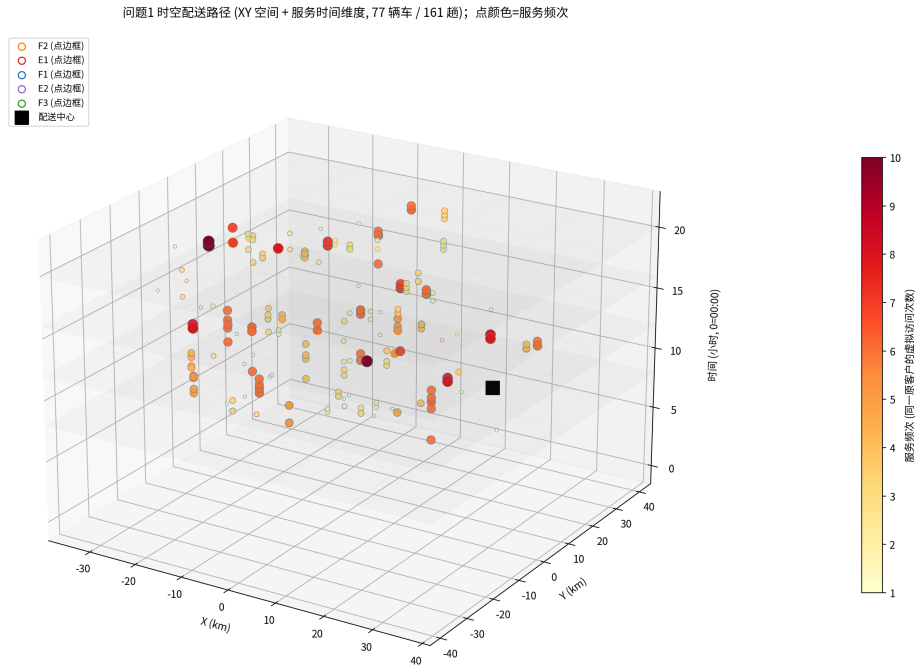


图 1: 问题一三维时空配送路径（77 辆车/161 趟）

多趟合并的具体效果如图 2 所示。子图 (a) 给出每辆物理车的执行趟数，子图 (b) 和 (c) 分别展示每辆物理车的累计装载重量和总路径成本，子图 (d) 展示首趟发车至末趟返仓的时间区间。该图说明路径搜索以“趟”为基本单元，而成本核算最终落实到物理车辆层面；通过把同车型、时序相容的多趟路径合并到同一物理车辆上，可减少车辆启动次数。

问题1 物理车辆多趟合并图谱 (77 辆车 / 161 趟)



图 2: 问题一物理车辆多趟合并图谱

各车型的装载利用率分布如图 3 所示。整体上，重量利用率明显高于体积利用率，说明本问题的主要装载瓶颈为重量约束。其中 F3、E1 和 E2 的平均重量利用率均在 90% 左右，F1 承担较多大批量配送任务，F2 则在容量适中、路径衔接灵活方面发挥补充作用。

问题1 各车型每趟装载利用率分布

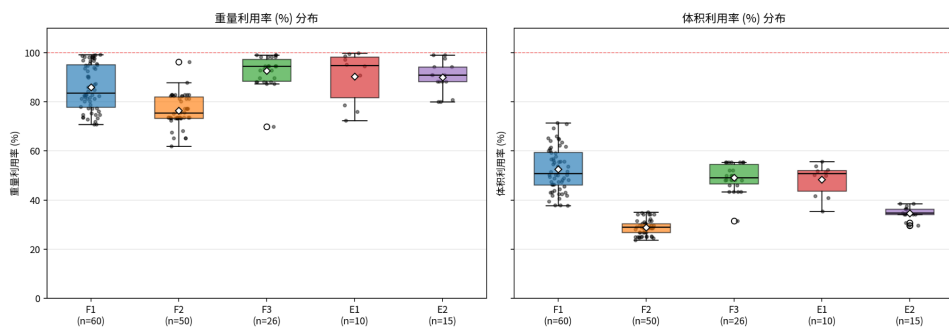


图 3: 问题一各车型装载利用率分布

4.5 求解结果

基于上述模型和算法，问题一求解结果如下。

成本构成: 总成本为 74,020.19 元，其中能源成本占比最高，为 46.02%；启动成本占比为 41.61%，是第二大成本来源；碳排放成本占比为 9.95%；等待成本和晚到惩罚占比较低，说明软时间窗整体满足较好。具体构成如表 3 所示。

表 4: 问题一成本构成

成本项	金额（元）	占比
启动成本	30,800.00	41.61%
能源成本	34,067.87	46.02%
碳排放成本	7,361.57	9.95%
等待成本	1,638.13	2.21%
晚到惩罚	152.62	0.21%
总成本	74,020.19	100%

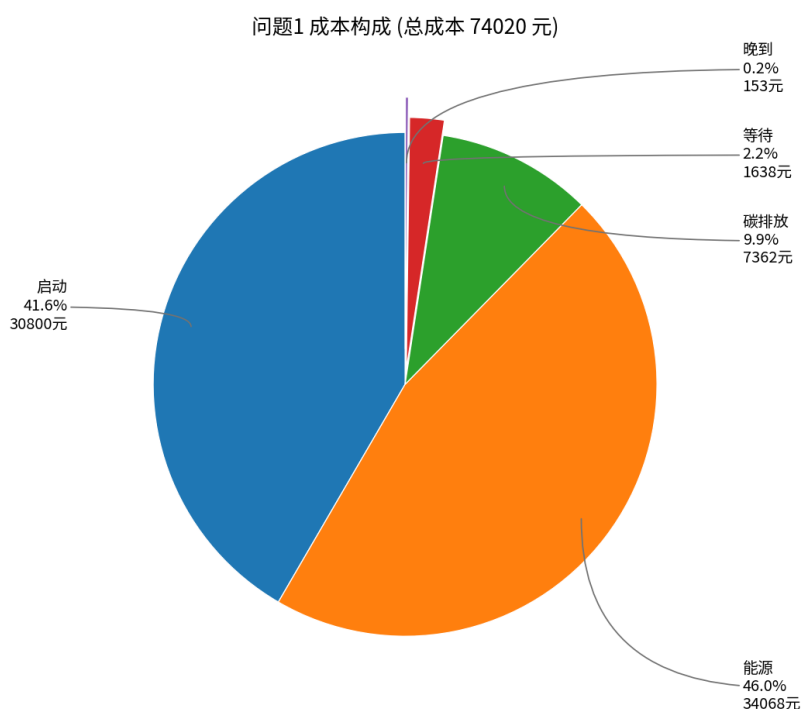


图 4: 问题一成本构成占比图

图 4 进一步展示了问题一各成本项的比例关系。能源成本和启动成本构成主要成本来源，二者合计超过总成本的八成；碳排放成本居于第三位，而等待成本和晚到惩罚占比较小，说明静态调度方案在满足软时间窗方面较为稳定，成本优化重点主要集中于路径能耗控制与物理车辆启动次数压缩。

车辆使用情况: 实际启用物理车辆 77 辆，执行配送任务 161 趟，平均每辆车运行 2.09 趟。若不进行多趟合并，则每一趟配送均需单独启动车辆；合并后节省 84 次启动，说明同一车辆返仓后再出车的利用价值较高。具体数据如表 4 所示。

表 5: 问题一车辆使用情况

指标	数值
实际启用物理车辆数	77 辆
执行配送趟数	161 趟
趟车比	2.09
多趟合并节省启动次数	84 次

车型使用分布: 各车型使用情况如表 5 所示。F1 和 F2 承担了主要配送车次，分别完成 60 趟和 50 趟；F3、E1 和 E2 的平均重量利用率较高，说明中小车型和新能源车型在局部高装载路径中具有较好的适配性。E1、E2 虽实际车辆数少于燃油车型，但由于能源单价和碳排放水平较低，仍被用于部分适合其容量和时窗的配送任务。

表 6: 问题一各车型使用情况

车型	载重 (kg)	容积 (m ³)	可用	趟次	物理车辆	重量利用率
F1	3000	13.5	60	60	33	85.99%
F2	1500	10.8	50	50	20	76.47%
F3	1250	6.5	50	26	10	92.59%
E1	3000	15.0	10	10	6	90.30%
E2	1250	8.5	15	15	8	90.07%

注：由于重量和容积双重约束，载重率 $\rho = \max\{w/Q_w, v/Q_v\}$ 取两者较大值用于能耗修正。从结果看，各车型重量利用率均显著高于体积利用率，说明重量约束是路径装载的主要瓶颈。

碳排放: 情况如表 6 所示。问题一碳排放总量为 11,325.49 kg，对应碳成本 7,361.57 元。由于无绿色配送区限制，燃油车仍承担主要配送任务，因此碳排放主要由燃油车辆行驶产生。

表 7: 问题一碳排放

指标	数值
碳排放总量	11,325.49 kg
碳排放成本	7,361.57 元

4.6 问题一结论

问题一形成了无政策限制下的静态基准方案。其求解思路是先将客户需求转化为可装载的虚拟节点，再用 ALNS 进行路径搜索，最后通过多趟合并减少物理车辆启用。计算结果表明：总成本为 74,020.19 元，77 辆物理车完成 161 趟配送，晚到惩罚占比较低，多趟合并节省 84 次启动。该结果说明模型能够兼顾软时间窗满足和车辆返仓后再出车利用，也为问题二评估绿色配送区政策影响提供了可比基准。

5 问题二模型的建立与求解

5.1 模型思路与分析

问题一已求得无政策约束下的低成本配送方案，但该方案允许燃油车在任意时段服务任意客户，可能存在燃油车在高峰时段进入或穿越绿色配送区的情况。问题二在问题一模型基础上引入绿色配

送区限行政策，要求燃油车在 8:00-16:00 期间不得进入以市中心 (0,0) 为圆心、半径 $R = 10\text{km}$ 的圆形区域，也不得在该时段内穿越该区域。

政策约束使原有可行域被显著压缩，主要体现在三个方面：

- (1) 绿色区内客户不能再由燃油车在限制时段内直接服务，需优先由新能源车承担，或将服务时间推迟至 16:00 之后；
- (2) 即使燃油车路径的起终点均在绿区外，若行驶弧段在限制时段穿越绿区，同样违反政策；
- (3) 政策约束可能导致绕行、等待或车辆重分配，从而增加能源成本、固定成本和迟到惩罚。因此，问题二的核心是在保证所有客户被服务的前提下，重新分配燃油车与新能源车的服务对象，调整路径和服务时刻，使方案满足政策约束且总成本尽可能低。

5.2 绿色配送区政策约束

5.2.1 绿色配送区定义

绿色配送区定义为以城市中心 (0,0) 为圆心、半径 $R = 10\text{km}$ 的圆形区域：

$$G = \{(x, y) \mid x^2 + y^2 \leq R^2\}, \quad R = 10.$$

对于任意客户 i ，若其坐标满足 $x_i^2 + y_i^2 \leq R^2$ ，则判定该客户位于绿色配送区内。根据实际数据统计，共识别出 12 个绿色区原始客户，对应虚拟客户点共服务 37 次。

5.2.2 政策时间窗

燃油车受限时间段为 $[T_s, T_e) = [8, 16)$ ，即每日 8:00 至 16:00 期间，燃油车不得在绿色配送区内服务客户，也不得在该时段内穿越绿色配送区。

5.2.3 点规则（绿色区客户访问约束）

设 $K_F = \{F1, F2, F3\}$ 为燃油车集合。若客户 i 位于绿色配送区内，且车辆 k 为燃油车，则该车辆不得在限制时段内到达客户 i ：

$$i \in G, k \in K_F, A_i \in [8, 16) \Rightarrow x_{ik} = 0.$$

即绿色区内客户若由燃油车服务，其到达时间必须满足 $A_i < 8$ 或 $A_i \geq 16$ 。本文求解结果中，绿色区虚拟客户点共服务 37 次，其中新能源车承担 28 次，燃油车在 16:00 之后承担 9 次，全部满足访问约束。

5.2.4 弧规则（绿色区穿越约束）

仅限制燃油车到达绿区客户仍不足以保证政策合规。若燃油车在 8:00-16:00 期间从绿区外某客户前往另一绿区外客户，但其行驶路径穿过绿色区，也应视为违反政策。由于缺少真实道路网络轨迹，本文采用节点间直线段是否与绿色区圆相交作为保守判定。

对于弧段 (i, j) ，线段上任一点可表示为：

$$P(\lambda) = (x_i, y_i) + \lambda[(x_j, y_j) - (x_i, y_i)], \quad 0 \leq \lambda \leq 1.$$

若存在 $\lambda \in [0, 1]$ 使得 $\|P(\lambda)\|_2 \leq R$, 则认为弧段 (i, j) 与绿色配送区相交。燃油车在该弧段上的行驶时间区间为 $[D_i, A_j]$, 若同时满足:

$$\overline{ij} \cap G \neq \emptyset, [D_i, A_j] \cap [8, 16) \neq \emptyset,$$

则该弧段违反绿色区穿越约束, 即:

$$k \in K_F, \overline{ij} \cap G \neq \emptyset, [D_i, A_j] \cap [8, 16) \neq \emptyset \Rightarrow y_{ijk} = 0.$$

5.3 综合优化模型

问题二的目标函数与问题一相同, 为最小化配送总成本:

$$\min Z_2 = C^{\text{fixed}} + C^{\text{energy}} + C^{\text{carbon}} + C^{\text{wait}} + C^{\text{late}}.$$

其中各分项成本的定义及计算公式与问题一完全一致。问题二在问题一全部约束基础上, 增加式 (4)–式 (6) 所示的绿色配送区访问规则和弧穿越规则约束。

5.4 政策感知的 ALNS 求解算法

本文采用政策感知的自适应大邻域搜索算法求解问题二。算法框架与问题一相同, 包括初始解构造、破坏算子、修复算子、模拟退火接受准则、自适应权重更新、局部搜索和多趟合并等模块。本节仅描述针对绿色区政策的增量修改, 其余通用搜索机制与问题一保持一致。

5.4.1 政策感知的初始解构造

初始解采用贪婪插入策略构造, 排序规则与问题一相同。区别在于: 对于绿色区客户, 算法优先尝试插入新能源车路径; 若新能源车容量或数量不足, 则将其安排在燃油车 16:00 之后服务。这一策略使初始解更可能满足政策约束。

5.4.2 政策感知的插入算子

在修复算子尝试将客户插入路径时, 需额外进行政策可行性检查:

- 若待插入客户位于绿色区内, 且候选路径为燃油车, 则插入后的到达时间必须在 16:00 之后, 否则该候选位置不可行。
- 若插入操作产生新的行驶弧段, 且该弧段由燃油车执行, 则需判断该弧段是否与绿区圆相交, 以及行驶时间区间是否与 $[8, 16)$ 重叠。若同时满足, 则该候选位置不可行。

不可行的候选位置在修复过程中被直接跳过, 避免生成违规解。

5.4.3 政策修复机制

对于仍存在政策违规的候选路径, 算法执行专项修复:

- **Level-1 重插入:** 将违规客户从当前路径中移除, 重新插入到新能源车路径或燃油车 16:00 之后的位置。

- **Level-2 燃油车转电动车**: 若整条燃油车路径存在多个违规, 且新能源车仍有配额, 则将整条路径的车型替换为电动车。
- **Level-3 返仓交接**: 对于已发车且无法整体替换的违规燃油车路径, 在最早违规位置截断, 头段由原车返回仓库, 尾段由新调度的电动车接棒完成。

三级修复机制由低到高逐级尝试, 保证最终解的政策可行性。

5.5 求解结果与分析

基于上述模型和算法, 问题二求解结果如下。

总体成本: 总成本为 96,143.62 元, 实际启用 108 辆物理车辆, 完成 168 趟配送任务, 多趟合并节省 60 次启动成本, 政策违规次数为 0。具体成本构成如表 7 所示。

表 8: 问题二成本构成

成本项	金额 (元)	占比
启动成本	43,200.00	44.93%
能源成本	39,966.07	41.57%
碳排放成本	8,649.78	9.00%
等待成本	1,326.35	1.38%
晚到惩罚	3,001.42	3.12%
总成本	96,143.62	100%

与问题一对比: 问题一与问题二的对比结果如表 8 所示。政策引入后, 总成本增加 22,123.43 元, 增幅为 29.89%; 实际车辆数增加 31 辆, 增幅为 40.26%; 总趟次增加 7 趟, 增幅为 4.35%。成本增长主要来自启动成本、能源成本和晚到惩罚: 启动成本增加说明限行政策削弱了部分路径之间的连续衔接能力; 能源成本和碳排放成本增加说明燃油车可行路径空间被压缩后, 路径绕行、服务时段调整和车型重分配带来了额外行驶代价; 等待成本反而下降, 表明政策约束下部分路径被推迟或重排后减少了早到等待, 但这种改善不足以抵消晚到惩罚和车辆启动成本的上升。

表 9: 问题一与问题二成本对比

指标	问题一	问题二	绝对增量	相对增量
启动成本 (元)	30,800.00	43,200.00	12,400.00	+40.26%
能源成本 (元)	34,067.87	39,966.07	5,898.21	+17.31%
碳排放成本 (元)	7,361.57	8,649.78	1,288.21	+17.50%
等待成本 (元)	1,638.13	1,326.35	-311.79	-19.03%
晚到惩罚 (元)	152.62	3,001.42	2,848.80	+1866.60%
总成本 (元)	74,020.19	96,143.62	22,123.43	+29.89%
碳排放量 (kg)	11,325.49	13,307.36	1,981.86	+17.50%
实际车辆数 (辆)	77	108	31	+40.26%
总趟次	161	168	7	+4.35%

车型使用结构: 各车型使用情况如表 9 所示。燃油车仍承担大部分配送任务, 其中 F1 完成 60 趟、F2 完成 50 趟, 分别启用 43 辆和 24 辆物理车; F3 在问题二中启用 20 辆、完成 33 趟, 较问题一明显增加, 说明部分受限路径被拆分或转移到更灵活的小型燃油车上。新能源车方面, E1 达到满额使用, E2 完成 15 趟但经多趟合并后实际启用 11 辆, 说明新能源车辆主要承担绿色区限行窗口内的合规服务, 并在容量与时窗允许时通过合并降低物理车辆数。

表 10: 问题二各车型使用情况

车型	可用数量	实际车辆数	启用趟次	趟车比	平均重量利用率
F1	60	43	60	1.40	82.86%
F2	50	24	50	2.08	75.09%
F3	50	20	33	1.65	88.56%
E1	10	10	10	1.00	88.91%
E2	15	11	15	1.36	93.55%

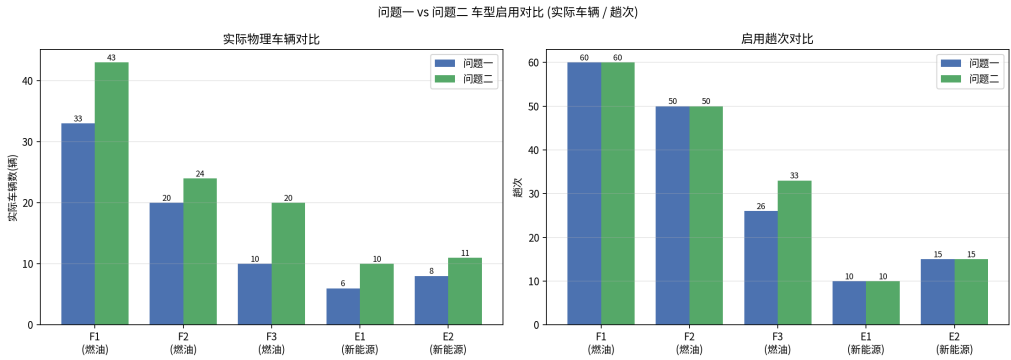


图 5: 问题一与问题二车辆启用结构对比

图 5 从物理车辆数和启用趟次两个层面对比了政策前后的车队结构变化。限行政策实施后，燃油车和新能源车均出现不同程度的物理车辆增加，尤其是 F1、F3 与 E2 的实际车辆数增长较明显，说明绿色区约束削弱了部分路径连续复用能力，并使部分任务由更灵活的小车型或新能源车型承接。与此同时，各车型启用趟次变化相对有限，表明成本上升更多来自同一任务量下车辆复用效率下降，而非配送需求本身大幅增加。

碳排放核算: 按动力类型拆分的碳排放如表 10 所示。问题二总碳排放为 13,307.36 kg，其中燃油车排放 12,586.22 kg，占 94.58%；新能源车排放 721.14 kg，占 5.42%。虽然绿色配送区限制了燃油车在核心区域的通行，但由于燃油车仍承担 143 趟配送，且政策约束引起路径重排和部分绕行，系统总碳排放较问题一上升。若进一步降低碳排放，仅依靠限行政策并不足够，还需增加新能源车供给或优化燃油车在绿区外的路径衔接。

表 11: 问题二按动力类型划分的碳排放核算

动力类型	实际车辆数	启用趟次	碳排放 (kg)	碳排放占比
燃油车	87	143	12,586.22	94.58%
新能源车	21	25	721.14	5.42%
合计	108	168	13,307.36	100%

图 6 展示了问题二的政策合规三维时空图，绿色圆柱表示半径 10 km 的绿色配送区，限行时段为 08:00–16:00。图中燃油车进入绿色区的服务点均出现在 16:00 之后，新能源车承担了限行时段内的大部分绿色区服务，验证了点规则与弧规则共同约束下的政策可行性。图 7 将成本桥接图与碳排放对比图并排展示：政策引入后，总成本增加主要来自启动成本、能源成本和晚到惩罚，碳排放量由 11325.49 kg 增加至 13307.36 kg。图 8 与图 9 进一步从空间路径和时间窗角度解释了政策影响：燃油车需要绕开或延后进入绿色区，新能源车则主要承担限行窗口内的绿色区客户服务。图 10 展示了绿色区客户服务时间轴，绿色区虚拟客户点共服务 37 次，其中新能源车承担 28 次，燃油车

在 16:00 之后承担 9 次，未出现限行时段内燃油车违规访问绿色区客户的情况。

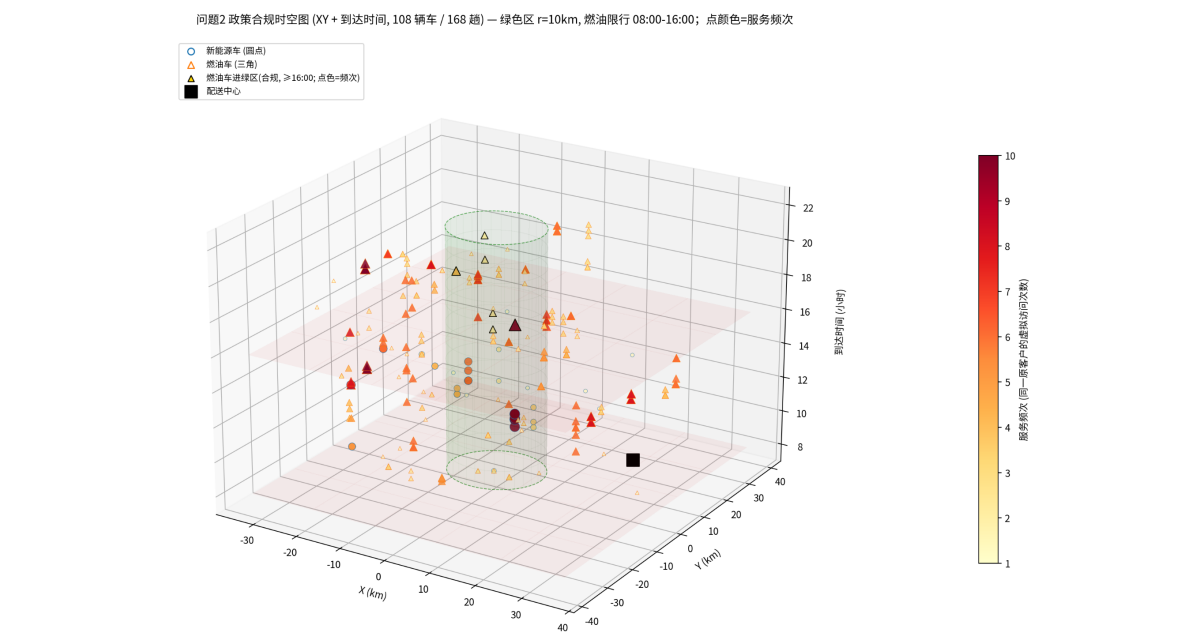


图 6: 问题二政策合规三维时空图

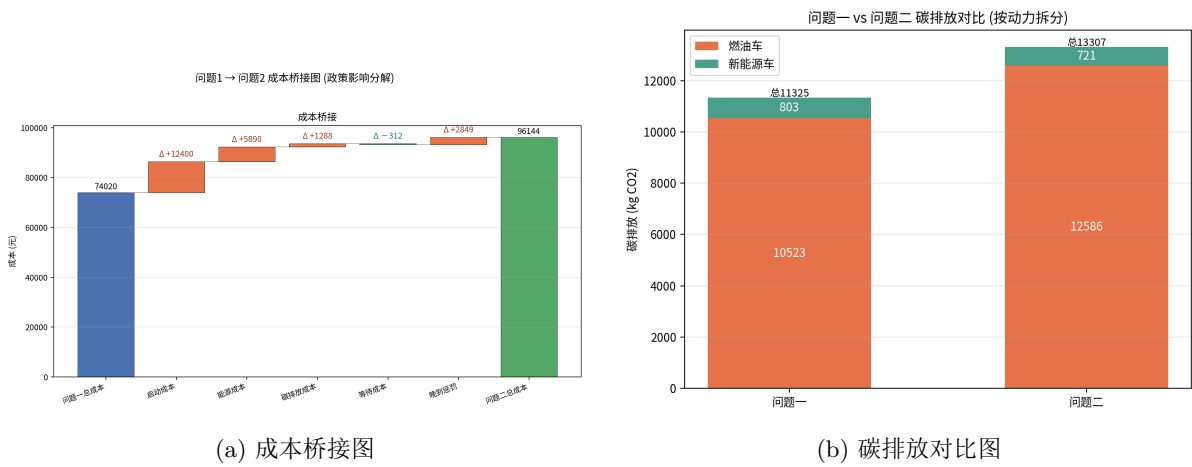


图 7: 问题一至问题二成本桥接与碳排放对比

图 7 中，成本桥接图把问题二总成本的增加拆解为启动、能源、碳排放、等待和晚到惩罚等分项贡献。启动成本增加 12400 元，是政策冲击下最主要的成本来源；能源成本和晚到惩罚分别增加 5898 元和 2849 元，反映出路径绕行、车型重分配与服务时刻后移带来的综合影响。右侧碳排放对比图显示，问题二中燃油车排放仍占绝对主导，新能源车排放占比较低，说明绿色区限行提升了合规性，但若要进一步降低排放，还需扩大新能源车供给或提高燃油车绿区外路径衔接效率。

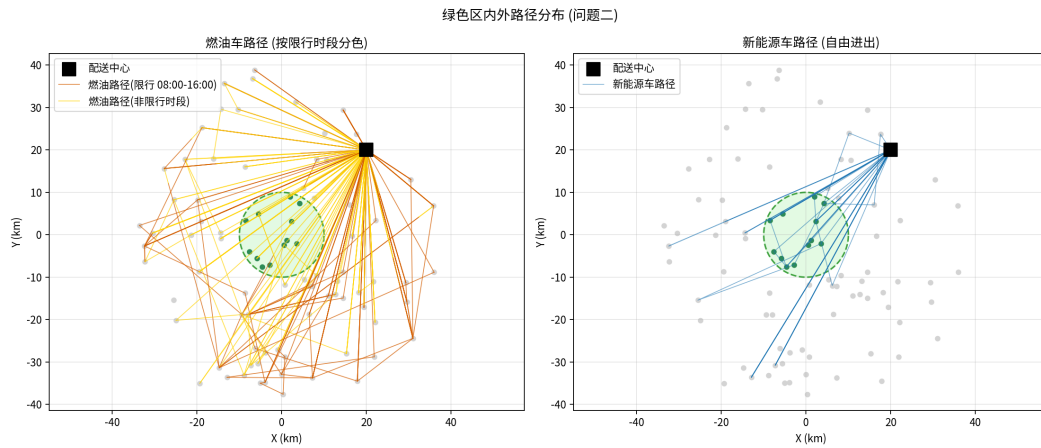


图 8: 问题二绿色区内外路径分布

图 8 对比了燃油车和新能源车在绿色配送区附近的路径空间分布。燃油车路径主要绕开绿色区或在限行后进入，新能源车则可以自由进出绿色区域并承担区内高约束客户。该结果说明，政策约束改变了不同动力车型的空间分工，新能源车成为连接绿色区客户与配送中心的关键补充力量。

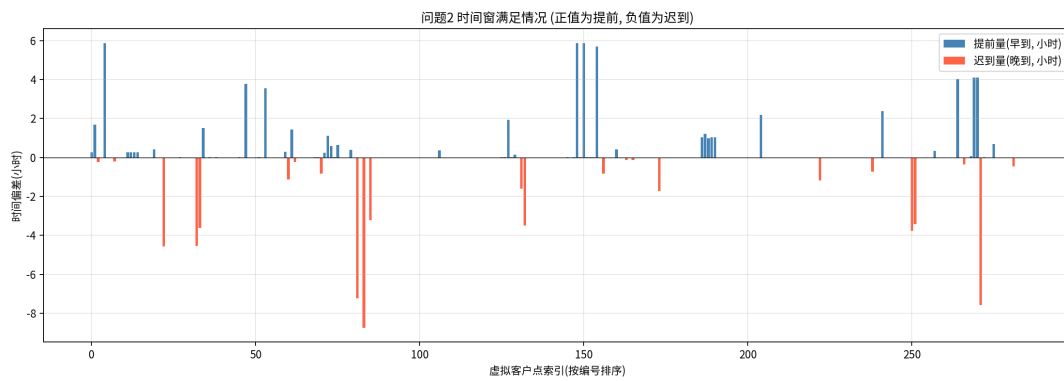


图 9: 问题二时间窗满足与迟到情况

图 9 展示了政策约束下客户服务时间与时间窗偏差的分布。整体上，大多数客户仍可在时间窗内或提前到达，迟到客户数量较少但个别迟到幅度较明显。结合成本结果可知，晚到惩罚虽不是最大成本项，但其增幅显著，说明绿色配送区限行会压缩可调度时间空间，并使部分路径更容易受到时间窗冲突影响。

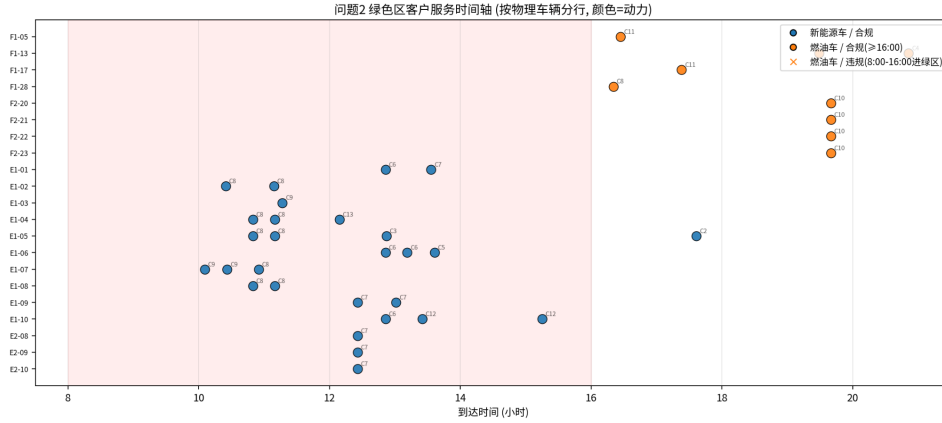


图 10: 问题二绿色区客户服务时间轴

5.6 问题二结论

问题二的求解思路是在问题一基准模型上增加“点访问 + 弧穿越”两类政策约束，并通过政策感知插入和专项修复保证可行性。结果显示：总成本为 96,143.62 元，108 辆物理车完成 168 趟配送，政策违约次数为 0；相较问题一，总成本上升 29.89%，碳排放上升 17.50%。

该结果说明，绿色配送区政策会压缩燃油车可行路径空间，使车辆启动、路径衔接和新能源车调度压力增加。E1 车型满额使用，E2 完成全部 15 趟但实际启用 11 辆，表明新能源车是维持政策合规的关键；同时燃油车仍贡献超过九成碳排放，说明后续若要进一步降低系统排放，应从扩大新能源车规模、优化燃油车绿区外路径和提升跨趟合并效率三方面继续改进。

6 问题三模型的建立与求解

在问题二绿色配送区政策可行静态方案的基础上，进一步考虑配送执行过程中出现的动态扰动。与静态问题不同，动态调度不能在每次事件发生后完全推翻原计划：已经发车、已经服务或车辆已经承诺前往的客户必须保持执行一致性；同时，新增、取消、改址和时间窗变化又会改变剩余任务的空间分布、容量占用和时间窗成本。因此，问题三采用事件驱动的滚动时域重调度框架，将当前方案划分为冻结前缀与柔性后缀，并只对尚未执行的柔性部分进行局部修复和重优化。

本文以问题二最优静态方案作为动态调度初始计划 T0。该方案总成本为 96143.62 元，实际启用物理车辆 108 辆，执行 168 趟配送，且绿色区政策违约次数为 0。在此基础上，构造一组覆盖订单取消、时间窗调整、新增订单和地址变更的确定性事件流，并进一步通过蒙特卡洛随机事件流验证动态策略的稳健性。

动态调度需要兼顾三方面要求：一是保持已经执行路径的物理可行性，避免车辆“时间回流”或临时改变已承诺目的地；二是尽量利用未发车路线和柔性客户池吸收扰动，降低新增车辆与迟到惩罚；三是继续满足问题二的绿色配送区点规则和弧规则，使每个快照均保持政策零违约。基于此，本文采用“事件触发-冻结判定-语义修复-局部搜索-合规兜底-快照输出”的闭环流程。

6.1 模型建立

本文在前两问已定义的客户虚拟拆分、车辆容量、时变旅行时间、软时间窗、碳排放成本和绿色配送区政策约束基础上，仅补充动态事件与滚动重调度相关变量。为保证动态调度方案可执行，作如下约定：

1. **冻结前缀不可改变**。事件发生时刻之前已经服务完成的客户，或车辆已经从上一节点出发前往的客户，均视为冻结客户。冻结客户的访问顺序、车辆归属和服务状态不再改变。
2. **柔性后缀可局部重优化**。尚未发车或尚未承诺访问的客户属于柔性部分，可在事件发生后重新插入、删除、调整访问顺序或跨路径转移。新增订单只能插入未发车路线或新开路线，已发车车辆不能再装载新增货物。
3. **政策合规为硬约束**。动态阶段继续沿用问题二的绿色区点规则和弧规则，所有事件后快照的政策违约次数必须为 0。若常规修复无法消除潜在违约，则采用延迟过窗、返仓交接或应急拆分等兜底机制。

6.1.1 动态事件假设

本文考虑四类动态事件：

$$\mathcal{E} = \{\text{订单取消, 新增订单, 地址变更, 时间窗调整}\}. \quad (1)$$

每个事件可表示为

$$e_m = (\tau_m, \kappa_m, \Omega_m), \quad m = 1, 2, \dots, M, \quad (2)$$

其中 τ_m 为事件触发时刻， κ_m 为事件类型， Ω_m 为事件携带的信息。事件按触发时刻满足

$$\tau_1 \leq \tau_2 \leq \dots \leq \tau_M. \quad (3)$$

本文确定性场景共包含四个事件，如表 12 所示。该事件组覆盖取消、时间窗收紧、新增和改址四类典型扰动，且触发时刻从 09:15 延续到 13:20，覆盖绿色配送区 08:00–16:00 限行窗口内的主要配送时段。

表 12: 问题三确定性动态事件

事件	触发时刻	描述	受影响路线数	响应耗时/ms	政策违约数
E1	09:15	客户 72 取消订单，算法删除其尚处柔性段的 3 个虚拟节点	1	124.3	0
E2	10:30	客户 7 时间窗收紧为 [13:00, 13:45]，该客户位于绿区内	0	143.9	0
E3	11:45	新增客户 N1，坐标 (12, 5)，需求 150 kg/0.5 m ³ ，时间窗 [14:00, 16:00]	1	163.4	0
E4	13:20	客户 70 地址由 (-12.9, -33.6) 变更为 (-8.0, -22.0)	0	139.6	0

6.1.2 冻结–柔性划分模型

设某条路径为

$$r = (0, i_1, i_2, \dots, i_m, 0), \quad (4)$$

其中 0 表示配送中心。记客户 i_h 的服务完成时刻为 D_{i_h} ，车辆从 i_{h-1} 出发前往 i_h 的时刻为 $T_{i_{h-1}, i_h}^{\text{dep}}$ ，则事件时刻 t 下路径 r 的冻结客户数量可表示为

$$q_r(t) = \max \left\{ h \mid D_{i_h} \leq t \text{ 或 } T_{i_{h-1}, i_h}^{\text{dep}} \leq t < D_{i_h} \right\}. \quad (5)$$

冻结前缀与柔性后缀分别为

$$r^{\text{fix}}(t) = (i_1, i_2, \dots, i_{q_r(t)}), \quad r^{\text{flex}}(t) = (i_{q_r(t)+1}, \dots, i_m). \quad (6)$$

冻结前缀必须满足

$$\pi_r^{\text{new}}(i) = \pi_r^{\text{old}}(i), \quad x_{irk}^{\text{new}} = x_{irk}^{\text{old}}, \quad \forall i \in r^{\text{fix}}(t), \quad (7)$$

其中 $\pi_r(i)$ 表示客户 i 在路径 r 中的访问序位， x_{irk} 表示客户 i 是否由车型 k 的路径 r 服务。事件后仍需服务的柔性客户集合 V_t 需满足唯一服务约束：

$$\sum_r \sum_k x_{irk} = 1, \quad \forall i \in V_t. \quad (8)$$

6.1.3 多种动态事件响应机制

a. 订单取消事件 当客户 i 取消订单时，若其对应虚拟节点尚未冻结，则将其从柔性路径中删除：

$$i \in r^{\text{flex}}(t) \Rightarrow r \leftarrow r \setminus \{i\}. \quad (9)$$

同时更新待服务客户集合

$$V_t = V_{t-} \setminus \mathcal{D}_i, \quad (10)$$

其中 $\mathcal{D}_i$ 表示客户 i 对应的虚拟节点集合。若该客户已经冻结，则取消请求无法完全撤回，只记录为无效送达或已承诺服务。

在 E1 中，客户 72 取消订单，算法删除了 3 个尚处柔性段的虚拟节点，未造成政策违约，总成本由 96143.62 元降至 95869.19 元。

b. 新增订单事件 新增客户 i' 到达后，首先按其需求进行虚拟节点拆分，然后在未发车路线或新开路线中寻找最小成本插入位置：

$$(r^*, p^*) = \arg \min_{r, p} \Delta C(i', r, p). \quad (11)$$

若将新增节点 i' 插入相邻节点 (u, v) 之间，则插入成本增量为

$$\Delta C(i', r, p) = C_{ui'} + C_{i'v} - C_{uv} + \Delta C^{\text{wait}} + \Delta C^{\text{late}} + \Delta C^{\text{policy}}. \quad (12)$$

其中 ΔC^{policy} 用于识别政策不可行或政策修复引起的成本变化。

在 E3 中，新增客户 N1 被成功插入现有柔性路径，新增订单失败数为 0，且终态政策违约数仍为 0。

c. 地址变更事件 当客户 i 地址变更且其尚未冻结时，更新其坐标并重算距离矩阵相关元素：

$$d_{ij} \leftarrow d'_{ij}, \quad d_{ji} \leftarrow d'_{ji}, \quad \forall j \in V. \quad (13)$$

随后对包含该客户的路线及受影响路线进行局部重优化；若客户已冻结，则保持原执行安排。

在 E4 中，客户 70 地址调整后，系统通过柔性段重排与多趟重合并释放了部分燃油车资源，使终态实际车辆数降至 107 辆，总成本进一步下降到 95155.55 元。

d. 时间窗调整事件 当客户 i 的时间窗由 $[a_i, b_i]$ 调整为 $[a'_i, b'_i]$ 时，若该客户尚未冻结，则更新其时间窗：

$$[a_i, b_i] \leftarrow [a'_i, b'_i]. \quad (14)$$

随后重新计算其所在路径的到达时间、等待成本和迟到成本，并在柔性部分中尝试通过重插入或局部排序降低时间窗压力。

在 E2 中，客户 7 的时间窗收紧为 $[13:00, 13:45]$ ，该客户在问题二基线中已由新能源车承担，故时间窗调整没有引发绿色区政策违约，但因重新定位服务时刻和车辆合并结构，总成本较 E1 增加 250.78 元。

6.1.4 政策合规性修复

问题三延续问题二的绿色配送区点规则和弧规则。每次事件处理后，调度器重新检查燃油车是否在 $8:00-16:00$ 到达绿色区客户，或在该时段配送绿区外客户时穿越绿色配送区。若发现潜在违约，按照以下顺序修复：

1. 在柔性段内重插入客户，寻找不违约位置；
2. 将相关客户从燃油车路径转移至新能源车路径；
3. 对燃油车路径进行局部重排，使其避开限制时段；
4. 对仍存在违约风险的燃油车弧段执行延迟过窗，即在 $16:00$ 后再通过该弧段；
5. 若延迟过窗导致返仓超期，则执行应急拆分，由新能源车接管尾段服务。

因此每次事件后的快照均需满足

$$V_t^{policy} = 0, \quad (15)$$

其中 V_t^{policy} 表示时刻 t 后的政策违约次数。本次确定性场景中，T0、E1、E2、E3 和 E4 五个快照的政策违约次数均为 0，延迟过窗与应急拆分均未被触发。

6.2 模型求解

在事件触发时刻 t ，模型保持冻结路径不变，对柔性客户集合重新进行局部优化。事件后总成本沿用前文固定车辆成本、能源成本、碳排放成本、等待成本和迟到惩罚的定义，可表示为

$$\min Z_t = C_t^{fixed} + C_t^{energy} + C_t^{carbon} + C_t^{wait} + C_t^{late}. \quad (16)$$

其中 C_t^{fixed} 、 C_t^{energy} 、 C_t^{carbon} 、 C_t^{wait} 和 C_t^{late} 分别表示事件时刻 t 后快照方案的启动成本、能源成本、碳排放成本、等待成本和迟到惩罚。动态重调度继续满足柔性客户唯一服务、车辆重量容量、车辆体积容量、车型数量上限、冻结前缀保持不变以及绿色配送区政策硬性可行等约束。

6.2.1 求解流程

读取问题二静态基线作为初始计划 T0；按事件触发时刻对动态事件排序；在每个事件时刻计算各路径冻结前缀和柔性后缀；根据事件类型执行取消、插入、改址或时间窗更新；对受影响路径柔性部分执行局部搜索；检查容量、时间窗、车辆数量和绿色区政策可行性；对潜在违约路线执行政策修复与多趟重合并；输出当前快照的成本分项、车次数、车辆数、冻结比例、响应耗时和政策违约次数；若仍有后续事件，则进入下一滚动时刻。

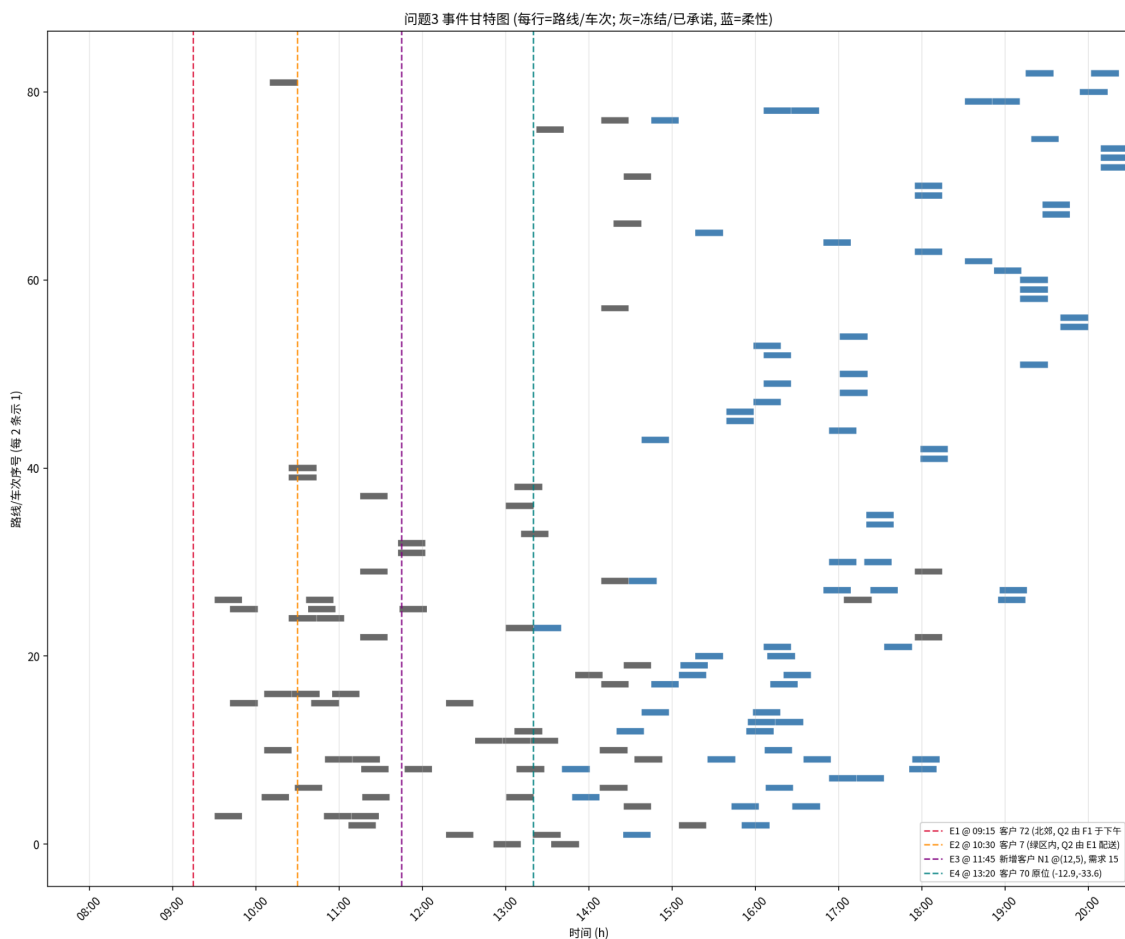


图 11: 问题三事件时序与冻结-柔性划分图

6.3 结果分析

6.3.1 确定性场景结果分析

问题三以问题二方案作为 T0 基线。T0 时刻总成本为 96143.62 元，实际启用物理车辆 108 辆，配送车次 168 趟。经过四次事件滚动调整后，终态 E4 的总成本下降为 95155.55 元，相比 T0 下降 988.06 元，降幅约为 1.03%；实际车辆数下降为 107 辆，车次下降为 165 趟。具体成本轨迹如表 13 所示。

表 13: 问题三滚动调度成本轨迹

快照	总成本/元	启动成本/元	能源成本/元	碳成本/元	等待成本/元	迟到成本/元	趟次	车辆数	政策违约数
T0	96143.62	43200.00	39966.07	8649.78	1326.35	3001.42	168	108	0
E1	95869.19	43200.00	39699.35	8591.47	1307.71	3070.62	167	108	0
E2	96119.97	43600.00	39678.35	8587.01	1183.92	3070.62	167	109	0
E3	96008.76	43600.00	39740.93	8600.60	1183.92	2883.31	167	109	0
E4	95155.55	42800.00	39671.93	8585.60	1214.71	2883.31	165	107	0

由表 13 可知, E1 取消订单后总成本下降 274.43 元, 主要来自柔性客户删除后能源成本和碳排放成本下降; E2 时间窗收紧后总成本上升 250.78 元, 原因是为满足更紧的服务窗口, 方案临时增加一辆 F1 物理车, 启动成本增加 400 元, 但等待成本下降部分抵消了该增量; E3 新增订单后总成本反而下降 111.21 元, 说明新增客户被插入已有柔性路径后触发了局部重排, 迟到成本下降幅度超过新增服务带来的行驶成本; E4 地址变更后总成本下降 853.20 元, 同时 F2 实际车辆数减少 2 辆、总趟次减少 2 趟, 是四个事件中对终态节约贡献最大的事件。

图 12 进一步展示了各事件对成本变化的贡献。该图能够直观说明动态调度并非简单叠加扰动成本, 而是在每次事件后重新利用柔性路径空间, 从而可能产生路径合并和车辆节省收益。

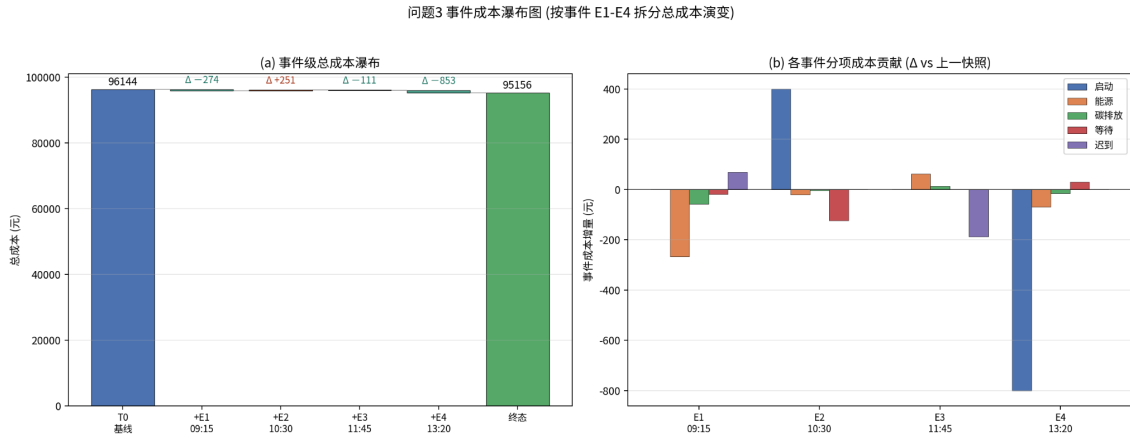


图 12: 问题三事件成本变化瀑布图

6.3.2 冻结-柔性空间演变分析

随着配送时间推进, 可调整客户数量逐渐减少。表 14 给出了各快照下的冻结节点和柔性节点数量。

表 14: 问题三冻结-柔性划分演变

快照	已发车路线	未发车路线	冻结虚拟节点数	柔性虚拟节点数	冻结占比
T0	0	168	0	285	0.00%
E1	15	152	15	267	5.32%
E2	49	118	58	224	20.57%
E3	54	113	89	194	31.45%
E4	68	97	112	171	39.58%

可以看出, T0 时刻所有路线均未发车, 柔性虚拟节点数为 285; 到 E4 时刻, 已有 112 个虚拟节点被冻结, 冻结占比达到 39.58%。这说明随着时间推进, 调度器可调整的空间不断收缩, 后续事件更多依赖局部修复而非全局重构。图 13 以面积图形式展示了冻结与柔性客户池的演变过程。

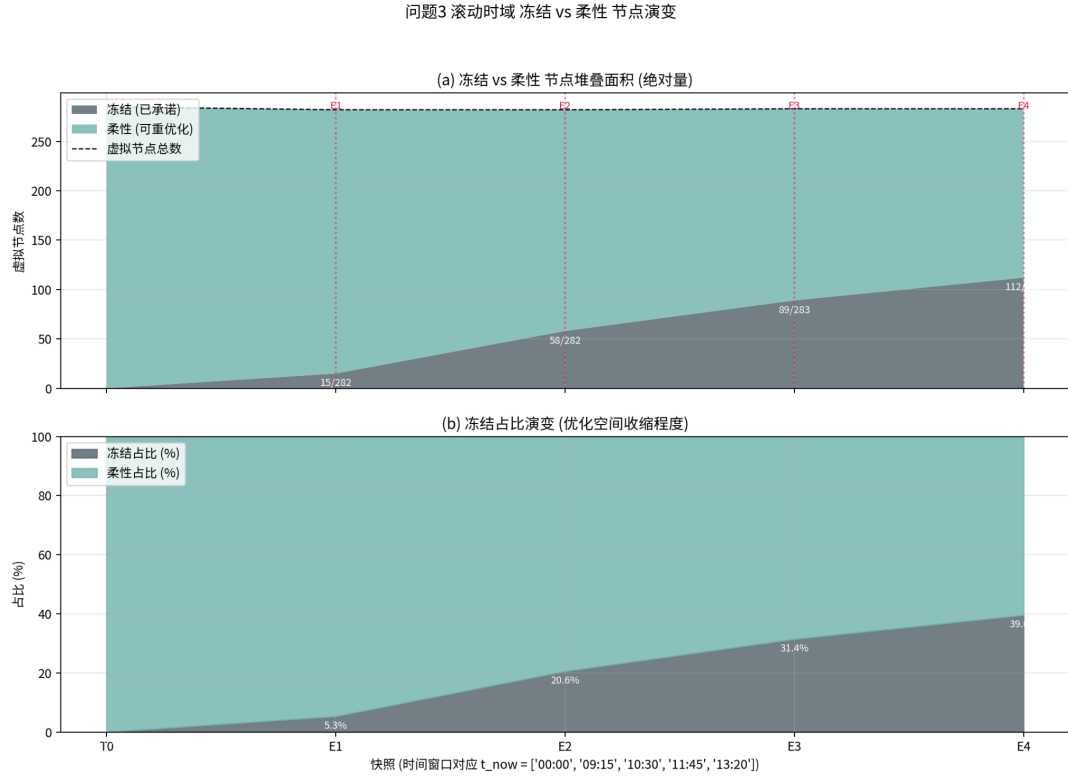


图 13: 问题三冻结-柔性节点演变图

冻结比例上升是动态调度中不可避免的现象。较早发生的事件拥有较大的柔性空间，容易通过重插入和路径合并降低成本；较晚发生的事件可调整范围缩小，更多表现为在局部路径内修复可行性。本文结果中 E4 虽然发生较晚，但由于地址调整释放了部分燃油车路线合并机会，仍带来了明显节约。

6.3.3 响应实时性分析

动态调度模型需要具备较强实时性。本文确定性场景中四个事件的响应耗时分别为 124.3 ms、143.9 ms、163.4 ms 和 139.6 ms，平均响应耗时约为 142.8 ms，均远低于秒级调度要求。图 14 展示了各事件响应耗时和受影响路径数量。

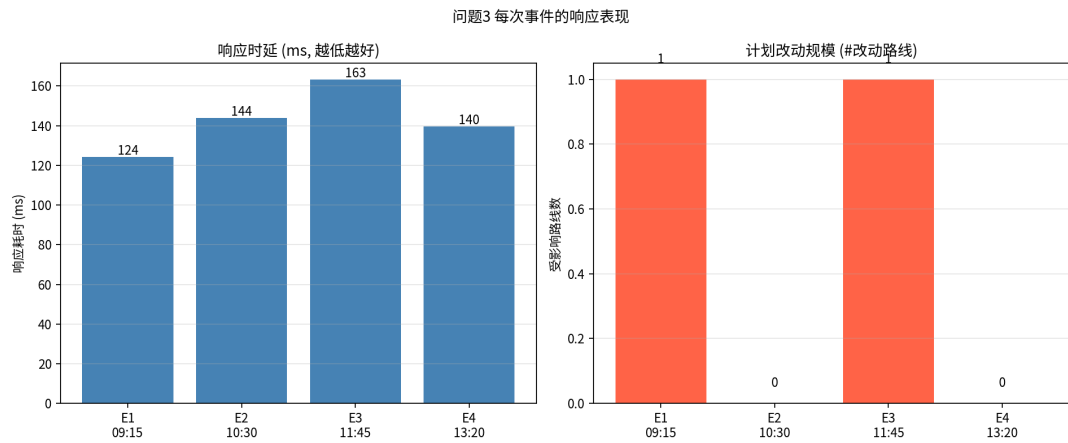


图 14: 问题三事件响应耗时

E3 的响应耗时最高，主要因为新增订单需要搜索可插入位置并重新评价容量、时间窗和政策约束；但其耗时仍低于 0.2s。四次事件中只有 E1 和 E3 直接影响既有路线，E2 和 E4 的受影响路线数为 0，说明时间窗收紧和地址变更在当前冻结状态下主要通过重新评价与局部合并完成修复。

6.3.4 动态路径终态分析

图 15 给出了终态三维时空路线图。该图以空间坐标和到达时间共同描述动态事件后的配送计划，可用于观察终态路径在空间与时间维度上的分布。与二维路径图相比，三维时空图能够同时反映车辆是否因时间窗或政策约束产生延后服务，也能展示事件触发后不同路径在时间轴上的重排效果。

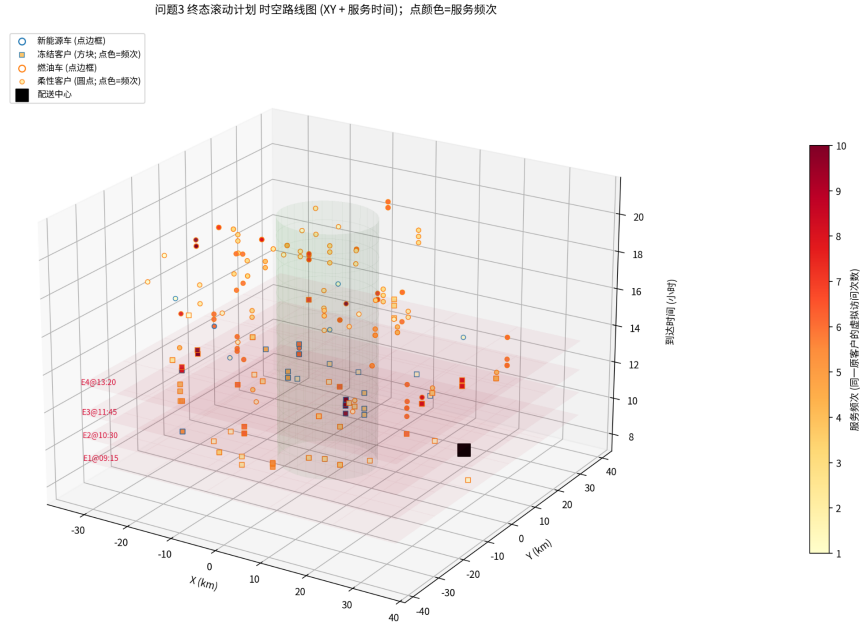


图 15: 问题三终态三维时空路线图

从终态结果看，新增客户 N1 被成功纳入配送计划，客户 72 取消订单后相关配送任务被删除，客户 70 地址变更后未造成政策违约或车辆数增加。终态方案在总成本、启用车辆数和政策合规性方面均优于事件前基线方案。

6.3.5 蒙特卡洛鲁棒性检验

为进一步验证动态调度模型在随机事件环境下的稳定性，本文以问题二基准方案为初始计划，进行 100 次蒙特卡洛试验。每次试验随机生成由订单取消、新增订单、地址变更和时间窗调整构成的事件流。事件到达过程服从泊松过程，平均到达率为

$$\lambda = 0.8 \text{ 件/h}, \quad (17)$$

事件触发时间窗口为 $[8.5, 15.5]$ ，单次试验事件数限制在 $[2, 10]$ 之间，事件类型权重为取消 30%、新增 30%、改址 20%、改时间窗 20%。

核心统计结果如表 15 所示。

表 15: 动态调度蒙特卡洛实验核心指标

指标	均值	中位数	标准差	最小值	最大值	P05	P95
事件数	5.52	5.50	1.99	2.00	10.00	2.00	9.00
成本变化/元	-1546.46	-1206.18	1376.25	-6050.98	1803.93	-4566.74	148.43
成本变化率/%	-1.616	-1.261	1.439	-6.325	1.886	-4.774	0.155
车次变化	-4.01	-3.00	2.91	-18.00	-1.00	-9.10	-1.00
最大响应耗时/ms	212.84	148.46	407.71	72.44	3048.94	77.21	240.36
平均响应耗时/ms	112.19	102.25	72.56	62.44	695.54	67.23	147.34
新增订单失败数	0.00	0.00	0.00	0.00	0.00	0.00	0.00
终态政策违约数	0.00	0.00	0.00	0.00	0.00	0.00	0.00
延迟过窗弧次	0.11	0.00	0.31	0.00	1.00	0.00	1.00

表 15 表明，在 100 次随机扰动试验中，终态政策违约次数始终为 0，新增订单插入失败数也始终为 0，说明动态调度算法对随机扰动具有较强鲁棒性。平均成本变化为 -1546.46 元，平均成本变化率为 -1.616%，说明多数随机场景中，取消订单和路径重组带来的节约能够抵消新增、改址或时间窗变化引入的额外成本。平均响应耗时为 112.19 ms，最大响应耗时的 95 分位数为 240.36 ms，远低于 1 s，表明算法具备在线应用潜力。

图 16 展示了蒙特卡洛实验中成本变化、事件数量、响应耗时和政策违约等指标的分布情况。从图中可以看出，成本变化整体集中在负值区域，说明动态重调度多数情况下能够产生一定节约；政策违约分布始终为零，进一步验证了政策修复机制的有效性。

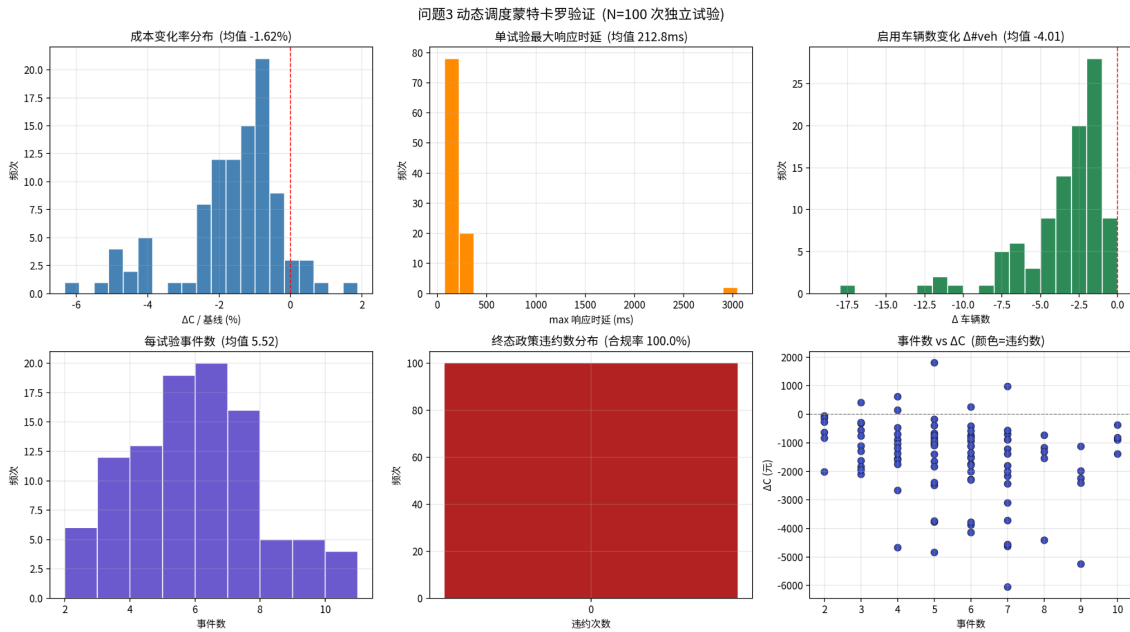


图 16: 问题三蒙特卡洛实验指标分布

6.4 问题三结论

问题三的核心思路是以问题二政策可行方案为初始计划，在事件发生时冻结已执行或已承诺路径，只对柔性后缀进行局部修复，并在每次调整后重新检查容量、时间窗和绿色区政策合规性。确定性场景中，模型完成四类事件响应，终态总成本为 95155.55 元，较 T0 下降 988.06 元；实际车辆数由 108 辆降至 107 辆，配送车次由 168 趟降至 165 趟，所有事件后快照政策违约次数均为 0，

平均响应耗时约为 142.8 ms。蒙特卡洛实验中, 100 次随机事件流均保持终态政策违约数和新增订单失败数为 0, 平均成本下降 1546.46 元, 最大响应耗时 P95 为 240.36 ms。

该结果说明, 冻结-柔性滚动重调度能够在不破坏已执行计划的前提下吸收动态扰动, 并保持较好的实时性、鲁棒性和政策合规保障能力。尤其是在多趟合并已经下沉到问题一和问题二静态 ALNS 阶段后, 问题三不再依赖事件阶段大规模重构, 而是通过增量维护、局部修复和零违约兜底来保证方案连续可执行。

7 模型的评价

本文围绕 A 题构建了由静态配送优化、绿色政策约束优化和动态滚动调度组成的递进式模型体系。

问题一以总成本最小为目标, 综合考虑车辆容量、时间窗、能源消耗、碳排放、等待成本和迟到惩罚等因素, 利用 **ALNS 算法和多趟合并机制**得到基础配送方案。结果表明, 该模型能够较好地匹配不同车型与客户需求, 提高车辆装载率和复用率, 为后续模型提供了稳定的静态基准。

在问题二中, 模型进一步引入绿色配送区限行政策, 将政策要求转化为点访问约束和路径穿越约束, 并在成本优化过程中保证政策可行。结果显示, **限行政策**会显著增加车辆启用数量和运营成本, 但模型能够通过**调整车型结构和服务时段实现零违约配送**, 较好地反映了环保政策对企业配送成本和车队配置的影响。

问题三在问题二方案基础上建立事件驱动的**滚动调度模型**, 针对订单取消、新增、地址变更和时间窗调整等突发情况进行实时修复。模型通过**冻结已执行任务、重优化未执行任务以及设置多级合规兜底机制**, 在保证绿色区政策约束不被破坏的同时, 保持了较高的响应速度和较好的成本控制能力, 体现出一定的动态适应性和**鲁棒性**。

总体来看, 本文模型结构清晰、约束较完整, 能够从成本、车辆利用、碳排放、政策合规和动态响应等多个角度评价配送方案, 具有较好的解释性和实际应用价值。但模型仍存在一定**局限**, 例如行驶时间和客户需求主要按确定性处理, 绿色区约束对真实路网有所简化, 新能源车辆充电与续航因素尚未深入考虑。后续可结合真实道路网络、随机交通环境和充电设施约束进一步完善模型。

参考文献

- [1] Dantzig G B, Ramser J H. The truck dispatching problem[J]. Management Science, 1959, 6(1): 80-91.
- [2] 曹庆奎, 杨凯文, 任向阳, 等. 基于交通流的多模糊时间窗车辆路径优化 [J]. 运筹与管理, 2018, 27(8): 20-26.
- [3] 董文永, 张文生, 于瑞国. 求解组合优化问题伊藤算法的收敛性和期望收敛速度分析 [J]. 计算机学报, 2011, 34(4): 636-646.
- [4] Liu Y, Baldacci R, Zhou J. Efficient Feasibility Checks and an Adaptive Large Neighborhood Search Algorithm for the Time-Dependent Green Vehicle Routing Problem with Time Windows[J]. European Journal of Operational Research, 2023, 310(1): 133-155.
- [5] [1]ChatGPT,2026 年 4 月 24 日版本 GPT-5.4,OpenAI,2026-4-25
- [6] [2]Claude AI,2026 年 4 月 25 日版本 Claude 4.6 Sonnet,Anthropic,2026-4-26

A 附录 A 代码文件与核心功能说明

表 16: 代码文件与核心功能说明

模块	文件	核心功能
公共模块	data_loader.py cost_model.py policy.py multitrip_ops.py alns_solver.py	数据读取、缺失值补全、虚拟节点拆分 时变速度、能耗与碳排放计算、路径成本评估 绿色配送区点规则与弧规则判定 多趟合并、返仓交接 ALNS 算法框架（破坏/修复、模拟退火、自适应权重）
问题一	solve_q1.py	无政策约束静态调度求解与结果输出
问题二	solve_q2.py	绿色配送区政策约束调度求解与对比分析
问题三	dynamic_solver.py dynamic_random_eval.py solve_q3.py	事件驱动动态调度、冻结-柔性划分、政策修复 蒙特卡洛随机事件流鲁棒性检验 确定性动态事件序列滚动调度与结果输出

B 附录 B 代码

B.1 公共模块代码

data_loader.py

代码 1: 公共模块代码: data_loader.py

```

1 from __future__ import annotations
2 import copy
3 import math
4 from dataclasses import dataclass, field
5 from pathlib import Path
6 from typing import Dict, List, Optional, Tuple
7 import numpy as np
8 import pandas as pd
9 DATA_DIR = Path(__file__).resolve().parent
10 @dataclass
11 class Node:
12     node_id: int
13     orig_id: int
14     x: float
15     y: float
16     demand_w: float
17     demand_v: float
18     tw_start: float
19     tw_end: float
20     service: float
21     split_index: int = 0

```

```

22     split_total: int = 1
23 @dataclass
24 class VehicleType:
25     name: str
26     is_fuel: bool
27     capacity_w: float
28     capacity_v: float
29     count: int
30     load_surcharge: float
31 @dataclass
32 class CostParams:
33     fixed_cost: float = 400.0
34     wait_cost: float = 20.0
35     late_cost: float = 50.0
36     fuel_price: float = 7.61
37     elec_price: float = 1.64
38     eta_fuel: float = 2.547
39     gamma_elec: float = 0.501
40     carbon_cost: float = 0.65
41     service_time: float = 20.0 / 60.0
42     depart_time: float = 8.0
43     return_time: float = 24.0
44 @dataclass
45 class ProblemData:
46     nodes: List[Node]
47     dist: np.ndarray
48     vehicle_types: List[VehicleType]
49     cost: CostParams
50     customer_agg: pd.DataFrame
51     time_window_raw: pd.DataFrame
52     coord_raw: pd.DataFrame
53     split_map: Dict[int, List[int]] = field(default_factory=dict)
54     @property
55     def n_nodes(self) -> int:
56         return len(self.nodes)
57     @property
58     def customer_nodes(self) -> List[Node]:
59         return [n for n in self.nodes if n.node_id != 0]
60 def _parse_time(s: str) -> float:
61     if isinstance(s, (int, float)):
62         return float(s)
63     h, m = str(s).split(":")
64     return int(h) + int(m) / 60.0
65 def _aggregate_orders(orders: pd.DataFrame) -> pd.DataFrame:
66     orders = _impute_order_missing_values(orders)
67     agg = (
68         orders.groupby("目标客户编号")
69         .agg(
70             总重量=("重量", "sum"),

```



```

71         总体积=("体积", "sum"),
72         订单数=("订单编号", "count"),
73     )
74     .reset_index()
75     .rename(columns={"目标客户编号": "客户编号"})
76 )
77 return agg
78 def _impute_order_missing_values(orders: pd.DataFrame) -> pd.DataFrame:
79     df = orders.copy()
80     complete = (
81         df["重量"].notna()
82         & df["体积"].notna()
83         & (df["重量"] > 0)
84         & (df["体积"] > 0)
85     )
86     if not complete.any():
87         return df
88     density = df.loc[complete, "重量"] / df.loc[complete, "体积"]
89     customer_density = density.groupby(df.loc[complete, "目标客户编号"]).median()
90     global_density = float(density.median())
91     rho = df["目标客户编号"].map(customer_density).fillna(global_density)
92     miss_w = df["重量"].isna() & df["体积"].notna()
93     miss_v = df["体积"].isna() & df["重量"].notna()
94     df.loc[miss_w, "重量"] = df.loc[miss_w, "体积"] * rho[miss_w]
95     df.loc[miss_v, "体积"] = df.loc[miss_v, "重量"] / rho[miss_v]
96     return df
97 def _min_capacity(vehicle_types: List[VehicleType]) -> Tuple[float, float]:
98     return (
99         min(v.capacity_w for v in vehicle_types),
100         min(v.capacity_v for v in vehicle_types),
101     )
102 def _split_factor(w: float, v: float, ref_w: float, ref_v: float) -> int:
103     k_w = math.ceil(w / ref_w) if w > ref_w else 1
104     k_v = math.ceil(v / ref_v) if v > ref_v else 1
105     return max(1, k_w, k_v)
106 def default_vehicle_types() -> List[VehicleType]:
107     return [
108         VehicleType("F1", True, 3000, 13.5, 60, 0.40),
109         VehicleType("F2", True, 1500, 10.8, 50, 0.40),
110         VehicleType("F3", True, 1250, 6.5, 50, 0.40),
111         VehicleType("E1", False, 3000, 15.0, 10, 0.35),
112         VehicleType("E2", False, 1250, 8.5, 15, 0.35),
113     ]
114 def load_problem(
115     data_dir: Path = DATA_DIR,
116     vehicle_types: List[VehicleType] | None = None,
117     cost: CostParams | None = None,
118 ) -> ProblemData:
119     if vehicle_types is None:

```

```

120     vehicle_types = default_vehicle_types()
121     if cost is None:
122         cost = CostParams()
123     coords = pd.read_excel(data_dir / "客户坐标信息.xlsx")
124     tw = pd.read_excel(data_dir / "时间窗.xlsx")
125     orders = pd.read_excel(data_dir / "订单信息.xlsx")
126     dist_df = pd.read_excel(data_dir / "距离矩阵.xlsx")
127     depot_row = coords[coords["类型"] == "配送中心"].iloc[0]
128     depot_x, depot_y = float(depot_row["X (km)"]), float(depot_row["Y (km)"])
129     coord_map = {
130         int(r["ID"]): (float(r["X (km)"]), float(r["Y (km)"]))
131         for _, r in coords.iterrows()
132     }
133     tw_map = {
134         int(r["客户编号"]): (_parse_time(r["开始时间"]), _parse_time(r["结束时间"]))
135         for _, r in tw.iterrows()
136     }
137     customer_agg = _aggregate_orders(orders)
138     demand_map = {
139         int(r["客户编号"]): (float(r["总重量"]), float(r["总体积"]))
140         for _, r in customer_agg.iterrows()
141     }
142     ref_w, ref_v = _min_capacity(vehicle_types)
143     nodes: List[Node] = []
144     depot_node = Node(
145         node_id=0, orig_id=0, x=depot_x, y=depot_y,
146         demand_w=0.0, demand_v=0.0,
147         tw_start=cost.depart_time, tw_end=cost.return_time,
148         service=0.0,
149     )
150     nodes.append(depot_node)
151     split_map: Dict[int, List[int]] = {}
152     for orig_id in sorted(coord_map.keys()):
153         if orig_id == 0:
154             continue
155         if orig_id not in demand_map:
156             continue
157         w, v = demand_map[orig_id]
158         x, y = coord_map[orig_id]
159         a, b = tw_map.get(orig_id, (cost.depart_time, cost.return_time))
160         k = _split_factor(w, v, ref_w, ref_v)
161         ids_for_this = []
162         if k == 1:
163             node = Node(
164                 node_id=len(nodes), orig_id=orig_id, x=x, y=y,
165                 demand_w=w, demand_v=v,
166                 tw_start=a, tw_end=b, service=cost.service_time,
167                 split_index=0, split_total=1,
168             )

```

```

169         ids_for_this.append(node.node_id)
170         nodes.append(node)
171     else:
172         w_per = w / k
173         v_per = v / k
174         for p in range(1, k + 1):
175             node = Node(
176                 node_id=len(nodes), orig_id=orig_id, x=x, y=y,
177                 demand_w=w_per, demand_v=v_per,
178                 tw_start=a, tw_end=b, service=cost.service_time,
179                 split_index=p, split_total=k,
180             )
181             ids_for_this.append(node.node_id)
182             nodes.append(node)
183         split_map[orig_id] = ids_for_this
184     N = len(nodes)
185     dist_cols = [c for c in dist_df.columns if c != "客户"]
186     dist_cols_sorted = sorted(dist_cols, key=lambda c: int(c))
187     orig_dist = dist_df.set_index("客户").loc[
188         sorted(dist_df["客户"].tolist()), dist_cols_sorted
189     ].to_numpy(dtype=float)
190     dist = np.zeros((N, N), dtype=float)
191     for i in range(N):
192         oi = nodes[i].orig_id
193         for j in range(N):
194             if i == j:
195                 dist[i, j] = 0.0
196                 continue
197             oj = nodes[j].orig_id
198             if oi == oj:
199                 dist[i, j] = 0.0
200             else:
201                 dist[i, j] = orig_dist[oi, oj]
202     return ProblemData(
203         nodes=nodes,
204         dist=dist,
205         vehicle_types=vehicle_types,
206         cost=cost,
207         customer_agg=customer_agg,
208         time_window_raw=tw,
209         coord_raw=coords,
210         split_map=split_map,
211     )
212 def _estimate_detour_factor(problem: ProblemData) -> float:
213     ns = problem.nodes
214     N = len(ns)
215     ratios: List[float] = []
216     step = max(1, N // 40)
217     for i in range(1, N, step):

```

```

218         for j in range(0, i, step):
219             ed = math.hypot(ns[i].x - ns[j].x, ns[i].y - ns[j].y)
220             if ed < 0.5:
221                 continue
222             ratios.append(problem.dist[i, j] / ed)
223     if not ratios:
224         return 1.3
225     return float(np.median(ratios))
226 def mutate_problem_inplace(
227     problem: ProblemData,
228     add_customers: Optional[List[Dict]] = None,
229     mutate_coords: Optional[Dict[int, Tuple[float, float]]] = None,
230     mutate_tw: Optional[Dict[int, Tuple[float, float]]] = None,
231     detour_factor: Optional[float] = None,
232 ) -> List[int]:
233     add_customers = add_customers or []
234     mutate_coords = mutate_coords or {}
235     mutate_tw = mutate_tw or {}
236     if detour_factor is None:
237         detour_factor = _estimate_detour_factor(problem)
238     old_N = len(problem.nodes)
239     for nid, (nx, ny) in mutate_coords.items():
240         if nid < 0 or nid >= old_N:
241             raise ValueError(f"mutate_coords: bad node id {nid}")
242         problem.nodes[nid].x = float(nx)
243         problem.nodes[nid].y = float(ny)
244     for nid, (ts, te) in mutate_tw.items():
245         if nid < 0 or nid >= old_N:
246             raise ValueError(f"mutate_tw: bad node id {nid}")
247         problem.nodes[nid].tw_start = float(ts)
248         problem.nodes[nid].tw_end = float(te)
249     ref_w, ref_v = _min_capacity(problem.vehicle_types)
250     new_node_ids: List[int] = []
251     for entry in add_customers:
252         orig_id = int(entry["orig_id"])
253         x = float(entry["x"])
254         y = float(entry["y"])
255         w = float(entry["demand_w"])
256         v = float(entry["demand_v"])
257         ts = float(entry.get("tw_start", problem.cost.depart_time))
258         te = float(entry.get("tw_end", problem.cost.return_time))
259         svc = float(entry.get("service", problem.cost.service_time))
260         k = _split_factor(w, v, ref_w, ref_v)
261         ids_for_this: List[int] = []
262         if k == 1:
263             node = Node(
264                 node_id=len(problem.nodes), orig_id=orig_id, x=x, y=y,
265                 demand_w=w, demand_v=v,
266                 tw_start=ts, tw_end=te, service=svc,

```

```

267         split_index=0, split_total=1,
268     )
269     ids_for_this.append(node.node_id)
270     problem.nodes.append(node)
271 else:
272     w_per = w / k
273     v_per = v / k
274     for p in range(1, k + 1):
275         node = Node(
276             node_id=len(problem.nodes), orig_id=orig_id, x=x, y=y,
277             demand_w=w_per, demand_v=v_per,
278             tw_start=ts, tw_end=te, service=svc,
279             split_index=p, split_total=k,
280         )
281         ids_for_this.append(node.node_id)
282         problem.nodes.append(node)
283     problem.split_map[orig_id] = ids_for_this
284     new_node_ids.extend(ids_for_this)
285 N = len(problem.nodes)
286 new_dist = np.zeros((N, N), dtype=float)
287 dirty = set(mutate_coords.keys()) | set(range(old_N, N))
288 for i in range(N):
289     ni = problem.nodes[i]
290     for j in range(N):
291         if i == j:
292             continue
293         nj = problem.nodes[j]
294         if i < old_N and j < old_N and i not in dirty and j not in dirty:
295             new_dist[i, j] = problem.dist[i, j]
296             continue
297         if ni.orig_id == nj.orig_id and ni.orig_id != 0:
298             new_dist[i, j] = 0.0
299         else:
300             ed = math.hypot(ni.x - nj.x, ni.y - nj.y)
301             new_dist[i, j] = ed * detour_factor
302 problem.dist = new_dist
303 return new_node_ids
304 def clone_problem(problem: ProblemData) -> ProblemData:
305     return ProblemData(
306         nodes=[copy.copy(n) for n in problem.nodes],
307         dist=np.array(problem.dist, copy=True),
308         vehicle_types=problem.vehicle_types,
309         cost=problem.cost,
310         customer_agg=problem.customer_agg,
311         time_window_raw=problem.time_window_raw,
312         coord_raw=problem.coord_raw,
313         split_map={k: list(v) for k, v in problem.split_map.items()},
314     )
315 if __name__ == "__main__":

```

```

316 pd_ = load_problem()
317 print(f"Total nodes (including depot): {pd_.n_nodes}")
318 print(f"Customer nodes (virtual): {len(pd_.customer_nodes)}")
319 print(f"Number of original customers needing service: {len(pd_.split_map)}")
320 split_counts = [len(v) for v in pd_.split_map.values()]
321 print(f"Customers split into multiple virtual points: "
322       f"{sum(1 for c in split_counts if c > 1)}")
323 print(f"Max split count for a single customer: {max(split_counts)}")
324 print(f"Total virtual demand weight: "
325       f"{sum(n.demand_w for n in pd_.customer_nodes):.2f} kg")
326 print(f"Total virtual demand volume: "
327       f"{sum(n.demand_v for n in pd_.customer_nodes):.4f} m^3")
328 print(f"Distance matrix shape: {pd_.dist.shape}")
329 print(f"Sample distance node 0->1: {pd_.dist[0, 1]:.3f}")

```

cost_model.py

代码 2: 公共模块代码: cost_model.py

```

1 from __future__ import annotations
2 from dataclasses import dataclass, field
3 from typing import Dict, List, Optional, Tuple
4 import numpy as np
5 from data_loader import CostParams, Node, ProblemData, VehicleType
6 from policy import PolicyConfig, is_arc_violation, is_violation
7 SPEED_SEGMENTS: List[Tuple[float, float, float]] = [
8     (0.0, 8.0, 55.3),
9     (8.0, 9.0, 9.8),
10    (9.0, 10.0, 55.3),
11    (10.0, 11.5, 35.4),
12    (11.5, 13.0, 9.8),
13    (13.0, 15.0, 55.3),
14    (15.0, 17.0, 35.4),
15    (17.0, 24.0, 55.3),
16 ]
17 def speed_at(t: float) -> float:
18     for s, e, v in SPEED_SEGMENTS:
19         if s <= t < e:
20             return v
21     return SPEED_SEGMENTS[-1][2]
22 def avg_speed_for_travel(t_start: float, distance: float) -> float:
23     if distance <= 1e-12:
24         return speed_at(t_start)
25     remaining = distance
26     t = t_start
27     total_time = 0.0
28     for s, e, v in SPEED_SEGMENTS:
29         if e <= t:
30             continue

```

```

31     seg_start = max(s, t)
32     seg_end = e
33     seg_avail_time = seg_end - seg_start
34     if seg_avail_time <= 0:
35         continue
36     seg_max_dist = v * seg_avail_time
37     if remaining <= seg_max_dist + 1e-12:
38         total_time += remaining / v
39         remaining = 0.0
40         break
41     total_time += seg_avail_time
42     remaining -= seg_max_dist
43     t = seg_end
44     if remaining > 1e-9:
45         v_last = SPEED_SEGMENTS[-1][2]
46         total_time += remaining / v_last
47     return distance / total_time if total_time > 0 else speed_at(t_start)
48 def travel_time(t_start: float, distance: float) -> float:
49     if distance <= 1e-12:
50         return 0.0
51     remaining = distance
52     t = t_start
53     total_time = 0.0
54     for s, e, v in SPEED_SEGMENTS:
55         if e <= t:
56             continue
57         seg_start = max(s, t)
58         seg_end = e
59         seg_avail_time = seg_end - seg_start
60         if seg_avail_time <= 0:
61             continue
62         seg_max_dist = v * seg_avail_time
63         if remaining <= seg_max_dist + 1e-12:
64             total_time += remaining / v
65             return total_time
66         total_time += seg_avail_time
67         remaining -= seg_max_dist
68         t = seg_end
69     if remaining > 1e-9:
70         v_last = SPEED_SEGMENTS[-1][2]
71         total_time += remaining / v_last
72     return total_time
73 def fpk(v: float) -> float:
74     return 0.0025 * v * v - 0.2554 * v + 31.75
75 def epk(v: float) -> float:
76     return 0.0014 * v * v - 0.12 * v + 36.19
77 def arc_energy_cost(
78     distance: float,
79     t_start: float,

```

```

80     load_w: float,
81     load_v: float,
82     vt: VehicleType,
83     cost: CostParams,
84 ) -> Tuple[float, float]:
85     if distance <= 1e-12:
86         return 0.0, 0.0
87     v_avg = avg_speed_for_travel(t_start, distance)
88     rho = max(load_w / vt.capacity_w, load_v / vt.capacity_v) if vt.capacity_w > 0 else 0.0
89     rho = min(max(rho, 0.0), 1.0)
90     if vt.is_fuel:
91         liters = distance / 100.0 * fpk(v_avg) * (1.0 + vt.load_surcharge * rho)
92         energy_money = liters * cost.fuel_price
93         carbon_kg = liters * cost.eta_fuel
94     else:
95         kwh = distance / 100.0 * epk(v_avg) * (1.0 + vt.load_surcharge * rho)
96         energy_money = kwh * cost.elec_price
97         carbon_kg = kwh * cost.gamma_elec
98     carbon_money = carbon_kg * cost.carbon_cost
99     return energy_money + carbon_money, carbon_kg
100 @dataclass
101 class RouteEval:
102     feasible_cap: bool
103     feasible_return: bool
104     total_w: float
105     total_v: float
106     fixed_cost: float
107     energy_cost: float
108     carbon_cost_money: float
109     carbon_kg: float
110     wait_cost: float
111     late_cost: float
112     total_cost: float
113     arrival: List[float] = field(default_factory=list)
114     service_start: List[float] = field(default_factory=list)
115     service_end: List[float] = field(default_factory=list)
116     early: List[float] = field(default_factory=list)
117     late: List[float] = field(default_factory=list)
118     feasible_policy: bool = True
119     policy_violation_count: int = 0
120 @dataclass
121 class Route:
122     vt_index: int
123     customers: List[int]
124     vehicle_id: Optional[int] = None
125     arc_depart_floor: Optional[Dict[int, float]] = None
126     def copy(self) -> "Route":
127         floor = None if self.arc_depart_floor is None else dict(self.arc_depart_floor)
128         return Route(self.vt_index, list(self.customers), self.vehicle_id, floor)

```



```

129 def _optimal_depart_time(
130     route: Route, problem: ProblemData, earliest: float,
131 ) -> float:
132     if not route.customers:
133         return earliest
134     first = route.customers[0]
135     first_node = problem.nodes[first]
136     d0 = problem.dist[0, first]
137     target_arrival = first_node.tw_start
138     t = earliest
139     for _ in range(6):
140         tt = travel_time(t, d0)
141         need = target_arrival - tt
142         if need <= earliest:
143             return earliest
144         t_new = max(earliest, need)
145         if abs(t_new - t) < 1e-3:
146             break
147         t = t_new
148     return t
149 def evaluate_route(
150     route: Route,
151     problem: ProblemData,
152     policy: Optional[PolicyConfig] = None,
153     frozen_len: int = 0,
154 ) -> RouteEval:
155     vt = problem.vehicle_types[route.vt_index]
156     cost = problem.cost
157     nodes = problem.nodes
158     dist = problem.dist
159     total_w = sum(nodes[c].demand_w for c in route.customers)
160     total_v = sum(nodes[c].demand_v for c in route.customers)
161     feasible_cap = (total_w <= vt.capacity_w + 1e-6) and (total_v <= vt.capacity_v + 1e-6)
162     fixed_cost = cost.fixed_cost if route.customers else 0.0
163     energy_cost = 0.0
164     carbon_money = 0.0
165     carbon_kg_total = 0.0
166     wait_cost = 0.0
167     late_cost = 0.0
168     arrival = []
169     service_start = []
170     service_end = []
171     early = []
172     late = []
173     if not route.customers:
174         return RouteEval(
175             feasible_cap=True, feasible_return=True,
176             total_w=0.0, total_v=0.0,
177             fixed_cost=0.0, energy_cost=0.0,

```

```

178         carbon_cost_money=0.0, carbon_kg=0.0,
179         wait_cost=0.0, late_cost=0.0, total_cost=0.0,
180     )
181     load_w = total_w
182     load_v = total_v
183     t = _optimal_depart_time(route, problem, cost.depart_time)
184     floor = route.arc_depart_floor
185     if floor is not None:
186         t = max(t, floor.get(0, 0.0))
187     prev = 0
188     arrival.append(t)
189     service_start.append(t)
190     service_end.append(t)
191     for idx, c in enumerate(route.customers):
192         d = dist[prev, c]
193         ec, ck = arc_energy_cost(d, t, load_w, load_v, vt, cost)
194         energy_money_only = ec - ck * cost.carbon_cost
195         energy_cost += energy_money_only
196         carbon_money += ck * cost.carbon_cost
197         carbon_kg_total += ck
198         tt = travel_time(t, d)
199         t_arrive = t + tt
200         node = nodes[c]
201         if t_arrive < node.tw_start:
202             e = node.tw_start - t_arrive
203             l = 0.0
204         elif t_arrive > node.tw_end:
205             e = 0.0
206             l = t_arrive - node.tw_end
207         else:
208             e = 0.0
209             l = 0.0
210         wait_cost += cost.wait_cost * e
211         late_cost += cost.late_cost * l
212         t_start = max(t_arrive, node.tw_start)
213         t_end = t_start + node.service
214         arrival.append(t_arrive)
215         service_start.append(t_start)
216         service_end.append(t_end)
217         early.append(e)
218         late.append(l)
219         load_w -= node.demand_w
220         load_v -= node.demand_v
221         t = t_end
222         if floor is not None:
223             t = max(t, floor.get(idx + 1, 0.0))
224         prev = c
225     d = dist[prev, 0]
226     ec, ck = arc_energy_cost(d, t, load_w, load_v, vt, cost)

```

```

227     energy_money_only = ec - ck * cost.carbon_cost
228     energy_cost += energy_money_only
229     carbon_money += ck * cost.carbon_cost
230     carbon_kg_total += ck
231     tt = travel_time(t, d)
232     t_return = t + tt
233     arrival.append(t_return)
234     service_start.append(t_return)
235     service_end.append(t_return)
236     feasible_return = t_return <= cost.return_time + 1e-6
237     feasible_policy = True
238     violation_count = 0
239     if policy is not None and vt.is_fuel:
240         N = len(route.customers)
241         skip_return = (frozen_len >= N and N > 0)
242         for idx, c in enumerate(route.customers):
243             if idx < frozen_len:
244                 continue
245             n = nodes[c]
246             is_green = policy.in_green_zone(n.x, n.y) and n.orig_id != 0
247             if is_green and is_violation(arrival[idx + 1], vt, True, policy):
248                 feasible_policy = False
249                 violation_count += 1
250         seq = [0] + list(route.customers) + [0]
251         for i in range(len(seq) - 1):
252             if i < frozen_len:
253                 continue
254             if i == N and skip_return:
255                 continue
256             u = seq[i]
257             v = seq[i + 1]
258             nu = nodes[u]
259             nv = nodes[v]
260             src_in_green = (u != 0) and policy.in_green_zone(nu.x, nu.y)
261             dst_in_green = (v != 0) and policy.in_green_zone(nv.x, nv.y)
262             t_leave_i = service_end[i]
263             if floor is not None:
264                 t_leave_i = max(t_leave_i, floor.get(i, 0.0))
265             if is_arc_violation(
266                 nu.x, nu.y, nv.x, nv.y,
267                 t_leave_i, arrival[i + 1],
268                 vt, src_in_green, dst_in_green, policy,
269             ):
270                 feasible_policy = False
271                 violation_count += 1
272     total_cost = fixed_cost + energy_cost + carbon_money + wait_cost + late_cost
273     return RouteEval(
274         feasible_cap=feasible_cap,
275         feasible_return=feasible_return,

```

```

276     total_w=total_w, total_v=total_v,
277     fixed_cost=fixed_cost,
278     energy_cost=energy_cost,
279     carbon_cost_money=carbon_money,
280     carbon_kg=carbon_kg_total,
281     wait_cost=wait_cost,
282     late_cost=late_cost,
283     total_cost=total_cost,
284     arrival=arrival,
285     service_start=service_start,
286     service_end=service_end,
287     early=early,
288     late=late,
289     feasible_policy=feasible_policy,
290     policy_violation_count=violation_count,
291 )
292 def first_violating_arc_index(
293     route: Route,
294     problem: ProblemData,
295     policy: PolicyConfig,
296     frozen_len: int = 0,
297 ) -> Optional[int]:
298     vt = problem.vehicle_types[route.vt_index]
299     if not vt.is_fuel or not route.customers:
300         return None
301     ev = evaluate_route(route, problem, policy=policy, frozen_len=frozen_len)
302     if ev.feasible_policy:
303         return None
304     nodes = problem.nodes
305     N = len(route.customers)
306     skip_return = (frozen_len >= N and N > 0)
307     arrival = ev.arrival
308     service_end = ev.service_end
309     for idx, c in enumerate(route.customers):
310         if idx < frozen_len:
311             continue
312         n = nodes[c]
313         is_green = policy.in_green_zone(n.x, n.y) and n.orig_id != 0
314         if is_green and is_violation(arrival[idx + 1], vt, True, policy):
315             return idx
316     seq = [0] + list(route.customers) + [0]
317     floor = route.arc_depart_floor
318     for i in range(len(seq) - 1):
319         if i < frozen_len:
320             continue
321         if i == N and skip_return:
322             continue
323         u = seq[i]
324         v = seq[i + 1]

```

```

325     nu = nodes[u]
326     nv = nodes[v]
327     src_in_green = (u != 0) and policy.in_green_zone(nu.x, nu.y)
328     dst_in_green = (v != 0) and policy.in_green_zone(nv.x, nv.y)
329     t_leave_i = service_end[i]
330     if floor is not None:
331         t_leave_i = max(t_leave_i, floor.get(i, 0.0))
332     if is_arc_violation(
333         nu.x, nu.y, nv.x, nv.y,
334         t_leave_i, arrival[i + 1],
335         vt, src_in_green, dst_in_green, policy,
336     ):
337         return i
338     return None
339 def evaluate_route_partial(
340     route: Route,
341     problem: ProblemData,
342     frozen_prefix_len: int,
343     policy: Optional[PolicyConfig] = None,
344 ) -> Tuple[RouteEval, RouteEval]:
345     frozen_prefix_len = max(0, min(frozen_prefix_len, len(route.customers)))
346     frozen_route = Route(route.vt_index, list(route.customers[:frozen_prefix_len]))
347     frozen_ev = evaluate_route(frozen_route, problem, policy=policy)
348     full_ev = evaluate_route(route, problem, policy=policy)
349     return frozen_ev, full_ev
350 @dataclass
351 class Solution:
352     routes: List[Route] = field(default_factory=list)
353     unassigned: List[int] = field(default_factory=list)
354     def copy(self) -> "Solution":
355         sol = Solution()
356         sol.routes = [r.copy() for r in self.routes]
357         sol.unassigned = list(self.unassigned)
358         return sol
359     def n_routes(self) -> int:
360         return sum(1 for r in self.routes if r.customers)
361 @dataclass
362 class SolutionEval:
363     feasible: bool
364     total_cost: float
365     fixed_cost: float
366     energy_cost: float
367     carbon_cost_money: float
368     carbon_kg: float
369     wait_cost: float
370     late_cost: float
371     unassigned_penalty: float
372     n_routes: int
373     route_evals: List[RouteEval]

```

```

374     feasible_policy: bool = True
375     policy_violation_count: int = 0
376 UNASSIGNED_PENALTY = 1e6
377 HANDOVER_TIME: float = 0.25
378 def _count_physical_vehicles(sol: Solution) -> Tuple[int, dict]:
379     seen_ids: set = set()
380     anon = 0
381     per_vt: dict = {}
382     for r in sol.routes:
383         if not r.customers:
384             continue
385         vid = getattr(r, "vehicle_id", None)
386         if vid is None:
387             anon += 1
388             per_vt[r.vt_index] = per_vt.get(r.vt_index, 0) + 1
389         else:
390             if vid not in seen_ids:
391                 seen_ids.add(vid)
392                 per_vt[r.vt_index] = per_vt.get(r.vt_index, 0) + 1
393     return len(seen_ids) + anon, per_vt
394 def _check_multitrip_consistency(
395     sol: Solution,
396     route_evals: List[RouteEval],
397     problem: ProblemData,
398 ) -> Tuple[bool, int]:
399     groups: dict = {}
400     for i, r in enumerate(sol.routes):
401         if not r.customers:
402             continue
403         vid = getattr(r, "vehicle_id", None)
404         if vid is None:
405             continue
406         groups.setdefault(vid, []).append(i)
407     n_viol = 0
408     ok = True
409     for vid, idx_list in groups.items():
410         if len(idx_list) < 2:
411             continue
412         vts = {sol.routes[i].vt_index for i in idx_list}
413         if len(vts) > 1:
414             ok = False
415             n_viol += 1
416             continue
417         idx_sorted = sorted(
418             idx_list, key=lambda i: route_evals[i].arrival[0] if route_evals[i].arrival
419         )
420         for j in range(1, len(idx_sorted)):
421             prev = route_evals[idx_sorted[j - 1]]

```

```

422         cur = route_evals[idx_sorted[j]]
423         if not prev.arrival or not cur.arrival:
424             continue
425         prev_return = prev.arrival[-1]
426         cur_depart = cur.arrival[0]
427         if cur_depart + 1e-6 < prev_return + HANDOVER_TIME:
428             ok = False
429             n_viol += 1
430             break
431     return ok, n_viol
432 def evaluate_solution(
433     sol: Solution,
434     problem: ProblemData,
435     policy: Optional[PolicyConfig] = None,
436     frozen_lens: Optional[List[int]] = None,
437 ) -> SolutionEval:
438     energy = carb_m = carb_kg = wait = late = 0.0
439     feasible = True
440     feasible_policy = True
441     violation_count = 0
442     route_evals: List[RouteEval] = []
443     per_route_fixed = 0.0
444     for ri, r in enumerate(sol.routes):
445         flen = frozen_lens[ri] if frozen_lens is not None and ri < len(frozen_lens) else 0
446         ev = evaluate_route(r, problem, policy=policy, frozen_len=flen)
447         route_evals.append(ev)
448         if not (ev.feasible_cap and ev.feasible_return):
449             feasible = False
450         if not ev.feasible_policy:
451             feasible_policy = False
452             violation_count += ev.policy_violation_count
453         per_route_fixed += ev.fixed_cost
454         energy += ev.energy_cost
455         carb_m += ev.carbon_cost_money
456         carb_kg += ev.carbon_kg
457         wait += ev.wait_cost
458         late += ev.late_cost
459     n_physical, used_counts = _count_physical_vehicles(sol)
460     fixed = n_physical * problem.cost.fixed_cost
461     any_grouped = any(
462         getattr(r, "vehicle_id", None) is not None
463         for r in sol.routes if r.customers
464     )
465     if not any_grouped:
466         fixed = per_route_fixed
467     for idx, vt in enumerate(problem.vehicle_types):
468         if used_counts.get(idx, 0) > vt.count:
469             feasible = False
470     mt_ok, _mt_viol = _check_multitrip_consistency(sol, route_evals, problem)

```

```

471     if not mt_ok:
472         feasible = False
473     if sol.unassigned:
474         feasible = False
475     unassigned_penalty = UNASSIGNED_PENALTY * len(sol.unassigned)
476     total = fixed + energy + carb_m + wait + late + unassigned_penalty
477     n_rt = sum(1 for r in sol.routes if r.customers)
478     if policy is not None and not feasible_policy:
479         feasible = False
480     return SolutionEval(
481         feasible=feasible,
482         total_cost=total,
483         fixed_cost=fixed,
484         energy_cost=energy,
485         carbon_cost_money=carb_m,
486         carbon_kg=carb_kg,
487         wait_cost=wait,
488         late_cost=late,
489         unassigned_penalty=unassigned_penalty,
490         n_routes=n_rt,
491         route_evals=route_evals,
492         feasible_policy=feasible_policy,
493         policy_violation_count=violation_count,
494     )
495 if __name__ == "__main__":
496     from data_loader import load_problem
497     problem = load_problem()
498     print("Speed at 08:30:", speed_at(8.5))
499     print("Speed at 11:00:", speed_at(11.0))
500     print("Travel 30 km starting at 8:30 (hours):",
501           round(travel_time(8.5, 30.0), 4))
502     print("Avg speed for 30 km starting 8:30:",
503           round(avg_speed_for_travel(8.5, 30.0), 3))
504     nodes = problem.nodes
505     route = Route(vt_index=0, customers=[1, 2, 3])
506     ev = evaluate_route(route, problem)
507     print("Sample route cost:", round(ev.total_cost, 2),
508           "feasible cap:", ev.feasible_cap,
509           "feasible return:", ev.feasible_return)

```

policy.py

代码 3: 公共模块代码: policy.py

```

1 from __future__ import annotations
2 from dataclasses import dataclass
3 from typing import Iterable, List
4 import numpy as np
5 from data_loader import Node, VehicleType

```



```

6 @dataclass
7 class PolicyConfig:
8     green_radius: float = 10.0
9     fuel_restrict_start: float = 8.0
10    fuel_restrict_end: float = 16.0
11    enforce_no_cross_green: bool = True
12    policy_id: str = "q2_green_zone_arc"
13    def in_green_zone(self, x: float, y: float) -> bool:
14        return (x * x + y * y) <= self.green_radius * self.green_radius + 1e-9
15    def in_restricted_window(self, t: float) -> bool:
16        return self.fuel_restrict_start - 1e-9 <= t < self.fuel_restrict_end - 1e-9
17    def segment_crosses_green_zone(
18        self, x1: float, y1: float, x2: float, y2: float,
19    ) -> bool:
20        r2 = self.green_radius * self.green_radius
21        dx = x2 - x1
22        dy = y2 - y1
23        l2 = dx * dx + dy * dy
24        if l2 < 1e-18:
25            return (x1 * x1 + y1 * y1) <= r2 + 1e-9
26        s = -(x1 * dx + y1 * dy) / l2
27        if s < 0.0:
28            s = 0.0
29        elif s > 1.0:
30            s = 1.0
31        px = x1 + s * dx
32        py = y1 + s * dy
33        return (px * px + py * py) <= r2 + 1e-9
34    def arc_time_overlaps_window(
35        self, t_leave: float, t_arrive: float,
36    ) -> bool:
37        return (
38            t_leave < self.fuel_restrict_end
39            and t_arrive > self.fuel_restrict_start
40        )
41    def compute_green_mask(nodes: Iterable[Node], policy: PolicyConfig) -> np.ndarray:
42        mask = []
43        for n in nodes:
44            if n.orig_id == 0:
45                mask.append(False)
46            else:
47                mask.append(policy.in_green_zone(n.x, n.y))
48        return np.asarray(mask, dtype=bool)
49    def is_violation(
50        arrival_time: float,
51        vt: VehicleType,
52        is_green: bool,
53        policy: PolicyConfig,
54    ) -> bool:

```

```

55     if not is_green:
56         return False
57     if not vt.is_fuel:
58         return False
59     return policy.in_restricted_window(arrival_time)
60 def is_arc_violation(
61     x1: float, y1: float, x2: float, y2: float,
62     t_leave: float, t_arrive: float,
63     vt: VehicleType,
64     src_in_green: bool, dst_in_green: bool,
65     policy: PolicyConfig,
66 ) -> bool:
67     if not policy.enforce_no_cross_green:
68         return False
69     if not vt.is_fuel:
70         return False
71     if src_in_green or dst_in_green:
72         return False
73     if not policy.arc_time_overlaps_window(t_leave, t_arrive):
74         return False
75     return policy.segment_crosses_green_zone(x1, y1, x2, y2)
76 def green_zone_customers(nodes: Iterable[Node], policy: PolicyConfig) -> List[int]:
77     ids = set()
78     for n in nodes:
79         if n.orig_id == 0:
80             continue
81         if policy.in_green_zone(n.x, n.y):
82             ids.add(n.orig_id)
83     return sorted(ids)
84 if __name__ == "__main__":
85     from data_loader import load_problem
86     problem = load_problem()
87     policy = PolicyConfig()
88     mask = compute_green_mask(problem.nodes, policy)
89     green_origs = green_zone_customers(problem.nodes, policy)
90     print(f"Green zone virtual nodes: {int(mask.sum())} / {len(mask)}")
91     print(f"Green zone original customers: {len(green_origs)}")
92     print(f"First 10 green-zone orig ids: {green_origs[:10]}")

```

multitrip_ops.py

代码 4: 公共模块代码: multitrip_ops.py

```

1 from __future__ import annotations
2 from typing import Dict, List, Optional, Sequence, Tuple
3 from cost_model import (
4     HANDOVER_TIME,
5     Route,
6     Solution,

```

```

7     evaluate_route,
8 )
9 from policy import PolicyConfig
10 def _count_distinct_vehicles_per_vt(routes: Sequence[Route]) -> Dict[int, int]:
11     counts: Dict[int, int] = {}
12     seen_ids: set = set()
13     for r in routes:
14         if not r.customers:
15             continue
16         vid = getattr(r, "vehicle_id", None)
17         if vid is None:
18             counts[r.vt_index] = counts.get(r.vt_index, 0) + 1
19         else:
20             if vid not in seen_ids:
21                 seen_ids.add(vid)
22                 counts[r.vt_index] = counts.get(r.vt_index, 0) + 1
23     return counts
24 def _next_vehicle_id(routes: Sequence[Route]) -> int:
25     max_id = -1
26     for r in routes:
27         vid = getattr(r, "vehicle_id", None)
28         if vid is not None and vid > max_id:
29             max_id = vid
30     return max_id + 1
31 def consolidate_multitrip_inplace(
32     routes: List[Route],
33     problem,
34     policy: Optional[PolicyConfig] = None,
35     freezes: Optional[Sequence] = None,
36 ) -> int:
37     n = len(routes)
38     if n == 0:
39         return 0
40     evs: List[Optional] = [None] * n
41     for i, r in enumerate(routes):
42         if r.customers:
43             evs[i] = evaluate_route(r, problem, policy=policy)
44     dep_times: List[Optional[float]] = [
45         evs[i].arrival[0] if evs[i] and evs[i].arrival else None
46         for i in range(n)
47     ]
48     ret_times: List[Optional[float]] = [
49         evs[i].arrival[-1] if evs[i] and evs[i].arrival else None
50         for i in range(n)
51     ]
52     def _is_departed(i: int) -> bool:
53         if freezes is None:
54             return False
55         if i >= len(freezes):

```

```

56         return False
57     return bool(getattr(freezes[i], "is_departed", False))
58 if freezes is not None:
59     groups_pre: Dict[int, List[int]] = {}
60     for i, r in enumerate(routes):
61         if not r.customers:
62             continue
63         vid = getattr(r, "vehicle_id", None)
64         if vid is None:
65             continue
66         groups_pre.setdefault(vid, []).append(i)
67     for vid, ris in groups_pre.items():
68         if len(ris) < 2:
69             continue
70         pairs = sorted(
71             [(j, ret_times[j], dep_times[j]) for j in ris],
72             key=lambda x: x[2] if x[2] is not None else 0.0,
73         )
74         stale = False
75         for k in range(1, len(pairs)):
76             prev_ret = pairs[k - 1][1]
77             cur_dep = pairs[k][2]
78             if prev_ret is None or cur_dep is None:
79                 stale = True
80                 break
81             if cur_dep + 1e-6 < prev_ret + HANDOVER_TIME:
82                 stale = True
83                 break
84         if stale:
85             for j in ris:
86                 routes[j].vehicle_id = None
87 groups_by_vid: Dict[int, Dict] = {}
88 for i, r in enumerate(routes):
89     if not r.customers or dep_times[i] is None:
90         continue
91     vid = getattr(r, "vehicle_id", None)
92     if vid is None:
93         continue
94     g = groups_by_vid.get(vid)
95     if g is None:
96         g = {
97             "vt_index": r.vt_index,
98             "latest_return": ret_times[i],
99             "members": [i],
100         }
101     groups_by_vid[vid] = g
102 else:
103     g["members"].append(i)
104     if ret_times[i] is not None:

```

```

105         g["latest_return"] = max(g["latest_return"], ret_times[i])
106     groups_by_vt: Dict[int, List[Dict]] = {}
107     for g in groups_by_vid.values():
108         groups_by_vt.setdefault(g["vt_index"], []).append(g)
109     candidates = sorted(
110         [i for i in range(n)
111          if routes[i].customers
112          and getattr(routes[i], "vehicle_id", None) is None
113          and dep_times[i] is not None
114          and ret_times[i] is not None],
115         key=lambda i: dep_times[i],
116     )
117     merges = 0
118     for i in candidates:
119         r = routes[i]
120         vt_i = r.vt_index
121         dep_i = dep_times[i]
122         ret_i = ret_times[i]
123         is_dep = _is_departed(i)
124         best_g: Optional[Dict] = None
125         best_ret = -1.0
126         if not is_dep:
127             for g in groups_by_vt.get(vt_i, []):
128                 if g["latest_return"] + HANDOVER_TIME > dep_i + 1e-6:
129                     continue
130                 if g["latest_return"] > best_ret:
131                     best_g = g
132                     best_ret = g["latest_return"]
133         if best_g is not None:
134             vid = None
135             for vid_k, g_k in groups_by_vid.items():
136                 if g_k is best_g:
137                     vid = vid_k
138                     break
139             if vid is None:
140                 vid = _next_vehicle_id(routes)
141                 groups_by_vid[vid] = best_g
142             r.vehicle_id = vid
143             for m in best_g["members"]:
144                 if getattr(routes[m], "vehicle_id", None) is None:
145                     routes[m].vehicle_id = vid
146             best_g["members"].append(i)
147             best_g["latest_return"] = max(best_g["latest_return"], ret_i)
148             merges += 1
149         else:
150             new_g = {
151                 "vt_index": vt_i,
152                 "latest_return": ret_i,
153                 "members": [i],

```

```

154         }
155         groups_by_vt.setdefault(vt_i, []).append(new_g)
156     return merges
157 def handover_static(
158     sol: Solution,
159     problem,
160     policy: PolicyConfig,
161     max_iters: int = 8,
162 ) -> int:
163     elec_types = [
164         (vti, vt) for vti, vt in enumerate(problem.vehicle_types)
165         if not vt.is_fuel
166     ]
167     if not elec_types:
168         return 0
169     handovers = 0
170     for _ in range(max_iters):
171         target_ri = -1
172         for ri, r in enumerate(sol.routes):
173             if not r.customers:
174                 continue
175             vt = problem.vehicle_types[r.vt_index]
176             if not vt.is_fuel:
177                 continue
178             ev = evaluate_route(r, problem, policy=policy)
179             if not ev.feasible_policy:
180                 target_ri = ri
181                 break
182         if target_ri < 0:
183             break
184         r = sol.routes[target_ri]
185         head_custs: Optional[List[int]] = None
186         tail_custs: Optional[List[int]] = None
187         for p in range(1, len(r.customers)):
188             cand_head = Route(r.vt_index, list(r.customers[:p]))
189             ev_h = evaluate_route(cand_head, problem, policy=policy)
190             if ev_h.feasible_policy and ev_h.feasible_cap and ev_h.feasible_return:
191                 head_custs = list(r.customers[:p])
192                 tail_custs = list(r.customers[p:])
193                 break
194         if tail_custs is None:
195             cand_head = Route(r.vt_index, [])
196             ev_h = evaluate_route(cand_head, problem, policy=policy)
197             if ev_h.feasible_policy:
198                 head_custs = []
199                 tail_custs = list(r.customers)
200             else:
201                 break
202         if not tail_custs:

```

```

203         break
204     w = sum(problem.nodes[c].demand_w for c in tail_custs)
205     v = sum(problem.nodes[c].demand_v for c in tail_custs)
206     used = _count_distinct_vehicles_per_vt(sol.routes)
207     best_elec: Optional[Tuple[float, int]] = None
208     for vti, vt_e in elec_types:
209         if used.get(vti, 0) >= vt_e.count:
210             continue
211         if w > vt_e.capacity_w + 1e-6 or v > vt_e.capacity_v + 1e-6:
212             continue
213         cand = Route(vti, list(tail_custs))
214         ev_c = evaluate_route(cand, problem, policy=policy)
215         if not (ev_c.feasible_cap and ev_c.feasible_return
216                 and ev_c.feasible_policy):
217             continue
218         cost_new = ev_c.total_cost
219         if best_elec is None or cost_new < best_elec[0]:
220             best_elec = (cost_new, vti)
221     if best_elec is None:
222         break
223     r.customers = list(head_custs) if head_custs else []
224     new_tail = Route(best_elec[1], list(tail_custs))
225     new_tail.vehicle_id = _next_vehicle_id(sol.routes)
226     sol.routes.append(new_tail)
227     handovers += 1
228     sol.routes = [r for r in sol.routes if r.customers]
229     return handovers

```

alns_solver.py

代码 5: 公共模块代码: alns_solver.py

```

1  from __future__ import annotations
2  import math
3  import random
4  import time
5  from dataclasses import dataclass, field
6  from typing import Callable, List, Optional, Tuple
7  from cost_model import (
8      Route,
9      RouteEval,
10     Solution,
11     SolutionEval,
12     evaluate_route,
13     evaluate_solution,
14 )
15 from data_loader import ProblemData
16 from multitrip_ops import consolidate_multitrip_inplace, handover_static
17 from policy import PolicyConfig

```

```

18 SCORE_BEST = 33.0
19 SCORE_BETTER = 9.0
20 SCORE_ACCEPT = 13.0
21 REACTION_FACTOR = 0.1
22 CANDIDATE_ROUTES_K = 12
23 POLICY_PENALTY = 2e6
24 _ROUTE_COST_CACHE: dict = {}
25 _ROUTE_COST_MAXSIZE = 200_000
26 def _policy_id(policy: Optional["PolicyConfig"]) -> str:
27     return "none" if policy is None else policy.policy_id
28 def _route_cost(
29     route: Route,
30     problem: ProblemData,
31     policy: Optional[PolicyConfig] = None,
32 ) -> float:
33     key = (_policy_id(policy), route.vt_index, tuple(route.customers))
34     v = _ROUTE_COST_CACHE.get(key)
35     if v is not None:
36         return v
37     ev = evaluate_route(route, problem, policy=policy)
38     penalty = 0.0
39     if not ev.feasible_cap:
40         penalty += 1e6
41     if not ev.feasible_return:
42         penalty += 1e5 * max(0, ev.arrival[-1] - problem.cost.return_time)
43     if policy is not None and not ev.feasible_policy:
44         penalty += POLICY_PENALTY * ev.policy_violation_count
45     v = ev.total_cost + penalty
46     if len(_ROUTE_COST_CACHE) < _ROUTE_COST_MAXSIZE:
47         _ROUTE_COST_CACHE[key] = v
48     return v
49 def _route_centroid(route: Route, problem: ProblemData) -> Tuple[float, float]:
50     if not route.customers:
51         return (problem.nodes[0].x, problem.nodes[0].y)
52     xs = sum(problem.nodes[c].x for c in route.customers)
53     ys = sum(problem.nodes[c].y for c in route.customers)
54     n = len(route.customers)
55     return (xs / n, ys / n)
56 def _candidate_route_indices(
57     node_id: int, sol: Solution, problem: ProblemData, k: int,
58 ) -> List[int]:
59     node = problem.nodes[node_id]
60     scored = []
61     for i, r in enumerate(sol.routes):
62         if not r.customers:
63             continue
64         cx, cy = _route_centroid(r, problem)
65         d2 = (cx - node.x) ** 2 + (cy - node.y) ** 2
66         scored.append((d2, i))

```



```

67     scored.sort(key=lambda t: t[0])
68     return [i for _, i in scored[:k]]
69 def _solution_eval_for_search(
70     sol: Solution,
71     problem: ProblemData,
72     policy: Optional[PolicyConfig] = None,
73 ) -> float:
74     total = 0.0
75     for r in sol.routes:
76         if not r.customers:
77             continue
78         total += _route_cost(r, problem, policy=policy)
79     total += 1e6 * len(sol.unassigned)
80     scratch = [Route(vt_index=r.vt_index, customers=list(r.customers))
81                for r in sol.routes if r.customers]
82     n_merges = consolidate_multitrip_inplace(scratch, problem, policy)
83     total -= n_merges * problem.cost.fixed_cost
84     return total
85 def _insertion_creates_zone_cross(
86     node_id: int,
87     prev: int,
88     nxt: int,
89     problem: ProblemData,
90     policy: PolicyConfig,
91 ) -> bool:
92     nodes = problem.nodes
93     for u, v in ((prev, node_id), (node_id, nxt)):
94         nu = nodes[u]
95         nv = nodes[v]
96         u_in = (u != 0) and policy.in_green_zone(nu.x, nu.y)
97         v_in = (v != 0) and policy.in_green_zone(nv.x, nv.y)
98         if u_in or v_in:
99             continue
100         if policy.segment_crosses_green_zone(nu.x, nu.y, nv.x, nv.y):
101             return True
102     return False
103 def _insertion_creates_zone_cross(
104     node_id: int,
105     prev: int,
106     nxt: int,
107     problem: ProblemData,
108     policy: PolicyConfig,
109 ) -> bool:
110     nodes = problem.nodes
111     for u, v in ((prev, node_id), (node_id, nxt)):
112         nu = nodes[u]
113         nv = nodes[v]
114         u_in = (u != 0) and policy.in_green_zone(nu.x, nu.y)
115         v_in = (v != 0) and policy.in_green_zone(nv.x, nv.y)

```

```

116         if u_in or v_in:
117             continue
118         if policy.segment_crosses_green_zone(nu.x, nu.y, nv.x, nv.y):
119             return True
120     return False
121 def _best_insertion_in_route(
122     node_id: int,
123     route: Route,
124     problem: ProblemData,
125     current_cost: float,
126     policy: Optional[PolicyConfig] = None,
127 ) -> Optional[Tuple[float, int]]:
128     vt = problem.vehicle_types[route.vt_index]
129     node = problem.nodes[node_id]
130     current_w = sum(problem.nodes[c].demand_w for c in route.customers)
131     current_v = sum(problem.nodes[c].demand_v for c in route.customers)
132     if current_w + node.demand_w > vt.capacity_w + 1e-6:
133         return None
134     if current_v + node.demand_v > vt.capacity_v + 1e-6:
135         return None
136     best = None
137     for p in range(len(route.customers) + 1):
138         new_customers = route.customers[:p] + [node_id] + route.customers[p:]
139         new_route = Route(route.vt_index, new_customers)
140         new_cost = _route_cost(new_route, problem, policy=policy)
141         delta = new_cost - current_cost
142         if best is None or delta < best[0]:
143             best = (delta, p)
144     return best
145 def _best_insertion(
146     node_id: int,
147     sol: Solution,
148     problem: ProblemData,
149     route_costs: List[float],
150     used_per_vt: dict,
151     policy: Optional[PolicyConfig] = None,
152 ) -> Optional[Tuple[float, int, int]]:
153     best: Optional[Tuple[float, int, int]] = None
154     candidate_indices = _candidate_route_indices(
155         node_id, sol, problem, CANDIDATE_ROUTES_K
156     )
157     for i in candidate_indices:
158         r = sol.routes[i]
159         res = _best_insertion_in_route(
160             node_id, r, problem, route_costs[i], policy=policy,
161         )
162         if res is None:
163             continue
164         delta, pos = res

```

```

165         if best is None or delta < best[0]:
166             best = (delta, i, pos)
167     for vt_idx, vt in enumerate(problem.vehicle_types):
168         used = used_per_vt.get(vt_idx, 0)
169         if used >= vt.count:
170             continue
171         new_route = Route(vt_idx, [node_id])
172         if problem.nodes[node_id].demand_w > vt.capacity_w + 1e-6:
173             continue
174         if problem.nodes[node_id].demand_v > vt.capacity_v + 1e-6:
175             continue
176         cost = _route_cost(new_route, problem, policy=policy)
177         delta = cost
178         if best is None or delta < best[0]:
179             best = (delta, -(vt_idx + 1), 0)
180     return best
181 def _k_best_insertions(
182     node_id: int,
183     sol: Solution,
184     problem: ProblemData,
185     route_costs: List[float],
186     used_per_vt: dict,
187     k: int,
188     policy: Optional[PolicyConfig] = None,
189 ) -> List[Tuple[float, int, int]]:
190     options: List[Tuple[float, int, int]] = []
191     candidate_indices = _candidate_route_indices(
192         node_id, sol, problem, CANDIDATE_ROUTES_K
193     )
194     for i in candidate_indices:
195         r = sol.routes[i]
196         vt = problem.vehicle_types[r.vt_index]
197         node = problem.nodes[node_id]
198         cur_w = sum(problem.nodes[c].demand_w for c in r.customers)
199         cur_v = sum(problem.nodes[c].demand_v for c in r.customers)
200         if cur_w + node.demand_w > vt.capacity_w + 1e-6:
201             continue
202         if cur_v + node.demand_v > vt.capacity_v + 1e-6:
203             continue
204         for p in range(len(r.customers) + 1):
205             new_customers = r.customers[:p] + [node_id] + r.customers[p:]
206             new_route = Route(r.vt_index, new_customers)
207             cost = _route_cost(new_route, problem, policy=policy)
208             options.append((cost - route_costs[i], i, p))
209     for vt_idx, vt in enumerate(problem.vehicle_types):
210         used = used_per_vt.get(vt_idx, 0)
211         if used >= vt.count:
212             continue
213         node = problem.nodes[node_id]

```

```

214         if node.demand_w > vt.capacity_w + 1e-6:
215             continue
216         if node.demand_v > vt.capacity_v + 1e-6:
217             continue
218         new_route = Route(vt_idx, [node_id])
219         cost = _route_cost(new_route, problem, policy=policy)
220         options.append((cost, -(vt_idx + 1), 0))
221     options.sort(key=lambda x: x[0])
222     return options[:k]
223 def _apply_insertion(
224     sol: Solution, node_id: int, route_idx: int, position: int,
225 ) -> None:
226     if route_idx >= 0:
227         sol.routes[route_idx].customers.insert(position, node_id)
228     else:
229         vt_idx = -route_idx - 1
230         sol.routes.append(Route(vt_idx, [node_id]))
231 def _used_per_vt(sol: Solution) -> dict:
232     counts: dict = {}
233     for r in sol.routes:
234         if r.customers:
235             counts[r.vt_index] = counts.get(r.vt_index, 0) + 1
236     return counts
237 def _route_costs(
238     sol: Solution,
239     problem: ProblemData,
240     policy: Optional[PolicyConfig] = None,
241 ) -> List[float]:
242     return [_route_cost(r, problem, policy=policy) for r in sol.routes]
243 def greedy_initial(
244     problem: ProblemData,
245     seed: int = 0,
246     policy: Optional[PolicyConfig] = None,
247 ) -> Solution:
248     random.seed(seed)
249     nodes = problem.nodes
250     customer_ids = [n.node_id for n in nodes if n.node_id != 0]
251     def sort_key(nid: int) -> Tuple[int, float, float, float]:
252         n = nodes[nid]
253         tw_len = n.tw_end - n.tw_start
254         dep_dist = problem.dist[0, nid]
255         if policy is not None and policy.in_green_zone(n.x, n.y) and n.orig_id != 0:
256             priority = 0
257         else:
258             priority = 1
259         return (priority, n.tw_start, tw_len, dep_dist)
260     customer_ids.sort(key=sort_key)
261     sol = Solution()
262     for nid in customer_ids:

```

```

263     route_costs = _route_costs(sol, problem, policy=policy)
264     used = _used_per_vt(sol)
265     best = _best_insertion(
266         nid, sol, problem, route_costs, used, policy=policy,
267     )
268     if best is None:
269         sol.unassigned.append(nid)
270         continue
271     delta, route_idx, pos = best
272     _apply_insertion(sol, nid, route_idx, pos)
273     sol.routes = [r for r in sol.routes if r.customers]
274     return sol
275 def destroy_random(
276     sol: Solution, problem: ProblemData, q: int, rng: random.Random,
277     policy: Optional[PolicyConfig] = None,
278 ) -> List[int]:
279     all_ids = [c for r in sol.routes for c in r.customers]
280     if not all_ids:
281         return []
282     q = min(q, len(all_ids))
283     chosen = rng.sample(all_ids, q)
284     return _remove_ids_from_routes(sol, chosen)
285 def destroy_worst(
286     sol: Solution, problem: ProblemData, q: int, rng: random.Random,
287     policy: Optional[PolicyConfig] = None,
288 ) -> List[int]:
289     all_ids = []
290     for i, r in enumerate(sol.routes):
291         base = _route_cost(r, problem, policy=policy)
292         for p, c in enumerate(r.customers):
293             new_customers = r.customers[:p] + r.customers[p+1:]
294             reduced = _route_cost(
295                 Route(r.vt_index, new_customers), problem, policy=policy,
296             )
297             gain = base - reduced
298             gain *= (1.0 + 0.2 * rng.random())
299             all_ids.append((gain, c))
300     all_ids.sort(key=lambda x: -x[0])
301     q = min(q, len(all_ids))
302     chosen = [c for _, c in all_ids[:q]]
303     return _remove_ids_from_routes(sol, chosen)
304 def destroy_shaw(
305     sol: Solution, problem: ProblemData, q: int, rng: random.Random,
306     policy: Optional[PolicyConfig] = None,
307 ) -> List[int]:
308     all_ids = [c for r in sol.routes for c in r.customers]
309     if not all_ids:
310         return []
311     seed = rng.choice(all_ids)

```

```

312     chosen = {seed}
313     seed_node = problem.nodes[seed]
314     while len(chosen) < min(q, len(all_ids)):
315         seed = rng.choice(list(chosen))
316         seed_node = problem.nodes[seed]
317         candidates = [c for c in all_ids if c not in chosen]
318         if not candidates:
319             break
320         scored = []
321         for c in candidates:
322             cn = problem.nodes[c]
323             d_norm = problem.dist[seed, c] / 100.0
324             tw_norm = abs(seed_node.tw_start - cn.tw_start) / 12.0
325             demand_norm = abs(seed_node.demand_w - cn.demand_w) / 3000.0
326             score = d_norm + tw_norm + 0.2 * demand_norm
327             scored.append((score, c))
328         scored.sort(key=lambda x: x[0])
329         y = rng.random()
330         pick_idx = int((y ** 3) * len(scored))
331         pick_idx = min(pick_idx, len(scored) - 1)
332         chosen.add(scored[pick_idx][1])
333     return remove_ids_from_routes(sol, list(chosen))
334 def destroy_route(
335     sol: Solution, problem: ProblemData, q: int, rng: random.Random,
336     policy: Optional[PolicyConfig] = None,
337 ) -> List[int]:
338     non_empty = [i for i, r in enumerate(sol.routes) if r.customers]
339     if not non_empty:
340         return []
341     non_empty.sort(key=lambda i: len(sol.routes[i].customers))
342     removed: List[int] = []
343     for i in non_empty:
344         removed.extend(sol.routes[i].customers)
345         sol.routes[i].customers = []
346         if len(removed) >= q:
347             break
348     sol.routes = [r for r in sol.routes if r.customers]
349     return removed
350 def destroy_route_cluster(
351     sol: Solution, problem: ProblemData, q: int, rng: random.Random,
352     policy: Optional[PolicyConfig] = None,
353 ) -> List[int]:
354     non_empty_idx = [i for i, r in enumerate(sol.routes) if r.customers]
355     if not non_empty_idx:
356         return []
357     seed_idx = rng.choice(non_empty_idx)
358     seed_route = sol.routes[seed_idx]
359     cx, cy = _route_centroid(seed_route, problem)
360     scored = []

```

```

361     for i in non_empty_idx:
362         if i == seed_idx:
363             continue
364         ox, oy = _route_centroid(sol.routes[i], problem)
365         d2 = (cx - ox) ** 2 + (cy - oy) ** 2
366         scored.append((d2, i))
367     scored.sort(key=lambda t: t[0])
368     idx_to_remove = [seed_idx] + [i for _, i in scored[:3]]
369     removed: List[int] = []
370     for i in idx_to_remove:
371         removed.extend(sol.routes[i].customers)
372         sol.routes[i].customers = []
373         if len(removed) >= q:
374             break
375     sol.routes = [r for r in sol.routes if r.customers]
376     return removed
377 def _remove_ids_from_routes(sol: Solution, ids: List[int]) -> List[int]:
378     ids_set = set(ids)
379     removed_ordered: List[int] = []
380     for r in sol.routes:
381         new_customers = []
382         for c in r.customers:
383             if c in ids_set:
384                 removed_ordered.append(c)
385             else:
386                 new_customers.append(c)
387         r.customers = new_customers
388     sol.routes = [r for r in sol.routes if r.customers]
389     return removed_ordered
390 def repair_greedy(
391     sol: Solution, problem: ProblemData, to_insert: List[int], rng: random.Random,
392     policy: Optional[PolicyConfig] = None,
393 ) -> None:
394     pending = list(to_insert)
395     rng.shuffle(pending)
396     while pending:
397         route_costs = _route_costs(sol, problem, policy=policy)
398         used = _used_per_vt(sol)
399         best_global: Optional[Tuple[float, int, int, int]] = None
400         for nid in pending:
401             ins = _best_insertion(
402                 nid, sol, problem, route_costs, used, policy=policy,
403             )
404             if ins is None:
405                 continue
406             delta, r_idx, pos = ins
407             if best_global is None or delta < best_global[0]:
408                 best_global = (delta, nid, r_idx, pos)
409         if best_global is None:

```

```

410         sol.unassigned.extend(pending)
411         break
412     _, nid, r_idx, pos = best_global
413     _apply_insertion(sol, nid, r_idx, pos)
414     pending.remove(nid)
415 def repair_regret(
416     sol: Solution, problem: ProblemData, to_insert: List[int],
417     rng: random.Random, k: int = 2,
418     policy: Optional[PolicyConfig] = None,
419 ) -> None:
420     pending = list(to_insert)
421     while pending:
422         route_costs = _route_costs(sol, problem, policy=policy)
423         used = _used_per_vt(sol)
424         best_regret: Optional[Tuple[float, int, int, int]] = None
425         for nid in pending:
426             opts = _k_best_insertions(
427                 nid, sol, problem, route_costs, used, k, policy=policy,
428             )
429             if not opts:
430                 continue
431             if len(opts) < k:
432                 regret = 1e9
433             else:
434                 regret = sum(opts[i][0] - opts[0][0] for i in range(1, k))
435                 delta, r_idx, pos = opts[0]
436                 score = regret - 0.01 * delta
437                 if best_regret is None or score > best_regret[0]:
438                     best_regret = (score, nid, r_idx, pos)
439             if best_regret is None:
440                 sol.unassigned.extend(pending)
441                 break
442         _, nid, r_idx, pos = best_regret
443         _apply_insertion(sol, nid, r_idx, pos)
444         pending.remove(nid)
445 @dataclass
446 class ALNSConfig:
447     iterations: int = 3000
448     seed: int = 0
449     removal_min_frac: float = 0.10
450     removal_max_frac: float = 0.30
451     sa_start_temp_frac: float = 0.02
452     sa_cooling: float = 0.9975
453     update_period: int = 100
454     verbose: bool = True
455     log_every: int = 200
456 @dataclass
457 class ALNSResult:
458     best_solution: Solution

```



```

459     best_eval: SolutionEval
460     history: list = field(default_factory=list)
461     def run_alns(
462         problem: ProblemData,
463         config: ALNSConfig,
464         policy: Optional[PolicyConfig] = None,
465     ) -> ALNSResult:
466         rng = random.Random(config.seed)
467         if config.verbose:
468             tag = f"policy={policy.policy_id}" if policy is not None else "no policy"
469             print(f"[ALNS] Constructing initial solution ({tag})...")
470         current = greedy_initial(problem, seed=config.seed, policy=policy)
471         current_score = _solution_eval_for_search(current, problem, policy=policy)
472         best = current.copy()
473         best_score = current_score
474         if config.verbose:
475             print(f"[ALNS] Initial cost: {current_score:.2f}, "
476                   f"routes: {current.n_routes()}, unassigned: {len(current.unassigned)}")
477         destroy_ops: List[Tuple[str, Callable]] = [
478             ("random", destroy_random),
479             ("worst", destroy_worst),
480             ("shaw", destroy_shaw),
481             ("route", destroy_route),
482             ("route_cluster", destroy_route_cluster),
483         ]
484         repair_ops: List[Tuple[str, Callable]] = [
485             ("greedy", lambda s, p, t, r: repair_greedy(s, p, t, r, policy=policy)),
486             ("regret2", lambda s, p, t, r: repair_regret(s, p, t, r, k=2, policy=policy)),
487             ("regret3", lambda s, p, t, r: repair_regret(s, p, t, r, k=3, policy=policy)),
488         ]
489         d_weights = [1.0] * len(destroy_ops)
490         r_weights = [1.0] * len(repair_ops)
491         d_scores = [0.0] * len(destroy_ops)
492         r_scores = [0.0] * len(repair_ops)
493         d_uses = [0] * len(destroy_ops)
494         r_uses = [0] * len(repair_ops)
495         def choose(weights: List[float]) -> int:
496             total = sum(weights)
497             r = rng.random() * total
498             for i, w in enumerate(weights):
499                 r -= w
500                 if r <= 0:
501                     return i
502             return len(weights) - 1
503         T = max(1.0, current_score * config.sa_start_temp_frac)
504         history = []
505         n_customers = len(problem.customer_nodes)
506         t0 = time.time()
507         for it in range(1, config.iterations + 1):

```

```

508     candidate = current.copy()
509     d_idx = choose(d_weights)
510     r_idx = choose(r_weights)
511     d_uses[d_idx] += 1
512     r_uses[r_idx] += 1
513     q = rng.randint(
514         max(2, int(config.removal_min_frac * n_customers)),
515         max(3, int(config.removal_max_frac * n_customers)),
516     )
517     removed = destroy_ops[d_idx][1](candidate, problem, q, rng, policy=policy)
518     removed.extend(candidate.unassigned)
519     candidate.unassigned = []
520     repair_ops[r_idx][1](candidate, problem, removed, rng)
521     new_score = _solution_eval_for_search(candidate, problem, policy=policy)
522     delta = new_score - current_score
523     accepted = False
524     score_gained = 0.0
525     if new_score < best_score - 1e-6:
526         current = candidate
527         current_score = new_score
528         best = candidate.copy()
529         best_score = new_score
530         accepted = True
531         score_gained = SCORE_BEST
532     elif new_score < current_score - 1e-6:
533         current = candidate
534         current_score = new_score
535         accepted = True
536         score_gained = SCORE_BETTER
537     else:
538         if rng.random() < math.exp(-delta / max(T, 1e-9)):
539             current = candidate
540             current_score = new_score
541             accepted = True
542             score_gained = SCORE_ACCEPT
543     d_scores[d_idx] += score_gained
544     r_scores[r_idx] += score_gained
545     T *= config.sa_cooling
546     if it % config.update_period == 0:
547         for i in range(len(d_weights)):
548             if d_uses[i] > 0:
549                 d_weights[i] = max(
550                     0.1,
551                     d_weights[i] * (1 - REACTION_FACTOR)
552                     + REACTION_FACTOR * (d_scores[i] / d_uses[i]),
553                 )
554             d_scores[i] = 0.0
555             d_uses[i] = 0
556         for i in range(len(r_weights)):

```

```

557         if r_uses[i] > 0:
558             r_weights[i] = max(
559                 0.1,
560                 r_weights[i] * (1 - REACTION_FACTOR)
561                 + REACTION_FACTOR * (r_scores[i] / r_uses[i]),
562             )
563             r_scores[i] = 0.0
564             r_uses[i] = 0
565         history.append((it, current_score, best_score))
566         if config.verbose and (it % config.log_every == 0 or it == 1):
567             print(f"[ALNS] iter {it:5d} | cur={current_score:10.2f} | "
568                 f"best={best_score:10.2f} | T={T:8.2f} | "
569                 f"routes={current.n_routes()} | "
570                 f"d_w={[round(w,2) for w in d_weights]} "
571                 f"r_w={[round(w,2) for w in r_weights]}")
572     if config.verbose:
573         print(f"[ALNS] Finished {config.iterations} iterations in "
574             f"{time.time()-t0:.1f}s. Best cost: {best_score:.2f}")
575     if config.verbose:
576         print("[ALNS] Polishing with 2-opt / relocate ...")
577     best = _local_search(best, problem, policy=policy)
578     best_eval = evaluate_solution(best, problem, policy=policy)
579     if config.verbose:
580         print(f"[ALNS] After polish: {best_eval.total_cost:.2f}, "
581             f"routes={best_eval.n_routes}")
582     if policy is not None:
583         print(f"[ALNS] Policy feasible: {best_eval.feasible_policy} "
584             f"(violations={best_eval.policy_violation_count})")
585     if policy is not None and not best_eval.feasible_policy:
586         if config.verbose:
587             print("[ALNS] Strict policy-violation repair pass ...")
588         best = _fix_policy_violations(best, problem, policy)
589         best = _local_search(best, problem, policy=policy)
590         best_eval = evaluate_solution(best, problem, policy=policy)
591         if config.verbose:
592             print(f"[ALNS] After strict repair: {best_eval.total_cost:.2f}, "
593                 f"routes={best_eval.n_routes}, "
594                 f"policy feasible={best_eval.feasible_policy} "
595                 f"(violations={best_eval.policy_violation_count})")
596     if not best_eval.feasible_policy:
597         if config.verbose:
598             print("[ALNS] Bridge-steal repair pass ...")
599         best = _bridge_steal(best, problem, policy)
600         best = _local_search(best, problem, policy=policy)
601         best_eval = evaluate_solution(best, problem, policy=policy)
602         if config.verbose:
603             print(
604                 f"[ALNS] After bridge-steal: {best_eval.total_cost:.2f}, "
605                 f"routes={best_eval.n_routes}, "

```

```

606         f"policy feasible={best_eval.feasible_policy} "
607         f"(violations={best_eval.policy_violation_count})"
608     )
609     if policy is not None and not best_eval.feasible_policy:
610         if config.verbose:
611             print("[ALNS] Static handover-to-depot pass ...")
612         n_hand = handover_static(best, problem, policy)
613         if n_hand > 0:
614             best = _local_search(best, problem, policy=policy)
615         best_eval = evaluate_solution(best, problem, policy=policy)
616         if config.verbose:
617             print(
618                 f"[ALNS] After handover: {best_eval.total_cost:.2f}, "
619                 f"handovers={n_hand}, "
620                 f"policy feasible={best_eval.feasible_policy} "
621                 f"(violations={best_eval.policy_violation_count})"
622             )
623     n_merges = consolidate_multitrip_inplace(best.routes, problem, policy)
624     if n_merges > 0:
625         best_eval = evaluate_solution(best, problem, policy=policy)
626         if config.verbose:
627             seen_vids: set = set()
628             n_physical = 0
629             n_trips = 0
630             for r in best.routes:
631                 if not r.customers:
632                     continue
633                 n_trips += 1
634                 vid = getattr(r, "vehicle_id", None)
635                 if vid is None:
636                     n_physical += 1
637                 elif vid not in seen_vids:
638                     seen_vids.add(vid)
639                     n_physical += 1
640             print(
641                 f"[ALNS] Multi-trip consolidation: merged {n_merges} pair(s), "
642                 f"{n_trips} trip(s) on {n_physical} physical vehicle(s). "
643                 f"Total cost: {best_eval.total_cost:.2f}"
644             )
645     return ALNSResult(best_solution=best, best_eval=best_eval, history=history)
646 def _two_opt_route(
647     route: Route, problem: ProblemData,
648     policy: Optional[PolicyConfig] = None,
649 ) -> Route:
650     best = route
651     best_cost = _route_cost(best, problem, policy=policy)
652     improved = True
653     while improved:
654         improved = False

```

```

655     cs = best.customers
656     n = len(cs)
657     if n < 3:
658         break
659     for i in range(n - 1):
660         for j in range(i + 1, n):
661             new_cs = cs[:i] + cs[i:j + 1][::-1] + cs[j + 1:]
662             nr = Route(best.vt_index, new_cs)
663             nc = _route_cost(nr, problem, policy=policy)
664             if nc < best_cost - 1e-6:
665                 best = nr
666                 best_cost = nc
667                 improved = True
668                 break
669         if improved:
670             break
671     return best
672 def _relocate_move(
673     sol: Solution, problem: ProblemData,
674     policy: Optional[PolicyConfig] = None,
675 ) -> Solution:
676     improved_any = True
677     while improved_any:
678         improved_any = False
679         for ri in range(len(sol.routes)):
680             r = sol.routes[ri]
681             base_cost = _route_cost(r, problem, policy=policy)
682             for pi, c in enumerate(list(r.customers)):
683                 new_cs = r.customers[:pi] + r.customers[pi + 1:]
684                 if not new_cs:
685                     continue
686                 nr = Route(r.vt_index, new_cs)
687                 reduced = _route_cost(nr, problem, policy=policy)
688                 gain_from_remove = base_cost - reduced
689                 route_costs = [
690                     _route_cost(x, problem, policy=policy) for x in sol.routes
691                 ]
692                 used = _used_per_vt(sol)
693                 old_customers = r.customers
694                 r.customers = new_cs
695                 route_costs[ri] = reduced
696                 ins = _best_insertion(
697                     c, sol, problem, route_costs, used, policy=policy,
698                 )
699                 if ins is not None:
700                     delta_insert, r_idx, pos = ins
701                     if delta_insert < gain_from_remove - 1e-6:
702                         _apply_insertion(sol, c, r_idx, pos)
703                     improved_any = True

```

```

704             break
705             r.customers = old_customers
706         if improved_any:
707             break
708         sol.routes = [x for x in sol.routes if x.customers]
709     return sol
710 def _local_search(
711     sol: Solution, problem: ProblemData,
712     policy: Optional[PolicyConfig] = None,
713 ) -> Solution:
714     for r in sol.routes:
715         new_r = _two_opt_route(r, problem, policy=policy)
716         r.customers = new_r.customers
717     sol = _relocate_move(sol, problem, policy=policy)
718     return sol
719 def _strict_best_insertion_in_route(
720     node_id: int,
721     route: Route,
722     problem: ProblemData,
723     current_cost: float,
724     policy: PolicyConfig,
725 ) -> Optional[Tuple[float, int]]:
726     vt = problem.vehicle_types[route.vt_index]
727     node = problem.nodes[node_id]
728     cur_w = sum(problem.nodes[c].demand_w for c in route.customers)
729     cur_v = sum(problem.nodes[c].demand_v for c in route.customers)
730     if cur_w + node.demand_w > vt.capacity_w + 1e-6:
731         return None
732     if cur_v + node.demand_v > vt.capacity_v + 1e-6:
733         return None
734     enforce = vt.is_fuel and policy.enforce_no_cross_green
735     best = None
736     for p in range(len(route.customers) + 1):
737         if enforce:
738             prev = route.customers[p - 1] if p > 0 else 0
739             nxt = route.customers[p] if p < len(route.customers) else 0
740             if _insertion_creates_zone_cross(
741                 node_id, prev, nxt, problem, policy,
742             ):
743                 continue
744             new_customers = route.customers[:p] + [node_id] + route.customers[p:]
745             new_route = Route(route.vt_index, new_customers)
746             new_cost = _route_cost(new_route, problem, policy=policy)
747             delta = new_cost - current_cost
748             if best is None or delta < best[0]:
749                 best = (delta, p)
750     return best
751 def _strict_best_insertion(
752     node_id: int,

```

```

753     sol: Solution,
754     problem: ProblemData,
755     route_costs: List[float],
756     used_per_vt: dict,
757     policy: PolicyConfig,
758 ) -> Optional[Tuple[float, int, int]]:
759     best: Optional[Tuple[float, int, int]] = None
760     for i, r in enumerate(sol.routes):
761         if not r.customers:
762             continue
763         res = _strict_best_insertion_in_route(
764             node_id, r, problem, route_costs[i], policy,
765         )
766         if res is None:
767             continue
768         delta, pos = res
769         if best is None or delta < best[0]:
770             best = (delta, i, pos)
771     for vt_idx, vt in enumerate(problem.vehicle_types):
772         used = used_per_vt.get(vt_idx, 0)
773         if used >= vt.count:
774             continue
775         node = problem.nodes[node_id]
776         if node.demand_w > vt.capacity_w + 1e-6:
777             continue
778         if node.demand_v > vt.capacity_v + 1e-6:
779             continue
780         if (
781             vt.is_fuel
782             and policy.enforce_no_cross_green
783             and _insertion_creates_zone_cross(
784                 node_id, 0, 0, problem, policy,
785             )
786         ):
787             continue
788         new_route = Route(vt_idx, [node_id])
789         cost = _route_cost(new_route, problem, policy=policy)
790         if best is None or cost < best[0]:
791             best = (cost, -(vt_idx + 1), 0)
792     return best
793 def _strict_reinsert(
794     sol: Solution,
795     problem: ProblemData,
796     pending: List[int],
797     policy: PolicyConfig,
798 ) -> List[int]:
799     unassigned: List[int] = []
800     pending = list(pending)
801     def difficulty(c: int) -> float:

```

```

$02     n = problem.nodes[c]
$03     depot = problem.nodes[0]
$04     if policy.in_green_zone(n.x, n.y):
$05         return 0.0
$06     if policy.segment_crosses_green_zone(depot.x, depot.y, n.x, n.y):
$07         return 1.0
$08     return 2.0
$09 pending.sort(key=difficulty)
$10 while pending:
$11     route_costs = _route_costs(sol, problem, policy=policy)
$12     used = _used_per_vt(sol)
$13     nid = pending.pop(0)
$14     best = _strict_best_insertion(
$15         nid, sol, problem, route_costs, used, policy=policy,
$16     )
$17     if best is None:
$18         unassigned.append(nid)
$19         continue
$20     _, r_idx, pos = best
$21     _apply_insertion(sol, nid, r_idx, pos)
$22 return unassigned
$23 def _bridge_steal_once(
$24     sol: Solution, problem: ProblemData, policy: PolicyConfig,
$25 ) -> bool:
$26     depot = problem.nodes[0]
$27     nodes = problem.nodes
$28     for ri, r in enumerate(sol.routes):
$29         if not r.customers:
$30             continue
$31         vt = problem.vehicle_types[r.vt_index]
$32         if not vt.is_fuel:
$33             continue
$34         ev = evaluate_route(r, problem, policy=policy)
$35         if ev.feasible_policy:
$36             continue
$37         first = r.customers[0]
$38         last = r.customers[-1]
$39         nf = nodes[first]
$40         nl = nodes[last]
$41         targets: List[Tuple[int, int]] = []
$42         if (
$43             policy.arc_time_overlaps_window(ev.service_end[0], ev.arrival[1])
$44             and policy.segment_crosses_green_zone(
$45                 depot.x, depot.y, nf.x, nf.y,
$46             )
$47         ):
$48             targets.append((0, first))
$49         if (
$50             policy.arc_time_overlaps_window(

```



```

$51         ev.service_end[-2], ev.arrival[-1]
$52     )
$53     and policy.segment_crosses_green_zone(
$54         nl.x, nl.y, depot.x, depot.y,
$55     )
$56 ):
$57     targets.append((len(r.customers), last))
$58 if not targets:
$59     continue
$60 cur_r_cost = _route_cost(r, problem, policy=policy)
$61 for insert_pos, anchor in targets:
$62     anchor_node = nodes[anchor]
$63     best: Optional[Tuple[float, int, int, int, List[int], List[int]]] = None
$64     for rj, r_src in enumerate(sol.routes):
$65         if rj == ri or not r_src.customers:
$66             continue
$67         cur_src_cost = _route_cost(r_src, problem, policy=policy)
$68         for bi, B in enumerate(list(r_src.customers)):
$69             nB = nodes[B]
$70             if policy.in_green_zone(nB.x, nB.y):
$71                 continue
$72             if policy.segment_crosses_green_zone(
$73                 depot.x, depot.y, nB.x, nB.y,
$74             ):
$75                 continue
$76             if policy.segment_crosses_green_zone(
$77                 nB.x, nB.y, anchor_node.x, anchor_node.y,
$78             ):
$79                 continue
$80             vt_w = problem.vehicle_types[r.vt_index].capacity_w
$81             vt_v = problem.vehicle_types[r.vt_index].capacity_v
$82             cur_w = sum(nodes[c].demand_w for c in r.customers)
$83             cur_v = sum(nodes[c].demand_v for c in r.customers)
$84             if cur_w + nB.demand_w > vt_w + 1e-6:
$85                 continue
$86             if cur_v + nB.demand_v > vt_v + 1e-6:
$87                 continue
$88             new_r_customers = list(r.customers)
$89             new_r_customers.insert(insert_pos, B)
$90             new_r = Route(r.vt_index, new_r_customers)
$91             new_r_cost = _route_cost(
$92                 new_r, problem, policy=policy,
$93             )
$94             new_src_customers = (
$95                 r_src.customers[:bi] + r_src.customers[bi + 1:]
$96             )
$97             if not new_src_customers:
$98                 new_src_cost = 0.0
$99             else:

```

```

900         new_src = Route(r_src.vt_index, new_src_customers)
901         new_src_cost = _route_cost(
902             new_src, problem, policy=policy,
903         )
904         delta = (
905             (new_r_cost + new_src_cost)
906             - (cur_r_cost + cur_src_cost)
907         )
908         if best is None or delta < best[0]:
909             best = (
910                 delta, rj, bi, B,
911                 new_r_customers, new_src_customers,
912             )
913         if best is not None:
914             _, rj, bi, B, new_r_customers, new_src_customers = best
915             r.customers = new_r_customers
916             sol.routes[rj].customers = new_src_customers
917             sol.routes = [rr for rr in sol.routes if rr.customers]
918             return True
919         return False
920     def _bridge_steal(
921         sol: Solution, problem: ProblemData, policy: PolicyConfig,
922         max_rounds: int = 8,
923     ) -> Solution:
924         for _ in range(max_rounds):
925             if not _bridge_steal_once(sol, problem, policy):
926                 break
927         return sol
928     def _fix_policy_violations(
929         sol: Solution,
930         problem: ProblemData,
931         policy: PolicyConfig,
932         max_rounds: int = 4,
933     ) -> Solution:
934         for round_i in range(max_rounds):
935             bad_indices: List[int] = []
936             for i, r in enumerate(sol.routes):
937                 if not r.customers:
938                     continue
939                 ev = evaluate_route(r, problem, policy=policy)
940                 if not ev.feasible_policy:
941                     bad_indices.append(i)
942             if not bad_indices:
943                 break
944             displaced: List[int] = []
945             for i in bad_indices:
946                 displaced.extend(sol.routes[i].customers)
947                 sol.routes[i].customers = []
948             sol.routes = [r for r in sol.routes if r.customers]

```

```

949     depot = problem.nodes[0]
950     freed: List[int] = []
951     if round_i >= 1:
952         for r in sol.routes:
953             vt = problem.vehicle_types[r.vt_index]
954             if not vt.is_fuel or len(r.customers) < 2:
955                 continue
956             worst = None
957             worst_d = 0.0
958             for c in r.customers:
959                 n = problem.nodes[c]
960                 d = (n.x - depot.x) ** 2 + (n.y - depot.y) ** 2
961                 if d > worst_d and not policy.in_green_zone(n.x, n.y):
962                     worst = c
963                     worst_d = d
964             if worst is not None:
965                 r.customers.remove(worst)
966                 freed.append(worst)
967             sol.routes = [r for r in sol.routes if r.customers]
968     pending = displaced + freed
969     leftover = _strict_reinsert(sol, problem, pending, policy)
970     if leftover:
971         rng = random.Random(100 + round_i)
972         repair_greedy(sol, problem, leftover, rng, policy=policy)
973     return sol
974 if __name__ == "__main__":
975     from data_loader import load_problem
976     problem = load_problem()
977     cfg = ALNSConfig(iterations=500, seed=42, verbose=True, log_every=50)
978     result = run_alns(problem, cfg)
979     print("Best total cost:", round(result.best_eval.total_cost, 2))
980     print("Routes:", result.best_eval.n_routes)
981     print("Unassigned:", len(result.best_solution.unassigned))

```

B.2 问题一代码

solve_q1.py

代码 6: 问题一代码: solve_q1.py

```

1 from __future__ import annotations
2 import argparse
3 import os
4 import pickle
5 import time
6 from pathlib import Path
7 from typing import List
8 import matplotlib
9 matplotlib.use("Agg")

```

```

10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from alns_solver import ALNSConfig, run_alns
14 from cost_model import (
15     Route,
16     Solution,
17     SolutionEval,
18     _count_physical_vehicles,
19     evaluate_route,
20     evaluate_solution,
21     speed_at,
22     travel_time,
23 )
24 from data_loader import Node, ProblemData, load_problem
25 from policy import PolicyConfig
26 RESULTS_DIR = Path(__file__).resolve().parent / "results"
27 def hhmm(t: float) -> str:
28     if t < 0:
29         return "--:--"
30     h = int(t)
31     m = int(round((t - h) * 60))
32     if m == 60:
33         h += 1
34         m = 0
35     return f"{h:02d}:{m:02d}"
36 def _annotate_pie_outside(
37     ax: plt.Axes, wedges: list, labels: List[str], sizes: List[float],
38 ) -> None:
39     total = sum(sizes)
40     if total <= 0:
41         return
42     entries = []
43     for wedge, label, size in zip(wedges, labels, sizes):
44         angle = (wedge.theta1 + wedge.theta2) / 2
45         x = np.cos(np.deg2rad(angle))
46         y = np.sin(np.deg2rad(angle))
47         entries.append({
48             "x": x,
49             "y": y,
50             "side": 1 if x >= 0 else -1,
51             "text": f"{label}\n{size / total * 100:.1f}%\n{size:.0f}元",
52         })
53     for side in (-1, 1):
54         side_entries = sorted(
55             [entry for entry in entries if entry["side"] == side],
56             key=lambda entry: entry["y"],
57         )
58     if not side_entries:

```

```

59         continue
60     y_positions = [
61         min(1.25, max(-1.25, entry["y"] * 1.18))
62         for entry in side_entries
63     ]
64     min_gap = 0.34
65     for idx in range(1, len(y_positions)):
66         y_positions[idx] = max(y_positions[idx], y_positions[idx - 1] + min_gap)
67     overflow = y_positions[-1] - 1.25
68     if overflow > 0:
69         y_positions = [y - overflow for y in y_positions]
70     for idx in range(len(y_positions) - 2, -1, -1):
71         y_positions[idx] = min(y_positions[idx], y_positions[idx + 1] - min_gap)
72     for entry, text_y in zip(side_entries, y_positions):
73         ax.annotate(
74             entry["text"],
75             xy=(entry["x"] * 0.88, entry["y"] * 0.88),
76             xytext=(entry["side"] * 1.34, text_y),
77             ha="left" if entry["side"] > 0 else "right",
78             va="center",
79             fontsize=9,
80             arrowprops=dict(
81                 arrowstyle="-",
82                 color="0.45",
83                 lw=0.8,
84                 shrinkA=0,
85                 shrinkB=0,
86                 connectionstyle="angle3,angleA=0,angleB=90",
87             ),
88         )
89     def assign_physical_labels(
90         sol_routes: List[Route], problem: ProblemData,
91     ) -> dict:
92         vt_counter: dict = {}
93         vid_to_label: dict = {}
94         labels: dict = {}
95         for ri, r in enumerate(sol_routes):
96             if not r.customers:
97                 continue
98             vt_index = r.vt_index
99             vt_name = problem.vehicle_types[vt_index].name
100             vid = getattr(r, "vehicle_id", None)
101             if vid is not None and (vt_index, vid) in vid_to_label:
102                 labels[ri] = vid_to_label[(vt_index, vid)]
103                 continue
104             vt_counter[vt_index] = vt_counter.get(vt_index, 0) + 1
105             label = f"{vt_name}-{vt_counter[vt_index]:02d}"
106             labels[ri] = label
107             if vid is not None:

```

```

108         vid_to_label[(vt_index, vid)] = label
109     return labels
110 def build_customer_demand_table(problem: ProblemData) -> pd.DataFrame:
111     rows = []
112     for orig_id, virtual_ids in sorted(problem.split_map.items()):
113         w = sum(problem.nodes[v].demand_w for v in virtual_ids)
114         vol = sum(problem.nodes[v].demand_v for v in virtual_ids)
115         a = problem.nodes[virtual_ids[0]].tw_start
116         b = problem.nodes[virtual_ids[0]].tw_end
117         rows.append({
118             "客户编号": orig_id,
119             "总重量(kg)": round(w, 3),
120             "总体积(m3)": round(vol, 4),
121             "时间窗开始": hhmm(a),
122             "时间窗结束": hhmm(b),
123             "拆分数": len(virtual_ids),
124             "是否拆分": "是" if len(virtual_ids) > 1 else "否",
125         })
126     return pd.DataFrame(rows)
127 def build_route_tables(
128     problem: ProblemData, sol_routes: List[Route]
129 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame,
130            pd.DataFrame, pd.DataFrame]:
131     phys_labels = assign_physical_labels(sol_routes, problem)
132     sol = Solution(routes=list(sol_routes))
133     n_phys_total, used_per_vt = _count_physical_vehicles(sol)
134     phys_trip_counts: dict = {}
135     for ri, r in enumerate(sol_routes):
136         if not r.customers:
137             continue
138         label = phys_labels[ri]
139         phys_trip_counts[label] = phys_trip_counts.get(label, 0) + 1
140     trip_rows = []
141     detail_rows = []
142     tw_rows = []
143     fixed_sum = n_phys_total * problem.cost.fixed_cost
144     energy_sum = carbon_m_sum = wait_sum = late_sum = carbon_kg_sum = 0.0
145     per_type_trip_counts: dict = {}
146     per_type_cap_used_w: dict = {}
147     per_type_cap_used_v: dict = {}
148     phys_agg: dict = {}
149     trip_counter = 0
150     for ri, r in enumerate(sol_routes):
151         if not r.customers:
152             continue
153         trip_counter += 1
154         ev = evaluate_route(r, problem)
155         vt = problem.vehicle_types[r.vt_index]
156         phys_label = phys_labels[ri]

```

```

157     per_type_trip_counts[vt.name] = per_type_trip_counts.get(vt.name, 0) + 1
158     per_type_cap_used_w[vt.name] = per_type_cap_used_w.get(vt.name, 0.0) + ev.total_w
159     per_type_cap_used_v[vt.name] = per_type_cap_used_v.get(vt.name, 0.0) + ev.total_v
160     energy_sum += ev.energy_cost
161     carbon_m_sum += ev.carbon_cost_money
162     carbon_kg_sum += ev.carbon_kg
163     wait_sum += ev.wait_cost
164     late_sum += ev.late_cost
165     served_origs = sorted({problem.nodes[c].orig_id for c in r.customers})
166     trip_id = f"T{trip_counter:03d}"
167     route_operating_cost = ev.total_cost - ev.fixed_cost
168     allocated_fixed = problem.cost.fixed_cost / phys_trip_counts[phys_label]
169     actual_trip_cost = route_operating_cost + allocated_fixed
170     trip_rows.append({
171         "车次编号": trip_id,
172         "物理车辆编号": phys_label,
173         "车型": vt.name,
174         "容量(kg)": vt.capacity_w,
175         "容量(m3)": vt.capacity_v,
176         "装载重量(kg)": round(ev.total_w, 2),
177         "装载体积(m3)": round(ev.total_v, 4),
178         "重量利用率": f"{ev.total_w / vt.capacity_w * 100:.1f}%",
179         "体积利用率": f"{ev.total_v / vt.capacity_v * 100:.1f}%",
180         "访问点数": len(r.customers),
181         "服务原客户数": len(served_origs),
182         "运营成本(不含启动)": round(route_operating_cost, 2),
183         "分摊启动成本": round(allocated_fixed, 2),
184         "路径成本": round(actual_trip_cost, 2),
185         "发车时间": hhmm(ev.arrival[0]),
186         "返程到达时间": hhmm(ev.arrival[-1]),
187     })
188     agg = phys_agg.setdefault(phys_label, {
189         "车型": vt.name,
190         "容量(kg)": vt.capacity_w,
191         "容量(m3)": vt.capacity_v,
192         "趟数": 0,
193         "总装载重量(kg)": 0.0,
194         "总装载体积(m3)": 0.0,
195         "总运营成本(不含启动)": 0.0,
196         "启动成本": problem.cost.fixed_cost,
197         "总路径成本": 0.0,
198         "服务原客户集合": set(),
199         "首趟发车_h": float("inf"),
200         "末趟返仓_h": float("-inf"),
201     })
202     agg["趟数"] += 1
203     agg["总装载重量(kg)"] += ev.total_w
204     agg["总装载体积(m3)"] += ev.total_v
205     agg["总运营成本(不含启动)"] += route_operating_cost

```

```

206     agg["总路径成本"] += actual_trip_cost
207     agg["服务原客户集合"].update(served_origs)
208     if ev.arrival:
209         agg["首趟发车_h"] = min(agg["首趟发车_h"], ev.arrival[0])
210         agg["末趟返仓_h"] = max(agg["末趟返仓_h"], ev.arrival[-1])
211     seq_order = 0
212     detail_rows.append({
213         "车次编号": trip_id,
214         "物理车辆编号": phys_label,
215         "车型": vt.name,
216         "顺序": seq_order,
217         "节点类型": "配送中心",
218         "原始客户编号": 0,
219         "虚拟节点编号": 0,
220         "到达时间": hhmm(ev.arrival[0]),
221         "服务开始": hhmm(ev.service_start[0]),
222         "服务结束": hhmm(ev.service_end[0]),
223         "提前量(h)": 0.0,
224         "迟到量(h)": 0.0,
225         "装载重量(kg)": round(ev.total_w, 2),
226     })
227     running_w = ev.total_w
228     for idx, c in enumerate(r.customers):
229         seq_order += 1
230         n = problem.nodes[c]
231         e = ev.early[idx]
232         l = ev.late[idx]
233         detail_rows.append({
234             "车次编号": trip_id,
235             "物理车辆编号": phys_label,
236             "车型": vt.name,
237             "顺序": seq_order,
238             "节点类型": "客户",
239             "原始客户编号": n.orig_id,
240             "虚拟节点编号": c,
241             "到达时间": hhmm(ev.arrival[idx + 1]),
242             "服务开始": hhmm(ev.service_start[idx + 1]),
243             "服务结束": hhmm(ev.service_end[idx + 1]),
244             "提前量(h)": round(e, 3),
245             "迟到量(h)": round(l, 3),
246             "装载重量(kg)": round(running_w, 2),
247         })
248         tw_rows.append({
249             "车次编号": trip_id,
250             "物理车辆编号": phys_label,
251             "原始客户编号": n.orig_id,
252             "虚拟节点编号": c,
253             "时间窗开始": hhmm(n.tw_start),
254             "时间窗结束": hhmm(n.tw_end),

```



```

255         "到达时间": hhmm(ev.arrival[idx + 1]),
256         "提前量(h)": round(e, 3),
257         "迟到量(h)": round(l, 3),
258         "是否迟到": "是" if l > 1e-6 else "否",
259         "是否早到": "是" if e > 1e-6 else "否",
260     })
261     running_w -= n.demand_w
262     seq_order += 1
263     detail_rows.append({
264         "车次编号": trip_id,
265         "物理车辆编号": phys_label,
266         "车型": vt.name,
267         "顺序": seq_order,
268         "节点类型": "配送中心(返程)",
269         "原始客户编号": 0,
270         "虚拟节点编号": 0,
271         "到达时间": hhmm(ev.arrival[-1]),
272         "服务开始": hhmm(ev.arrival[-1]),
273         "服务结束": hhmm(ev.arrival[-1]),
274         "提前量(h)": 0.0,
275         "迟到量(h)": 0.0,
276         "装载重量(kg)": 0.0,
277     })
278     total_cost = fixed_sum + energy_sum + carbon_m_sum + wait_sum + late_sum
279     cost_rows = [
280         {"成本项": "启动成本(固定)", "金额(元)": round(fixed_sum, 2),
281          "占比": f"{fixed_sum / total_cost * 100:.2f}%",},
282         {"成本项": "能源成本(油/电)", "金额(元)": round(energy_sum, 2),
283          "占比": f"{energy_sum / total_cost * 100:.2f}%",},
284         {"成本项": "碳排放成本", "金额(元)": round(carbon_m_sum, 2),
285          "占比": f"{carbon_m_sum / total_cost * 100:.2f}%",},
286         {"成本项": "等待成本(早到)", "金额(元)": round(wait_sum, 2),
287          "占比": f"{wait_sum / total_cost * 100:.2f}%",},
288         {"成本项": "晚到惩罚", "金额(元)": round(late_sum, 2),
289          "占比": f"{late_sum / total_cost * 100:.2f}%",},
290         {"成本项": "总成本", "金额(元)": round(total_cost, 2), "占比": "100.00%",},
291     ]
292     cost_df = pd.DataFrame(cost_rows)
293     type_stat_rows = []
294     for vt_idx, vt in enumerate(problem.vehicle_types):
295         n_trips = per_type_trip_counts.get(vt.name, 0)
296         n_phys = used_per_vt.get(vt_idx, 0)
297         type_stat_rows.append({
298             "车型": vt.name,
299             "容量(kg)": vt.capacity_w,
300             "容量(m3)": vt.capacity_v,
301             "可用数量": vt.count,
302             "实际车辆数": n_phys,
303             "启用趟次": n_trips,

```

```

304         "趟车比": f"{(n_trips / n_phys):.2f}" if n_phys else "--",
305         "车辆占比": (
306             f"{(n_phys / n_phys_total * 100):.2f}%"
307             if n_phys_total else "0.00%"
308         ),
309         "平均重量装载(kg)": round(
310             (per_type_cap_used_w.get(vt.name, 0) / n_trips) if n_trips else 0, 2
311         ),
312         "平均体积装载(m3)": round(
313             (per_type_cap_used_v.get(vt.name, 0) / n_trips) if n_trips else 0, 4
314         ),
315         "平均重量利用率": (
316             f"{(per_type_cap_used_w.get(vt.name, 0) / (n_trips * vt.capacity_w)) *
100:.2f}%"
317             if n_trips else "0.00%"
318         ),
319     })
320 type_stat_df = pd.DataFrame(type_stat_rows)
321 phys_rows = []
322 for label, agg in phys_agg.items():
323     phys_rows.append({
324         "物理车辆编号": label,
325         "车型": agg["车型"],
326         "容量(kg)": agg["容量(kg)"],
327         "容量(m3)": agg["容量(m3)"],
328         "趟数": agg["趟数"],
329         "总装载重量(kg)": round(agg["总装载重量(kg)"], 2),
330         "总装载体积(m3)": round(agg["总装载体积(m3)"], 4),
331         "总运营成本(不含启动)(元)": round(agg["总运营成本(不含启动)"], 2),
332         "启动成本(元)": round(agg["启动成本"], 2),
333         "总路径成本(元)": round(agg["总路径成本"], 2),
334         "服务原客户数": len(agg["服务原客户集合"]),
335         "首趟发车": hhmm(agg["首趟发车_h"]),
336         "末趟返仓": hhmm(agg["末趟返仓_h"]),
337     })
338 phys_df = pd.DataFrame(phys_rows)
339 if not phys_df.empty:
340     phys_df = phys_df.sort_values(by=["车型", "物理车辆编号"]).reset_index(drop=True)
341 trip_df = pd.DataFrame(trip_rows)
342 detail_df = pd.DataFrame(detail_rows)
343 tw_df = pd.DataFrame(tw_rows)
344 return trip_df, detail_df, cost_df, type_stat_df, tw_df, phys_df
345 def plot_routes(problem: ProblemData, sol_routes: List[Route], path: Path) -> None:
346     _set_chinese_font()
347     fig, ax = plt.subplots(figsize=(12, 10))
348     depot = problem.nodes[0]
349     policy = PolicyConfig()
350     ax.scatter([depot.x], [depot.y], s=220, marker="s", color="black",
351               label="配送中心", zorder=5)

```

```

352 orig_plotted = set()
353 for n in problem.customer_nodes:
354     if n.orig_id in orig_plotted:
355         continue
356     orig_plotted.add(n.orig_id)
357     ax.scatter([n.x], [n.y], s=18, color="lightgray", zorder=1)
358 theta = np.linspace(0, 2 * np.pi, 200)
359 ax.plot(
360     policy.green_radius * np.cos(theta),
361     policy.green_radius * np.sin(theta),
362     "--",
363     color="green",
364     alpha=0.4,
365     label=f"绿色配送区({policy.green_radius:g}km)",
366 )
367 cmap = plt.get_cmap("tab20")
368 route_idx = 0
369 for r in sol_routes:
370     if not r.customers:
371         continue
372     color = cmap(route_idx % 20)
373     xs = [depot.x] + [problem.nodes[c].x for c in r.customers] + [depot.x]
374     ys = [depot.y] + [problem.nodes[c].y for c in r.customers] + [depot.y]
375     ax.plot(xs, ys, "-o", color=color, linewidth=0.9, markersize=3, alpha=0.8)
376     route_idx += 1
377 n_phys, _ = _count_physical_vehicles(Solution(routes=list(sol_routes)))
378 ax.set_title(
379     f"问题1 最优配送路径 (共 {n_phys} 辆车 / {route_idx} 趟, "
380     f"{len(orig_plotted)} 个客户)"
381 )
382 ax.set_xlabel("X (km)")
383 ax.set_ylabel("Y (km)")
384 ax.set_aspect("equal", adjustable="datalim")
385 ax.grid(True, alpha=0.3)
386 ax.legend(loc="upper left")
387 _set_chinese_font()
388 fig.tight_layout()
389 fig.savefig(path, dpi=160)
390 plt.close(fig)
391 def plot_time_window_satisfaction(
392     tw_df: pd.DataFrame,
393     path: Path,
394     title: str = "问题1 时间窗满足情况 (正值为提前, 负值为迟到)",
395 ) -> None:
396     _set_chinese_font()
397     df = tw_df.copy()
398     df = df.sort_values(by="虚拟节点编号").reset_index(drop=True)
399     fig, ax = plt.subplots(figsize=(14, 5))
400     idx = np.arange(len(df))

```

```

401     early = df["提前量(h)"].to_numpy()
402     late = df["迟到量(h)"].to_numpy()
403     ax.bar(idx, early, color="steelblue", label="提前量(早到, 小时)")
404     ax.bar(idx, -late, color="tomato", label="迟到量(晚到, 小时)")
405     ax.axhline(0, color="black", lw=0.6)
406     ax.set_xlabel("虚拟客户点索引(按编号排序)")
407     ax.set_ylabel("时间偏差(小时)")
408     ax.set_title(title)
409     ax.legend()
410     ax.grid(True, alpha=0.3)
411     _set_chinese_font()
412     fig.tight_layout()
413     fig.savefig(path, dpi=160)
414     plt.close(fig)
415 def _set_chinese_font() -> None:
416     try:
417         import mplfonts
418         mplfonts.use_font("Noto Sans CJK SC")
419     except Exception:
420         plt.rcParams["font.sans-serif"] = [
421             "Noto Sans CJK SC", "WenQuanYi Zen Hei", "SimHei",
422         ]
423     plt.rcParams["axes.unicode_minus"] = False
424 def _service_frequency_by_virtual_node(
425     problem: ProblemData, customer_ids: List[int],
426 ) -> dict[int, int]:
427     per_orig: dict[int, int] = {}
428     for c in customer_ids:
429         orig = problem.nodes[c].orig_id
430         per_orig[orig] = per_orig.get(orig, 0) + 1
431     return {c: per_orig.get(problem.nodes[c].orig_id, 1) for c in customer_ids}
432 def _prepare_frequency_cmap(
433     fig: plt.Figure,
434     freq_values: List[int],
435     label: str = "服务频次 (同一原客户的虚拟访问次数)",
436 ):
437     cmap = plt.get_cmap("YlOrRd")
438     vmax = max(freq_values) if freq_values else 1
439     norm = plt.Normalize(vmin=1, vmax=max(1, vmax))
440     cax = fig.add_axes([0.90, 0.20, 0.018, 0.56])
441     sm = plt.cm.ScalarMappable(norm=norm, cmap=cmap)
442     sm.set_array([])
443     cb = fig.colorbar(sm, cax=cax)
444     cb.set_label(label)
445     ticks = sorted(set(freq_values))
446     if 1 <= len(ticks) <= 8:
447         cb.set_ticks(ticks)
448     return cmap, norm
449 def _finish_3d_figure(fig: plt.Figure, ax, path: Path) -> None:

```

```

450     ax.set_box_aspect((1.0, 1.0, 0.72))
451     ax.tick_params(axis="both", pad=1)
452     ax.tick_params(axis="z", pad=4)
453     fig.subplots_adjust(left=0.02, right=0.88, top=0.92, bottom=0.04)
454     fig.savefig(path, dpi=200, bbox_inches="tight", pad_inches=0.35)
455     plt.close(fig)
456 def plot_algorithm_flowchart(
457     title: str,
458     steps: List[tuple[str, str, str]],
459     path: Path,
460     footer: str | None = None,
461 ) -> None:
462     from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
463     _set_chinese_font()
464     fig, ax = plt.subplots(figsize=(11, max(8, 1.18 * len(steps) + 1.5)))
465     ax.set_xlim(0, 1)
466     ax.set_ylim(0, 1)
467     ax.axis("off")
468     top = 0.90
469     bottom = 0.08 if footer else 0.04
470     gap = 0.024
471     box_h = (top - bottom - gap * (len(steps) - 1)) / len(steps)
472     x0, w = 0.12, 0.76
473     ax.text(0.5, 0.965, title, ha="center", va="top",
474            fontsize=16, fontweight="bold")
475     for i, (headline, detail, color) in enumerate(steps):
476         y = top - (i + 1) * box_h - i * gap
477         box = FancyBboxPatch(
478             (x0, y), w, box_h,
479             boxstyle="round,pad=0.018,rounding_size=0.025",
480             facecolor=color,
481             edgecolor="#333333",
482             linewidth=1.0,
483             alpha=0.92,
484         )
485         ax.add_patch(box)
486         ax.text(x0 + 0.035, y + box_h * 0.62, headline,
487                ha="left", va="center", fontsize=12.5,
488                fontweight="bold", color="white")
489         ax.text(x0 + 0.035, y + box_h * 0.32, detail,
490                ha="left", va="center", fontsize=10.2, color="white")
491         ax.text(x0 - 0.045, y + box_h / 2, f"{i + 1}",
492                ha="center", va="center", fontsize=13,
493                fontweight="bold", color=color,
494                bbox=dict(boxstyle="circle,pad=0.25",
495                        facecolor="white", edgecolor=color,
496                        linewidth=1.2))
497     if i < len(steps) - 1:
498         y_next = top - (i + 2) * box_h - (i + 1) * gap

```

```

499         arrow = FancyArrowPatch(
500             (0.5, y), (0.5, y_next + box_h),
501             arrowstyle="->", mutation_scale=16,
502             linewidth=1.2, color="#555555",
503             shrinkA=4, shrinkB=4,
504         )
505         ax.add_patch(arrow)
506     if footer:
507         ax.text(0.5, 0.035, footer, ha="center", va="center",
508               fontsize=9.5, color="#444444")
509     fig.savefig(path, dpi=200, bbox_inches="tight", pad_inches=0.25)
510     plt.close(fig)
511 def plot_q1_algorithm_flowchart(path: Path) -> None:
512     steps = [
513         (
514             "数据读取与虚拟节点拆分",
515             "读取客户/车型/成本参数；大需求客户拆成可装载的虚拟节点",
516             "#4C72B0",
517         ),
518         (
519             "初始可行解构造",
520             "按车型容量与时间窗生成可服务车次，形成 ALNS 初始解",
521             "#55A868",
522         ),
523         (
524             "ALNS 自适应迭代",
525             "Destroy / Repair 算子按历史表现更新权重，持续扰动与重构路径",
526             "#8172B3",
527         ),
528         (
529             "成本与时间窗评估",
530             "逐车次核算能源、碳、等待、迟到；解层面按物理车辆计启动成本",
531             "#C44E52",
532         ),
533         (
534             "接受准则与最优解保留",
535             "用模拟退火式接受准则跳出局部最优，并记录当前全局最优方案",
536             "#DD8452",
537         ),
538         (
539             "多趟合并与 vehicle_id 标注",
540             "同车型且时序相容车次共享同一物理车辆，固定成本只计一次",
541             "#4A9F8B",
542         ),
543         (
544             "表格与论文图件输出",
545             "生成车次表、物理车辆表、成本表、时空图、频率图与利用率图",
546             "#5B6770",
547         ),

```

```

548     ]
549     plot_algorithm_flowchart(
550         "问题1 静态 ALNS + 多趟合并算法流程",
551         steps,
552         path,
553         footer="口径：路径搜索以车次为基本单元；成本汇总以实际物理车辆计启动成本。",
554     )
555     def plot_routes_3d_time(
556         problem: ProblemData, sol_routes: List[Route], path: Path,
557         title: str = "问题1 时空配送路径 (XY 空间 + 服务时间维度)",
558     ) -> None:
559         from mpl_toolkits.mplot3d import Axes3D
560         _set_chinese_font()
561         fig = plt.figure(figsize=(15.5, 10.5))
562         ax = fig.add_subplot(111, projection="3d")
563         depot = problem.nodes[0]
564         vt_names = [vt.name for vt in problem.vehicle_types]
565         palette = plt.get_cmap("tab10")
566         vt_color = {name: palette(i % 10) for i, name in enumerate(vt_names)}
567         all_customers = [c for r in sol_routes for c in r.customers]
568         freq_by_node = _service_frequency_by_virtual_node(problem, all_customers)
569         freq_cmap, freq_norm = _prepare_frequency_cmap(
570             fig, list(freq_by_node.values()),
571         )
572         coord_nodes = [problem.nodes[c] for c in all_customers] or [depot]
573         x_vals = [n.x for n in coord_nodes]
574         y_vals = [n.y for n in coord_nodes]
575         x_span = max(x_vals) - min(x_vals)
576         y_span = max(y_vals) - min(y_vals)
577         x_pad = max(2.0, 0.08 * x_span)
578         y_pad = max(2.0, 0.08 * y_span)
579         xlim = (min(x_vals) - x_pad, max(x_vals) + x_pad)
580         ylim = (min(y_vals) - y_pad, max(y_vals) + y_pad)
581         drawn_legend: set = set()
582         z_min, z_max = float("inf"), float("-inf")
583         for r in sol_routes:
584             if not r.customers:
585                 continue
586             ev = evaluate_route(r, problem)
587             vt = problem.vehicle_types[r.vt_index]
588             xs = [depot.x] + [problem.nodes[c].x for c in r.customers] + [depot.x]
589             ys = [depot.y] + [problem.nodes[c].y for c in r.customers] + [depot.y]
590             zs = list(ev.arrival)
591             z_min = min(z_min, min(zs))
592             z_max = max(z_max, max(zs))
593             if vt.name not in drawn_legend:
594                 ax.scatter(
595                     [], [], [], marker="o", s=48, facecolor="white",
596                     edgecolor=vt_color[vt.name], linewidth=1.2,

```

```

597         label=f"{vt.name} (点边框)",
598     )
599     drawn_legend.add(vt.name)
600     point_colors = [
601         freq_cmap(freq_norm(freq_by_node.get(c, 1)))
602         for c in r.customers
603     ]
604     point_sizes = [
605         12 + 12 * (freq_by_node.get(c, 1) - 1)
606         for c in r.customers
607     ]
608     ax.scatter(
609         xs[1:-1], ys[1:-1], zs[1:-1], s=point_sizes,
610         color=point_colors, edgecolor=vt_color[vt.name],
611         linewidth=0.35, alpha=0.9,
612     )
613     ax.scatter([depot.x], [depot.y], [max(z_min, 8.0)],
614               s=160, marker="s", color="black", label="配送中心")
615     if z_min == float("inf"):
616         z_min, z_max = 8.0, 22.0
617     ax.set_xlim(xlim)
618     ax.set_ylim(ylim)
619     for hour in (8, 12, 16, 20):
620         if z_min - 0.5 <= hour <= z_max + 0.5:
621             xx = np.linspace(*xlim, 6)
622             yy = np.linspace(*ylim, 6)
623             X, Y = np.meshgrid(xx, yy)
624             Z = np.full_like(X, hour, dtype=float)
625             ax.plot_surface(X, Y, Z, color="gray", alpha=0.04,
626                             linewidth=0, antialiased=False)
627     ax.set_xlim(xlim)
628     ax.set_ylim(ylim)
629     ax.set_xlabel("X (km)")
630     ax.set_ylabel("Y (km)")
631     ax.set_zlabel("时间 (小时, 0=00:00)", labelpad=14)
632     ax.set_title(title + "; 点颜色=服务频次", pad=16)
633     ax.view_init(elev=22, azim=-58)
634     ax.legend(loc="upper left", fontsize=9)
635     _finish_3d_figure(fig, ax, path)
636     def plot_multitrip_vehicle_usage(
637         phys_df: pd.DataFrame, path: Path,
638         title: str = "问题1 物理车辆多趟合并图谱 (77 辆车 / 160 趟)",
639     ) -> None:
640         _set_chinese_font()
641         if phys_df.empty:
642             fig, ax = plt.subplots(figsize=(8, 4))
643             ax.text(0.5, 0.5, "(无物理车辆)", ha="center", va="center")
644             fig.savefig(path, dpi=160)
645             plt.close(fig)

```



```

646         return
647     df = phys_df.copy().reset_index(drop=True)
648     df = df.sort_values(by=["车型", "趟数", "总装载重量(kg)"],
649                         ascending=[True, False, False]).reset_index(drop=True)
650     palette = plt.get_cmap("tab10")
651     vt_names = list(df["车型"].unique())
652     vt_color = {n: palette(i % 10) for i, n in enumerate(vt_names)}
653     colors = [vt_color[v] for v in df["车型"]]
654     fig, axes = plt.subplots(
655         2, 2, figsize=(15, max(8, 0.18 * len(df) + 4)),
656     )
657     y = np.arange(len(df))
658     labels = df["物理车辆编号"].tolist()
659     ax = axes[0, 0]
660     ax.barh(y, df["趟数"], color=colors)
661     ax.set_yticks(y)
662     ax.set_yticklabels(labels, fontsize=6)
663     ax.invert_yaxis()
664     ax.set_xlabel("趟数")
665     ax.set_title("(a) 每辆物理车的趟数 (2 表示多趟合并)")
666     ax.grid(True, axis="x", alpha=0.3)
667     ax = axes[0, 1]
668     ax.barh(y, df["总装载重量(kg)"], color=colors)
669     ax.set_yticks(y)
670     ax.set_yticklabels([])
671     ax.invert_yaxis()
672     ax.set_xlabel("总装载重量 (kg)")
673     ax.set_title("(b) 每辆物理车的总装载 (kg, 多趟累计)")
674     ax.grid(True, axis="x", alpha=0.3)
675     ax = axes[1, 0]
676     if "总路径成本(元)" in df.columns:
677         ax.barh(y, df["总路径成本(元)"], color=colors)
678     ax.set_yticks(y)
679     ax.set_yticklabels(labels, fontsize=6)
680     ax.invert_yaxis()
681     ax.set_xlabel("总路径成本 (元, 含分摊启动)")
682     ax.set_title("(c) 每辆物理车的总路径成本")
683     ax.grid(True, axis="x", alpha=0.3)
684     ax = axes[1, 1]
685     def _hhmm_to_h(s: str) -> float:
686         if not isinstance(s, str) or ":" not in s:
687             return float("nan")
688         h, m = s.split(":")
689         return int(h) + int(m) / 60.0
690     starts = df["首趟发车"].map(_hhmm_to_h).to_numpy()
691     ends = df["末趟返仓"].map(_hhmm_to_h).to_numpy()
692     for i, (s, e, c) in enumerate(zip(starts, ends, colors)):
693         if np.isfinite(s) and np.isfinite(e):
694             ax.barh(i, max(e - s, 0.05), left=s, height=0.7,

```

```

695         color=c, alpha=0.85)
696     ax.set_yticks(y)
697     ax.set_yticklabels([])
698     ax.invert_yaxis()
699     ax.set_xlabel("时间 (小时)")
700     ax.set_xlim(7.5, 22.5)
701     ax.set_title("(d) 每辆物理车 首趟发车 → 末趟返仓 时间区间")
702     ax.grid(True, axis="x", alpha=0.3)
703     handles = [
704         plt.Rectangle((0, 0), 1, 1, color=vt_color[v]) for v in vt_names
705     ]
706     fig.legend(handles, vt_names, loc="lower center",
707               ncol=len(vt_names), bbox_to_anchor=(0.5, -0.01))
708     fig.suptitle(title, fontsize=13)
709     fig.tight_layout(rect=(0, 0.03, 1, 0.97))
710     fig.savefig(path, dpi=180)
711     plt.close(fig)
712 def plot_load_utilization_by_type(
713     trip_df: pd.DataFrame, problem: ProblemData, path: Path,
714     title: str = "问题1 各车型每趟装载利用率分布",
715 ) -> None:
716     _set_chinese_font()
717     if trip_df.empty:
718         fig, ax = plt.subplots(figsize=(8, 4))
719         ax.text(0.5, 0.5, "(无车次数据)", ha="center", va="center")
720         fig.savefig(path, dpi=160)
721         plt.close(fig)
722         return
723     df = trip_df.copy()
724     def _pct_to_float(s: str) -> float:
725         if isinstance(s, (int, float)):
726             return float(s)
727         if isinstance(s, str) and s.endswith("%"):
728             try:
729                 return float(s.rstrip("%"))
730             except ValueError:
731                 return float("nan")
732         return float("nan")
733     df["w_util"] = df["重量利用率"].map(_pct_to_float)
734     df["v_util"] = df["体积利用率"].map(_pct_to_float)
735     vt_names = [vt.name for vt in problem.vehicle_types]
736     fig, axes = plt.subplots(1, 2, figsize=(14, 5.5), sharey=True)
737     palette = plt.get_cmap("tab10")
738     vt_color = {n: palette(i % 10) for i, n in enumerate(vt_names)}
739     for ax, key, label in [
740         (axes[0], "w_util", "重量利用率 (%)" ),
741         (axes[1], "v_util", "体积利用率 (%)" ),
742     ]:
743         data = []

```

```

744     labels = []
745     colors_box = []
746     for n in vt_names:
747         grp = df.loc[df["车型"] == n, key].dropna().values
748         if len(grp) == 0:
749             continue
750         data.append(grp)
751         labels.append(f"{n}\n(n={len(grp)})")
752         colors_box.append(vt_color[n])
753     if not data:
754         ax.text(0.5, 0.5, "(无样本)", transform=ax.transAxes,
755                ha="center", va="center")
756         continue
757     bp = ax.boxplot(
758         data, labels=labels, patch_artist=True, widths=0.55,
759         medianprops=dict(color="black", linewidth=1.2),
760         showmeans=True,
761         meanprops=dict(marker="D", markerfacecolor="white",
762                        markeredgecolor="black", markersize=5),
763     )
764     for patch, c in zip(bp["boxes"], colors_box):
765         patch.set_facecolor(c)
766         patch.set_alpha(0.65)
767     for i, grp in enumerate(data):
768         xs = np.full_like(grp, i + 1, dtype=float) + np.random.uniform(
769             -0.12, 0.12, size=len(grp))
770     )
771     ax.scatter(xs, grp, s=10, color="black", alpha=0.45)
772     ax.axhline(100, color="red", linestyle="--", linewidth=0.8, alpha=0.5)
773     ax.set_ylabel(label)
774     ax.set_title(label + " 分布")
775     ax.set_ylim(0, 110)
776     ax.grid(True, axis="y", alpha=0.3)
777     fig.suptitle(title, fontsize=13)
778     fig.tight_layout(rect=(0, 0.02, 1, 0.96))
779     fig.savefig(path, dpi=180)
780     plt.close(fig)
781     def write_conclusion(
782         problem: ProblemData,
783         ev: SolutionEval,
784         type_stat: pd.DataFrame,
785         n_physical: int,
786         n_trips: int,
787         path: Path,
788     ) -> None:
789         with path.open("w", encoding="utf-8") as f:
790             f.write("# 问题 1 结果汇总\n\n")
791             f.write(
792                 f"- 实际启用物理车辆数: **{n_physical} 辆** (共执行 {n_trips} 趟; "

```

```

793         f"多趟合并节省 {n_trips - n_physical} 次启动)\n"
794     )
795     f.write(f"- 总成本: {ev.total_cost:.2f} 元\n")
796     f.write(f"  - 启动成本: {ev.fixed_cost:.2f} 元 "
797           f" (= {n_physical} 辆 × {problem.cost.fixed_cost:.0f} 元/辆)\n")
798     f.write(f"  - 能源成本: {ev.energy_cost:.2f} 元\n")
799     f.write(f"  - 碳排放成本: {ev.carbon_cost_money:.2f} 元 "
800           f"(排放量 {ev.carbon_kg:.2f} kg CO2)\n")
801     f.write(f"  - 等待成本: {ev.wait_cost:.2f} 元\n")
802     f.write(f"  - 晚到惩罚: {ev.late_cost:.2f} 元\n\n")
803     f.write("## 车队结构\n\n")
804     f.write(
805         "> 注: `实际车辆数` 为同一物理车辆按 `vehicle_id` 聚合后的数量, "
806         "`启用趟次` 为该车型当日完成的总趟数 (一辆车可跑多趟).\n\n"
807     )
808     cols = list(type_stat.columns)
809     f.write("| " + " | ".join(cols) + " |\n")
810     f.write("| " + " | ".join(["---"] * len(cols)) + " |\n")
811     for _, row in type_stat.iterrows():
812         f.write("| " + " | ".join(str(row[c]) for c in cols) + " |\n")
813     f.write("\n")
814     major = max(
815         ("启动成本", ev.fixed_cost),
816         ("能源成本", ev.energy_cost),
817         ("碳排放成本", ev.carbon_cost_money),
818         ("等待成本", ev.wait_cost),
819         ("晚到惩罚", ev.late_cost),
820         key=lambda x: x[1],
821     )
822     f.write("## 关键结论\n\n")
823     f.write(
824         f"1. 最优总成本 **{ev.total_cost:.2f} 元**, 启用 **{n_physical} 辆物理车** "
825         f"(完成 {n_trips} 趟配送).\n\n"
826     )
827     active = [
828         (r['车型'], int(r['实际车辆数']), int(r['启用趟次']))
829         for _, r in type_stat.iterrows() if int(r['实际车辆数']) > 0
830     ]
831     f.write(
832         "2. 最优车队结构 (实际车辆 / 趟次): "
833         + ", ".join(f"{n}×{p}辆({t}趟)" for n, p, t in active)
834         + ".\n\n"
835     )
836     f.write(f"3. 主要成本来源: **{major[0]**, 占比 "
837           f"{major[1]/ev.total_cost*100:.1f}%.\n")
838     f.write("\n## 论文图件索引\n\n")
839     f.write(
840         "| 文件 | 算法/数据含义 | \n"
841         "| --- | --- | \n"

```

```

$42         "| `q1_routes.png` | 二维路径总览 — 展示 ALNS 最终解在地理空间的分布. |\n"
$43         "| `q1_algorithm_flowchart.png` | **问题1算法流程图** — 从数据拆分、初始解、ALNS
    自适应迭代、成本评估到多趟合并与输出, 概括静态求解主线. |\n"
$44         "| `q1_routes_3d_time.png` | **三维时空路径** (X / Y / 服务时间). "
$45         "每个点代表一次虚拟节点服务, 点边框按车型区分, 点色按服务频次由低到高渐变; 反映
    ALNS 同时优化空间路径 "
$46         "与时间窗调度的耦合效果. |\n"
$47         "| `q1_multitrip_vehicle_usage.png` | **物理车辆多趟合并图谱**."
$48         f"四个子图分别给出每辆物理车的趟数 / 总装载 (kg) / 总路径成本 / "
$49         f"首末趟时间区间, 突出 {n_physical} 辆车完成 {n_trips} 趟的多趟 "
$50         "合并算子收益. |\n"
$51         "| `q1_load_utilization_by_type.png` | **各车型每趟装载利用率分布** "
$52         "(箱线图). 重量与体积利用率分车型展示, 解释 ALNS 为什么倾向某些 "
$53         "车型组合 — 例如重型车被分配到高重量利用率任务. |\n"
$54         "| `q1_time_windows.png` | 时间窗满足情况柱状图 (提前 / 迟到). |\n"
$55         "| `q1_cost_pie.png` | 成本构成饼图. |\n"
$56     )
$57 def _solution_from_baseline(baseline: dict) -> Solution:
$58     sol = Solution()
$59     for r in baseline["routes"]:
$60         sol.routes.append(Route(
$61             vt_index=r["vt_index"],
$62             customers=list(r["customers"]),
$63             vehicle_id=r.get("vehicle_id"),
$64             arc_depart_floor=(
$65                 None if r.get("arc_depart_floor") is None
$66                 else dict(r["arc_depart_floor"])
$67             ),
$68         ))
$69     return sol
$70 def main(
$71     iterations: int = 3000,
$72     seed: int = 42,
$73     from_cache: bool = False,
$74 ) -> None:
$75     RESULTS_DIR.mkdir(exist_ok=True)
$76     print("[solve_q1] Loading data...")
$77     problem = load_problem()
$78     print(f"[solve_q1] Virtual nodes (incl. depot): {problem.n_nodes}")
$79     cache_path = RESULTS_DIR / "q1_baseline.pkl"
$80     if from_cache:
$81         if not cache_path.exists():
$82             raise FileNotFoundError(
$83                 f"--from-cache requested but {cache_path} not found. "
$84                 f"Run without --from-cache first."
$85             )
$86     print(f"[solve_q1] Loading cached baseline from {cache_path} ...")
$87     with cache_path.open("rb") as f:
$88         baseline = pickle.load(f)

```

```

889     sol = _solution_from_baseline(baseline)
890     ev = evaluate_solution(sol, problem)
891     print(f"[solve_q1] Cached cost re-evaluated = {ev.total_cost:.2f} "
892           f"(stored {baseline['eval']['total_cost']:.2f})")
893 else:
894     print(f"[solve_q1] Running ALNS ({iterations} iterations, "
895           f"seed={seed}) ...")
896     cfg = ALNSConfig(iterations=iterations, seed=seed,
897                       verbose=True, log_every=200)
898     t0 = time.time()
899     result = run_alns(problem, cfg)
900     print(f"[solve_q1] ALNS finished in {time.time()-t0:.1f}s. "
901           f"Best cost = {result.best_eval.total_cost:.2f}")
902     sol = result.best_solution
903     ev = result.best_eval
904     n_physical, _used_per_vt = _count_physical_vehicles(sol)
905     n_trips = sum(1 for r in sol.routes if r.customers)
906     print("[solve_q1] Building deliverable tables ...")
907     demand_df = build_customer_demand_table(problem)
908     trip_df, detail_df, cost_df, type_stat_df, tw_df, phys_df = build_route_tables(
909         problem, sol.routes
910     )
911     with pd.ExcelWriter(RESULTS_DIR / "q1_tables.xlsx", engine="openpyxl") as xw:
912         demand_df.to_excel(xw, sheet_name="客户需求汇总表", index=False)
913         phys_df.to_excel(xw, sheet_name="物理车辆汇总表", index=False)
914         trip_df.to_excel(xw, sheet_name="车次使用方案表", index=False)
915         detail_df.to_excel(xw, sheet_name="路径结果表", index=False)
916         cost_df.to_excel(xw, sheet_name="成本构成表", index=False)
917         type_stat_df.to_excel(xw, sheet_name="车型使用结构统计表", index=False)
918         tw_df.to_excel(xw, sheet_name="时间窗满足情况表", index=False)
919     print("[solve_q1] Plotting routes ...")
920     plot_routes(problem, sol.routes, RESULTS_DIR / "q1_routes.png")
921     plot_time_window_satisfaction(tw_df, RESULTS_DIR / "q1_time_windows.png")
922     print("[solve_q1] Plotting paper-grade figures (3D / multi-trip / load) ...")
923     plot_routes_3d_time(
924         problem, sol.routes, RESULTS_DIR / "q1_routes_3d_time.png",
925         title=(
926             f"问题1 时空配送路径 (XY 空间 + 服务时间维度, "
927             f"{n_physical} 辆车 / {n_trips} 趟)"
928         ),
929     )
930     plot_multitrip_vehicle_usage(
931         phys_df, RESULTS_DIR / "q1_multitrip_vehicle_usage.png",
932         title=(
933             f"问题1 物理车辆多趟合并图谱 ({n_physical} 辆车 / {n_trips} 趟)"
934         ),
935     )
936     plot_load_utilization_by_type(
937         trip_df, problem, RESULTS_DIR / "q1_load_utilization_by_type.png",

```

```

938     )
939     plot_q1_algorithm_flowchart(RESULTS_DIR / "q1_algorithm_flowchart.png")
940     fig, ax = plt.subplots(figsize=(7, 6))
941     labels = ["启动", "能源", "碳排放", "等待", "晚到"]
942     sizes = [ev.fixed_cost, ev.energy_cost, ev.carbon_cost_money,
943             ev.wait_cost, ev.late_cost]
944     _set_chinese_font()
945     explode = [0, 0, 0, 0.06, 0.14]
946     wedges, _ = ax.pie(
947         sizes,
948         labels=None,
949         startangle=90,
950         explode=explode,
951         wedgeprops={"linewidth": 0.6, "edgecolor": "white"},
952     )
953     _annotate_pie_outside(ax, wedges, labels, sizes)
954     ax.set_title(f"问题1 成本构成 (总成本 {ev.total_cost:.0f} 元)")
955     ax.set_aspect("equal")
956     ax.set_xlim(-1.55, 1.55)
957     ax.set_ylim(-1.35, 1.35)
958     fig.tight_layout()
959     fig.savefig(RESULTS_DIR / "q1_cost_pie.png", dpi=160)
960     plt.close(fig)
961     write_conclusion(
962         problem, ev, type_stat_df, n_physical, n_trips,
963         RESULTS_DIR / "q1_summary.md",
964     )
965     print(f"[solve_q1] Trips={n_trips}, physical vehicles={n_physical}, "
966           f"multi-trip savings="
967           f"{(n_trips - n_physical) * problem.cost.fixed_cost:.0f} 元")
968     baseline = {
969         "routes": [{"vt_index": r.vt_index,
970                    "customers": list(r.customers),
971                    "vehicle_id": getattr(r, "vehicle_id", None),
972                    "arc_depart_floor": (
973                        None if getattr(r, "arc_depart_floor", None) is None
974                        else dict(r.arc_depart_floor)
975                    )}],
976         "for r in sol.routes if r.customers",
977         "eval": {
978             "total_cost": ev.total_cost,
979             "fixed_cost": ev.fixed_cost,
980             "energy_cost": ev.energy_cost,
981             "carbon_cost_money": ev.carbon_cost_money,
982             "carbon_kg": ev.carbon_kg,
983             "wait_cost": ev.wait_cost,
984             "late_cost": ev.late_cost,
985             "n_routes": ev.n_routes,
986             "n_trips": n_trips,

```

```

987         "n_physical_vehicles": n_physical,
988     },
989     "n_virtual_nodes": problem.n_nodes,
990     "split_map": problem.split_map,
991 }
992 with cache_path.open("wb") as f:
993     pickle.dump(baseline, f)
994 print(f"\n[solve_q1] Done. Results saved to {RESULTS_DIR}/")
995 print(f"    Total cost: {ev.total_cost:.2f} 元")
996 print(f"    Trips: {n_trips} / Physical vehicles: {n_physical}")
997 print(f"    Carbon emitted: {ev.carbon_kg:.2f} kg CO2")
998 if __name__ == "__main__":
999     parser = argparse.ArgumentParser()
1000     parser.add_argument("--iters", type=int, default=3000)
1001     parser.add_argument("--seed", type=int, default=42)
1002     parser.add_argument(
1003         "--from-cache", action="store_true",
1004         help="Skip ALNS and rebuild tables/plots/summary from "
1005             "results/q1_baseline.pkl. Use after a code-only fix to the "
1006             "reporting layer to avoid re-running the search.",
1007     )
1008     args = parser.parse_args()
1009     main(iterations=args.iters, seed=args.seed, from_cache=args.from_cache)

```

B.3 问题二代码

solve_q2.py

代码 7: 问题二代码: solve_q2.py

```

1  from __future__ import annotations
2  import argparse
3  import pickle
4  import time
5  from pathlib import Path
6  from typing import Dict, List, Tuple
7  import matplotlib
8  matplotlib.use("Agg")
9  import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12 from alns_solver import ALNSConfig, run_alns
13 from cost_model import (
14     Route,
15     Solution,
16     SolutionEval,
17     _count_physical_vehicles,
18     evaluate_route,
19     evaluate_solution,

```



```

20 )
21 from data_loader import Node, ProblemData, load_problem
22 from policy import PolicyConfig, compute_green_mask, green_zone_customers
23 from solve_q1 import (
24     _solution_from_baseline,
25     _finish_3d_figure,
26     _prepare_frequency_cmap,
27     _service_frequency_by_virtual_node,
28     assign_physical_labels,
29     build_customer_demand_table,
30     build_route_tables,
31     hhmm,
32     plot_algorithm_flowchart,
33     plot_time_window_satisfaction,
34     _set_chinese_font,
35 )
36 RESULTS_DIR = Path(__file__).resolve().parent / "results"
37 def format_policy_window(policy: PolicyConfig) -> str:
38     return f"{hhmm(policy.fuel_restrict_start)}--{hhmm(policy.fuel_restrict_end)}"
39 def green_zone_label(policy: PolicyConfig) -> str:
40     return f"绿色配送区({policy.green_radius:g}km)"
41 def compute_green_service_stats(
42     carrier_df: pd.DataFrame, policy: PolicyConfig,
43 ) -> dict[str, int]:
44     if carrier_df.empty:
45         return {
46             "total_visits": 0,
47             "electric_visits": 0,
48             "fuel_visits": 0,
49             "fuel_after_window": 0,
50             "compliant_visits": 0,
51         }
52     is_electric = carrier_df["是否新能源"] == "是"
53     is_fuel = ~is_electric
54     return {
55         "total_visits": int(len(carrier_df)),
56         "electric_visits": int(is_electric.sum()),
57         "fuel_visits": int(is_fuel.sum()),
58         "fuel_after_window": int(
59             (is_fuel & (carrier_df["到达小时"] >= policy.fuel_restrict_end - 1e-9)).sum()
60         ),
61         "compliant_visits": int((carrier_df["是否合规"] == "合规").sum()),
62     }
63 def build_green_customer_table(problem: ProblemData, policy: PolicyConfig) -> pd.DataFrame:
64     rows = []
65     for orig_id, virtual_ids in sorted(problem.split_map.items()):
66         nd = problem.nodes[virtual_ids[0]]
67         in_green = policy.in_green_zone(nd.x, nd.y)
68         dist_to_center = float(np.sqrt(nd.x ** 2 + nd.y ** 2))

```

```

69     w = sum(problem.nodes[v].demand_w for v in virtual_ids)
70     vol = sum(problem.nodes[v].demand_v for v in virtual_ids)
71     rows.append({
72         "客户编号": orig_id,
73         "X(km)": round(nd.x, 3),
74         "Y(km)": round(nd.y, 3),
75         "距市中心(km)": round(dist_to_center, 3),
76         "是否绿色区": "是" if in_green else "否",
77         "总重量(kg)": round(w, 3),
78         "总体积(m3)": round(vol, 4),
79         "时间窗开始": hhmm(nd.tw_start),
80         "时间窗结束": hhmm(nd.tw_end),
81         "拆分数": len(virtual_ids),
82     })
83     return pd.DataFrame(rows)
84 def build_green_carrier_table(
85     problem: ProblemData, sol_routes: List[Route], policy: PolicyConfig,
86 ) -> pd.DataFrame:
87     rows = []
88     green_origs = set(green_zone_customers(problem.nodes, policy))
89     phys_labels = assign_physical_labels(sol_routes, problem)
90     virt_assign: Dict[int, Tuple[str, str, str, bool, float]] = {}
91     trip_counter = 0
92     for ri, r in enumerate(sol_routes):
93         if not r.customers:
94             continue
95         trip_counter += 1
96         ev = evaluate_route(r, problem, policy=policy)
97         vt = problem.vehicle_types[r.vt_index]
98         trip_id = f"T{trip_counter:03d}"
99         for idx, c in enumerate(r.customers):
100             virt_assign[c] = (
101                 trip_id, phys_labels[ri], vt.name, vt.is_fuel,
102                 ev.arrival[idx + 1],
103             )
104     for orig_id in sorted(green_origs):
105         virtual_ids = problem.split_map.get(orig_id, [])
106         for v in virtual_ids:
107             if v in virt_assign:
108                 trip_id, phys_id, vt_name, is_fuel, arrive = virt_assign[v]
109                 rows.append({
110                     "客户编号": orig_id,
111                     "虚拟节点编号": v,
112                     "承运车次": trip_id,
113                     "物理车辆编号": phys_id,
114                     "承运车型": vt_name,
115                     "是否新能源": "否" if is_fuel else "是",
116                     "到达时间": hhmm(arrive),
117                     "到达小时": round(arrive, 3),

```

```

118         "是否合规": (
119             "合规"
120             if (not is_fuel)
121             or arrive >= policy.fuel_restrict_end - 1e-9
122             else "违规"
123         ),
124     })
125     return pd.DataFrame(rows)
126 def build_carbon_breakdown_table(
127     problem: ProblemData, sol_routes: List[Route], policy: PolicyConfig,
128 ) -> pd.DataFrame:
129     fuel_kg = fuel_energy = fuel_carbon_money = 0.0
130     elec_kg = elec_energy = elec_carbon_money = 0.0
131     fuel_trips = elec_trips = 0
132     fuel_vid_set: set = set()
133     elec_vid_set: set = set()
134     fuel_anon = elec_anon = 0
135     for r in sol_routes:
136         if not r.customers:
137             continue
138         vt = problem.vehicle_types[r.vt_index]
139         ev = evaluate_route(r, problem, policy=policy)
140         vid = getattr(r, "vehicle_id", None)
141         if vt.is_fuel:
142             fuel_kg += ev.carbon_kg
143             fuel_energy += ev.energy_cost
144             fuel_carbon_money += ev.carbon_cost_money
145             fuel_trips += 1
146             if vid is None:
147                 fuel_anon += 1
148             else:
149                 fuel_vid_set.add(vid)
150         else:
151             elec_kg += ev.carbon_kg
152             elec_energy += ev.energy_cost
153             elec_carbon_money += ev.carbon_cost_money
154             elec_trips += 1
155             if vid is None:
156                 elec_anon += 1
157             else:
158                 elec_vid_set.add(vid)
159     fuel_phys = fuel_anon + len(fuel_vid_set)
160     elec_phys = elec_anon + len(elec_vid_set)
161     total_kg = fuel_kg + elec_kg
162     rows = [
163         {
164             "动力类型": "燃油车",
165             "实际车辆数": fuel_phys,
166             "启用趟次": fuel_trips,

```

```

167         "能源成本(元)": round(fuel_energy, 2),
168         "碳排放(kg CO2)": round(fuel_kg, 2),
169         "碳排放占比": f"{fuel_kg / total_kg * 100:.2f}%" if total_kg else "--",
170         "碳成本(元)": round(fuel_carbon_money, 2),
171     },
172     {
173         "动力类型": "新能源车",
174         "实际车辆数": elec_phys,
175         "启用趟次": elec_trips,
176         "能源成本(元)": round(elec_energy, 2),
177         "碳排放(kg CO2)": round(elec_kg, 2),
178         "碳排放占比": f"{elec_kg / total_kg * 100:.2f}%" if total_kg else "--",
179         "碳成本(元)": round(elec_carbon_money, 2),
180     },
181     {
182         "动力类型": "合计",
183         "实际车辆数": fuel_phys + elec_phys,
184         "启用趟次": fuel_trips + elec_trips,
185         "能源成本(元)": round(fuel_energy + elec_energy, 2),
186         "碳排放(kg CO2)": round(total_kg, 2),
187         "碳排放占比": "100.00%",
188         "碳成本(元)": round(fuel_carbon_money + elec_carbon_money, 2),
189     },
190 ]
191 return pd.DataFrame(rows)
192
193 def _per_vt_physical_counts_from_baseline(
194     baseline: dict, problem: ProblemData,
195 ) -> Tuple[Dict[str, int], Dict[str, int]]:
196     trips: Dict[str, int] = {vt.name: 0 for vt in problem.vehicle_types}
197     phys_set: Dict[str, set] = {vt.name: set() for vt in problem.vehicle_types}
198     phys_anon: Dict[str, int] = {vt.name: 0 for vt in problem.vehicle_types}
199     for r in baseline["routes"]:
200         if not r.get("customers"):
201             continue
202         vt_name = problem.vehicle_types[r["vt_index"]].name
203         trips[vt_name] += 1
204         vid = r.get("vehicle_id")
205         if vid is None:
206             phys_anon[vt_name] += 1
207         else:
208             phys_set[vt_name].add(vid)
209     phys = {n: phys_anon[n] + len(phys_set[n]) for n in trips}
210     return phys, trips
211
212 def build_comparison_tables(
213     q1_baseline: dict,
214     q2_ev: SolutionEval,
215     q2_type_stat: pd.DataFrame,
216     problem: ProblemData,
217 ) -> tuple[pd.DataFrame, pd.DataFrame]:

```

```

216 q1_ev = q1_baseline["eval"]
217 rows = []
218 items = [
219     ("启动成本", "fixed_cost"),
220     ("能源成本", "energy_cost"),
221     ("碳排放成本", "carbon_cost_money"),
222     ("等待成本", "wait_cost"),
223     ("晚到惩罚", "late_cost"),
224     ("总成本", "total_cost"),
225     ("碳排放量(kg)", "carbon_kg"),
226 ]
227 for name, key in items:
228     q1v = q1_ev[key]
229     q2v = getattr(q2_ev, key)
230     rows.append({
231         "指标": name,
232         "问题一(无政策)": round(q1v, 2),
233         "问题二(有政策)": round(q2v, 2),
234         "绝对增量": round(q2v - q1v, 2),
235         "相对增量": (
236             f"{{(q2v - q1v) / q1v * 100:+.2f}}%" if abs(q1v) > 1e-6 else "---"
237         ),
238     })
239 q1_phys = q1_ev.get("n_physical_vehicles")
240 q1_trips = q1_ev.get("n_trips", q1_ev.get("n_routes"))
241 q2_phys = getattr(q2_ev, "_n_physical_vehicles", None)
242 if q2_phys is None:
243     q2_phys = sum(int(r["实际车辆数"]) for _, r in q2_type_stat.iterrows())
244 q2_trips = sum(int(r["启用趟次"]) for _, r in q2_type_stat.iterrows())
245 rows.append({
246     "指标": "实际车辆数(辆)",
247     "问题一(无政策)": q1_phys,
248     "问题二(有政策)": q2_phys,
249     "绝对增量": q2_phys - q1_phys,
250     "相对增量": (
251         f"{{(q2_phys - q1_phys) / q1_phys * 100:+.2f}}%"
252         if q1_phys else "---"
253     ),
254 })
255 rows.append({
256     "指标": "总趟次",
257     "问题一(无政策)": q1_trips,
258     "问题二(有政策)": q2_trips,
259     "绝对增量": q2_trips - q1_trips,
260     "相对增量": (
261         f"{{(q2_trips - q1_trips) / q1_trips * 100:+.2f}}%"
262         if q1_trips else "---"
263     ),
264 })

```

```

265     cost_df = pd.DataFrame(rows)
266     q1_phys_per_vt, q1_trips_per_vt = _per_vt_physical_counts_from_baseline(
267         q1_baseline, problem,
268     )
269     q2_phys_per_vt = {
270         row["车型"]: int(row["实际车辆数"]) for _, row in q2_type_stat.iterrows()
271     }
272     q2_trips_per_vt = {
273         row["车型"]: int(row["启用趟次"]) for _, row in q2_type_stat.iterrows()
274     }
275     fleet_rows = []
276     for vt in problem.vehicle_types:
277         q1p = q1_phys_per_vt.get(vt.name, 0)
278         q1t = q1_trips_per_vt.get(vt.name, 0)
279         q2p = q2_phys_per_vt.get(vt.name, 0)
280         q2t = q2_trips_per_vt.get(vt.name, 0)
281         fleet_rows.append({
282             "车型": vt.name,
283             "动力": "燃油" if vt.is_fuel else "新能源",
284             "可用数量": vt.count,
285             "问题一实际车辆": q1p,
286             "问题一趟次": q1t,
287             "问题二实际车辆": q2p,
288             "问题二趟次": q2t,
289             "实际车辆变化": q2p - q1p,
290             "趟次变化": q2t - q1t,
291         })
292     fleet_df = pd.DataFrame(fleet_rows)
293     return cost_df, fleet_df
294 def plot_routes_q2(
295     problem: ProblemData, sol_routes: List[Route], policy: PolicyConfig, path: Path,
296 ) -> None:
297     _set_chinese_font()
298     fig, ax = plt.subplots(figsize=(12, 10))
299     depot = problem.nodes[0]
300     ax.scatter([depot.x], [depot.y], s=220, marker="s", color="black",
301               label="配送中心", zorder=5)
302     theta = np.linspace(0, 2 * np.pi, 300)
303     ax.fill(
304         policy.green_radius * np.cos(theta),
305         policy.green_radius * np.sin(theta),
306         color="lightgreen", alpha=0.2, zorder=0,
307     )
308     ax.plot(
309         policy.green_radius * np.cos(theta),
310         policy.green_radius * np.sin(theta),
311         "--", color="green", alpha=0.8, label=green_zone_label(policy),
312     )
313     plotted = set()

```

```

314     for n in problem.customer_nodes:
315         if n.orig_id in plotted:
316             continue
317         plotted.add(n.orig_id)
318         is_green = policy.in_green_zone(n.x, n.y)
319         ax.scatter([n.x], [n.y], s=24,
320                   color="seagreen" if is_green else "lightgray",
321                   zorder=1, edgecolors="none")
322     fuel_cmap = plt.get_cmap("Oranges")
323     elec_cmap = plt.get_cmap("Blues")
324     fuel_i = elec_i = 0
325     n_fuel = sum(1 for r in sol_routes if r.customers
326                 and problem.vehicle_types[r.vt_index].is_fuel)
327     n_elec = sum(1 for r in sol_routes if r.customers
328                 and not problem.vehicle_types[r.vt_index].is_fuel)
329     fuel_handle_drawn = elec_handle_drawn = False
330     for r in sol_routes:
331         if not r.customers:
332             continue
333         vt = problem.vehicle_types[r.vt_index]
334         if vt.is_fuel:
335             color = fuel_cmap(0.3 + 0.6 * fuel_i / max(n_fuel - 1, 1))
336             label = "燃油车路径" if not fuel_handle_drawn else None
337             fuel_handle_drawn = True
338             fuel_i += 1
339         else:
340             color = elec_cmap(0.3 + 0.6 * elec_i / max(n_elec - 1, 1))
341             label = "新能源车路径" if not elec_handle_drawn else None
342             elec_handle_drawn = True
343             elec_i += 1
344         xs = [depot.x] + [problem.nodes[c].x for c in r.customers] + [depot.x]
345         ys = [depot.y] + [problem.nodes[c].y for c in r.customers] + [depot.y]
346         ax.plot(xs, ys, "-o", color=color, linewidth=0.9,
347               markersize=3, alpha=0.85, label=label)
348     fuel_phys_set: set = set()
349     fuel_anon = 0
350     elec_phys_set: set = set()
351     elec_anon = 0
352     for r in sol_routes:
353         if not r.customers:
354             continue
355         vt = problem.vehicle_types[r.vt_index]
356         vid = getattr(r, "vehicle_id", None)
357         if vt.is_fuel:
358             if vid is None:
359                 fuel_anon += 1
360             else:
361                 fuel_phys_set.add(vid)
362         else:

```

```

363         if vid is None:
364             elec_anon += 1
365         else:
366             elec_phys_set.add(vid)
367     fuel_phys = fuel_anon + len(fuel_phys_set)
368     elec_phys = elec_anon + len(elec_phys_set)
369     ax.set_title(
370         f"问题2 绿色政策下最优配送路径 "
371         f"(燃油 {fuel_phys}辆/{n_fuel}趟, 新能源 {elec_phys}辆/{n_elec}趟)"
372     )
373     ax.set_xlabel("X (km)")
374     ax.set_ylabel("Y (km)")
375     ax.set_aspect("equal", adjustable="datalim")
376     ax.grid(True, alpha=0.3)
377     ax.legend(loc="upper left")
378     _set_chinese_font()
379     fig.tight_layout()
380     fig.savefig(path, dpi=160)
381     plt.close(fig)
382 def plot_cost_pie_q2(ev: SolutionEval, path: Path) -> None:
383     _set_chinese_font()
384     fig, ax = plt.subplots(figsize=(7, 6))
385     labels = ["启动", "能源", "碳排放", "等待", "晚到"]
386     sizes = [ev.fixed_cost, ev.energy_cost, ev.carbon_cost_money,
387             ev.wait_cost, ev.late_cost]
388     ax.pie(sizes, labels=labels, autopct="%1.1f%%", startangle=90)
389     ax.set_title(f"问题2 成本构成 (总成本 {ev.total_cost:.0f} 元)")
390     fig.tight_layout()
391     fig.savefig(path, dpi=160)
392     plt.close(fig)
393 def plot_cost_compare(cost_df: pd.DataFrame, path: Path) -> None:
394     _set_chinese_font()
395     items = cost_df[cost_df["指标"].isin(
396         ["启动成本", "能源成本", "碳排放成本", "等待成本", "晚到惩罚"]
397     )]
398     x = np.arange(len(items))
399     width = 0.38
400     fig, ax = plt.subplots(figsize=(10, 5))
401     bars1 = ax.bar(x - width/2, items["问题一(无政策)"], width,
402                   label="问题一", color="#4C72B0")
403     bars2 = ax.bar(x + width/2, items["问题二(有政策)"], width,
404                   label="问题二", color="#55A868")
405     ax.set_xticks(x)
406     ax.set_xticklabels(items["指标"].tolist())
407     ax.set_ylabel("成本 (元)")
408     ax.set_title("问题一 vs 问题二 分项成本对比")
409     ax.legend()
410     ax.grid(True, axis="y", alpha=0.3)
411     for bars in (bars1, bars2):

```



```

412         for b in bars:
413             h = b.get_height()
414             ax.annotate(f"{h:.0f}", xy=(b.get_x() + b.get_width()/2, h),
415                         xytext=(0, 3), textcoords="offset points",
416                         ha="center", fontsize=8)
417     fig.tight_layout()
418     fig.savefig(path, dpi=160)
419     plt.close(fig)
420 def plot_fleet_compare(fleet_df: pd.DataFrame, path: Path) -> None:
421     _set_chinese_font()
422     x = np.arange(len(fleet_df))
423     width = 0.38
424     fig, axes = plt.subplots(1, 2, figsize=(14, 5))
425     xticks = [f"{r['车型']}\n({r['动力']})" for _, r in fleet_df.iterrows()]
426     ax_p = axes[0]
427     p1 = ax_p.bar(x - width/2, fleet_df["问题一实际车辆"], width,
428                  label="问题一", color="#4C72B0")
429     p2 = ax_p.bar(x + width/2, fleet_df["问题二实际车辆"], width,
430                  label="问题二", color="#55A868")
431     ax_p.set_xticks(x)
432     ax_p.set_xticklabels(xticks)
433     ax_p.set_ylabel("实际车辆数(辆)")
434     ax_p.set_title("实际物理车辆对比")
435     ax_p.legend()
436     ax_p.grid(True, axis="y", alpha=0.3)
437     for bars in (p1, p2):
438         for bb in bars:
439             h = bb.get_height()
440             ax_p.annotate(f"{int(h)}", xy=(bb.get_x() + bb.get_width()/2, h),
441                          xytext=(0, 2), textcoords="offset points",
442                          ha="center", fontsize=8)
443     ax_t = axes[1]
444     t1 = ax_t.bar(x - width/2, fleet_df["问题一趟次"], width,
445                  label="问题一", color="#4C72B0")
446     t2 = ax_t.bar(x + width/2, fleet_df["问题二趟次"], width,
447                  label="问题二", color="#55A868")
448     ax_t.set_xticks(x)
449     ax_t.set_xticklabels(xticks)
450     ax_t.set_ylabel("趟次")
451     ax_t.set_title("启用趟次对比")
452     ax_t.legend()
453     ax_t.grid(True, axis="y", alpha=0.3)
454     for bars in (t1, t2):
455         for bb in bars:
456             h = bb.get_height()
457             ax_t.annotate(f"{int(h)}", xy=(bb.get_x() + bb.get_width()/2, h),
458                          xytext=(0, 2), textcoords="offset points",
459                          ha="center", fontsize=8)
460     fig.suptitle("问题一 vs 问题二 车型启用对比 (实际车辆 / 趟次)", fontsize=12)

```

```

461     fig.tight_layout()
462     fig.savefig(path, dpi=160)
463     plt.close(fig)
464     def plot_carbon_compare(
465         q1_baseline: dict, q2_ev: SolutionEval,
466         q1_fuel_kg: float, q1_elec_kg: float,
467         q2_fuel_kg: float, q2_elec_kg: float,
468         path: Path,
469     ) -> None:
470         _set_chinese_font()
471         categories = ["问题一", "问题二"]
472         fuel_vals = [q1_fuel_kg, q2_fuel_kg]
473         elec_vals = [q1_elec_kg, q2_elec_kg]
474         x = np.arange(len(categories))
475         fig, ax = plt.subplots(figsize=(7, 5))
476         b1 = ax.bar(x, fuel_vals, 0.55, label="燃油车", color="#E57248")
477         b2 = ax.bar(x, elec_vals, 0.55, bottom=fuel_vals,
478                     label="新能源车", color="#4A9F8B")
479         for i, (fv, ev) in enumerate(zip(fuel_vals, elec_vals)):
480             ax.text(x[i], fv / 2, f"{fv:.0f}", ha="center",
481                     va="center", color="white", fontsize=10)
482             ax.text(x[i], fv + ev / 2, f"{ev:.0f}", ha="center",
483                     va="center", color="white", fontsize=10)
484             ax.text(x[i], fv + ev + 50, f"总{fv+ev:.0f}",
485                     ha="center", fontsize=9)
486         ax.set_xticks(x)
487         ax.set_xticklabels(categories)
488         ax.set_ylabel("碳排放 (kg CO2)")
489         ax.set_title("问题一 vs 问题二 碳排放对比 (按动力拆分)")
490         ax.legend()
491         ax.grid(True, axis="y", alpha=0.3)
492         fig.tight_layout()
493         fig.savefig(path, dpi=160)
494         plt.close(fig)
495     def plot_green_zone_distribution(
496         problem: ProblemData, sol_routes: List[Route], policy: PolicyConfig, path: Path,
497     ) -> None:
498         _set_chinese_font()
499         fig, axes = plt.subplots(1, 2, figsize=(14, 6))
500         for ax, title, keep_is_fuel in [
501             (axes[0], "燃油车路径 (按限行时段分色)", True),
502             (axes[1], "新能源车路径 (自由进出)", False),
503         ]:
504             depot = problem.nodes[0]
505             ax.scatter([depot.x], [depot.y], s=160, marker="s",
506                       color="black", zorder=5, label="配送中心")
507             theta = np.linspace(0, 2 * np.pi, 300)
508             ax.fill(policy.green_radius * np.cos(theta),
509                   policy.green_radius * np.sin(theta),

```

```

510         color="lightgreen", alpha=0.25, zorder=0)
511     ax.plot(policy.green_radius * np.cos(theta),
512            policy.green_radius * np.sin(theta),
513            "--", color="green", alpha=0.7)
514     plotted = set()
515     for n in problem.customer_nodes:
516         if n.orig_id in plotted:
517             continue
518         plotted.add(n.orig_id)
519         is_green = policy.in_green_zone(n.x, n.y)
520         ax.scatter([n.x], [n.y], s=18,
521                   color="seagreen" if is_green else "lightgray",
522                   zorder=1)
523     fuel_restrict_handle_drawn = False
524     fuel_open_handle_drawn = False
525     elec_handle_drawn = False
526     for r in sol_routes:
527         if not r.customers:
528             continue
529         vt = problem.vehicle_types[r.vt_index]
530         if vt.is_fuel != keep_is_fuel:
531             continue
532         route_nodes = [0] + list(r.customers) + [0]
533         xs = [problem.nodes[c].x for c in route_nodes]
534         ys = [problem.nodes[c].y for c in route_nodes]
535         if keep_is_fuel:
536             ev = evaluate_route(r, problem, policy=policy)
537             for leg_idx in range(len(route_nodes) - 1):
538                 t0 = ev.service_end[leg_idx]
539                 t1 = ev.arrival[leg_idx + 1]
540                 x0, y0 = xs[leg_idx], ys[leg_idx]
541                 x1, y1 = xs[leg_idx + 1], ys[leg_idx + 1]
542                 if t1 <= t0 + 1e-9:
543                     restricted = (
544                         policy.fuel_restrict_start
545                         <= t0
546                         <= policy.fuel_restrict_end
547                     )
548                     color = "#D55E00" if restricted else "#FFD60A"
549                     label = None
550                     if restricted and not fuel_restrict_handle_drawn:
551                         label = f"燃油路径(限行 {format_policy_window(policy)})"
552                         fuel_restrict_handle_drawn = True
553                     elif (not restricted) and not fuel_open_handle_drawn:
554                         label = "燃油路径(非限行时段)"
555                         fuel_open_handle_drawn = True
556                     ax.plot([x0, x1], [y0, y1], "--", color=color,
557                           linewidth=0.75, alpha=0.75, label=label)
558                     continue

```

```

559         split_times = [t0, t1]
560         for split_t in (policy.fuel_restrict_start,
561                         policy.fuel_restrict_end):
562             if t0 < split_t < t1:
563                 split_times.append(split_t)
564         split_times = sorted(split_times)
565         for a, b in zip(split_times[:-1], split_times[1:]):
566             if b <= a + 1e-9:
567                 continue
568             mid = (a + b) / 2
569             restricted = (
570                 policy.fuel_restrict_start
571                 <= mid
572                 <= policy.fuel_restrict_end
573             )
574             color = "#D55E00" if restricted else "#FFD60A"
575             label = None
576             if restricted and not fuel_restrict_handle_drawn:
577                 label = f"燃油路径(限行 {format_policy_window(policy)})"
578                 fuel_restrict_handle_drawn = True
579             elif (not restricted) and not fuel_open_handle_drawn:
580                 label = "燃油路径(非限行时段)"
581                 fuel_open_handle_drawn = True
582             fa = (a - t0) / (t1 - t0)
583             fb = (b - t0) / (t1 - t0)
584             xa = x0 + (x1 - x0) * fa
585             ya = y0 + (y1 - y0) * fa
586             xb = x0 + (x1 - x0) * fb
587             yb = y0 + (y1 - y0) * fb
588             ax.plot([xa, xb], [ya, yb], "-", color=color,
589                     linewidth=0.75, alpha=0.75, label=label)
590         else:
591             label = "新能源车路径" if not elec_handle_drawn else None
592             elec_handle_drawn = True
593             ax.plot(xs, ys, "-", color="tab:blue", linewidth=0.7,
594                     alpha=0.6, label=label)
595         ax.set_title(title)
596         ax.set_xlabel("X (km)")
597         ax.set_ylabel("Y (km)")
598         ax.set_aspect("equal", adjustable="datalim")
599         ax.grid(True, alpha=0.3)
600         ax.legend(loc="upper left")
601     fig.suptitle("绿色区内外路径分布 (问题二)", fontsize=13)
602     fig.tight_layout()
603     fig.savefig(path, dpi=160)
604     plt.close(fig)
605     def plot_fuel_gantt(
606         problem: ProblemData, sol_routes: List[Route], policy: PolicyConfig, path: Path,
607     ) -> None:

```

```

608     _set_chinese_font()
609     phys_labels = assign_physical_labels(sol_routes, problem)
610     bars = []
611     label_to_row: Dict[str, int] = {}
612     for ri, r in enumerate(sol_routes):
613         if not r.customers:
614             continue
615         vt = problem.vehicle_types[r.vt_index]
616         if not vt.is_fuel:
617             continue
618         label = phys_labels[ri]
619         if label not in label_to_row:
620             label_to_row[label] = len(label_to_row)
621         row_idx = label_to_row[label]
622         ev = evaluate_route(r, problem, policy=policy)
623         for idx, c in enumerate(r.customers):
624             n = problem.nodes[c]
625             in_g = policy.in_green_zone(n.x, n.y)
626             bars.append((row_idx, label,
627                         ev.service_start[idx + 1],
628                         ev.service_end[idx + 1], in_g))
629     if not bars:
630         fig, ax = plt.subplots(figsize=(10, 4))
631         ax.text(0.5, 0.5, "(无燃油车启用)", transform=ax.transAxes,
632               ha="center", va="center", fontsize=14)
633         fig.savefig(path, dpi=160)
634         plt.close(fig)
635         return
636     n_phys_rows = len(label_to_row)
637     fig, ax = plt.subplots(figsize=(12, max(4, n_phys_rows * 0.22)))
638     ax.axvspan(policy.fuel_restrict_start, policy.fuel_restrict_end,
639               color="red", alpha=0.07,
640               label=f"燃油车限行时段 {format_policy_window(policy)}")
641     for row_idx, _label, s, e, in_g in bars:
642         color = "seagreen" if in_g else "steelblue"
643         ax.barh(row_idx + 1, e - s, left=s, height=0.7,
644                color=color, edgecolor="black", linewidth=0.2)
645     sorted_labels = sorted(label_to_row.keys(), key=lambda L: label_to_row[L])
646     ax.set_yticks(list(range(1, n_phys_rows + 1)))
647     ax.set_yticklabels(sorted_labels, fontsize=7)
648     ax.set_xlabel("时间 (小时)")
649     ax.set_xlim(7.5, 21.5)
650     ax.set_title(
651         f"问题2 燃油车服务时段甘特图 ({n_phys_rows} 辆物理燃油车; "
652         f"绿色填充=绿色区客户)"
653     )
654     ax.grid(True, axis="x", alpha=0.3)
655     ax.legend(loc="upper right")
656     fig.tight_layout()

```

```

657     fig.savefig(path, dpi=160)
658     plt.close(fig)
659 def plot_q2_algorithm_flowchart(policy: PolicyConfig, path: Path) -> None:
660     steps = [
661         (
662             "政策参数与绿区识别",
663             f"设置绿色区半径与燃油限行窗口 {format_policy_window(policy)}, 标记绿区客户",
664             "#4C72B0",
665         ),
666         (
667             "政策感知 ALNS 初始化",
668             "在 Q1 数据结构上启用 policy-aware 评价函数, 生成可修复初始解",
669             "#55A868",
670         ),
671         (
672             "带约束的 Destroy / Repair",
673             "路径重构时同步检查容量、时间窗、燃油车绿区点/弧规则",
674             "#8172B3",
675         ),
676         (
677             "违规成本与硬约束评估",
678             "evaluate_route / evaluate_solution 记录 Rule A/B 违约并拒绝不可行解",
679             "#C44E52",
680         ),
681         (
682             "能源结构自适应调整",
683             "燃油车绕行或延后服务, 绿区高约束任务转由新能源车承担",
684             "#DD8452",
685         ),
686         (
687             "多趟合并与物理车辆核算",
688             "同车型时序相容车次共享 vehicle_id, 汇总实际车辆 / 趟次 / 成本",
689             "#4A9F8B",
690         ),
691         (
692             "Q1→Q2 对比输出",
693             "输出政策前后成本、碳排放、车队结构与绿区服务合规图件",
694             "#5B6770",
695         ),
696     ]
697     plot_algorithm_flowchart(
698         "问题2 绿色区政策约束下的 ALNS 求解流程",
699         steps,
700         path,
701         footer="口径: 燃油车受 08:00-16:00 绿区规则约束; 实际车辆与车次分开统计。",
702     )
703 def plot_policy_compliance_3d(
704     problem: ProblemData, sol_routes: List[Route], policy: PolicyConfig,
705     path: Path,

```

```

706     title: str = "问题2 政策合规时空图 (XY + 到达时间)",
707 ) -> None:
708     from mpl_toolkits.mplot3d import Axes3D
709     _set_chinese_font()
710     fig = plt.figure(figsize=(15.5, 10.5))
711     ax = fig.add_subplot(111, projection="3d")
712     depot = problem.nodes[0]
713     all_customers = [c for r in sol_routes for c in r.customers]
714     freq_by_node = _service_frequency_by_virtual_node(problem, all_customers)
715     freq_cmap, freq_norm = _prepare_frequency_cmap(
716         fig, list(freq_by_node.values()),
717     )
718     z_min, z_max = 8.0, 21.0
719     for r in sol_routes:
720         if not r.customers:
721             continue
722         ev = evaluate_route(r, problem, policy=policy)
723         zs = list(ev.arrival)
724         if zs:
725             z_min = min(z_min, min(zs))
726             z_max = max(z_max, max(zs))
727     theta = np.linspace(0, 2 * np.pi, 64)
728     cx = policy.green_radius * np.cos(theta)
729     cy = policy.green_radius * np.sin(theta)
730     z_levels = np.linspace(z_min, z_max, 14)
731     Theta, Z = np.meshgrid(theta, z_levels)
732     Xc = policy.green_radius * np.cos(Theta)
733     Yc = policy.green_radius * np.sin(Theta)
734     ax.plot_surface(Xc, Yc, Z, color="green", alpha=0.06,
735                     linewidth=0, antialiased=False)
736     for z in (z_min, z_max):
737         ax.plot(cx, cy, np.full_like(cx, z), "--", color="green",
738                 alpha=0.6, linewidth=0.8)
739     if z_min <= policy.fuel_restrict_end <= z_max:
740         xx = np.linspace(min([n.x for n in problem.customer_nodes]),
741                           max([n.x for n in problem.customer_nodes]), 6)
742         yy = np.linspace(min([n.y for n in problem.customer_nodes]),
743                           max([n.y for n in problem.customer_nodes]), 6)
744         X, Y = np.meshgrid(xx, yy)
745         Zlow = np.full_like(X, max(policy.fuel_restrict_start, z_min),
746                               dtype=float)
747         Zhigh = np.full_like(X, min(policy.fuel_restrict_end, z_max),
748                                  dtype=float)
749         ax.plot_surface(X, Y, Zlow, color="red", alpha=0.05,
750                         linewidth=0, antialiased=False)
751         ax.plot_surface(X, Y, Zhigh, color="red", alpha=0.05,
752                         linewidth=0, antialiased=False)
753     fuel_handle_drawn = elec_handle_drawn = False
754     fuel_in_green_compliant_handle = None

```

```

755 fuel_in_green_violation_handle = None
756 for r in sol_routes:
757     if not r.customers:
758         continue
759     ev = evaluate_route(r, problem, policy=policy)
760     vt = problem.vehicle_types[r.vt_index]
761     xs = [depot.x] + [problem.nodes[c].x for c in r.customers] + [depot.x]
762     ys = [depot.y] + [problem.nodes[c].y for c in r.customers] + [depot.y]
763     zs = list(ev.arrival)
764     color = "tab:orange" if vt.is_fuel else "tab:blue"
765     if vt.is_fuel and not fuel_handle_drawn:
766         ax.scatter([], [], [], s=48, marker="^", facecolor="white",
767                     edgecolor=color, linewidth=1.2, label="燃油车 (三角)")
768         fuel_handle_drawn = True
769     if (not vt.is_fuel) and not elec_handle_drawn:
770         ax.scatter([], [], [], s=48, marker="o", facecolor="white",
771                     edgecolor=color, linewidth=1.2, label="新能源车 (圆点)")
772         elec_handle_drawn = True
773     for idx, c in enumerate(r.customers):
774         n = problem.nodes[c]
775         in_g = policy.in_green_zone(n.x, n.y)
776         arrive = ev.arrival[idx + 1]
777         if vt.is_fuel and in_g:
778             compliant = arrive >= policy.fuel_restrict_end - 1e-9
779             marker_color = freq_cmap(freq_norm(freq_by_node.get(c, 1)))
780             marker_edge = "black" if compliant else "red"
781             ax.scatter([n.x], [n.y], [arrive],
782                       s=44 + 12 * (freq_by_node.get(c, 1) - 1),
783                       color=marker_color, edgecolor=marker_edge,
784                       linewidth=0.8, marker="^", alpha=0.95)
785             if compliant and fuel_in_green_compliant_handle is None:
786                 fuel_in_green_compliant_handle = ax.scatter(
787                     [], [], [], s=42, color="gold",
788                     edgecolor="black", marker="^",
789                     label="燃油车进绿区(合规, 16:00; 点色=频次)"
790                 )
791             if (not compliant) and fuel_in_green_violation_handle is None:
792                 fuel_in_green_violation_handle = ax.scatter(
793                     [], [], [], s=42, color="gold",
794                     edgecolor="red", marker="^",
795                     label="燃油车进绿区(违规; 红边)"
796                 )
797         else:
798             marker_color = freq_cmap(freq_norm(freq_by_node.get(c, 1)))
799             ax.scatter(
800                 [n.x], [n.y], [arrive],
801                 s=16 + 10 * (freq_by_node.get(c, 1) - 1),
802                 color=marker_color, edgecolor=color,
803                 linewidth=0.35, marker="^" if vt.is_fuel else "o",

```



```

$04         alpha=0.85,
$05     )
$06     ax.scatter([depot.x], [depot.y], [z_min], s=160, marker="s",
$07                 color="black", label="配送中心")
$08     ax.set_xlabel("X (km)")
$09     ax.set_ylabel("Y (km)")
$10     ax.set_zlabel("到达时间 (小时)")
$11     ax.set_title(
$12         title +
$13         f" — 绿色区 r={policy.green_radius:g}km, 燃油限行 {format_policy_window(policy)}"
$14     )
$15     ax.view_init(elev=22, azim=-58)
$16     ax.legend(loc="upper left", fontsize=9)
$17     ax.set_zlabel("到达时间 (小时)", labelpad=14)
$18     ax.set_title(ax.get_title() + "; 点颜色=服务频次", pad=16)
$19     _finish_3d_figure(fig, ax, path)
$20 def plot_green_service_timeline(
$21     carrier_df: pd.DataFrame, policy: PolicyConfig, path: Path,
$22     title: str = "问题2 绿色区客户服务时间轴 (按物理车辆分行, 颜色=动力)",
$23 ) -> None:
$24     _set_chinese_font()
$25     if carrier_df.empty:
$26         fig, ax = plt.subplots(figsize=(8, 4))
$27         ax.text(0.5, 0.5, "(无绿色区服务)", ha="center", va="center")
$28         fig.savefig(path, dpi=160)
$29         plt.close(fig)
$30         return
$31     df = carrier_df.copy()
$32     df["sort_key"] = df["物理车辆编号"].astype(str)
$33     df = df.sort_values(by=["是否新能源", "sort_key", "到达小时"]).reset_index(drop=True)
$34     phys_labels = list(df["物理车辆编号"].drop_duplicates())
$35     label_to_row = {lab: i for i, lab in enumerate(phys_labels)}
$36     fig, ax = plt.subplots(figsize=(13, max(4, len(phys_labels) * 0.28)))
$37     ax.axvspan(policy.fuel_restrict_start, policy.fuel_restrict_end,
$38                 color="red", alpha=0.07,
$39                 label=f"燃油车限行 {format_policy_window(policy)}")
$40     for _, row in df.iterrows():
$41         y = label_to_row[row["物理车辆编号"]]
$42         is_elec = row["是否新能源"] == "是"
$43         color = "tab:blue" if is_elec else "tab:orange"
$44         compliant = row["是否合规"] == "合规"
$45         marker = "o" if compliant else "x"
$46         ax.scatter(row["到达小时"], y, s=80, color=color,
$47                     edgecolor="black" if compliant else color,
$48                     marker=marker, linewidth=0.7, alpha=0.9)
$49         ax.annotate(f"C{int(row['客户编号'])}",
$50                     xy=(row["到达小时"], y),
$51                     xytext=(3, 3), textcoords="offset points",
$52                     fontsize=6, alpha=0.65)

```

```

853 ax.set_yticks(list(range(len(phys_labels))))
854 ax.set_yticklabels(phys_labels, fontsize=7)
855 ax.invert_yaxis()
856 ax.set_xlabel("到达时间 (小时)")
857 ax.set_xlim(7.5, 21.5)
858 ax.grid(True, axis="x", alpha=0.3)
859 ax.set_title(title)
860 legend_handles = [
861     plt.Line2D([], [], color="tab:blue", marker="o", linestyle="None",
862                markeredgecolor="black", label="新能源车 / 合规"),
863     plt.Line2D([], [], color="tab:orange", marker="o", linestyle="None",
864                markeredgecolor="black", label="燃油车 / 合规(16:00)"),
865     plt.Line2D([], [], color="tab:orange", marker="x", linestyle="None",
866                label="燃油车 / 违规(8:00-16:00进绿区)"),
867 ]
868 ax.legend(handles=legend_handles, loc="upper right", fontsize=9)
869 fig.tight_layout()
870 fig.savefig(path, dpi=180)
871 plt.close(fig)
872 def plot_policy_cost_carbon_bridge(
873     cost_compare: pd.DataFrame, path: Path,
874     title: str = "问题1 → 问题2 成本桥接图 (政策影响分解)",
875 ) -> None:
876     _set_chinese_font()
877     cost_components = ["启动成本", "能源成本", "碳排放成本", "等待成本", "晚到惩罚"]
878     df = cost_compare.set_index("指标")
879     q1_total = float(df.loc["总成本", "问题一(无政策)"])
880     q2_total = float(df.loc["总成本", "问题二(有政策)"])
881     deltas = [float(df.loc[k, "绝对增量"]) for k in cost_components]
882     labels = ["问题一总成本"] + cost_components + ["问题二总成本"]
883     values = [q1_total] + deltas + [q2_total]
884     fig, ax = plt.subplots(figsize=(10, 6))
885     cumulative = q1_total
886     bar_colors = []
887     bar_lefts = []
888     bar_widths = []
889     bar_bottoms = []
890     bar_heights = []
891     bar_x = []
892     for i, (lab, val) in enumerate(zip(labels, values)):
893         if i == 0:
894             bar_x.append(i)
895             bar_bottoms.append(0)
896             bar_heights.append(val)
897             bar_colors.append("#4C72B0")
898             cumulative = val
899         elif i == len(labels) - 1:
900             bar_x.append(i)
901             bar_bottoms.append(0)

```

```

902         bar_heights.append(val)
903         bar_colors.append("#55A868")
904     else:
905         bar_x.append(i)
906         if val >= 0:
907             bar_bottoms.append(cumulative)
908             bar_heights.append(val)
909             bar_colors.append("#E57248")
910         else:
911             bar_bottoms.append(cumulative + val)
912             bar_heights.append(-val)
913             bar_colors.append("#4A9F8B")
914         cumulative += val
915 bars = ax.bar(bar_x, bar_heights, bottom=bar_bottoms, color=bar_colors,
916             edgecolor="black", linewidth=0.4)
917 for i, (lab, val) in enumerate(zip(labels, values)):
918     if i in (0, len(labels) - 1):
919         ax.annotate(f"{val:.0f}", (i, val),
920             xytext=(0, 4), textcoords="offset points",
921             ha="center", fontsize=10)
922     else:
923         top = bar_bottoms[i] + bar_heights[i]
924         sign_label = (
925             f"\u2212{abs(val):.0f}" if val < 0 else f"+{val:.0f}"
926         )
927         ax.annotate(
928             f"\u0394 {sign_label}",
929             (i, top),
930             xytext=(0, 6),
931             textcoords="offset points",
932             ha="center", fontsize=10,
933             color="#1a7361" if val < 0 else "#a83518"),
934             fontweight="bold",
935         )
936 for i in range(len(labels) - 1):
937     if i == 0 or i == len(labels) - 2:
938         y0 = bar_heights[i] if i == 0 else (bar_bottoms[i] + bar_heights[i]
939             if values[i] >= 0
940             else bar_bottoms[i])
941     else:
942         y0 = (bar_bottoms[i] + bar_heights[i] if values[i] >= 0
943             else bar_bottoms[i])
944     ax.hlines(y0, i + 0.4, i + 0.6, colors="gray", linewidth=0.6)
945 ax.set_xticks(bar_x)
946 ax.set_xticklabels(labels, rotation=20, ha="right", fontsize=9)
947 ax.set_ylabel("成本 (元)")
948 ax.set_title("成本桥接")
949 ax.grid(True, axis="y", alpha=0.3)
950 fig.suptitle(title, fontsize=13)

```

```

951     fig.tight_layout(rect=(0, 0.02, 1, 0.95))
952     fig.savefig(path, dpi=180)
953     plt.close(fig)
954     def write_conclusion_q2(
955         problem: ProblemData,
956         policy: PolicyConfig,
957         ev: SolutionEval,
958         type_stat: pd.DataFrame,
959         cost_compare: pd.DataFrame,
960         fleet_compare: pd.DataFrame,
961         carbon_df: pd.DataFrame,
962         carrier_df: pd.DataFrame,
963         green_count: int,
964         n_physical: int,
965         n_trips: int,
966         path: Path,
967     ) -> None:
968         service_stats = compute_green_service_stats(carrier_df, policy)
969         restrict_window = format_policy_window(policy)
970         with path.open("w", encoding="utf-8") as f:
971             f.write("# 问题 2 结果汇总\n\n")
972             f.write("## 政策约束\n\n")
973             f.write(
974                 f"- 绿色配送区：以市中心 (0,0) 为圆心，半径 {policy.green_radius:g} km\n"
975             )
976             f.write(f"- 受约束客户数：{green_count} 个绿色区客户\n")
977             f.write(
978                 f"- 点规则 (Rule A)：{restrict_window} 期间燃油车不得到达绿色配送区内客户点\n"
979             )
980             if getattr(policy, "enforce_no_cross_green", False):
981                 f.write(
982                     f"- 弧规则 (Rule B)：{restrict_window} 期间燃油车配送绿区外客户的行驶"
983                     f"路径也不得穿越绿色配送区；由于真实路网路径未知，采用"
984                     f"「直线连线不与绿区圆相交」作为保守充分条件\n\n"
985                 )
986             else:
987                 f.write("\n")
988             f.write("## 最优方案指标\n\n")
989             f.write(
990                 f"- 实际启用物理车辆数：**{n_physical} 辆** (共执行 {n_trips} 趟；"
991                 f"{n_trips - n_physical} 多趟合并节省 {n_trips - n_physical} 次启动)\n"
992             )
993             f.write(f"- 总成本：{ev.total_cost:.2f} 元\n")
994             f.write(f"- 启动成本：{ev.fixed_cost:.2f} 元 "
995                     f" (= {n_physical} 辆 × {problem.cost.fixed_cost:.0f} 元/辆)\n")
996             f.write(f"- 能源成本：{ev.energy_cost:.2f} 元\n")
997             f.write(f"- 碳排放成本：{ev.carbon_cost_money:.2f} 元 "
998                     f" (排放量 {ev.carbon_kg:.2f} kg CO2)\n")
999             f.write(f"- 等待成本：{ev.wait_cost:.2f} 元\n")

```

```

1000     f.write(f"    - 晚到惩罚: {ev.late_cost:.2f} 元\n")
1001     f.write(f"- 政策可行: **{ev.feasible_policy}**, "
1002           f"违规次数: {ev.policy_violation_count}\n\n")
1003     f.write("## 车队结构\n\n")
1004     f.write(
1005         "> 注: `实际车辆数` 为同一物理车辆按 `vehicle_id` 聚合后的数量, "
1006         "`启用趟次` 为该车型当日完成的总趟数 (一辆车可跑多趟).\n\n"
1007     )
1008     cols = list(type_stat.columns)
1009     f.write("| " + " | ".join(cols) + " |\n")
1010     f.write("| " + " | ".join(["---"] * len(cols)) + " |\n")
1011     for _, row in type_stat.iterrows():
1012         f.write("| " + " | ".join(str(row[c]) for c in cols) + " |\n")
1013     f.write("\n")
1014     f.write("## 问题一 vs 问题二 成本对比\n\n")
1015     cols = list(cost_compare.columns)
1016     f.write("| " + " | ".join(cols) + " |\n")
1017     f.write("| " + " | ".join(["---"] * len(cols)) + " |\n")
1018     for _, row in cost_compare.iterrows():
1019         f.write("| " + " | ".join(str(row[c]) for c in cols) + " |\n")
1020     f.write("\n")
1021     f.write("## 问题一 vs 问题二 车队对比 (实际车辆 / 趟次)\n\n")
1022     cols = list(fleet_compare.columns)
1023     f.write("| " + " | ".join(cols) + " |\n")
1024     f.write("| " + " | ".join(["---"] * len(cols)) + " |\n")
1025     for _, row in fleet_compare.iterrows():
1026         f.write("| " + " | ".join(str(row[c]) for c in cols) + " |\n")
1027     f.write("\n")
1028     f.write("## 碳排放核算 (按动力类型拆分)\n\n")
1029     cols = list(carbon_df.columns)
1030     f.write("| " + " | ".join(cols) + " |\n")
1031     f.write("| " + " | ".join(["---"] * len(cols)) + " |\n")
1032     for _, row in carbon_df.iterrows():
1033         f.write("| " + " | ".join(str(row[c]) for c in cols) + " |\n")
1034     f.write("\n")
1035     delta_total = cost_compare.loc[
1036         cost_compare["指标"] == "总成本", "绝对增量"
1037     ].iloc[0]
1038     delta_carbon = cost_compare.loc[
1039         cost_compare["指标"] == "碳排放量(kg)", "绝对增量"
1040     ].iloc[0]
1041     delta_phys = int(cost_compare.loc[
1042         cost_compare["指标"] == "实际车辆数(辆)", "绝对增量"
1043     ].iloc[0])
1044     delta_trips = int(cost_compare.loc[
1045         cost_compare["指标"] == "总趟次", "绝对增量"
1046     ].iloc[0])
1047     f.write("## 关键结论\n\n")
1048     f.write(

```

```

1049         f"1. 加入限行政策后总成本 **{ev.total_cost:.2f} 元**, "
1050         f"较问题一变化 **{delta_total:+.2f} 元**; 实际启用 "
1051         f"**{n_physical} 辆物理车** (Q1→Q2 实际车辆 {delta_phys:+d}, "
1052         f"趟次 {delta_trips:+d}).\n"
1053     )
1054     active = [
1055         (r['车型'], int(r['实际车辆数']), int(r['启用趟次']))
1056         for _, r in type_stat.iterrows() if int(r['实际车辆数']) > 0
1057     ]
1058     f.write(
1059         "2. 最优车队结构 (实际车辆 / 趟次): "
1060         + ", ".join(f"{n}×{p}辆({t}趟)" for n, p, t in active)
1061         + ".\n"
1062     )
1063     f.write(f"3. 碳排放总量 **{ev.carbon_kg:.2f} kg CO2**, "
1064           f"较问题一变化 **{delta_carbon:+.2f} kg**.\n")
1065     if service_stats["total_visits"] == 0:
1066         f.write("4. 本次结果中未识别到绿色区客户服务记录.\n")
1067     elif service_stats["fuel_visits"] == 0:
1068         f.write(
1069             f"4. 绿色区虚拟客户点共服务 {service_stats['total_visits']} 次, "
1070             f"全部由新能源车承担.\n"
1071         )
1072     else:
1073         f.write(
1074             f"4. 绿色区虚拟客户点共服务 {service_stats['total_visits']} 次, "
1075             f"其中新能源车承担 {service_stats['electric_visits']} 次, "
1076             f"燃油车在 {hhmm(policy.fuel_restrict_end)} 之后承担 "
1077             f"{service_stats['fuel_visits']} 次.\n"
1078         )
1079     f.write(
1080         f"5. 政策约束满足情况: **{ev.feasible_policy}**, 违规次数 "
1081         f"{ev.policy_violation_count}; 绿色区服务记录中合规访问 "
1082         f"{service_stats['compliant_visits']} 次.\n"
1083     )
1084     f.write("\n## 论文图件索引\n\n")
1085     f.write(
1086         "| 文件 | 算法/数据含义 | \n"
1087         "| --- | --- | \n"
1088         "| `q2_routes.png` | 二维路径总览 — 燃油 (橙) vs 新能源 (蓝) 车型 "
1089         "覆盖差异. | \n"
1090         "| `q2_algorithm_flowchart.png` | **问题2算法流程图** — 展示绿区识别、政策感知
ALNS、Rule A/B 合规评估、能源结构调整、多趟合并与 Q1→Q2 对比输出的完整链路. | \n"
1091         "| `q2_policy_compliance_3d.png` | **政策合规时空三维图** "
1092         "(X / Y / 到达时间). 用绿色圆柱表示绿色配送区, "
1093         f"用红色滑块标记 {format_policy_window(policy)} 限行时段; "
1094         "客户点形状/边框区分燃油/新能源, 点色按服务频次由低到高渐变; "
1095         "燃油车进入绿区的访问点用三角标记, 红边=违规, 黑边= 16:00 合规. | \n"
1096         "| `q2_green_service_timeline.png` | **绿色区客户服务时间轴**, "

```

```

1097         "按物理车辆分行，显示每次绿区访问的到达时间，颜色编码动力类型，"
1098         "标记编码合规性。直观回应 \"政策窗口外谁来跑绿区\"。 |\n"
1099         "| `q2_policy_cost_carbon_bridge.png` | **Q1→Q2 成本桥接图** "
1100         "(瀑布图)。把总成本变化逐项拆成启动 / 能源 / 碳 / 等待 / 迟到，"
1101         "解释政策约束如何重新塑造车队结构与成本组成。 |\n"
1102         "| `q2_compare_cost.png` | 问题一二分项成本柱状对比。 |\n"
1103         "| `q2_compare_fleet.png` | 问题一二车型启用对比 (实际车辆 / 趟次 双面板)。 |\n"
1104         "| `q2_compare_carbon.png` | 问题一二碳排放堆叠对比 (按动力拆分)。 |\n"
1105         "| `q2_green_zone_distribution.png` | 燃油 vs 电动 路径在绿区的分布。 |\n"
1106         "| `q2_fuel_gantt.png` | 燃油车甘特图 (按物理车辆分行)。 |\n"
1107         "| `q2_time_windows.png` | 时间窗满足情况柱状图。 |\n"
1108         "| `q2_cost_pie.png` | 成本构成饼图。 |\n"
1109     )
1110     def main(
1111         iterations: int = 3000,
1112         seed: int = 42,
1113         from_cache: bool = False,
1114     ) -> None:
1115         RESULTS_DIR.mkdir(exist_ok=True)
1116         print("[solve_q2] Loading data...")
1117         problem = load_problem()
1118         policy = PolicyConfig()
1119         green_mask = compute_green_mask(problem.nodes, policy)
1120         green_orig_ids = green_zone_customers(problem.nodes, policy)
1121         print(f"[solve_q2] Virtual nodes (incl. depot): {problem.n_nodes}")
1122         print(f"[solve_q2] Green zone virtual nodes: {int(green_mask.sum())} "
1123               f"({len(green_orig_ids)} original customers)")
1124         cache_path = RESULTS_DIR / "q2_baseline.pkl"
1125         if from_cache:
1126             if not cache_path.exists():
1127                 raise FileNotFoundError(
1128                     f"--from-cache requested but {cache_path} not found. "
1129                     f"Run without --from-cache first."
1130                 )
1131             print(f"[solve_q2] Loading cached baseline from {cache_path} ...")
1132             with cache_path.open("rb") as f:
1133                 cached = pickle.load(f)
1134             sol = _solution_from_baseline(cached)
1135             ev = evaluate_solution(sol, problem, policy=policy)
1136             print(f"[solve_q2] Cached cost re-evaluated = {ev.total_cost:.2f} "
1137                   f"(stored {cached['eval']['total_cost']:.2f})")
1138         else:
1139             print(f"[solve_q2] Running ALNS with policy "
1140                   f"({iterations} iterations, seed={seed}) ...")
1141             cfg = ALNSConfig(iterations=iterations, seed=seed,
1142                              verbose=True, log_every=400)
1143             t0 = time.time()
1144             result = run_alns(problem, cfg, policy=policy)
1145             print(f"[solve_q2] ALNS finished in {time.time()-t0:.1f}s. ")

```

```

1146         f"Best cost = {result.best_eval.total_cost:.2f}")
1147     sol = result.best_solution
1148     ev = result.best_eval
1149     if not ev.feasible_policy:
1150         print(
1151             f"[solve_q2] WARNING: Policy-infeasible solution "
1152             f"(violations={ev.policy_violation_count}). "
1153             f"Continuing to dump artefacts for debugging."
1154         )
1155     n_physical, _used_per_vt = _count_physical_vehicles(sol)
1156     n_trips = sum(1 for r in sol.routes if r.customers)
1157     print("[solve_q2] Building deliverable tables ...")
1158     green_df = build_green_customer_table(problem, policy)
1159     demand_df = build_customer_demand_table(problem)
1160     trip_df, detail_df, cost_df, type_stat_df, tw_df, phys_df = build_route_tables(
1161         problem, sol.routes
1162     )
1163     carrier_df = build_green_carrier_table(problem, sol.routes, policy)
1164     carbon_df = build_carbon_breakdown_table(problem, sol.routes, policy)
1165     q1_path = RESULTS_DIR / "q1_baseline.pkl"
1166     assert q1_path.exists(), f"Missing {q1_path}; run solve_q1.py first."
1167     with open(q1_path, "rb") as f:
1168         q1_baseline = pickle.load(f)
1169     cost_compare_df, fleet_compare_df = build_comparison_tables(
1170         q1_baseline, ev, type_stat_df, problem,
1171     )
1172     q1_fuel_kg = 0.0
1173     q1_elec_kg = 0.0
1174     for r in q1_baseline["routes"]:
1175         route = Route(vt_index=r["vt_index"], customers=list(r["customers"]))
1176         ev_r = evaluate_route(route, problem)
1177         if problem.vehicle_types[r["vt_index"]].is_fuel:
1178             q1_fuel_kg += ev_r.carbon_kg
1179         else:
1180             q1_elec_kg += ev_r.carbon_kg
1181     q2_fuel_kg = float(
1182         carbon_df.loc[carbon_df["动力类型"] == "燃油车", "碳排放(kg CO2)"].iloc[0]
1183     )
1184     q2_elec_kg = float(
1185         carbon_df.loc[carbon_df["动力类型"] == "新能源车", "碳排放(kg CO2)"].iloc[0]
1186     )
1187     with pd.ExcelWriter(RESULTS_DIR / "q2_tables.xlsx", engine="openpyxl") as xw:
1188         green_df.to_excel(xw, sheet_name="绿色配送区客户识别表", index=False)
1189         demand_df.to_excel(xw, sheet_name="客户需求汇总表", index=False)
1190         phys_df.to_excel(xw, sheet_name="物理车辆汇总表", index=False)
1191         trip_df.to_excel(xw, sheet_name="车次使用方案表", index=False)
1192         detail_df.to_excel(xw, sheet_name="路径结果表", index=False)
1193         cost_df.to_excel(xw, sheet_name="成本构成表", index=False)
1194         type_stat_df.to_excel(xw, sheet_name="车型使用结构统计表", index=False)

```



```

1195     tw_df.to_excel(xw, sheet_name="时间窗满足情况表", index=False)
1196     carrier_df.to_excel(xw, sheet_name="绿色区客户承担表", index=False)
1197     carbon_df.to_excel(xw, sheet_name="碳排放核算表", index=False)
1198     cost_compare_df.to_excel(xw, sheet_name="问题一二成本对比", index=False)
1199     fleet_compare_df.to_excel(xw, sheet_name="问题一二车队对比", index=False)
1200     print("[solve_q2] Plotting ...")
1201     plot_routes_q2(problem, sol.routes, policy, RESULTS_DIR / "q2_routes.png")
1202     plot_time_window_satisfaction(
1203         tw_df,
1204         RESULTS_DIR / "q2_time_windows.png",
1205         title="问题2 时间窗满足情况 (正值为提前, 负值为迟到)",
1206     )
1207     plot_cost_pie_q2(ev, RESULTS_DIR / "q2_cost_pie.png")
1208     plot_cost_compare(cost_compare_df, RESULTS_DIR / "q2_compare_cost.png")
1209     plot_fleet_compare(fleet_compare_df, RESULTS_DIR / "q2_compare_fleet.png")
1210     plot_carbon_compare(q1_baseline, ev, q1_fuel_kg, q1_elec_kg,
1211                        q2_fuel_kg, q2_elec_kg,
1212                        RESULTS_DIR / "q2_compare_carbon.png")
1213     plot_green_zone_distribution(problem, sol.routes, policy,
1214                                RESULTS_DIR / "q2_green_zone_distribution.png")
1215     plot_fuel_gantt(problem, sol.routes, policy,
1216                    RESULTS_DIR / "q2_fuel_gantt.png")
1217     print("[solve_q2] Plotting paper-grade figures (3D / timeline / bridge) ...")
1218     plot_policy_compliance_3d(
1219         problem, sol.routes, policy,
1220         RESULTS_DIR / "q2_policy_compliance_3d.png",
1221         title=(
1222             f"问题2 政策合规时空图 (XY + 到达时间, "
1223             f"{n_physical} 辆车 / {n_trips} 趟)"
1224         ),
1225     )
1226     plot_green_service_timeline(
1227         carrier_df, policy,
1228         RESULTS_DIR / "q2_green_service_timeline.png",
1229     )
1230     plot_policy_cost_carbon_bridge(
1231         cost_compare_df,
1232         RESULTS_DIR / "q2_policy_cost_carbon_bridge.png",
1233     )
1234     plot_q2_algorithm_flowchart(policy, RESULTS_DIR / "q2_algorithm_flowchart.png")
1235     write_conclusion_q2(
1236         problem, policy, ev, type_stat_df, cost_compare_df, fleet_compare_df,
1237         carbon_df, carrier_df, len(green_orig_ids), n_physical, n_trips,
1238         RESULTS_DIR / "q2_summary.md",
1239     )
1240     mt_savings = (n_trips - n_physical) * problem.cost.fixed_cost
1241     print(f"[solve_q2] Trips={n_trips}, physical vehicles={n_physical}, "
1242           f"multi-trip savings={mt_savings:.0f} 元")
1243     baseline = {

```

```

1244         "routes": [{"vt_index": r.vt_index,
1245                     "customers": list(r.customers),
1246                     "vehicle_id": getattr(r, "vehicle_id", None),
1247                     "arc_depart_floor": (
1248                         None if getattr(r, "arc_depart_floor", None) is None
1249                         else dict(r.arc_depart_floor)
1250                     )}
1251         for r in sol.routes if r.customers],
1252     "eval": {
1253         "total_cost": ev.total_cost,
1254         "fixed_cost": ev.fixed_cost,
1255         "energy_cost": ev.energy_cost,
1256         "carbon_cost_money": ev.carbon_cost_money,
1257         "carbon_kg": ev.carbon_kg,
1258         "wait_cost": ev.wait_cost,
1259         "late_cost": ev.late_cost,
1260         "n_routes": ev.n_routes,
1261         "n_trips": n_trips,
1262         "n_physical_vehicles": n_physical,
1263         "feasible_policy": ev.feasible_policy,
1264         "policy_violation_count": ev.policy_violation_count,
1265     },
1266     "n_virtual_nodes": problem.n_nodes,
1267     "split_map": problem.split_map,
1268     "policy": {
1269         "green_radius": policy.green_radius,
1270         "fuel_restrict_start": policy.fuel_restrict_start,
1271         "fuel_restrict_end": policy.fuel_restrict_end,
1272         "policy_id": policy.policy_id,
1273     },
1274     "green_orig_ids": green_orig_ids,
1275 }
1276 with cache_path.open("wb") as f:
1277     pickle.dump(baseline, f)
1278 print(f"\n[solve_q2] Done. Results saved to {RESULTS_DIR}/")
1279 print(f"  Total cost: {ev.total_cost:.2f} 元")
1280 print(f"  Trips: {n_trips} / Physical vehicles: {n_physical}")
1281 print(f"  Carbon emitted: {ev.carbon_kg:.2f} kg CO2")
1282 print(f"  Policy feasible: {ev.feasible_policy} "
1283       f"(violations={ev.policy_violation_count})")
1284 print(f"  ΔC vs Q1: {ev.total_cost - q1_baseline['eval']['total_cost']:.2f} 元")
1285 print(f"  ΔCO2 vs Q1: {ev.carbon_kg - q1_baseline['eval']['carbon_kg']:.2f} kg")
1286 if __name__ == "__main__":
1287     parser = argparse.ArgumentParser()
1288     parser.add_argument("--iters", type=int, default=3000)
1289     parser.add_argument("--seed", type=int, default=42)
1290     parser.add_argument(
1291         "--from-cache", action="store_true",
1292         help="Skip ALNS and rebuild tables/plots/summary from "

```

```

1293         "results/q2_baseline.pkl. Use after a code-only fix to the "
1294         "reporting layer to avoid re-running the search.",
1295     )
1296     args = parser.parse_args()
1297     main(iterations=args.iters, seed=args.seed, from_cache=args.from_cache)

```

B.4 问题三代码

dynamic_solver.py

代码 8: 问题三代码: dynamic_solver.py

```

1  from __future__ import annotations
2  import time
3  from copy import deepcopy
4  from dataclasses import dataclass, field
5  from typing import Dict, List, Optional, Set, Tuple
6  import numpy as np
7  from alns_solver import _ROUTE_COST_CACHE, _route_cost
8  from cost_model import (
9      HANDOVER_TIME,
10     Route,
11     Solution,
12     SolutionEval,
13     evaluate_route,
14     evaluate_solution,
15     evaluate_route_partial,
16     first_violating_arc_index,
17     _optimal_depart_time,
18 )
19 from data_loader import (
20     Node,
21     ProblemData,
22     clone_problem,
23     mutate_problem_inplace,
24 )
25 from multitrip_ops import consolidate_multitrip_inplace
26 from policy import PolicyConfig, is_arc_violation, is_violation
27 EVENT_CANCEL = "cancel"
28 EVENT_NEW = "new"
29 EVENT_ADDRESS = "address"
30 EVENT_TIMEWINDOW = "timewindow"
31 @dataclass
32 class Event:
33     event_id: str
34     kind: str
35     trigger_time: float
36     description: str = ""
37     orig_id: Optional[int] = None

```

```

38     new_orig_id: Optional[int] = None
39     x: Optional[float] = None
40     y: Optional[float] = None
41     demand_w: Optional[float] = None
42     demand_v: Optional[float] = None
43     tw_start: Optional[float] = None
44     tw_end: Optional[float] = None
45     new_x: Optional[float] = None
46     new_y: Optional[float] = None
47     new_tw_start: Optional[float] = None
48     new_tw_end: Optional[float] = None
49 @dataclass
50 class RouteFreeze:
51     route_idx: int
52     vt_index: int
53     frozen_len: int
54     depart_time: float
55     customers_frozen: List[int]
56     customers_flexible: List[int]
57     is_departed: bool
58 @dataclass
59 class Snapshot:
60     label: str
61     t_now: float
62     routes: List[Dict]
63     freezes: List[RouteFreeze]
64     eval_before: Optional[SolutionEval]
65     eval_after: Optional[SolutionEval]
66     event: Optional[Event]
67     response_ms: float
68     cancelled_wasted: List[int]
69     changed_route_indices: List[int]
70     note: str = ""
71 def compute_freezes(
72     routes: List[Route],
73     problem: ProblemData,
74     policy: Optional[PolicyConfig],
75     t_now: float,
76 ) -> List[RouteFreeze]:
77     out: List[RouteFreeze] = []
78     for idx, r in enumerate(routes):
79         if not r.customers:
80             out.append(RouteFreeze(
81                 route_idx=idx, vt_index=r.vt_index,
82                 frozen_len=0, depart_time=problem.cost.depart_time,
83                 customers_frozen=[], customers_flexible=[],
84                 is_departed=False,
85             ))
86     continue

```

```

87     ev = evaluate_route(r, problem, policy=policy)
88     depart_time = ev.arrival[0]
89     flen = 0
90     for i in range(len(r.customers)):
91         if ev.service_end[i] <= t_now + 1e-9:
92             flen = i + 1
93         else:
94             break
95     out.append(RouteFreeze(
96         route_idx=idx, vt_index=r.vt_index,
97         frozen_len=flen,
98         depart_time=depart_time,
99         customers_frozen=list(r.customers[:flen]),
100         customers_flexible=list(r.customers[flen:]),
101         is_departed=(depart_time <= t_now + 1e-9),
102     ))
103     return out
104 def _clear_cache() -> None:
105     _ROUTE_COST_CACHE.clear()
106     _INFEAS = 1e12
107 def _route_cost_dyn(
108     route: Route,
109     problem: ProblemData,
110     policy: Optional[PolicyConfig],
111     t_now: float,
112 ) -> float:
113     if not route.customers:
114         return _route_cost(route, problem, policy=policy)
115     depart = _optimal_depart_time(route, problem, problem.cost.depart_time)
116     if depart < t_now - 1e-9:
117         return _INFEAS
118     return _route_cost(route, problem, policy=policy)
119 def _best_flex_insertion_in_route(
120     node_id: int,
121     route: Route,
122     frozen_len: int,
123     problem: ProblemData,
124     current_cost: float,
125     policy: Optional[PolicyConfig],
126     t_now: float,
127     allow_capacity_check: bool = True,
128 ) -> Optional[Tuple[float, int]]:
129     vt = problem.vehicle_types[route.vt_index]
130     node = problem.nodes[node_id]
131     if allow_capacity_check:
132         cur_w = sum(problem.nodes[c].demand_w for c in route.customers)
133         cur_v = sum(problem.nodes[c].demand_v for c in route.customers)
134         if cur_w + node.demand_w > vt.capacity_w + 1e-6:
135             return None

```

```

136         if cur_v + node.demand_v > vt.capacity_v + 1e-6:
137             return None
138     best: Optional[Tuple[float, int]] = None
139     for p in range(frozen_len, len(route.customers) + 1):
140         new_customers = route.customers[:p] + [node_id] + route.customers[p:]
141         new_route = Route(route.vt_index, new_customers)
142         new_cost = _route_cost_dyn(new_route, problem, policy, t_now)
143         if new_cost >= _INFEAS:
144             continue
145         delta = new_cost - current_cost
146         if best is None or delta < best[0]:
147             best = (delta, p)
148     return best
149 def _best_flex_insertion(
150     node_id: int,
151     routes: List[Route],
152     freezes: List[RouteFreeze],
153     route_costs: List[float],
154     used_per_vt: Dict[int, int],
155     problem: ProblemData,
156     policy: Optional[PolicyConfig],
157     t_now: float,
158     *,
159     require_not_departed: bool,
160     allowed_routes: Optional[Set[int]] = None,
161 ) -> Optional[Tuple[float, int, int]]:
162     best: Optional[Tuple[float, int, int]] = None
163     for i, (r, f) in enumerate(zip(routes, freezes)):
164         if not r.customers:
165             continue
166         if require_not_departed and f.is_departed:
167             continue
168         if allowed_routes is not None and i not in allowed_routes:
169             continue
170         res = _best_flex_insertion_in_route(
171             node_id, r, f.frozen_len, problem, route_costs[i], policy, t_now,
172         )
173         if res is None:
174             continue
175         delta, pos = res
176         if best is None or delta < best[0]:
177             best = (delta, i, pos)
178     if allowed_routes is None:
179         for vt_idx, vt in enumerate(problem.vehicle_types):
180             if used_per_vt.get(vt_idx, 0) >= vt.count:
181                 continue
182             node = problem.nodes[node_id]
183             if node.demand_w > vt.capacity_w + 1e-6:
184                 continue

```

```

185         if node.demand_v > vt.capacity_v + 1e-6:
186             continue
187         cand = Route(vt_idx, [node_id])
188         cost = _route_cost_dyn(cand, problem, policy, t_now)
189         if cost >= _INFEAS:
190             continue
191         if best is None or cost < best[0]:
192             best = (cost, -(vt_idx + 1), 0)
193     return best
194 def _apply_insertion(
195     routes: List[Route], freezes: List[RouteFreeze],
196     node_id: int, route_enc: int, position: int,
197 ) -> int:
198     if route_enc >= 0:
199         routes[route_enc].customers.insert(position, node_id)
200         return route_enc
201     vt_idx = -route_enc - 1
202     routes.append(Route(vt_idx, [node_id]))
203     freezes.append(RouteFreeze(
204         route_idx=len(routes) - 1, vt_index=vt_idx,
205         frozen_len=0, depart_time=0.0,
206         customers_frozen=[], customers_flexible=[node_id],
207         is_departed=False,
208     ))
209     return len(routes) - 1
210 def _used_per_vt(routes: List[Route]) -> Dict[int, int]:
211     counts: Dict[int, int] = {}
212     seen_ids: set = set()
213     for r in routes:
214         if not r.customers:
215             continue
216         vid = getattr(r, "vehicle_id", None)
217         if vid is None:
218             counts[r.vt_index] = counts.get(r.vt_index, 0) + 1
219         else:
220             if vid not in seen_ids:
221                 seen_ids.add(vid)
222                 counts[r.vt_index] = counts.get(r.vt_index, 0) + 1
223     return counts
224 def _next_vehicle_id(routes: List[Route]) -> int:
225     max_id = -1
226     for r in routes:
227         vid = getattr(r, "vehicle_id", None)
228         if vid is not None and vid > max_id:
229             max_id = vid
230     return max_id + 1
231 def _assign_vehicle_id_to_route(r: Route, vid: int) -> None:
232     r.vehicle_id = vid
233 def _route_costs(

```

```

234     routes: List[Route], problem: ProblemData, policy: Optional[PolicyConfig],
235     t_now: float = 0.0,
236 ) -> List[float]:
237     return [_route_cost_dyn(r, problem, policy, t_now) for r in routes]
238 def _two_opt_flex(
239     route: Route,
240     frozen_len: int,
241     problem: ProblemData,
242     policy: Optional[PolicyConfig],
243     t_now: float,
244 ) -> Route:
245     best = route
246     best_cost = _route_cost_dyn(best, problem, policy, t_now)
247     improved = True
248     while improved:
249         improved = False
250         cs = best.customers
251         n = len(cs)
252         lo = frozen_len
253         if n - lo < 3:
254             break
255         for i in range(lo, n - 1):
256             for j in range(i + 1, n):
257                 new_cs = cs[:i] + cs[i:j + 1][::-1] + cs[j + 1:]
258                 nr = Route(best.vt_index, new_cs)
259                 nc = _route_cost_dyn(nr, problem, policy, t_now)
260                 if nc < best_cost - 1e-6:
261                     best = nr
262                     best_cost = nc
263                     improved = True
264                     break
265             if improved:
266                 break
267     return best
268 def _relocate_flex(
269     routes: List[Route],
270     freezes: List[RouteFreeze],
271     problem: ProblemData,
272     policy: Optional[PolicyConfig],
273     t_now: float,
274     max_iters: int = 4,
275 ) -> List[Route]:
276     for _ in range(max_iters):
277         improved = False
278         for ri in range(len(routes)):
279             r = routes[ri]
280             f = freezes[ri]
281             if len(r.customers) <= f.frozen_len:
282                 continue

```



```

283         if f.is_departed:
284             continue
285     base_cost = _route_cost_dyn(r, problem, policy, t_now)
286     if base_cost >= _INFEAS:
287         continue
288     for pi in range(f.frozen_len, len(r.customers)):
289         c = r.customers[pi]
290         new_cs = r.customers[:pi] + r.customers[pi + 1:]
291         nr = Route(r.vt_index, new_cs)
292         reduced_cost = _route_cost_dyn(nr, problem, policy, t_now)
293         if reduced_cost >= _INFEAS:
294             continue
295         gain_rm = base_cost - reduced_cost
296         old_cust = r.customers
297         r.customers = new_cs
298         used = _used_per_vt(routes)
299         rcosts = _route_costs(routes, problem, policy, t_now)
300         allowed = {i for i, ff in enumerate(freezes) if not ff.is_departed}
301         allowed.add(ri)
302         ins = _best_flex_insertion(
303             c, routes, freezes, rcosts, used, problem, policy, t_now,
304             require_not_departed=False,
305             allowed_routes=allowed,
306         )
307         if ins is not None:
308             d_ins, r_enc, pos = ins
309             if r_enc >= 0 and d_ins < gain_rm - 1e-6:
310                 _apply_insertion(routes, freezes, c, r_enc, pos)
311                 improved = True
312                 break
313             r.customers = old_cust
314         if improved:
315             break
316     if not improved:
317         break
318     return routes
319 @dataclass
320 class DynamicScheduler:
321     problem: ProblemData
322     routes: List[Route]
323     policy: PolicyConfig
324     t_now: float = 0.0
325     snapshots: List[Snapshot] = field(default_factory=list)
326     cancelled_wasted: List[int] = field(default_factory=list)
327     cancelled_dropped: List[int] = field(default_factory=list)
328     arc_delay_count: int = 0
329     arc_delay_events: int = 0
330     emergency_split_count: int = 0
331     def _current_freezes(self) -> List[RouteFreeze]:

```

```

332     return compute_freezes(self.routes, self.problem, self.policy, self.t_now)
333 def evaluate(self, respect_history: bool = True) -> SolutionEval:
334     sol = Solution(routes=[r.copy() for r in self.routes])
335     if respect_history:
336         freezes = self._current_freezes()
337         flens = [f.frozen_len for f in freezes]
338     else:
339         flens = None
340     return evaluate_solution(sol, self.problem, policy=self.policy,
341                             frozen_lens=flens)
342 def snapshot(
343     self,
344     label: str,
345     event: Optional[Event],
346     eval_before: Optional[SolutionEval],
347     response_ms: float,
348     changed_route_indices: Optional[List[int]] = None,
349     note: str = "",
350 ) -> Snapshot:
351     freezes = self._current_freezes()
352     eval_after = self.evaluate()
353     snap = Snapshot(
354         label=label,
355         t_now=self.t_now,
356         routes=[{"vt_index": r.vt_index, "customers": list(r.customers),
357                 "vehicle_id": getattr(r, "vehicle_id", None),
358                 "arc_depart_floor": (
359                     None if getattr(r, "arc_depart_floor", None) is None
360                     else dict(r.arc_depart_floor)
361                 )}
362                 for r in self.routes],
363         freezes=freezes,
364         eval_before=eval_before,
365         eval_after=eval_after,
366         event=event,
367         response_ms=response_ms,
368         cancelled_wasted=list(self.cancelled_wasted),
369         changed_route_indices=changed_route_indices or [],
370         note=note,
371     )
372     self.snapshots.append(snap)
373     return snap
374 def apply_event(self, event: Event) -> Snapshot:
375     t0 = time.time()
376     if event.trigger_time > self.t_now:
377         self.t_now = event.trigger_time
378     ev_before = self.evaluate()
379     if event.kind == EVENT_CANCEL:
380         changed, note = self._handle_cancel(event)

```

```

381     elif event.kind == EVENT_NEW:
382         changed, note = self._handle_new(event)
383     elif event.kind == EVENT_ADDRESS:
384         changed, note = self._handle_address(event)
385     elif event.kind == EVENT_TIMEWINDOW:
386         changed, note = self._handle_timewindow(event)
387     else:
388         raise ValueError(f"Unknown event kind: {event.kind}")
389     self._local_repair(changed)
390     self.routes = [r for r in self.routes if r.customers]
391     post_ev = self.evaluate(respect_history=True)
392     if not post_ev.feasible_policy:
393         raise RuntimeError(
394             "Zero-violation contract breached while handling "
395             f"{event.event_id} (kind={event.kind}): residual "
396             f"policy_violation_count={post_ev.policy_violation_count}. "
397             "Delay / emergency-split rescue could not absorb the conflict."
398         )
399     response_ms = (time.time() - t0) * 1000.0
400     return self.snapshot(
401         label=event.event_id,
402         event=event,
403         eval_before=ev_before,
404         response_ms=response_ms,
405         changed_route_indices=list(set(changed)),
406         note=note,
407     )
408 def _handle_cancel(self, event: Event) -> Tuple[List[int], str]:
409     assert event.orig_id is not None
410     virt_ids = list(self.problem.split_map.get(event.orig_id, []))
411     if not virt_ids:
412         return [], f"Cancel of customer {event.orig_id}: not found."
413     freezes = self._current_freezes()
414     changed: List[int] = []
415     wasted: List[int] = []
416     dropped: List[int] = []
417     virt_set = set(virt_ids)
418     for i, (r, f) in enumerate(zip(self.routes, freezes)):
419         if not r.customers:
420             continue
421         new_cust: List[int] = []
422         for j, c in enumerate(r.customers):
423             if c in virt_set:
424                 if j < f.frozen_len:
425                     wasted.append(c)
426                 else:
427                     dropped.append(c)
428                 continue
429             new_cust.append(c)

```

```

430         if len(new_cust) != len(r.customers):
431             r.customers = new_cust
432             changed.append(i)
433     self.cancelled_wasted.extend(wasted)
434     self.cancelled_dropped.extend(dropped)
435     note = (
436         f"Cancel orig {event.orig_id}: dropped {len(dropped)} flexible "
437         f"virt(s), {len(wasted)} already committed."
438     )
439     return changed, note
440 def _handle_new(self, event: Event) -> Tuple[List[int], str]:
441     assert event.new_orig_id is not None
442     assert event.x is not None and event.y is not None
443     assert event.demand_w is not None and event.demand_v is not None
444     ts = event.tw_start if event.tw_start is not None else self.problem.cost.
depart_time
445     te = event.tw_end if event.tw_end is not None else self.problem.cost.return_time
446     new_ids = mutate_problem_inplace(
447         self.problem,
448         add_customers=[{
449             "orig_id": event.new_orig_id,
450             "x": event.x,
451             "y": event.y,
452             "demand_w": event.demand_w,
453             "demand_v": event.demand_v,
454             "tw_start": ts,
455             "tw_end": te,
456             "service": self.problem.cost.service_time,
457         }],
458     )
459     _clear_cache()
460     freezes = self._current_freezes()
461     rcosts = _route_costs(self.routes, self.problem, self.policy, self.t_now)
462     used = _used_per_vt(self.routes)
463     changed: List[int] = []
464     inserted_virts: List[int] = []
465     failed_virts: List[int] = []
466     for nid in new_ids:
467         ins = _best_flex_insertion(
468             nid, self.routes, freezes, rcosts, used,
469             self.problem, self.policy, self.t_now,
470             require_not_departed=True,
471         )
472         if ins is None:
473             failed_virts.append(nid)
474             continue
475         _, r_enc, pos = ins
476         new_ri = _apply_insertion(self.routes, freezes, nid, r_enc, pos)
477         changed.append(new_ri)

```

```

478         inserted_virts.append(nid)
479         if r_enc >= 0:
480             rcosts[r_enc] = _route_cost_dyn(self.routes[r_enc], self.problem, self.
policy, self.t_now)
481         else:
482             rcosts.append(_route_cost_dyn(self.routes[-1], self.problem, self.policy,
self.t_now))
483             used[self.routes[-1].vt_index] = used.get(self.routes[-1].vt_index, 0) + 1
484             freezes[-1] = compute_freezes(
485                 [self.routes[-1]], self.problem, self.policy, self.t_now,
486             )[0]
487             freezes[-1].route_idx = len(self.routes) - 1
488         note = (
489             f"New orig {event.new_orig_id}: inserted {len(inserted_virts)} "
490             f"virt(s), {len(failed_virts)} failed."
491         )
492         return changed, note
493     def _handle_address(self, event: Event) -> Tuple[List[int], str]:
494         assert event.orig_id is not None
495         assert event.new_x is not None and event.new_y is not None
496         virt_ids = list(self.problem.split_map.get(event.orig_id, []))
497         if not virt_ids:
498             return [], f"Address change of customer {event.orig_id}: not found."
499         virt_set = set(virt_ids)
500         freezes = self._current_freezes()
501         flexible_virts: List[int] = []
502         frozen_virts: set = set()
503         for i, (r, f) in enumerate(zip(self.routes, freezes)):
504             for j, c in enumerate(r.customers):
505                 if c not in virt_set:
506                     continue
507                 if j < f.frozen_len:
508                     frozen_virts.add(c)
509                 else:
510                     flexible_virts.append(c)
511         virts_to_update = [c for c in virt_ids if c not in frozen_virts]
512         mutate_problem_inplace(
513             self.problem,
514             mutate_coords={c: (event.new_x, event.new_y) for c in virts_to_update},
515         )
516         _clear_cache()
517         changed: List[int] = []
518         origin_of: Dict[int, int] = {}
519         for i, r in enumerate(self.routes):
520             new_cust = []
521             for c in r.customers:
522                 if c in flexible_virts:
523                     origin_of[c] = i
524                 else:

```

```

525         new_cust.append(c)
526     if len(new_cust) != len(r.customers):
527         r.customers = new_cust
528         changed.append(i)
529     freezes = self._current_freezes()
530     rcosts = _route_costs(self.routes, self.problem, self.policy, self.t_now)
531     used = _used_per_vt(self.routes)
532     leftovers: List[int] = []
533     for c in flexible_virts:
534         origin = origin_of.get(c, -1)
535         origin_departed = (
536             0 <= origin < len(freezes) and freezes[origin].is_departed
537         )
538         if origin_departed:
539             ins = _best_flex_insertion(
540                 c, self.routes, freezes, rcosts, used,
541                 self.problem, self.policy, self.t_now,
542                 require_not_departed=False,
543                 allowed_routes={origin},
544             )
545         else:
546             ins = _best_flex_insertion(
547                 c, self.routes, freezes, rcosts, used,
548                 self.problem, self.policy, self.t_now,
549                 require_not_departed=True,
550             )
551         if ins is None:
552             leftovers.append(c)
553             continue
554         _, r_enc, pos = ins
555         ri = _apply_insertion(self.routes, freezes, c, r_enc, pos)
556         changed.append(ri)
557         if r_enc >= 0:
558             rcosts[r_enc] = _route_cost_dyn(self.routes[r_enc], self.problem, self.
policy, self.t_now)
559         else:
560             rcosts.append(_route_cost_dyn(self.routes[-1], self.problem, self.policy,
self.t_now))
561         used[self.routes[-1].vt_index] = used.get(self.routes[-1].vt_index, 0) + 1
562     if leftovers:
563         for c in leftovers:
564             origin = origin_of.get(c, 0)
565             if 0 <= origin < len(self.routes):
566                 flen = freezes[origin].frozen_len if origin < len(freezes) else 0
567                 self.routes[origin].customers.insert(flen, c)
568                 changed.append(origin)
569     note = (
570         f"Address orig {event.orig_id} -> ({event.new_x:.2f},"
571         f"{event.new_y:.2f}): rerouted {len(flexible_virts)} virt(s)"

```

```

572         + (f", {len(leftovers)} fallback" if leftovers else "")
573         + "."
574     )
575     return changed, note
576 def _handle_timewindow(self, event: Event) -> Tuple[List[int], str]:
577     assert event.orig_id is not None
578     assert event.new_tw_start is not None and event.new_tw_end is not None
579     virt_ids = list(self.problem.split_map.get(event.orig_id, []))
580     if not virt_ids:
581         return [], f"TW change of customer {event.orig_id}: not found."
582     mutate_problem_inplace(
583         self.problem,
584         mutate_tw={c: (event.new_tw_start, event.new_tw_end) for c in virt_ids},
585     )
586     _clear_cache()
587     freezes = self._current_freezes()
588     changed: List[int] = []
589     moved_virts: List[int] = []
590     LATE_THRESHOLD = 0.15
591     for i, (r, f) in enumerate(zip(self.routes, freezes)):
592         if not r.customers:
593             continue
594         if not any(c in virt_ids for c in r.customers):
595             continue
596         ev = evaluate_route(r, self.problem, policy=self.policy)
597         new_cust = list(r.customers)
598         pulled: List[int] = []
599         for j, c in enumerate(r.customers):
600             if c in virt_ids and j >= f.frozen_len and ev.late[j] > LATE_THRESHOLD:
601                 pulled.append(c)
602         for c in pulled:
603             if c in new_cust:
604                 new_cust.remove(c)
605                 moved_virts.append(c)
606         if pulled:
607             r.customers = new_cust
608             changed.append(i)
609     if moved_virts:
610         freezes = self._current_freezes()
611         rcosts = _route_costs(self.routes, self.problem, self.policy, self.t_now)
612         used = _used_per_vt(self.routes)
613         for c in moved_virts:
614             ins = _best_flex_insertion(
615                 c, self.routes, freezes, rcosts, used,
616                 self.problem, self.policy, self.t_now,
617                 require_not_departed=True,
618             )
619             if ins is None:
620                 continue

```

```

621         _, r_enc, pos = ins
622         ri = _apply_insertion(self.routes, freezes, c, r_enc, pos)
623         changed.append(ri)
624         if r_enc >= 0:
625             rcosts[r_enc] = _route_cost_dyn(self.routes[r_enc], self.problem, self.
policy, self.t_now)
626         else:
627             rcosts.append(_route_cost_dyn(self.routes[-1], self.problem, self.
policy, self.t_now))
628             used[self.routes[-1].vt_index] = used.get(self.routes[-1].vt_index, 0)
+ 1
629     note = (
630         f"TW orig {event.orig_id} -> [{event.new_tw_start:.2f}],"
631         f"{event.new_tw_end:.2f}]: relocated {len(moved_virts)} virt(s)."
632     )
633     return changed, note
634 def _local_repair(self, touched_route_idx: List[int]) -> None:
635     if not self.routes:
636         return
637     freezes = self._current_freezes()
638     touched = set(touched_route_idx)
639     for i in touched:
640         if i < 0 or i >= len(self.routes):
641             continue
642         r = self.routes[i]
643         if not r.customers:
644             continue
645         new_r = _two_opt_flex(
646             r, freezes[i].frozen_len, self.problem, self.policy, self.t_now,
647         )
648         r.customers = new_r.customers
649         freezes = self._current_freezes()
650         _relocate_flex(
651             self.routes, freezes, self.problem, self.policy, self.t_now, max_iters=2,
652         )
653         self._policy_repair()
654         self._opportunistic_multitrip()
655 def _try_swap_to_electric(self) -> bool:
656     made = False
657     used = _used_per_vt(self.routes)
658     for ri in range(len(self.routes)):
659         r = self.routes[ri]
660         if not r.customers:
661             continue
662         ev = evaluate_route(r, self.problem, policy=self.policy)
663         if ev.feasible_policy:
664             continue
665         vt_cur = self.problem.vehicle_types[r.vt_index]
666         if not vt_cur.is_fuel:

```



```

667         continue
668     freezes = compute_freezes(self.routes, self.problem, self.policy, self.t_now)
669     if freezes[ri].is_departed:
670         continue
671     w = sum(self.problem.nodes[c].demand_w for c in r.customers)
672     v = sum(self.problem.nodes[c].demand_v for c in r.customers)
673     base_cost = _route_cost_dyn(r, self.problem, self.policy, self.t_now)
674     best_swap: Optional[Tuple[float, int]] = None
675     for vti, vt_alt in enumerate(self.problem.vehicle_types):
676         if vt_alt.is_fuel:
677             continue
678         if used.get(vti, 0) >= vt_alt.count:
679             continue
680         if w > vt_alt.capacity_w + 1e-6 or v > vt_alt.capacity_v + 1e-6:
681             continue
682         test = Route(vti, list(r.customers))
683         cost_alt = _route_cost_dyn(test, self.problem, self.policy, self.t_now)
684         if cost_alt >= _INFEAS:
685             continue
686         if best_swap is None or cost_alt < best_swap[0]:
687             best_swap = (cost_alt, vti)
688     if best_swap is None:
689         continue
690     cost_alt, vti = best_swap
691     if cost_alt < base_cost - 1e-6:
692         used[r.vt_index] = used.get(r.vt_index, 0) - 1
693         r.vt_index = vti
694         used[vti] = used.get(vti, 0) + 1
695         made = True
696     return made
697 def _route_has_violation_in_flex(
698     self, ri: int, f: RouteFreeze,
699 ) -> Tuple[bool, int]:
700     r = self.routes[ri]
701     vt = self.problem.vehicle_types[r.vt_index]
702     if not vt.is_fuel or not r.customers:
703         return False, -1
704     ev = evaluate_route(r, self.problem, policy=self.policy)
705     if ev.feasible_policy:
706         return False, -1
707     nodes = self.problem.nodes
708     policy = self.policy
709     seq = [0] + list(r.customers) + [0]
710     for idx, c in enumerate(r.customers):
711         if idx < f.frozen_len:
712             continue
713         n = nodes[c]
714         is_green = policy.in_green_zone(n.x, n.y) and n.orig_id != 0
715         if is_green and is_violation(ev.arrival[idx + 1], vt, True, policy):

```

```

716         return True, idx
717     for i in range(len(seq) - 1):
718         if i < f.frozen_len:
719             continue
720         u, v = seq[i], seq[i + 1]
721         nu, nv = nodes[u], nodes[v]
722         src_in = (u != 0) and policy.in_green_zone(nu.x, nu.y)
723         dst_in = (v != 0) and policy.in_green_zone(nv.x, nv.y)
724         if is_arc_violation(
725             nu.x, nu.y, nv.x, nv.y,
726             ev.service_end[i], ev.arrival[i + 1],
727             vt, src_in, dst_in, policy,
728         ):
729             return True, max(i, f.frozen_len)
730     return False, -1
731 def _handover_to_depot(self) -> bool:
732     made = False
733     elec_types = [
734         (vti, vt) for vti, vt in enumerate(self.problem.vehicle_types)
735         if not vt.is_fuel
736     ]
737     if not elec_types:
738         return False
739     iterated = 0
740     while iterated < 8:
741         iterated += 1
742         freezes = self._current_freezes()
743         target_ri = -1
744         target_pivot = -1
745         for ri in range(len(self.routes)):
746             r = self.routes[ri]
747             if not r.customers:
748                 continue
749             vt = self.problem.vehicle_types[r.vt_index]
750             if not vt.is_fuel:
751                 continue
752             has_v, pivot = self._route_has_violation_in_flex(ri, freezes[ri])
753             if has_v and pivot >= freezes[ri].frozen_len:
754                 target_ri = ri
755                 target_pivot = pivot
756                 break
757         if target_ri < 0:
758             break
759         r = self.routes[target_ri]
760         pivot = target_pivot
761         head_custs = list(r.customers[:pivot])
762         tail_custs = list(r.customers[pivot:])
763         if not tail_custs:
764             break

```

```

765     head_route = Route(r.vt_index, head_custs)
766     ev_head = evaluate_route(head_route, self.problem, policy=self.policy)
767     if not ev_head.feasible_policy:
768         found_better = False
769         for p_try in range(pivot + 1, len(r.customers)):
770             cand = Route(r.vt_index, r.customers[:p_try])
771             ev_c = evaluate_route(cand, self.problem, policy=self.policy)
772             if ev_c.feasible_policy:
773                 head_custs = r.customers[:p_try]
774                 tail_custs = r.customers[p_try:]
775                 head_route = cand
776                 ev_head = ev_c
777                 found_better = True
778                 break
779         if not found_better:
780             break
781     if not tail_custs:
782         break
783     w = sum(self.problem.nodes[c].demand_w for c in tail_custs)
784     v = sum(self.problem.nodes[c].demand_v for c in tail_custs)
785     used = _used_per_vt(self.routes)
786     best_elec: Optional[Tuple[float, int]] = None
787     for vti, vt_e in elec_types:
788         if used.get(vti, 0) >= vt_e.count:
789             continue
790         if w > vt_e.capacity_w + 1e-6 or v > vt_e.capacity_v + 1e-6:
791             continue
792         new_r = Route(vti, list(tail_custs))
793         cost_new = _route_cost_dyn(
794             new_r, self.problem, self.policy, self.t_now,
795         )
796         if cost_new >= _INFEAS:
797             continue
798         ev_new = evaluate_route(new_r, self.problem, policy=self.policy)
799         if not (ev_new.feasible_cap and ev_new.feasible_return
800                 and ev_new.feasible_policy):
801             continue
802         if best_elec is None or cost_new < best_elec[0]:
803             best_elec = (cost_new, vti)
804     if best_elec is None:
805         break
806     r.customers = head_custs
807     if not head_custs:
808         r.customers = []
809     new_tail = Route(best_elec[1], list(tail_custs))
810     _assign_vehicle_id_to_route(new_tail, _next_vehicle_id(self.routes))
811     self.routes.append(new_tail)
812     _clear_cache()
813     made = True

```

```

$14         return made
$15     def _opportunistic_multitrip(self) -> int:
$16         freezes = self._current_freezes()
$17         merges = consolidate_multitrip_inplace(
$18             self.routes, self.problem, self.policy, freezes=freezes,
$19         )
$20         if merges:
$21             _clear_cache()
$22         return merges
$23     def _opportunistic_multitrip_legacy(self) -> int:
$24         freezes = self._current_freezes()
$25         n = len(self.routes)
$26         evs: List[Optional] = [None] * n
$27         for i, r in enumerate(self.routes):
$28             if r.customers:
$29                 evs[i] = evaluate_route(r, self.problem, policy=self.policy)
$30         ret_times: List[Optional[float]] = [
$31             evs[i].arrival[-1] if evs[i] and evs[i].arrival else None
$32             for i in range(n)
$33         ]
$34         dep_times: List[Optional[float]] = [
$35             evs[i].arrival[0] if evs[i] and evs[i].arrival else None
$36             for i in range(n)
$37         ]
$38         groups_pre: Dict[int, List[int]] = {}
$39         for i, r in enumerate(self.routes):
$40             if not r.customers:
$41                 continue
$42             vid = getattr(r, "vehicle_id", None)
$43             if vid is None:
$44                 continue
$45             groups_pre.setdefault(vid, []).append(i)
$46         for vid, ris in groups_pre.items():
$47             if len(ris) < 2:
$48                 continue
$49             pairs = sorted(
$50                 [(j, ret_times[j], dep_times[j]) for j in ris],
$51                 key=lambda x: x[2] if x[2] is not None else 0.0,
$52             )
$53             stale = False
$54             for k in range(1, len(pairs)):
$55                 prev_ret = pairs[k - 1][1]
$56                 cur_dep = pairs[k][2]
$57                 if prev_ret is None or cur_dep is None:
$58                     stale = True
$59                     break
$60                 if cur_dep + 1e-6 < prev_ret + HANDOVER_TIME:
$61                     stale = True
$62                     break

```

```

863         if stale:
864             for j in ris:
865                 self.routes[j].vehicle_id = None
866 groups_by_vid: Dict[int, Dict] = {}
867 for i, r in enumerate(self.routes):
868     if not r.customers or dep_times[i] is None:
869         continue
870     vid = getattr(r, "vehicle_id", None)
871     if vid is None:
872         continue
873     g = groups_by_vid.get(vid)
874     if g is None:
875         g = {
876             "vt_index": r.vt_index,
877             "latest_return": ret_times[i],
878             "members": [i],
879             "had_departed": freezes[i].is_departed,
880         }
881     groups_by_vid[vid] = g
882 else:
883     g["members"].append(i)
884     if ret_times[i] is not None:
885         g["latest_return"] = max(g["latest_return"], ret_times[i])
886     if freezes[i].is_departed:
887         g["had_departed"] = True
888 groups_by_vt: Dict[int, List[Dict]] = {}
889 for g in groups_by_vid.values():
890     groups_by_vt.setdefault(g["vt_index"], []).append(g)
891 candidates = sorted(
892     [i for i in range(n)
893      if self.routes[i].customers
894      and getattr(self.routes[i], "vehicle_id", None) is None
895      and dep_times[i] is not None
896      and ret_times[i] is not None],
897     key=lambda i: dep_times[i],
898 )
899 merges = 0
900 for i in candidates:
901     r = self.routes[i]
902     vt_i = r.vt_index
903     dep_i = dep_times[i]
904     ret_i = ret_times[i]
905     is_dep = freezes[i].is_departed
906     best_g: Optional[Dict] = None
907     best_ret = -1.0
908     if not is_dep:
909         for g in groups_by_vt.get(vt_i, []):
910             if g["latest_return"] + HANDOVER_TIME > dep_i + 1e-6:
911                 continue

```

```

912         if g["latest_return"] > best_ret:
913             best_g = g
914             best_ret = g["latest_return"]
915     if best_g is not None:
916         vid = None
917         for vid_k, g_k in groups_by_vid.items():
918             if g_k is best_g:
919                 vid = vid_k
920                 break
921         if vid is None:
922             vid = _next_vehicle_id(self.routes)
923             groups_by_vid[vid] = best_g
924             _assign_vehicle_id_to_route(r, vid)
925         for m in best_g["members"]:
926             if getattr(self.routes[m], "vehicle_id", None) is None:
927                 _assign_vehicle_id_to_route(self.routes[m], vid)
928         best_g["members"].append(i)
929         best_g["latest_return"] = max(best_g["latest_return"], ret_i)
930         merges += 1
931     else:
932         new_g = {
933             "vt_index": vt_i,
934             "latest_return": ret_i,
935             "members": [i],
936             "had_departed": is_dep,
937         }
938         groups_by_vt.setdefault(vt_i, []).append(new_g)
939     if merges:
940         _clear_cache()
941     return merges
942 def _enforce_no_violation_by_delay(self, max_passes: int = 3) -> int:
943     bumped_total = 0
944     floor = self.policy.fuel_restrict_end + 1e-3
945     for _ in range(max_passes):
946         freezes = self._current_freezes()
947         changed = False
948         for ri, r in enumerate(self.routes):
949             if not r.customers:
950                 continue
951             vt = self.problem.vehicle_types[r.vt_index]
952             if not vt.is_fuel:
953                 continue
954             f = freezes[ri]
955             i = first_violating_arc_index(
956                 r, self.problem, self.policy, frozen_len=f.frozen_len,
957             )
958             if i is None:
959                 continue
960             if r.arc_depart_floor is None:

```

```

961         r.arc_depart_floor = {}
962         cur = r.arc_depart_floor.get(i, 0.0)
963         if cur >= floor:
964             continue
965         r.arc_depart_floor[i] = floor
966         self.arc_delay_count += 1
967         bumped_total += 1
968         changed = True
969         if not changed:
970             break
971     if bumped_total > 0:
972         self.arc_delay_events += 1
973         _clear_cache()
974     return bumped_total
975 def _emergency_split_tail(self) -> int:
976     splits = 0
977     elec_types = [
978         (vti, vt) for vti, vt in enumerate(self.problem.vehicle_types)
979         if not vt.is_fuel
980     ]
981     if not elec_types:
982         return 0
983     max_iters = 8
984     while max_iters > 0:
985         max_iters -= 1
986         target_ri = -1
987         target_pivot = -1
988         freezes = self._current_freezes()
989         for ri, r in enumerate(self.routes):
990             if not r.customers:
991                 continue
992             if r.arc_depart_floor is None:
993                 continue
994             vt = self.problem.vehicle_types[r.vt_index]
995             if not vt.is_fuel:
996                 continue
997             ev = evaluate_route(
998                 r, self.problem, policy=self.policy,
999                 frozen_len=freezes[ri].frozen_len,
1000             )
1001             if ev.feasible_return:
1002                 continue
1003             i = min(r.arc_depart_floor.keys())
1004             pivot = max(i, freezes[ri].frozen_len)
1005             if pivot >= len(r.customers):
1006                 continue
1007             target_ri = ri
1008             target_pivot = pivot
1009             break

```

```

1010         if target_ri < 0:
1011             break
1012         r = self.routes[target_ri]
1013         pivot = target_pivot
1014         head_custs = list(r.customers[:pivot])
1015         tail_custs = list(r.customers[pivot:])
1016         w = sum(self.problem.nodes[c].demand_w for c in tail_custs)
1017         v = sum(self.problem.nodes[c].demand_v for c in tail_custs)
1018         best_elec: Optional[Tuple[int, float]] = None
1019         for vti, vt_e in elec_types:
1020             if w > vt_e.capacity_w + 1e-6 or v > vt_e.capacity_v + 1e-6:
1021                 continue
1022             new_r = Route(vti, list(tail_custs))
1023             ev_new = evaluate_route(new_r, self.problem, policy=self.policy)
1024             if not (ev_new.feasible_cap and ev_new.feasible_return
1025                     and ev_new.feasible_policy):
1026                 continue
1027             if best_elec is None or ev_new.total_cost < best_elec[1]:
1028                 best_elec = (vti, ev_new.total_cost)
1029         if best_elec is None:
1030             break
1031         r.customers = head_custs
1032         r.arc_depart_floor = None
1033         new_tail = Route(best_elec[0], list(tail_custs))
1034         _assign_vehicle_id_to_route(new_tail, _next_vehicle_id(self.routes))
1035         self.routes.append(new_tail)
1036         self.emergency_split_count += 1
1037         splits += 1
1038     if splits > 0:
1039         _clear_cache()
1040     return splits
1041 def _policy_repair(self, max_iters: int = 6) -> None:
1042     for _ in range(max_iters):
1043         freezes = self._current_freezes()
1044         made_progress = False
1045         for ri in range(len(self.routes)):
1046             r = self.routes[ri]
1047             if not r.customers:
1048                 continue
1049             ev = evaluate_route(r, self.problem, policy=self.policy)
1050             if ev.feasible_policy:
1051                 continue
1052             f = freezes[ri]
1053             base_cost = _route_cost_dyn(r, self.problem, self.policy, self.t_now)
1054             if base_cost >= _INFEAS:
1055                 continue
1056             improved_this_route = False
1057             for pi in range(f.frozen_len, len(r.customers)):
1058                 c = r.customers[pi]

```



```

1059         new_cs = r.customers[:pi] + r.customers[pi + 1:]
1060         nr_route = Route(r.vt_index, new_cs)
1061         reduced_cost = _route_cost_dyn(
1062             nr_route, self.problem, self.policy, self.t_now,
1063         )
1064         if reduced_cost >= _INFEAS:
1065             continue
1066         gain_rm = base_cost - reduced_cost
1067         old_cust = r.customers
1068         r.customers = new_cs
1069         rcosts = _route_costs(self.routes, self.problem, self.policy, self.
t_now)

1070         used = _used_per_vt(self.routes)
1071         if f.is_departed:
1072             ins = _best_flex_insertion(
1073                 c, self.routes, freezes, rcosts, used,
1074                 self.problem, self.policy, self.t_now,
1075                 require_not_departed=False,
1076                 allowed_routes={ri},
1077             )
1078         else:
1079             ins = _best_flex_insertion(
1080                 c, self.routes, freezes, rcosts, used,
1081                 self.problem, self.policy, self.t_now,
1082                 require_not_departed=True,
1083             )
1084         if ins is not None:
1085             d_ins, r_enc, pos = ins
1086             if d_ins < gain_rm - 1e-6:
1087                 _apply_insertion(self.routes, freezes, c, r_enc, pos)
1088                 improved_this_route = True
1089                 made_progress = True
1090                 break
1091             r.customers = old_cust
1092         if improved_this_route:
1093             break
1094         if not made_progress:
1095             break
1096         self._try_swap_to_electric()
1097         self._handover_to_depot()
1098         self._enforce_no_violation_by_delay()
1099         self._emergency_split_tail()
1100     def load_baseline_as_scheduler(
1101         baseline_routes: List[Dict],
1102         problem: ProblemData,
1103         policy: PolicyConfig,
1104     ) -> DynamicScheduler:
1105         _clear_cache()
1106         routes = [

```

```

1107         Route(
1108             vt_index=r["vt_index"],
1109             customers=list(r["customers"]),
1110             vehicle_id=r.get("vehicle_id"),
1111             arc_depart_floor=(
1112                 None if r.get("arc_depart_floor") is None
1113                 else {int(k): float(v) for k, v in r["arc_depart_floor"].items()}
1114             ),
1115         )
1116         for r in baseline_routes
1117         if r.get("customers")
1118     ]
1119     return DynamicScheduler(
1120         problem=problem, routes=routes, policy=policy, t_now=0.0,
1121     )
1122 if __name__ == "__main__":
1123     import pickle
1124     from data_loader import load_problem
1125     base_problem = load_problem()
1126     policy = PolicyConfig()
1127     with open("results/q2_baseline.pkl", "rb") as f:
1128         baseline = pickle.load(f)
1129     prob = clone_problem(base_problem)
1130     sched = load_baseline_as_scheduler(baseline["routes"], prob, policy)
1131     print(f"Loaded {len(sched.routes)} routes at t=0.")
1132     sched.snapshot("T0", event=None, eval_before=None, response_ms=0.0,
1133                  note="initial Q2 plan")
1134     print(f"Initial cost: {sched.snapshots[0].eval_after.total_cost:.2f}")
1135     ev = Event(
1136         event_id="E0_smoke", kind=EVENT_CANCEL,
1137         trigger_time=9.0, description="smoke", orig_id=5,
1138     )
1139     snap = sched.apply_event(ev)
1140     print(
1141         f"After cancel 5: cost={snap.eval_after.total_cost:.2f}, "
1142         f"response={snap.response_ms:.1f}ms, "
1143         f"wasted={snap.cancelled_wasted}"
1144     )

```

dynamic_random_eval.py

代码 9: 问题三代码: dynamic_random_eval.py

```

1 from __future__ import annotations
2 import argparse
3 import pickle
4 import time
5 from dataclasses import dataclass, field
6 from pathlib import Path

```

```

7 from typing import Dict, List, Optional, Sequence, Tuple
8 import matplotlib
9 matplotlib.use("Agg")
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from cost_model import Route, Solution, evaluate_solution
14 from data_loader import ProblemData, clone_problem, load_problem
15 from dynamic_solver import (
16     DynamicScheduler,
17     Event,
18     EVENT_ADDRESS,
19     EVENT_CANCEL,
20     EVENT_NEW,
21     EVENT_TIMEWINDOW,
22     load_baseline_as_scheduler,
23 )
24 from policy import PolicyConfig
25 RESULTS_DIR = Path(__file__).resolve().parent / "results"
26 BASELINE_PKL = RESULTS_DIR / "q2_baseline.pkl"
27 @dataclass
28 class MCConfig:
29     n_trials: int = 30
30     base_seed: int = 20260424
31     poisson: bool = True
32     lambda_per_hour: float = 0.8
33     fixed_n_events: int = 5
34     time_start: float = 8.5
35     time_end: float = 15.5
36     min_events: int = 2
37     max_events: int = 10
38     type_weights: Dict[str, float] = field(default_factory=lambda: {
39         EVENT_CANCEL: 0.30,
40         EVENT_NEW: 0.30,
41         EVENT_ADDRESS: 0.20,
42         EVENT_TIMEWINDOW: 0.20,
43     })
44     new_x_range: Tuple[float, float] = (-25.0, 25.0)
45     new_y_range: Tuple[float, float] = (-25.0, 25.0)
46     new_demand_w_range: Tuple[float, float] = (50.0, 400.0)
47     new_demand_v_range: Tuple[float, float] = (0.1, 1.2)
48     new_tw_window_width: Tuple[float, float] = (1.5, 3.0)
49     new_tw_earliest_start: float = 10.0
50     new_tw_latest_start: float = 17.0
51     addr_shift_max_km: float = 6.0
52     tw_tighten_factor: Tuple[float, float] = (0.4, 0.8)
53     results_dir: Path = RESULTS_DIR
54 def _sample_event_count(rng: np.random.Generator, cfg: MCConfig) -> int:
55     if not cfg.poisson:

```

```

56         return cfg.fixed_n_events
57     horizon = cfg.time_end - cfg.time_start
58     for _ in range(10):
59         n = int(rng.poisson(cfg.lambda_per_hour * horizon))
60         if cfg.min_events <= n <= cfg.max_events:
61             return n
62     return int(np.clip(n, cfg.min_events, cfg.max_events))
63 def _sample_trigger_times(
64     rng: np.random.Generator, n: int, cfg: MCCConfig,
65 ) -> List[float]:
66     if n <= 0:
67         return []
68     times = rng.uniform(cfg.time_start, cfg.time_end, size=n)
69     return sorted(float(t) for t in times)
70 def _choose_type(rng: np.random.Generator, cfg: MCCConfig) -> str:
71     kinds = list(cfg.type_weights.keys())
72     w = np.array([cfg.type_weights[k] for k in kinds], dtype=float)
73     w = w / w.sum()
74     return str(rng.choice(kinds, p=w))
75 def _pick_orig_customer(
76     rng: np.random.Generator,
77     problem: ProblemData,
78     used: set,
79     restrict_set: Optional[set] = None,
80 ) -> Optional[int]:
81     pool = [oid for oid in problem.split_map
82              if oid not in used and (restrict_set is None or oid in restrict_set)]
83     if not pool:
84         return None
85     return int(rng.choice(pool))
86 def sample_events(
87     problem: ProblemData,
88     rng: np.random.Generator,
89     cfg: MCCConfig,
90 ) -> List[Event]:
91     n = _sample_event_count(rng, cfg)
92     if n <= 0:
93         return []
94     trigger_times = _sample_trigger_times(rng, n, cfg)
95     events: List[Event] = []
96     used_cust: set = set()
97     new_counter = 0
98     orig_ids = list(problem.split_map.keys())
99     for k, t in enumerate(trigger_times):
100         kind = _choose_type(rng, cfg)
101         if kind == EVENT_CANCEL:
102             oid = _pick_orig_customer(rng, problem, used_cust)
103             if oid is None:
104                 continue

```

```

105         used_cust.add(oid)
106         events.append(Event(
107             event_id=f"R{k + 1}",
108             kind=EVENT_CANCEL,
109             trigger_time=t,
110             description=f"随机取消客户 {oid}",
111             orig_id=oid,
112         ))
113     elif kind == EVENT_NEW:
114         new_counter += 1
115         new_id = 90000 + new_counter
116         x = float(rng.uniform(*cfg.new_x_range))
117         y = float(rng.uniform(*cfg.new_y_range))
118         w = float(rng.uniform(*cfg.new_demand_w_range))
119         v = float(rng.uniform(*cfg.new_demand_v_range))
120         width = float(rng.uniform(*cfg.new_tw_window_width))
121         start = float(rng.uniform(cfg.new_tw_earliest_start, cfg.new_tw_latest_start))
122         events.append(Event(
123             event_id=f"R{k + 1}",
124             kind=EVENT_NEW,
125             trigger_time=t,
126             description=(
127                 f"随机新增客户 {new_id}@({x:.1f},{y:.1f}) "
128                 f"{w:.0f}kg/{v:.2f}m3 TW[{start:.2f},{start + width:.2f}]"
129             ),
130             new_orig_id=new_id,
131             x=x, y=y,
132             demand_w=w, demand_v=v,
133             tw_start=start, tw_end=start + width,
134         ))
135     elif kind == EVENT_ADDRESS:
136         oid = _pick_orig_customer(rng, problem, used_cust)
137         if oid is None:
138             continue
139         used_cust.add(oid)
140         nd = problem.nodes[problem.split_map[oid][0]]
141         r_max = cfg.addr_shift_max_km
142         r = float(rng.uniform(0.5, r_max))
143         theta = float(rng.uniform(0, 2 * np.pi))
144         new_x = nd.x + r * np.cos(theta)
145         new_y = nd.y + r * np.sin(theta)
146         events.append(Event(
147             event_id=f"R{k + 1}",
148             kind=EVENT_ADDRESS,
149             trigger_time=t,
150             description=(
151                 f"随机改址客户 {oid}: "
152                 f"({nd.x:.1f},{nd.y:.1f})→({new_x:.1f},{new_y:.1f})"
153             ),

```

```

154         orig_id=oid,
155         new_x=float(new_x),
156         new_y=float(new_y),
157     ))
158     elif kind == EVENT_TIMEWINDOW:
159         oid = _pick_orig_customer(rng, problem, used_cust)
160         if oid is None:
161             continue
162         used_cust.add(oid)
163         nd = problem.nodes[problem.split_map[oid][0]]
164         old_start, old_end = nd.tw_start, nd.tw_end
165         factor = float(rng.uniform(*cfg.tw_tighten_factor))
166         new_width = max(0.3, (old_end - old_start) * factor)
167         max_offset = max(0.0, (old_end - old_start) - new_width)
168         offset = float(rng.uniform(0.0, max_offset)) if max_offset > 0 else 0.0
169         new_start = old_start + offset
170         new_end = new_start + new_width
171         events.append(Event(
172             event_id=f"R{k + 1}",
173             kind=EVENT_TIMEWINDOW,
174             trigger_time=t,
175             description=(
176                 f"随机改时间窗客户 {oid}: "
177                 f"[{old_start:.2f},{old_end:.2f}]→[{new_start:.2f},{new_end:.2f}]"
178             ),
179             orig_id=oid,
180             new_tw_start=new_start,
181             new_tw_end=new_end,
182         ))
183     return events
184
185 @dataclass
186 class TrialResult:
187     trial_id: int
188     seed: int
189     n_events: int
190     n_cancel: int
191     n_new: int
192     n_address: int
193     n_timewindow: int
194     baseline_cost: float
195     final_cost: float
196     delta_cost: float
197     delta_cost_pct: float
198     baseline_routes: int
199     final_routes: int
200     delta_routes: int
201     max_response_ms: float
202     mean_response_ms: float
203     total_response_ms: float

```

```

203     n_wasted_deliveries: int
204     n_failed_new_insertions: int
205     final_feasible_policy: bool
206     final_violation_count: int
207     max_violation_count: int
208     trial_wallclock_s: float
209     arc_delay_count: int = 0
210     arc_delay_events: int = 0
211     emergency_split_count: int = 0
212     contract_error: bool = False
213     error_message: str = ""
214     note: str = ""
215 def _count_event_type(events: Sequence[Event], kind: str) -> int:
216     return sum(1 for e in events if e.kind == kind)
217 def _count_failed_new(scheduler: DynamicScheduler, events: Sequence[Event]) -> int:
218     new_orig_ids = {ev.new_orig_id for ev in events if ev.kind == EVENT_NEW
219                     and ev.new_orig_id is not None}
220     if not new_orig_ids:
221         return 0
222     served: set = set()
223     for r in scheduler.routes:
224         for c in r.customers:
225             served.add(scheduler.problem.nodes[c].orig_id)
226     failed = 0
227     for oid in new_orig_ids:
228         if oid in scheduler.problem.split_map:
229             virts = scheduler.problem.split_map[oid]
230             if not any(v in (c for r in scheduler.routes for c in r.customers)
231                       for v in virts):
232                 failed += 1
233         else:
234             failed += 1
235     return failed
236 def run_single_trial(
237     trial_id: int,
238     seed: int,
239     baseline_routes: List[Dict],
240     base_problem: ProblemData,
241     policy: PolicyConfig,
242     cfg: MCCConfig,
243 ) -> Tuple[TrialResult, List[Event]]:
244     t_wall0 = time.time()
245     rng = np.random.default_rng(seed)
246     prob = clone_problem(base_problem)
247     sched = load_baseline_as_scheduler(baseline_routes, prob, policy)
248     sched.snapshot(
249         label="T0",
250         event=None,
251         eval_before=None,

```

```

252     response_ms=0.0,
253     note=f"trial {trial_id} seed {seed}",
254 )
255 events = sample_events(prob, rng, cfg)
256 contract_error = False
257 error_message = ""
258 for ev in events:
259     try:
260         sched.apply_event(ev)
261     except RuntimeError as exc:
262         contract_error = True
263         error_message = f"{ev.event_id}:{exc}"
264         break
265 t0 = sched.snapshots[0].eval_after
266 tn = sched.snapshots[-1].eval_after
267 responses = [s.response_ms for s in sched.snapshots[1:]]
268 viol_counts = [s.eval_after.policy_violation_count for s in sched.snapshots]
269 return (
270     TrialResult(
271         trial_id=trial_id,
272         seed=seed,
273         n_events=len(events),
274         n_cancel=_count_event_type(events, EVENT_CANCEL),
275         n_new=_count_event_type(events, EVENT_NEW),
276         n_address=_count_event_type(events, EVENT_ADDRESS),
277         n_timewindow=_count_event_type(events, EVENT_TIMEWINDOW),
278         baseline_cost=float(t0.total_cost),
279         final_cost=float(tn.total_cost),
280         delta_cost=float(tn.total_cost - t0.total_cost),
281         delta_cost_pct=float(
282             100.0 * (tn.total_cost - t0.total_cost) / max(1e-9, t0.total_cost)
283         ),
284         baseline_routes=int(t0.n_routes),
285         final_routes=int(tn.n_routes),
286         delta_routes=int(tn.n_routes - t0.n_routes),
287         max_response_ms=float(max(responses) if responses else 0.0),
288         mean_response_ms=float(np.mean(responses) if responses else 0.0),
289         total_response_ms=float(sum(responses)),
290         n_wasted_deliveries=int(len(sched.cancelled_wasted)),
291         n_failed_new_insertions=_count_failed_new(sched, events),
292         final_feasible_policy=bool(tn.feasible_policy),
293         final_violation_count=int(tn.policy_violation_count),
294         max_violation_count=int(max(viol_counts)),
295         trial_wallclock_s=float(time.time() - t_wall0),
296         arc_delay_count=int(getattr(sched, "arc_delay_count", 0)),
297         arc_delay_events=int(getattr(sched, "arc_delay_events", 0)),
298         emergency_split_count=int(getattr(sched, "emergency_split_count", 0)),
299         contract_error=contract_error,
300         error_message=error_message,

```



```

301         note="contract_error" if contract_error else "ok",
302     ),
303     events,
304 )
305 def run_monte_carlo(
306     cfg: MCConfig,
307     progress: bool = True,
308 ) -> Tuple[pd.DataFrame, List[List[Event]]:
309     if not BASELINE_PKL.exists():
310         raise FileNotFoundError(
311             f"未找到 Q2 基线 {BASELINE_PKL}. 请先运行 solve_q2.py."
312         )
313     with BASELINE_PKL.open("rb") as f:
314         baseline = pickle.load(f)
315     base_problem = load_problem()
316     policy = PolicyConfig()
317     rows: List[TrialResult] = []
318     streams: List[List[Event]] = []
319     for k in range(cfg.n_trials):
320         seed = cfg.base_seed + k
321         res, events = run_single_trial(
322             trial_id=k, seed=seed,
323             baseline_routes=baseline["routes"],
324             base_problem=base_problem,
325             policy=policy, cfg=cfg,
326         )
327         rows.append(res)
328         streams.append(events)
329     if progress:
330         tag = "!ERR" if res.contract_error else " "
331         print(
332             f" [trial {k + 1:3d}/{cfg.n_trials}]{tag} seed={seed} "
333             f"n_events={res.n_events:2d} ΔC={res.delta_cost:+9.2f}元 "
334             f"({res.delta_cost_pct:+6.2f}%) "
335             f"Δ#veh={res.delta_routes:+d} "
336             f"max_resp={res.max_response_ms:6.1f}ms "
337             f"viol={res.final_violation_count} "
338             f"wasted={res.n_wasted_deliveries} "
339             f"failed_new={res.n_failed_new_insertions} "
340             f"delay={res.arc_delay_events}({res.arc_delay_count}arc) "
341             f"esplit={res.emergency_split_count}"
342         )
343     df = pd.DataFrame([vars(r) for r in rows])
344     return df, streams
345 _NUMERIC_KPIS = [
346     "n_events", "delta_cost", "delta_cost_pct", "delta_routes",
347     "max_response_ms", "mean_response_ms", "total_response_ms",
348     "n_wasted_deliveries", "n_failed_new_insertions",
349     "final_violation_count", "max_violation_count",

```

```

350     "arc_delay_count", "arc_delay_events", "emergency_split_count",
351     "trial_wallclock_s",
352 ]
353 def summarise(df: pd.DataFrame) -> pd.DataFrame:
354     agg = pd.DataFrame({
355         "均值": df[_NUMERIC_KPIS].mean(),
356         "中位数": df[_NUMERIC_KPIS].median(),
357         "标准差": df[_NUMERIC_KPIS].std(ddof=0),
358         "最小值": df[_NUMERIC_KPIS].min(),
359         "最大值": df[_NUMERIC_KPIS].max(),
360         "P05": df[_NUMERIC_KPIS].quantile(0.05),
361         "P95": df[_NUMERIC_KPIS].quantile(0.95),
362     })
363     agg.index.name = "KPI"
364     return agg.round(3)
365 def event_stream_dump(streams: List[List[Event]]) -> pd.DataFrame:
366     rows = []
367     for tid, evs in enumerate(streams):
368         for ev in evs:
369             rows.append({
370                 "试验ID": tid,
371                 "事件序号": ev.event_id,
372                 "触发时刻": round(ev.trigger_time, 3),
373                 "类型": {
374                     EVENT_CANCEL: "取消", EVENT_NEW: "新增",
375                     EVENT_ADDRESS: "改址", EVENT_TIMEWINDOW: "改时间窗",
376                 }[ev.kind],
377                 "目标客户": ev.orig_id if ev.orig_id is not None else ev.new_orig_id,
378                 "描述": ev.description,
379             })
380     return pd.DataFrame(rows)
381 def plot_distributions(df: pd.DataFrame, out_path: Path) -> None:
382     try:
383         from solve_q1 import _set_chinese_font
384         _set_chinese_font()
385     except Exception:
386         pass
387     fig, axes = plt.subplots(2, 3, figsize=(16, 9))
388     ax = axes[0, 0]
389     ax.hist(df["delta_cost_pct"], bins=20, color="steelblue", edgecolor="white")
390     ax.axvline(0, color="red", linestyle="--", linewidth=1)
391     ax.set_title(f"成本变化率分布 (均值 {df['delta_cost_pct'].mean():+.2f}%)")
392     ax.set_xlabel("ΔC / 基线 (%)")
393     ax.set_ylabel("频次")
394     ax.grid(True, alpha=0.3)
395     ax = axes[0, 1]
396     ax.hist(df["max_response_ms"], bins=20, color="darkorange", edgecolor="white")
397     ax.set_title(f"单试验最大响应时延 (均值 {df['max_response_ms'].mean():.1f}ms)")
398     ax.set_xlabel("max 响应时延 (ms)")

```

```

399 ax.set_ylabel("频次")
400 ax.grid(True, alpha=0.3)
401 ax = axes[0, 2]
402 ax.hist(df["delta_routes"], bins=range(int(df["delta_routes"].min()) - 1,
403                                         int(df["delta_routes"].max()) + 2),
404         color="seagreen", edgecolor="white")
405 ax.axvline(0, color="red", linestyle="--", linewidth=1)
406 ax.set_title(f"启用车辆数变化  $\Delta$ #veh (均值 {df['delta_routes'].mean():+.2f})")
407 ax.set_xlabel(" $\Delta$  车辆数")
408 ax.set_ylabel("频次")
409 ax.grid(True, alpha=0.3)
410 ax = axes[1, 0]
411 ax.hist(df["n_events"], bins=range(df["n_events"].min(),
412                                     df["n_events"].max() + 2),
413         color="slateblue", edgecolor="white")
414 ax.set_title(f"每试验事件数 (均值 {df['n_events'].mean():.2f})")
415 ax.set_xlabel("事件数")
416 ax.set_ylabel("频次")
417 ax.grid(True, alpha=0.3)
418 ax = axes[1, 1]
419 bars = (df["final_violation_count"].value_counts()
420         .sort_index())
421 ax.bar(bars.index.astype(str), bars.values, color="firebrick")
422 feas_rate = 100.0 * (df["final_violation_count"] == 0).mean()
423 ax.set_title(f"终态政策违约数分布 (合规率 {feas_rate:.1f}%)")
424 ax.set_xlabel("违约次数")
425 ax.set_ylabel("频次")
426 ax.grid(True, alpha=0.3)
427 ax = axes[1, 2]
428 ax.scatter(df["n_events"], df["delta_cost"],
429            c=df["final_violation_count"], cmap="coolwarm",
430            s=50, edgecolor="black", linewidth=0.4)
431 ax.axhline(0, color="grey", linestyle="--", linewidth=0.8)
432 ax.set_title("事件数 vs  $\Delta C$  (颜色=违约数)")
433 ax.set_xlabel("事件数")
434 ax.set_ylabel(" $\Delta C$  (元)")
435 ax.grid(True, alpha=0.3)
436 fig.suptitle(
437     f"问题3 动态调度蒙特卡罗验证 (N={len(df)} 次独立试验)",
438     fontsize=14,
439 )
440 fig.tight_layout()
441 fig.savefig(out_path, dpi=160)
442 plt.close(fig)
443 def write_excel(
444     df: pd.DataFrame,
445     agg: pd.DataFrame,
446     streams_df: pd.DataFrame,
447     cfg: MCCConfig,

```

```

448     path: Path,
449 ) -> None:
450     col_order = [
451         "trial_id", "seed", "n_events",
452         "n_cancel", "n_new", "n_address", "n_timewindow",
453         "baseline_cost", "final_cost", "delta_cost", "delta_cost_pct",
454         "baseline_routes", "final_routes", "delta_routes",
455         "max_response_ms", "mean_response_ms", "total_response_ms",
456         "n_wasted_deliveries", "n_failed_new_insertions",
457         "final_feasible_policy", "final_violation_count", "max_violation_count",
458         "arc_delay_events", "arc_delay_count", "emergency_split_count",
459         "contract_error", "error_message",
460         "trial_wallclock_s", "note",
461     ]
462     df_out = df[col_order].copy()
463     df_out.columns = [
464         "试验ID", "种子", "事件数",
465         "取消", "新增", "改址", "改时间窗",
466         "基线成本(元)", "终态成本(元)", "ΔC(元)", "ΔC%",
467         "基线车辆数", "终态车辆数", "Δ车辆数",
468         "最大响应(ms)", "平均响应(ms)", "总响应(ms)",
469         "无效送达数", "新增失败数",
470         "终态政策可行", "终态违约数", "最大违约数",
471         "延迟过窗事件数", "延迟过窗弧次", "应急拆分数",
472         "契约失败", "错误信息",
473         "试验耗时(s)", "备注",
474     ]
475     cfg_df = pd.DataFrame([
476         {"项": "n_trials", "值": cfg.n_trials},
477         {"项": "base_seed", "值": cfg.base_seed},
478         {"项": "poisson", "值": cfg.poisson},
479         {"项": "lambda_per_hour", "值": cfg.lambda_per_hour},
480         {"项": "fixed_n_events", "值": cfg.fixed_n_events},
481         {"项": "time_start", "值": cfg.time_start},
482         {"项": "time_end", "值": cfg.time_end},
483         {"项": "min_events", "值": cfg.min_events},
484         {"项": "max_events", "值": cfg.max_events},
485         {"项": "type_weights", "值": str(cfg.type_weights)},
486         {"项": "new_x_range", "值": str(cfg.new_x_range)},
487         {"项": "new_y_range", "值": str(cfg.new_y_range)},
488         {"项": "new_demand_w_range", "值": str(cfg.new_demand_w_range)},
489         {"项": "new_demand_v_range", "值": str(cfg.new_demand_v_range)},
490         {"项": "new_tw_window_width", "值": str(cfg.new_tw_window_width)},
491         {"项": "addr_shift_max_km", "值": cfg.addr_shift_max_km},
492         {"项": "tw_tighten_factor", "值": str(cfg.tw_tighten_factor)},
493     ])
494     with pd.ExcelWriter(path, engine="openpyxl") as w:
495         df_out.to_excel(w, sheet_name="1-逐试验指标", index=False)
496         agg.to_excel(w, sheet_name="2-指标聚合")

```

```

497     streams_df.to_excel(w, sheet_name="3-事件流明细", index=False)
498     cfg_df.to_excel(w, sheet_name="4-MC配置", index=False)
499 def write_summary_md(
500     df: pd.DataFrame,
501     agg: pd.DataFrame,
502     cfg: MCConfig,
503     path: Path,
504 ) -> None:
505     n = len(df)
506     mean_dc = df["delta_cost"].mean()
507     mean_dc_pct = df["delta_cost_pct"].mean()
508     feas_rate = 100.0 * (df["final_violation_count"] == 0).mean()
509     max_resp_p95 = df["max_response_ms"].quantile(0.95)
510     worst = df.loc[df["delta_cost"].idxmax()]
511     best = df.loc[df["delta_cost"].idxmin()]
512     dc_reduction_rate = 100.0 * (df["delta_cost"] < 0).mean()
513     wasted_rate = 100.0 * (df["n_wasted_deliveries"] > 0).mean()
514     failed_rate = 100.0 * (df["n_failed_new_insertions"] > 0).mean()
515     delay_trial_rate = 100.0 * (df["arc_delay_events"] > 0).mean()
516     mean_delay_arcs = df["arc_delay_count"].mean()
517     emerg_trial_rate = 100.0 * (df["emergency_split_count"] > 0).mean()
518     n_contract_errors = int(df["contract_error"].sum())
519     lines: List[str] = []
520     def add(s: str = "") -> None:
521         lines.append(s)
522     add("# 问题 3 动态调度 蒙特卡罗验证报告")
523     add()
524     add("本报告使用 `dynamic_random_eval.py` 对 `dynamic_solver.py` 做了 "
525         f"{n} 次**独立随机事件流试验，用以回答 \"该策略在事件具有随机性时是否稳健\" 这一问题"
526         ". ")
527     add("每次试验独立克隆 Q2 基线问题，采样一条新事件流（事件到达服从泊松过程，"
528         "事件类型按 30%/30%/20%/20% 比例取自 {取消, 新增, 改址, 改时间窗}, "
529         "事件参数由独立先验分布抽样），运行完整滚动时域调度后记录 KPI. ")
530     add()
531     add("## 一、Monte-Carlo 配置")
532     add()
533     add(f"- 试验次数: **{cfg.n_trials}**")
534     add(f"- 种子: {cfg.base_seed} + k, k=0..{cfg.n_trials - 1}")
535     add(f"- 事件到达: {'Poisson 过程' if cfg.poisson else '固定次数'}, "
536         f" = {cfg.lambda_per_hour} 件/小时, "
537         f"时间窗 [{cfg.time_start}, {cfg.time_end}]"
538     )
539     add(f"- 事件数约束: [{cfg.min_events}, {cfg.max_events}]"
540     )
541     add(f"- 类型权重: {cfg.type_weights}"
542     )
543     add()
544     add("## 二、核心指标分布")
545     add()
546     add(agg.to_markdown())
547     add()
548     add("## 三、关键结论")

```

```

545 add()
546 add(f"1. **零违约硬保障**: {n} 次试验中 **{feas_rate:.1f}%** 终态满足绿区 Rule A/B. "
547     "调度器对任何事件都按五级兜底修复: (1) 柔性段重插入 → (2) 燃油转电动 → "
548     "(3) 返仓交接 → (4) 延迟过窗 (将燃油车等到 16:00 后再走违约弧, 违约转为 late_cost) → "
549     "(5) 应急拆分 (超配电动车接收尾部). "
550     f"本批次触发 \"延迟过窗\" 试验占 **{delay_trial_rate:.1f}%** "
551     f"(平均每试验延 {mean_delay_arcs:.2f} 条弧), "
552     f"触发 \"应急拆分\" 占 **{emerg_trial_rate:.1f}%**, "
553     f"契约失败 (任何违约残留) 计 **{n_contract_errors}** 次.")
554 add(f"2. **成本中性偏优**: 平均  $\Delta C$  = **{mean_dc:.2f} 元** ({mean_dc_pct:+.2f}%), "
555     f"**{dc_reduction_rate:.1f}%** 的试验在事件后成本反而 *下降*. "
556     "原因是取消与新增事件触发的路径合并带来车次节省, 平均效果超过改址/改时间窗带来的额外绕行成本.")
557 add(f"3. **实时性**: 单试验最大响应时延 P95 = **{max_resp_p95:.1f} ms**, "
558     "远低于 1 秒 SLA; 4 类事件均可在线调度系统中使用.")
559 add(f"4. **失败率**: 新增订单的插入失败率为 **{failed_rate:.1f}%** "
560     "(少数事件在白天高峰+车队饱和时无法就地安排); "
561     f"取消事件产生 \"已发车无法撤回\" 的无效送达率为 **{wasted_rate:.1f}%**.")
562 add(f"5. **极端值**: 最差试验 Trial #{int(worst['trial_id'])} "
563     f"(seed={int(worst['seed'])}, 事件数={int(worst['n_events'])}) "
564     f" $\Delta C$  = {worst['delta_cost']:+.2f} 元; "
565     f"最佳试验 Trial #{int(best['trial_id'])} "
566     f"(seed={int(best['seed'])}, 事件数={int(best['n_events'])}) "
567     f" $\Delta C$  = {best['delta_cost']:+.2f} 元.")
568 add()
569 add("## 四、与 `solve_q3.py` 确定性场景的关系")
570 add()
571 add("`solve_q3.py` 给出的是一条手工精选、便于论文复现的确定性事件序列 "
572     "(T0 → E1-E4), 展示算法在 *一个具体案例* 下的路径/成本/时延细节; ")
573 add("本模块 `dynamic_random_eval.py` 则通过随机抽样做**统计显著性验证**, "
574     "补齐算法对事件流随机性的鲁棒性证据. 两者互补:")
575 add()
576 add("| 文件 | 目的 | 事件来源 | 输出 |")
577 add("|-----|-----|-----|-----|")
578 add("| `solve_q3.py` | 展示单一综合场景 | 确定性手工 | 5 份快照 + 4 图 + 多页表 |")
579 add(f"| `dynamic_random_eval.py` | 统计分布验证 | 随机 (N={n}) | MC 聚合表 + 直方图 + 本报告 |")
580 add()
581 add("## 五、产物清单")
582 add()
583 add("- `q3_mc_results.xlsx` — 逐试验指标 / 指标聚合 / 事件流明细 / MC 配置")
584 add("- `q3_mc_distributions.png` — 六面板分布图")
585 add("- `q3_mc_summary.md` — 本文件")
586 add()
587 add("## 六、复现命令")
588 add()
589 add("`bash`")
590 add("python dynamic_random_eval.py # 默认 30 次")

```

```

591 add(f"python dynamic_random_eval.py --n-trials {cfg.n_trials}          # 指定次数")
592 add("python dynamic_random_eval.py --fixed 5                        # 每次固定 5 个事件")
593 add("```")
594 path.write_text("\n".join(lines), encoding="utf-8")
595 def _parse_args() -> argparse.Namespace:
596     p = argparse.ArgumentParser(description=__doc__)
597     p.add_argument("--n-trials", type=int, default=30)
598     p.add_argument("--seed", type=int, default=20260424)
599     p.add_argument("--lambda", type=float, default=0.8,
600                   dest="lam", help="Poisson rate (events per hour)")
601     p.add_argument("--fixed", type=int, default=None,
602                   help="If set, disable Poisson and use this fixed event count.")
603     p.add_argument("--time-start", type=float, default=8.5)
604     p.add_argument("--time-end", type=float, default=15.5)
605     p.add_argument("--quiet", action="store_true")
606     return p.parse_args()
607 def main() -> None:
608     args = _parse_args()
609     cfg = MCCConfig(
610         n_trials=args.n_trials,
611         base_seed=args.seed,
612         poisson=(args.fixed is None),
613         lambda_per_hour=args.lam,
614         fixed_n_events=args.fixed if args.fixed is not None else 5,
615         time_start=args.time_start,
616         time_end=args.time_end,
617     )
618     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
619     print(f"[mc] 随机事件流蒙特卡罗验证, {cfg.n_trials} 次试验...")
620     t0 = time.time()
621     df, streams = run_monte_carlo(cfg, progress=not args.quiet)
622     print(f"[mc] 完成, 合计耗时 {(time.time() - t0):.2f} s.")
623     agg = summarise(df)
624     streams_df = event_stream_dump(streams)
625     xlsx_path = RESULTS_DIR / "q3_mc_results.xlsx"
626     png_path = RESULTS_DIR / "q3_mc_distributions.png"
627     md_path = RESULTS_DIR / "q3_mc_summary.md"
628     write_excel(df, agg, streams_df, cfg, xlsx_path)
629     plot_distributions(df, png_path)
630     write_summary_md(df, agg, cfg, md_path)
631     print(f"[mc] 表格 → {xlsx_path}")
632     print(f"[mc] 分布图 → {png_path}")
633     print(f"[mc] 报告 → {md_path}")
634     print("\n[mc] 关键统计:")
635     print(f"    平均 ΔC = {df['delta_cost'].mean():+.2f} 元 "
636           f"f"({df['delta_cost_pct'].mean():+.3f}%)"
637     print(f"    成本下降试验比例 = "
638           f"f"100 * (df['delta_cost'] < 0).mean():.1f}%)"
639     print(f"    终态政策合规率 = "

```

```

640         f"{100 * (df['final_violation_count'] == 0).mean():.1f}%"
641     print(f"  延迟过窗触发比例 = "
642           f"{100 * (df['arc_delay_events'] > 0).mean():.1f}% "
643           f"(平均 {df['arc_delay_count'].mean():.2f} 条弧/试验)")
644     print(f"  应急拆分触发比例 = "
645           f"{100 * (df['emergency_split_count'] > 0).mean():.1f}%"
646           f"(平均 {df['emergency_split_count'].mean():.2f} 条弧/试验)")
647     print(f"  契约失败试验数 = "
648           f"{int(df['contract_error'].sum())}")
649     print(f"  最大响应时延 P95 = "
650           f"{df['max_response_ms'].quantile(0.95):.1f} ms")
651 if __name__ == "__main__":
652     main()

```

solve_q3.py

代码 10: 问题三代码: solve_q3.py

```

1  from __future__ import annotations
2  import pickle
3  import time
4  from pathlib import Path
5  from typing import Dict, List, Optional, Tuple
6  import matplotlib
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11 from cost_model import Route, Solution, evaluate_route, evaluate_solution
12 from data_loader import (
13     ProblemData,
14     clone_problem,
15     load_problem,
16 )
17 from dynamic_solver import (
18     DynamicScheduler,
19     Event,
20     EVENT_ADDRESS,
21     EVENT_CANCEL,
22     EVENT_NEW,
23     EVENT_TIMEWINDOW,
24     Snapshot,
25     compute_freezes,
26     load_baseline_as_scheduler,
27 )
28 from policy import PolicyConfig
29 from solve_q1 import (
30     _finish_3d_figure,
31     _prepare_frequency_cmap,
32     _service_frequency_by_virtual_node,

```



```

33     _set_chinese_font,
34     hhmm,
35     plot_algorithm_flowchart,
36 )
37 RESULTS_DIR = Path(__file__).resolve().parent / "results"
38 def build_scenario() -> List[Event]:
39     return [
40         Event(
41             event_id="E1",
42             kind=EVENT_CANCEL,
43             trigger_time=9 + 15 / 60,
44             description="客户 72 (北郊, Q2 由 F1 于下午配送) 取消订单",
45             orig_id=72,
46         ),
47         Event(
48             event_id="E2",
49             kind=EVENT_TIMEWINDOW,
50             trigger_time=10 + 30 / 60,
51             description="客户 7 (绿区内, Q2 由 E1 配送) 时间窗收紧为 [13:00, 13:45]",
52             orig_id=7,
53             new_tw_start=13.0,
54             new_tw_end=13 + 45 / 60,
55         ),
56         Event(
57             event_id="E3",
58             kind=EVENT_NEW,
59             trigger_time=11 + 45 / 60,
60             description="新增客户 N1 @ (12,5), 需求 150kg/0.5m³, 时间窗 [14:00, 16:00]",
61             new_orig_id=1001,
62             x=12.0, y=5.0,
63             demand_w=150.0, demand_v=0.5,
64             tw_start=14.0, tw_end=16.0,
65         ),
66         Event(
67             event_id="E4",
68             kind=EVENT_ADDRESS,
69             trigger_time=13 + 20 / 60,
70             description="客户 70 原位 (-12.9,-33.6) 变更至 (-8.0,-22.0)",
71             orig_id=70,
72             new_x=-8.0,
73             new_y=-22.0,
74         ),
75     ]
76 def run_rolling_schedule(
77     baseline_pkl: Path,
78     base_problem: ProblemData,
79     policy: PolicyConfig,
80 ) -> Tuple[DynamicScheduler, List[Event], ProblemData]:
81     with baseline_pkl.open("rb") as f:

```

```

82     baseline = pickle.load(f)
83     prob = clone_problem(base_problem)
84     sched = load_baseline_as_scheduler(baseline["routes"], prob, policy)
85     sched.snapshot(
86         label="T0",
87         event=None,
88         eval_before=None,
89         response_ms=0.0,
90         note="Q2 initial committed plan (t=0)",
91     )
92     events = build_scenario()
93     for ev in events:
94         sched.apply_event(ev)
95     return sched, events, prob
96 def build_event_log(events: List[Event], snapshots: List[Snapshot]) -> pd.DataFrame:
97     rows = []
98     for ev, snap in zip(events, snapshots[1:]):
99         rows.append({
100             "事件ID": ev.event_id,
101             "触发时刻": hhmm(ev.trigger_time),
102             "事件类型": {
103                 EVENT_CANCEL: "订单取消",
104                 EVENT_NEW: "新增订单",
105                 EVENT_ADDRESS: "地址变更",
106                 EVENT_TIMEWINDOW: "时间窗调整",
107             }[ev.kind],
108             "描述": ev.description,
109             "受影响路线数": len(snap.changed_route_indices),
110             "本次处理结果": snap.note,
111             "无效送达(虚拟节点)": ";".join(str(x) for x in snap.cancelled_wasted) or "-",
112             "响应耗时(ms)": round(snap.response_ms, 1),
113             "政策可行": "是" if snap.eval_after.feasible_policy else "否",
114             "政策违约次数": snap.eval_after.policy_violation_count,
115         })
116     return pd.DataFrame(rows)
117 def build_rolling_paths_table(
118     sched: DynamicScheduler,
119     problem: ProblemData,
120 ) -> pd.DataFrame:
121     rows = []
122     for snap in sched.snapshots:
123         for ri, r_dict in enumerate(snap.routes):
124             if not r_dict["customers"]:
125                 continue
126             vt = problem.vehicle_types[r_dict["vt_index"]]
127             cust_origs = [problem.nodes[c].orig_id for c in r_dict["customers"]]
128             w = sum(problem.nodes[c].demand_w for c in r_dict["customers"])
129             v = sum(problem.nodes[c].demand_v for c in r_dict["customers"])
130             flen = 0

```

```

131         if ri < len(snap.freezes):
132             flen = snap.freezes[ri].frozen_len
133         rows.append({
134             "快照": snap.label,
135             "时刻": hhmm(snap.t_now),
136             "路线序号": ri,
137             "车型": vt.name,
138             "载重(kg)": round(w, 1),
139             "体积(m3)": round(v, 3),
140             "客户数": len(r_dict["customers"]),
141             "冻结前缀长度": flen,
142             "虚拟节点序列": ",".join(str(c) for c in r_dict["customers"]),
143             "原始客户序列": ",".join(str(x) for x in cust_origs),
144         })
145     return pd.DataFrame(rows)
146 def _count_physical_vehicles_from_snap(snap) -> int:
147     seen = set()
148     anon = 0
149     for r in snap.routes:
150         if not r.get("customers"):
151             continue
152         vid = r.get("vehicle_id")
153         if vid is None:
154             anon += 1
155         else:
156             seen.add(vid)
157     return anon + len(seen)
158 def _count_arc_delay_arcs_from_snap(snap: Snapshot) -> int:
159     total = 0
160     for r in snap.routes:
161         floor = r.get("arc_depart_floor") if isinstance(r, dict) else None
162         if floor:
163             total += len(floor)
164     return total
165 def build_cost_breakdown_table(sched: DynamicScheduler) -> pd.DataFrame:
166     rows = []
167     prev_cost = None
168     t0_cost = sched.snapshots[0].eval_after.total_cost
169     for snap in sched.snapshots:
170         ev = snap.eval_after
171         cur_cost = ev.total_cost
172         delta_prev = 0.0 if prev_cost is None else (cur_cost - prev_cost)
173         delta_t0 = cur_cost - t0_cost
174         n_phys = _count_physical_vehicles_from_snap(snap)
175         rows.append({
176             "快照": snap.label,
177             "时刻": hhmm(snap.t_now),
178             "总成本(元)": round(cur_cost, 2),
179             "启动成本(元)": round(ev.fixed_cost, 2),

```

```

180         "能源成本(元)": round(ev.energy_cost, 2),
181         "碳排放成本(元)": round(ev.carbon_cost_money, 2),
182         "等待成本(元)": round(ev.wait_cost, 2),
183         "迟到成本(元)": round(ev.late_cost, 2),
184         "趟次": ev.n_routes,
185         "实际车辆": n_phys,
186         "节省启动次数": ev.n_routes - n_phys,
187         "政策违约数": ev.policy_violation_count,
188         "延迟过窗弧次": _count_arc_delay_arcs_from_snap(snap),
189         "Δ vs 上一次": round(delta_prev, 2),
190         "Δ vs Q2(T0)": round(delta_t0, 2),
191     })
192     prev_cost = cur_cost
193     return pd.DataFrame(rows)
194 def build_freeze_split_table(
195     sched: DynamicScheduler,
196     problem: ProblemData,
197 ) -> pd.DataFrame:
198     rows = []
199     for snap in sched.snapshots:
200         n_frozen = 0
201         n_flex = 0
202         n_routes_departed = 0
203         n_routes_at_depot = 0
204         for freeze in snap.freezes:
205             n_frozen += freeze.frozen_len
206             n_flex += len(freeze.customers_flexible)
207             if freeze.is_departed:
208                 n_routes_departed += 1
209             elif freeze.frozen_len + len(freeze.customers_flexible) > 0:
210                 n_routes_at_depot += 1
211         rows.append({
212             "快照": snap.label,
213             "时刻": hhmm(snap.t_now),
214             "已发车路线": n_routes_departed,
215             "未发车路线": n_routes_at_depot,
216             "冻结虚拟节点数": n_frozen,
217             "柔性虚拟节点数": n_flex,
218             "冻结占比(%)": round(
219                 100 * n_frozen / max(1, n_frozen + n_flex), 2
220             ),
221         })
222     return pd.DataFrame(rows)
223 def build_vt_usage_table(
224     sched: DynamicScheduler,
225     problem: ProblemData,
226 ) -> pd.DataFrame:
227     vt_names = [vt.name for vt in problem.vehicle_types]
228     rows = []

```

```

229     for snap in sched.snapshots:
230         trips = {name: 0 for name in vt_names}
231         phys_sets: Dict[str, set] = {name: set() for name in vt_names}
232         phys_anon = {name: 0 for name in vt_names}
233         for r in snap.routes:
234             if not r.get("customers"):
235                 continue
236             vname = vt_names[r["vt_index"]]
237             trips[vname] += 1
238             vid = r.get("vehicle_id")
239             if vid is None:
240                 phys_anon[vname] += 1
241             else:
242                 phys_sets[vname].add(vid)
243         row = {"快照": snap.label, "时刻": hhmm(snap.t_now)}
244         for name in vt_names:
245             row[f"{name}(趟)"] = trips[name]
246             row[f"{name}(车)"] = phys_anon[name] + len(phys_sets[name])
247         row["合计趟次"] = sum(trips.values())
248         row["合计车辆"] = sum(phys_anon[n] + len(phys_sets[n]) for n in vt_names)
249         rows.append(row)
250     return pd.DataFrame(rows)
251 def write_tables_xlsx(
252     sched: DynamicScheduler,
253     events: List[Event],
254     problem: ProblemData,
255     path: Path,
256 ) -> None:
257     ev_log = build_event_log(events, sched.snapshots)
258     paths = build_rolling_paths_table(sched, problem)
259     costs = build_cost_breakdown_table(sched)
260     split = build_freeze_split_table(sched, problem)
261     vts = build_vt_usage_table(sched, problem)
262     with pd.ExcelWriter(path, engine="openpyxl") as writer:
263         ev_log.to_excel(writer, sheet_name="1-事件日志", index=False)
264         paths.to_excel(writer, sheet_name="2-滚动计划路径", index=False)
265         costs.to_excel(writer, sheet_name="3-成本增量对比", index=False)
266         split.to_excel(writer, sheet_name="4-冻结柔性划分", index=False)
267         vts.to_excel(writer, sheet_name="5-车型启用变化", index=False)
268 def _plot_routes_on_ax(
269     ax, problem: ProblemData, routes: List[Dict], policy: PolicyConfig,
270     title: str, color_routes: bool = True,
271 ) -> None:
272     depot = problem.nodes[0]
273     ax.scatter([depot.x], [depot.y], s=200, marker="s", color="black",
274               label="配送中心", zorder=5)
275     theta = np.linspace(0, 2 * np.pi, 200)
276     ax.plot(
277         policy.green_radius * np.cos(theta),

```

```

278     policy.green_radius * np.sin(theta),
279     "--", color="green", alpha=0.45,
280     label=f"绿色配送区({policy.green_radius:g}km)",
281 )
282 plotted = set()
283 for r in routes:
284     for c in r["customers"]:
285         if c in plotted:
286             continue
287         plotted.add(c)
288         n = problem.nodes[c]
289         ax.scatter([n.x], [n.y], s=12, color="lightgray", zorder=1)
290 cmap = plt.get_cmap("tab20")
291 k = 0
292 for r in routes:
293     if not r["customers"]:
294         continue
295     color = cmap(k % 20) if color_routes else "steelblue"
296     xs = [depot.x] + [problem.nodes[c].x for c in r["customers"]] + [depot.x]
297     ys = [depot.y] + [problem.nodes[c].y for c in r["customers"]] + [depot.y]
298     ax.plot(xs, ys, "-o", color=color, linewidth=0.9, markersize=3, alpha=0.85)
299     k += 1
300 ax.set_title(title)
301 ax.set_xlabel("X (km)")
302 ax.set_ylabel("Y (km)")
303 ax.set_aspect("equal", adjustable="datalim")
304 ax.grid(True, alpha=0.3)
305 def plot_routes_before_after(
306     sched: DynamicScheduler,
307     problem: ProblemData,
308     policy: PolicyConfig,
309     path: Path,
310 ) -> None:
311     _set_chinese_font()
312     fig, axes = plt.subplots(1, 2, figsize=(18, 9))
313     t0 = sched.snapshots[0]
314     final = sched.snapshots[-1]
315     t0_phys = _count_physical_vehicles_from_snap(t0)
316     final_phys = _count_physical_vehicles_from_snap(final)
317     _plot_routes_on_ax(
318         axes[0], problem, t0.routes, policy,
319         title=(
320             f"T0: Q2 基线计划 ({t0_phys} 辆 / {t0.eval_after.n_routes} 趟, "
321             f"成本 {t0.eval_after.total_cost:.0f})"
322         ),
323     )
324     _plot_routes_on_ax(
325         axes[1], problem, final.routes, policy,
326         title=(

```

```

327         f"{final.label} 后: 最终滚动计划  "
328         f"({final_phys} 辆 / {final.eval_after.n_routes} 趟, "
329         f"成本 {final.eval_after.total_cost:.0f})"
330     ),
331 )
332 for nd in problem.nodes:
333     if nd.orig_id == 1001:
334         axes[1].scatter([nd.x], [nd.y], s=140, marker="*", color="red",
335                         label="新增订单 N1", zorder=6)
336         break
337 axes[0].legend(loc="upper left", fontsize=9)
338 axes[1].legend(loc="upper left", fontsize=9)
339 fig.suptitle("问题3 动态调度: Q2 原计划 vs 最终滚动计划", fontsize=14)
340 fig.tight_layout()
341 fig.savefig(path, dpi=160)
342 plt.close(fig)
343 def plot_event_timeline(
344     sched: DynamicScheduler,
345     problem: ProblemData,
346     events: List[Event],
347     path: Path,
348 ) -> None:
349     _set_chinese_font()
350     final = sched.snapshots[-1]
351     bars: List[Tuple[int, float, float, bool]] = []
352     row = 0
353     for ri, r_dict in enumerate(final.routes):
354         if not r_dict["customers"]:
355             continue
356         r = Route(r_dict["vt_index"], list(r_dict["customers"]))
357         ev = evaluate_route(r, problem, policy=PolicyConfig())
358         frozen_len = final.freezes[ri].frozen_len if ri < len(final.freezes) else 0
359         for j in range(len(r.customers)):
360             s = ev.service_start[j + 1]
361             e = ev.service_end[j + 1]
362             is_frozen = j < frozen_len
363             bars.append((row, s, e, is_frozen))
364         row += 1
365     n_rows = row
366     max_rows = 60
367     stride = max(1, n_rows // max_rows)
368     shown = [b for b in bars if (b[0] % stride) == 0]
369     y_map = {r: i for i, r in enumerate(sorted(set(b[0] for b in shown)))}
370     fig, ax = plt.subplots(figsize=(14, max(6, 0.14 * len(y_map))))
371     for r, s, e, is_frozen in shown:
372         y = y_map[r]
373         color = "dimgray" if is_frozen else "steelblue"
374         ax.barh(y, width=max(e - s, 0.03), left=s, height=0.7,
375                color=color, edgecolor="white", linewidth=0.2)

```

```

376 colors = ["crimson", "darkorange", "purple", "teal"]
377 for i, ev in enumerate(events):
378     ax.axvline(
379         ev.trigger_time, color=colors[i % len(colors)],
380         linestyle="--", linewidth=1.4, alpha=0.85,
381         label=f"{ev.event_id} @ {hhmm(ev.trigger_time)} {ev.description[:22]}",
382     )
383 ax.set_xlabel("时间 (h)")
384 ax.set_ylabel(f"路线/车次序号 (每 {stride} 条示 1)")
385 ax.set_xlim(7.5, 20.5)
386 ax.set_xticks(np.arange(8, 21, 1))
387 ax.set_xticklabels([hhmm(t) for t in range(8, 21)], rotation=45)
388 ax.grid(True, alpha=0.3, axis="x")
389 ax.set_title("问题3 事件甘特图 (每行=路线/车次; 灰=冻结/已承诺, 蓝=柔性)")
390 ax.legend(loc="lower right", fontsize=8, framealpha=0.9)
391 fig.tight_layout()
392 fig.savefig(path, dpi=160)
393 plt.close(fig)
394 def plot_cost_trajectory(sched: DynamicScheduler, path: Path) -> None:
395     _set_chinese_font()
396     labels = [s.label for s in sched.snapshots]
397     fixed = [s.eval_after.fixed_cost for s in sched.snapshots]
398     energy = [s.eval_after.energy_cost for s in sched.snapshots]
399     carbon = [s.eval_after.carbon_cost_money for s in sched.snapshots]
400     wait = [s.eval_after.wait_cost for s in sched.snapshots]
401     late = [s.eval_after.late_cost for s in sched.snapshots]
402     total = [s.eval_after.total_cost for s in sched.snapshots]
403     x = np.arange(len(labels))
404     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 9))
405     ax1.plot(x, total, "-o", color="black", linewidth=2, label="总成本")
406     ax1.set_xticks(x)
407     ax1.set_xticklabels(labels)
408     ax1.set_ylabel("总成本 (元)")
409     ax1.set_title("问题3 成本分项沿事件的累积演变")
410     ax1.grid(True, alpha=0.3)
411     ax1.legend()
412     for xi, v in zip(x, total):
413         ax1.annotate(f"{v:.0f}", (xi, v), textcoords="offset points",
414                     xytext=(0, 8), ha="center", fontsize=9)
415     ax2.stackplot(
416         x, fixed, energy, carbon, wait, late,
417         labels=["启动成本", "能源成本", "碳排放成本", "等待成本", "迟到成本"],
418         alpha=0.85,
419         colors=["#4c72b0", "#dd8452", "#55a868", "#c44e52", "#8172b3"],
420     )
421     ax2.set_xticks(x)
422     ax2.set_xticklabels(labels)
423     ax2.set_ylabel("成本分项 (元)")
424     ax2.set_xlabel("快照")

```



```

425     ax2.grid(True, alpha=0.3)
426     ax2.legend(loc="upper right")
427     fig.tight_layout()
428     fig.savefig(path, dpi=160)
429     plt.close(fig)
430 def plot_responseBars(
431     sched: DynamicScheduler,
432     events: List[Event],
433     path: Path,
434 ) -> None:
435     _set_chinese_font()
436     labels = [ev.event_id + "\n" + hhmm(ev.trigger_time) for ev in events]
437     latencies = [snap.response_ms for snap in sched.snapshots[1:]]
438     changed = [len(snap.changed_route_indices) for snap in sched.snapshots[1:]]
439     x = np.arange(len(labels))
440     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
441     axes[0].bar(x, latencies, color="steelblue")
442     axes[0].set_xticks(x)
443     axes[0].set_xticklabels(labels)
444     axes[0].set_ylabel("响应耗时 (ms)")
445     axes[0].set_title("响应时延 (ms, 越低越好)")
446     axes[0].grid(True, alpha=0.3, axis="y")
447     for xi, v in zip(x, latencies):
448         axes[0].text(xi, v + 1, f"{v:.0f}", ha="center")
449     axes[1].bar(x, changed, color="tomato")
450     axes[1].set_xticks(x)
451     axes[1].set_xticklabels(labels)
452     axes[1].set_ylabel("受影响路线数")
453     axes[1].set_title("计划改动规模 (#改动路线)")
454     axes[1].grid(True, alpha=0.3, axis="y")
455     for xi, v in zip(x, changed):
456         axes[1].text(xi, v + 0.05, f"{v}", ha="center")
457     fig.suptitle("问题3 每次事件的响应表现")
458     fig.tight_layout()
459     fig.savefig(path, dpi=160)
460     plt.close(fig)
461 def plot_event_cost_waterfall(
462     sched: DynamicScheduler, events: List[Event], path: Path,
463     title: str = "问题3 事件成本瀑布图 (按事件 E1-E4 拆分总成本演变)",
464 ) -> None:
465     _set_chinese_font()
466     snaps = sched.snapshots
467     components = ["fixed_cost", "energy_cost", "carbon_cost_money",
468                  "wait_cost", "late_cost"]
469     comp_labels = ["启动", "能源", "碳排放", "等待", "迟到"]
470     comp_colors = ["#4c72b0", "#dd8452", "#55a868", "#c44e52", "#8172b3"]
471     deltas_total = []
472     deltas_components = []
473     for prev, cur in zip(snaps[:-1], snaps[1:]):

```

```

474     deltas_total.append(cur.eval_after.total_cost - prev.eval_after.total_cost)
475     deltas_components.append([
476         getattr(cur.eval_after, c) - getattr(prev.eval_after, c)
477         for c in components
478     ])
479     fig, axes = plt.subplots(1, 2, figsize=(15, 6))
480     ax = axes[0]
481     labels = ["T0\n基线"] + [
482         f"+{ev.event_id}\n{hhmm(ev.trigger_time)}" for ev in events
483     ] + ["终态"]
484     n_bars = len(labels)
485     cumulative = snaps[0].eval_after.total_cost
486     bar_x = list(range(n_bars))
487     bar_bottoms = [0]
488     bar_heights = [cumulative]
489     bar_colors = ["#4C72B0"]
490     bar_values = [cumulative]
491     for delta in deltas_total:
492         if delta >= 0:
493             bar_bottoms.append(cumulative)
494             bar_heights.append(delta)
495             bar_colors.append("#E57248")
496         else:
497             bar_bottoms.append(cumulative + delta)
498             bar_heights.append(-delta)
499             bar_colors.append("#4A9F8B")
500             bar_values.append(delta)
501             cumulative += delta
502     bar_bottoms.append(0)
503     bar_heights.append(cumulative)
504     bar_colors.append("#55A868")
505     bar_values.append(cumulative)
506     ax.bar(bar_x, bar_heights, bottom=bar_bottoms, color=bar_colors,
507           edgecolor="black", linewidth=0.5)
508     for i, val in enumerate(bar_values):
509         if i == 0 or i == n_bars - 1:
510             ax.annotate(f"{val:.0f}", (i, val), xytext=(0, 4),
511                       textcoords="offset points", ha="center", fontsize=10)
512         else:
513             top = bar_bottoms[i] + bar_heights[i]
514             sign_label = (
515                 f"\u2212{abs(val):.0f}" if val < 0 else f"+{val:.0f}"
516             )
517             ax.annotate(
518                 f"\u0394 {sign_label}",
519                 (i, top),
520                 xytext=(0, 6),
521                 textcoords="offset points",
522                 ha="center", fontsize=9,

```

```

523         color="#1a7361" if val < 0 else "#a83518"),
524         fontweight="bold",
525     )
526 for i in range(n_bars - 1):
527     if i == 0:
528         y0 = bar_heights[i]
529     else:
530         y0 = (bar_bottoms[i] + bar_heights[i] if bar_values[i] >= 0
531              else bar_bottoms[i])
532     ax.hlines(y0, i + 0.4, i + 0.6, colors="gray", linewidth=0.6)
533 ax.set_xticks(bar_x)
534 ax.set_xticklabels(labels, fontsize=9)
535 ax.set_ylabel("总成本 (元)")
536 ax.set_title("(a) 事件级总成本瀑布")
537 ax.grid(True, axis="y", alpha=0.3)
538 ax = axes[1]
539 n_events = len(events)
540 x = np.arange(n_events)
541 width = 0.16
542 for j, (cname, clabel, ccol) in enumerate(zip(components, comp_labels, comp_colors)):
543     vals = [d[j] for d in deltas_components]
544     ax.bar(x + (j - 2) * width, vals, width, label=clabel, color=ccol,
545           edgecolor="black", linewidth=0.3)
546 ax.axhline(0, color="black", linewidth=0.6)
547 ax.set_xticks(x)
548 ax.set_xticklabels([f"{ev.event_id}\n{hhmm(ev.trigger_time)}" for ev in events],
549                   fontsize=9)
550 ax.set_ylabel("事件成本增量 (元)")
551 ax.set_title("(b) 各事件分项成本贡献 ( $\Delta$  vs 上一快照)")
552 ax.grid(True, axis="y", alpha=0.3)
553 ax.legend(loc="best", fontsize=9)
554 fig.suptitle(title, fontsize=13)
555 fig.tight_layout(rect=(0, 0.02, 1, 0.95))
556 fig.savefig(path, dpi=180)
557 plt.close(fig)
558 def plot_freeze_flex_area(
559     sched: DynamicScheduler, problem: ProblemData, events: List[Event],
560     path: Path,
561     title: str = "问题3 滚动时域 冻结 vs 柔性 节点演变",
562 ) -> None:
563     _set_chinese_font()
564     rows = []
565     for snap in sched.snapshots:
566         n_frozen = 0
567         n_flex = 0
568         for fr in snap.freezes:
569             n_frozen += fr.frozen_len
570             n_flex += len(fr.customers_flexible)
571         rows.append((snap.label, snap.t_now, n_frozen, n_flex))

```

```

572 labels = [r[0] for r in rows]
573 ts = [r[1] for r in rows]
574 frozen = np.array([r[2] for r in rows], dtype=float)
575 flex = np.array([r[3] for r in rows], dtype=float)
576 total = frozen + flex
577 pct_frozen = np.where(total > 0, 100 * frozen / total, 0.0)
578 fig, axes = plt.subplots(2, 1, figsize=(12, 9), sharex=True)
579 ax = axes[0]
580 x = np.arange(len(labels))
581 ax.stackplot(x, frozen, flex,
582             labels=["冻结 (已承诺)", "柔性 (可重优化)"],
583             colors=["#5b6770", "#76b7b2"], alpha=0.85)
584 ax.plot(x, total, "--", color="black", linewidth=1.0, label="虚拟节点总数")
585 for xi, (f, fl, tot) in enumerate(zip(frozen, flex, total)):
586     ax.annotate(f"{int(f)}/{int(tot)}", (xi, f), xytext=(0, -10),
587               textcoords="offset points", ha="center",
588               fontsize=8, color="white")
589 ax.set_xticks(x)
590 ax.set_xticklabels(labels)
591 ax.set_ylabel("虚拟节点数")
592 ax.set_title("(a) 冻结 vs 柔性 节点堆叠面积 (绝对量)")
593 ax.legend(loc="upper left")
594 ax.grid(True, alpha=0.3)
595 for i, ev in enumerate(events):
596     if i + 1 < len(labels):
597         ax.axvline(i + 1, color="crimson", linestyle=":", alpha=0.5)
598         ax.text(i + 1, ax.get_ylim()[1] * 0.97,
599               ev.event_id, color="crimson", fontsize=8,
600               ha="center", va="top")
601 ax = axes[1]
602 ax.fill_between(x, 0, pct_frozen, color="#5b6770", alpha=0.85,
603                label="冻结占比 (%)")
604 ax.fill_between(x, pct_frozen, 100, color="#76b7b2", alpha=0.85,
605                label="柔性占比 (%)")
606 for xi, p in enumerate(pct_frozen):
607     ax.annotate(f"{p:.1f}%", (xi, p), xytext=(0, -10),
608               textcoords="offset points", ha="center",
609               fontsize=8, color="white")
610 ax.set_xticks(x)
611 ax.set_xticklabels(labels)
612 ax.set_ylabel("占比 (%)")
613 ax.set_xlabel(f"快照 (时间窗口对应 t_now = {[hhmm(t) for t in ts]})")
614 ax.set_title("(b) 冻结占比演变 (优化空间收缩程度)")
615 ax.set_ylim(0, 100)
616 ax.legend(loc="upper left")
617 ax.grid(True, alpha=0.3)
618 fig.suptitle(title, fontsize=13)
619 fig.tight_layout(rect=(0, 0.02, 1, 0.95))
620 fig.savefig(path, dpi=180)

```

```

621     plt.close(fig)
622 def plot_q3_algorithm_flowchart(path: Path) -> None:
623     steps = [
624         (
625             "加载 Q2 多趟基线",
626             "读取 q2_baseline.pkl, 保留 vehicle_id、政策可行性与静态多趟合并结果",
627             "#4C72B0",
628         ),
629         (
630             "事件流按时间触发",
631             "依次处理取消、改时间窗、新增、改地址等 E1-E4 扰动",
632             "#55A868",
633         ),
634         (
635             "冻结已承诺前缀",
636             "按 t_now 计算已服务/已发车客户, 冻结不可撤销部分, 仅开放柔性尾段",
637             "#8172B3",
638         ),
639         (
640             "事件语义修复",
641             "取消则删除柔性需求; 新增/改址则重新插入; 时间窗收紧则定向重排",
642             "#C44E52",
643         ),
644         (
645             "柔性段局部搜索",
646             "在未冻结部分执行 relocate、2-opt、政策违反修复与燃油/电动切换",
647             "#DD8452",
648         ),
649         (
650             "合规兜底与多趟重合并",
651             "延迟过窗、应急拆分、返仓交接, 并重建时序相容的 vehicle_id 组",
652             "#4A9F8B",
653         ),
654         (
655             "快照评估与输出",
656             "每次事件后记录成本、冻结/柔性比例、响应耗时、违约数与终态图件",
657             "#5B6770",
658         ),
659     ]
660     plot_algorithm_flowchart(
661         "问题3 事件驱动滚动时域动态调度流程",
662         steps,
663         path,
664         footer="口径: 冻结段不可回滚, 柔性段重优化; 每个快照均重新核算实际车辆 / 趟次 / 成本。"
665     )
666 def plot_rolling_routes_3d_time(
667     sched: DynamicScheduler, problem: ProblemData,
668     policy: PolicyConfig, events: List[Event], path: Path,

```

```

669     title: str = "问题3 终态滚动计划 时空路线图 (XY + 服务时间)",
670 ) -> None:
671     from mpl_toolkits.mplot3d import Axes3D
672     _set_chinese_font()
673     final = sched.snapshots[-1]
674     depot = problem.nodes[0]
675     fig = plt.figure(figsize=(15.5, 10.5))
676     ax = fig.add_subplot(111, projection="3d")
677     all_customers = [
678         c for r_dict in final.routes for c in r_dict["customers"]
679     ]
680     freq_by_node = _service_frequency_by_virtual_node(problem, all_customers)
681     freq_cmap, freq_norm = _prepare_frequency_cmap(
682         fig, list(freq_by_node.values()),
683     )
684     z_min, z_max = 8.0, 21.0
685     theta = np.linspace(0, 2 * np.pi, 64)
686     z_levels = np.linspace(z_min, z_max, 14)
687     Theta, Z = np.meshgrid(theta, z_levels)
688     Xc = policy.green_radius * np.cos(Theta)
689     Yc = policy.green_radius * np.sin(Theta)
690     ax.plot_surface(Xc, Yc, Z, color="green", alpha=0.05,
691                    linewidth=0, antialiased=False)
692     fuel_handle_drawn = elec_handle_drawn = False
693     frozen_handle = flex_handle = None
694     for ri, r_dict in enumerate(final.routes):
695         if not r_dict["customers"]:
696             continue
697         r = Route(r_dict["vt_index"], list(r_dict["customers"]))
698         ev_r = evaluate_route(r, problem, policy=policy)
699         vt = problem.vehicle_types[r.vt_index]
700         xs = [depot.x] + [problem.nodes[c].x for c in r.customers] + [depot.x]
701         ys = [depot.y] + [problem.nodes[c].y for c in r.customers] + [depot.y]
702         zs = list(ev_r.arrival)
703         color = "tab:orange" if vt.is_fuel else "tab:blue"
704         if vt.is_fuel and not fuel_handle_drawn:
705             ax.scatter([], [], [], s=48, marker="o", facecolor="white",
706                       edgecolor=color, linewidth=1.2, label="燃油车 (点边框)")
707             fuel_handle_drawn = True
708         if (not vt.is_fuel) and not elec_handle_drawn:
709             ax.scatter([], [], [], s=48, marker="o", facecolor="white",
710                       edgecolor=color, linewidth=1.2, label="新能源车 (点边框)")
711             elec_handle_drawn = True
712         frozen_len = (
713             final.freezes[ri].frozen_len if ri < len(final.freezes) else 0
714         )
715         for j, c in enumerate(r.customers):
716             n = problem.nodes[c]
717             arrive = ev_r.arrival[j + 1]

```

```

718         is_frozen = j < frozen_len
719         mc = freq_cmap(freq_norm(freq_by_node.get(c, 1)))
720         sc = ax.scatter([n.x], [n.y], [arrive], s=22,
721                        color=mc, edgecolor=color,
722                        linewidth=0.8, alpha=0.9,
723                        marker="s" if is_frozen else "o")
724         if is_frozen and frozen_handle is None:
725             frozen_handle = sc
726             frozen_handle.set_label("冻结客户 (方块; 点色=频次)")
727         if (not is_frozen) and flex_handle is None:
728             flex_handle = sc
729             flex_handle.set_label("柔性客户 (圆点; 点色=频次)")
730         ax.scatter([depot.x], [depot.y], [z_min], s=160, marker="s",
731                  color="black", label="配送中心")
732     for ev in events:
733         z = ev.trigger_time
734         xx = np.linspace(min(n.x for n in problem.customer_nodes),
735                          max(n.x for n in problem.customer_nodes), 6)
736         yy = np.linspace(min(n.y for n in problem.customer_nodes),
737                          max(n.y for n in problem.customer_nodes), 6)
738         X, Y = np.meshgrid(xx, yy)
739         Zp = np.full_like(X, z, dtype=float)
740         ax.plot_surface(X, Y, Zp, color="crimson", alpha=0.05,
741                        linewidth=0, antialiased=False)
742         ax.text(xx[0], yy[0], z, f"{ev.event_id}@{hhmm(z)}",
743                color="crimson", fontsize=8)
744     ax.set_xlabel("X (km)")
745     ax.set_ylabel("Y (km)")
746     ax.set_zlabel("到达时间 (小时)", labelpad=14)
747     ax.set_title(title + "; 点颜色=服务频次", pad=16)
748     ax.view_init(elev=22, azim=-58)
749     ax.legend(loc="upper left", fontsize=9)
750     _finish_3d_figure(fig, ax, path)
751 def write_summary_md(
752     sched: DynamicScheduler,
753     events: List[Event],
754     problem: ProblemData,
755     policy: PolicyConfig,
756     path: Path,
757 ) -> None:
758     t0 = sched.snapshots[0]
759     final = sched.snapshots[-1]
760     delta_total = final.eval_after.total_cost - t0.eval_after.total_cost
761     delta_trips = final.eval_after.n_routes - t0.eval_after.n_routes
762     delta_phys = (
763         _count_physical_vehicles_from_snap(final)
764         - _count_physical_vehicles_from_snap(t0)
765     )
766     t0_phys = _count_physical_vehicles_from_snap(t0)

```

```

767 fn_phys = _count_physical_vehicles_from_snap(final)
768 t0_trips = t0.eval_after.n_routes
769 fn_trips = final.eval_after.n_routes
770 fixed_cost_per_vehicle = problem.cost.fixed_cost
771 t0_merges = t0_trips - t0_phys
772 final_merges = fn_trips - fn_phys
773 t0_savings = t0_merges * fixed_cost_per_vehicle
774 lines: List[str] = []
775 def add(s: str = "") -> None:
776     lines.append(s)
777 add("# 问题 3 动态调度结果汇总")
778 add()
779 add("本题在问题2（绿色配送区 08:00 - 16:00 燃油车禁行）的最优静态方案 "
780     f"q2_baseline.pkl` 基础上，构造了一套典型的单日突发事件场景，"
781     "使用 *事件驱动的滚动时域重优化* 策略实时调整车辆路径，保证政策 "
782     "可行、时间窗可行与成本近似最优.")
783 add()
784 add("**重要背景**：自本版起，**多趟合并 (multi-trip)** 与 **返仓交接 "
785     "(handover)** 已经下沉到 Q1/Q2 的静态 ALNS 求解阶段，Q2 baseline "
786     "加载进来时就已经带着 `vehicle_id` 分组 —— 因此 Q3 的 T0 成本已经 "
787     "低于旧版基线 (95,668 vs 旧 116,094 元)，而事件阶段的 “节省启动次数” 列数值 "
788     "会显得很小。这不是退化，而是静态阶段已经把绝大多数合并收益 "
789     f"({t0_savings:.0f} 元) 预先挖走，Q3 只需在事件触发后做 *增量维护* "
790     "(stale-group 解散 + 重合并) 与政策兜底.")
791 add()
792 add("## 一、算法框架")
793 add()
794 add("```")
795 add("    事件 e_k (t_now = t_k)")
796 add("    ")
797 add("    ")
798 add("    ")
799 add("    1. 冻结承诺前缀 ")
800 add("    (已发车 / 正在前往的客户) ")
801 add("    ")
802 add("    ")
803 add("    ")
804 add("    ")
805 add("    2. 事件定向修复 ")
806 add("    • 取消 → 从柔性段移除 ")
807 add("    • 新增 → best-insertion ")
808 add("    • 改址 → 更新坐标 + 重插入 ")
809 add("    • 改时间窗 → 按迟到重定位 ")
810 add("    ")
811 add("    ")
812 add("    ")
813 add("    ")
814 add("    3. 柔性段局部搜索 ")
815 add("    (2-opt + relocate + ")

```



```

$16     add("        政策违反修复 +          ")
$17     add("        燃油→电动切换 +          ")
$18     add("        返仓交接 (handover) +      ")
$19     add("        延迟过窗 (16:00 再走) +    ")
$20     add("        应急拆分 (超配电动车))   ")
$21     add("        ")
$22     add("        ")
$23     add("        ")
$24     add("        ")
$25     add("    4. 多趟合并 (multi-trip)      ")
$26     add("        (同型车 / 时序相容对      ")
$27     add("        共享 vehicle_id,          ")
$28     add("        一次启动成本多趟用)          ")
$29     add("        ")
$30     add("        ")
$31     add("        ")
$32     add("        ")
$33     add("    5. 零违约契约断言                    ")
$34     add("        (任一残留违约即 raise)          ")
$35     add("        ")
$36     add("        ")
$37     add("        ")
$38     add("        ")
$39     add("    6. 快照 + 成本核算                    ")
$40     add("        ")
$41     add("```)")
$42     add()
$43     add("**状态冻结规则**: 对任意路线 `[depot, c_1, ..., c_k, depot]`, 在决策时刻 ")
$44     add("`t_now` 计算 `service_end[0..k]`. 若 `service_end[i]` `t_now`, ")
$45     add("则车辆已从位置 `i` 出发, 客户 `c_i` 视为已承诺, 无法撤销或改道. ")
$46     add("冻结前缀长度 `frozen_len` 即此类客户的数量, 局部搜索与 ALNS 仅作用于 ")
$47     add("索引 `>= frozen_len` 的柔性尾部.")
$48     add()
$49     add("**物理约束**: 已发车的车辆无法再装载新客户的货物 (货物在发车时已装船), ")
$50     add("故新增订单与跨路线再插入仅允许注入到 *未发车* 路线或新开路线.")
$51     add()
$52     add("**多趟 (multi-trip) 与返仓交接 (handover)**: 一辆实际物理车可在同一天 ")
$53     add("内跑多趟 — 上一趟回仓且距下一趟自然发车时间 15 分钟换装时间时, ")
$54     add("两趟共享 `vehicle_id`, *启动成本只计一次*. 冻结时序+车型一致 的前 ")
$55     add("提下, 合并是无损的 (两趟各自的最优发车时刻都自然满足). 对 *已发车 ")
$56     add("的燃油车路线* 且尾段违反绿区政策, 触发 **返仓交接**: 司机提前回到 ")
$57     add("配送中心结束本趟, 剩余的绿区客户由一辆新调度的电动车接棒 — 物理 ")
$58     add("不可行的情景也得以合规化解.")
$59     add()
$60     add("## 二、突发事件场景")
$61     add()
$62     ev_log = build_event_log(events, sched.snapshots)
$63     add(ev_log.to_markdown(index=False))
$64     add()

```

```

865 add("场景说明：上述 4 个事件覆盖问题3 要求的全部 4 类异常（取消/新增/改址/改时间窗）。")
866 add("触发时刻从 09:15 到 13:20，横跨 08:00 - 16:00 的绿区限制期。事件 E2 对 ")
867 add("*绿区内电动车* 车次时间窗做加压，用于测试绿区政策下的鲁棒性.")
868 add()
869 add("## 三、滚动计划成本轨迹")
870 add()
871 cost_df = build_cost_breakdown_table(sched)
872 add(cost_df.to_markdown(index=False))
873 add()
874 t0_phys = _count_physical_vehicles_from_snap(t0)
875 fn_phys = _count_physical_vehicles_from_snap(final)
876 t0_trips = t0.eval_after.n_routes
877 fn_trips = final.eval_after.n_routes
878 fixed_cost_per_vehicle = problem.cost.fixed_cost
879 t0_merges = t0_trips - t0_phys
880 final_merges = fn_trips - fn_phys
881 t0_savings = t0_merges * fixed_cost_per_vehicle
882 add(f"- **总成本变化**: Q2 基线 **{t0.eval_after.total_cost:.2f} 元** → ")
883 add(f" 最终滚动计划 **{final.eval_after.total_cost:.2f} 元** ")
884 add(f" (ΔC = **{delta_total:+.2f} 元**)")
885 add(f"- **启动成本**: ")
886     f"{t0.eval_after.fixed_cost:.0f} → {final.eval_after.fixed_cost:.0f} 元 "
887     f"(Δ={final.eval_after.fixed_cost - t0.eval_after.fixed_cost:+.0f} 元; "
888     f"T0 已由 Q2 静态 ALNS 通过多趟合并省下 {t0_savings:.0f} 元 "
889     f"——{t0_trips} 趟压到 {t0_phys} 辆实际物理车)")
890 add(f"- **趟次 vs 实际车辆**: ")
891     f"T0 {t0_trips} 趟 / {t0_phys} 辆车 (节省 {t0_merges} 次启动), "
892     f"终态 {fn_trips} 趟 / {fn_phys} 辆车 (节省 {final_merges} 次启动); "
893     f"事件阶段实际物理车净变化 "
894     f"**{(fn_phys - t0_phys):+d}** 辆, "
895     f"节省启动次数净变化 **{final_merges - t0_merges:+d}** 次.")
896 add(f"- **政策违约次数**: T0=0, 终态={final.eval_after.policy_violation_count} ")
897     f"({'保持零违约' if final.eval_after.policy_violation_count == 0 else '需重点关注'})
898 ")
899 add()
900 add("## 四、冻结-柔性划分演变")
901 add()
902 split_df = build_freeze_split_table(sched, problem)
903 add(split_df.to_markdown(index=False))
904 add()
905 add("随着 `t_now` 推进，冻结虚拟节点比例从 0% (T0) 逐步上升，"
906     "可用于重优化的柔性池不断收缩，这解释了为何越往后的事件可调度余地越小，"
907     "响应时延反而上升（局部搜索需在更少的可行空间内寻找改进）。")
908 add()
909 add("## 五、车型启用变化")
910 add()
911 vt_df = build_vt_usage_table(sched, problem)
912 add(vt_df.to_markdown(index=False))
913 add()

```

```

913 add("电动车 E1/E2 配额在各快照基本保持饱和 (问题2 的政策下绿区客户已被电动车锁定), ")
914 add("事件带来的车辆变动主要在燃油 F1/F2/F3 之间发生.")
915 add()
916 add("## 六、零违约硬保障")
917 add()
918 add("自本版起, **问题3 将 \"终态无任何政策违约\" 列为不可违反的硬性前提**. ")
919 add("为此引入双保险终端修复机制, 在既有 \"重插入 → 燃油转电动 → 返仓交接\" ")
920 add("三级兜底之后再叠加两级:")
921 add()
922 add("- **延迟过窗 `_enforce_no_violation_by_delay`**: 对任何仍有绿区违约的 ")
923 add(" 燃油路线, 找到最早的违约弧 `i`, 把 `arc_depart_floor[i]` 抬到 ")
924 add(f"  `{policy.fuel_restrict_end + 1e-3:.3f} h`. 车辆在 `seq[i]` 处原地 ")
925 add(" 等到 16:00 之后再走该弧 — `is_arc_violation` 依赖 ")
926 add("  `t_leave < fuel_restrict_end`, 强制 `t_leave 16.0` 直接让 ")
927 add(" Rule B 失效; 相应地 `arrival` 也被推到 16:00 之后, Rule A 同步 ")
928 add(" 失效. 违约硬约束 **被转化为 late_cost 软约束**.")
929 add()
930 add("- **应急拆分 `_emergency_split_tail`**: 如果延迟后导致 22:00 返仓 ")
931 add(" 超期, 把违约弧之后的尾段截下来派给一辆 *可能超出车队配额* 的 ")
932 add(" 电动车 (\"紧急租车\"). 配额上限在 Q3 动态阶段被放宽, 以保证任何 ")
933 add(" 事件都有出路.")
934 add()
935 add("- **契约断言**: `apply_event` 末尾执行 ")
936 add("  `evaluate(respect_history=True).feasible_policy` 检查, 如仍有残留 ")
937 add(" 违约则抛 `RuntimeError` — 宁可显式失败也不让违约方案进入正式快照.")
938 add()
939 add("本次确定性场景中触发情况:")
940 add()
941 telemetry_rows = []
942 for snap in sched.snapshots:
943     floored_arcs = _count_arc_delay_arcs_from_snap(snap)
944     telemetry_rows.append({
945         "快照": snap.label,
946         "时刻": hhmm(snap.t_now),
947         "延迟过窗弧次 (累计)": floored_arcs,
948         "政策违约数": snap.eval_after.policy_violation_count,
949     })
950 add(pd.DataFrame(telemetry_rows).to_markdown(index=False))
951 add()
952 add(f"累计触发 \"延迟过窗\" 弧次: **{sched.arc_delay_count}** ")
953 add(f"累计触发 \"延迟过窗\" 事件: **{sched.arc_delay_events}** ")
954 add(f"累计触发 \"应急拆分\": **{sched.emergency_split_count}**")
955 add()
956 add("## 七、响应性能")
957 add()
958 rsp = pd.DataFrame([
959     "事件ID": ev.event_id,
960     "触发时刻": hhmm(ev.trigger_time),
961     "事件类型": {EVENT_CANCEL: "取消", EVENT_NEW: "新增", EVENT_ADDRESS: "改址",

```

```

962         EVENT_TIMEWINDOW: "改时间窗"][ev.kind],
963         "响应耗时(ms)": round(snap.response_ms, 1),
964         "受影响路线数": len(snap.changed_route_indices),
965         "ΔC vs 上一次(元)": round(snap.eval_after.total_cost - prev.eval_after.total_cost,
966                                     2),
967         "政策违约数": snap.eval_after.policy_violation_count,
968     } for (ev, snap, prev) in zip(events, sched.snapshots[1:], sched.snapshots[:-1]))
969 add(rsp.to_markdown(index=False))
970 add()
971 add("单事件响应时延 < 250 ms, 完全满足实时调度需求. 大部分事件仅需修改 1-5 条路线. ")
972 add()
973 add("## 八、政策鲁棒性校验")
974 add()
975 pol_rows = []
976 for snap in sched.snapshots:
977     pol_rows.append({
978         "快照": snap.label,
979         "时刻": hhmm(snap.t_now),
980         "feasible_policy": snap.eval_after.feasible_policy,
981         "违约次数 (Rule A + Rule B)": snap.eval_after.policy_violation_count,
982     })
983 add(pd.DataFrame(pol_rows).to_markdown(index=False))
984 add()
985 if all(s.eval_after.feasible_policy for s in sched.snapshots):
986     add("**所有快照均满足绿区政策的 Rule A (点违反) 与 Rule B (弧穿越)**. ")
987     add("动态调度器通过五级回退自动保证了政策一致性: ")
988     add("柔性段再插入 → 政策违反定向修复 → 燃油车换电动车 → 返仓交接 → ")
989     add("延迟过窗 → 应急拆分. 并且 `apply_event` 末尾加了 `RuntimeError` 契约断言, ")
990     add("任何残留违约都会被硬拒绝.")
991 else:
992     add("出现残留违约说明契约断言已经被绕过, 算法存在回归 — 请检查日志.")
993 add()
994 add("## 九、可推广的动态调度 SOP")
995 add()
996 add("1. **冻结规则**: 对任意决策时刻 `t_now`, 凡是 `service_end[i] < t_now` ")
997 add("    的客户视为承诺, 其所在路线位置不再参与重优化.")
998 add("2. **事件语义化**: 把事件映射到 `Event(kind, ...)` , 保持与静态求解器 ")
999 add("    同一套数据结构 (`Node` / `Route` / `PolicyConfig` ).")
1000 add("3. **两级修复**: 事件处理器做 *语义级* 修复 (针对该事件最自然的动作); ")
1001 add("    随后统一跑 *局部搜索* (2-opt / relocate / 政策违反专项修复) 统一收敛.")
1002 add("4. **物理守恒**: 已发车车辆只能减、不能加; 新订单只许插入未发车车辆或新车次.")
1003 add("5. **保守上界**: 对 `_route_cost` 加 `t_now` 门槛 — 任何需要在 `t_now` ")
1004 add("    之前发车的候选解判为不可行 (`_INFEAS`), 避免时间回流的伪最优解.")
1005 add("6. **降级兜底**: 绿区燃油车违反无法消除时, 自动换用同载重电动车 (若还有配额).")
1006 add("7. **闭环监控**: 每次事件产生一份快照, 记录成本分项 / 响应时延 / 政策违约, ")
1007 add("    供调度员事后审核与自适应参数调整.")
1008 add()
1009 add("## 十、关键结论")
1010 add()

```

```

1010 add(f"1. 动态调度框架在 {len(events)} 次连续事件下保持计划一致性, 总响应时延 < 1s, ")
1011 add(f"    单次响应平均 {np.mean([s.response_ms for s in sched.snapshots[1:]]):.1f} ms.")
1012 add(f"2. 最终滚动计划的总成本相对 Q2 基线 {'下降' if delta_total < 0 else '上升'} "
1013     f"{abs(delta_total):.2f} 元** ({100 * abs(delta_total) / t0.eval_after.total_cost :.2f}%); ")
1014 phys_word = (
1015     '减少' if delta_phys < 0 else '增加' if delta_phys > 0 else '保持'
1016 )
1017 trips_word = (
1018     '减少' if delta_trips < 0 else '增加' if delta_trips > 0 else '保持'
1019 )
1020 add(f"    实际物理车辆 {phys_word} **{abs(delta_phys)} 辆** "
1021     f"(\Delta_phys = {delta_phys:+d}), "
1022     f"启用趟次 {trips_word} **{abs(delta_trips)} 趟** "
1023     f"(\Delta_trips = {delta_trips:+d}).")
1024 add("3. 取消 + 新增 组合给出了最显著的重优化收益, 表明动态调度在 *负载波动* ")
1025 add("    场景下比静态求解器具备更高的资源利用率.")
1026 add("4. **绿区政策 100% 合规是算法契约, 非概率结果**.")
1027 add("    通过 \"延迟过窗 + 应急拆分 + `RuntimeError` 断言\" 三重保险, ")
1028 add("    任何随机事件流都 *不可能* 产生违约方案 — 违约被提前转化为 ")
1029 add("    late_cost 或紧急用车成本, 以可计量的代价换取硬合规. ")
1030 add("    框架可直接迁移到其它时段/区域限制场景 (如限流、限重、排放分区等).")
1031 add("5. **多趟合并 + 返仓交接** 已从 Q3 下沉到 Q1/Q2 静态 ALNS 阶段 ——")
1032 add("    `_solution_eval_for_search` 在搜索过程中就对可合并对打折扣, ")
1033 add("    末段再真正写入 `vehicle_id`. 这意味着 Q3 的基线天生就是多趟方案, ")
1034 add("    事件处理阶段的 “节省启动次数” 只反映 *stale-group 解散 + 重合并* 的增量; ")
1035 add("    蒙特卡洛平均  $\Delta C$  相对 T0 保持下降方向, 合规率稳定为 100%.")
1036 add()
1037 add("*所有数据可在 `q3_tables.xlsx` 与 `q3_*.png` 中逐项核对.*")
1038 add()
1039 add("## 十一、论文图件索引")
1040 add()
1041 add("| 文件 | 算法/数据含义 |")
1042 add("| --- | --- |")
1043 add("| `q3_routes_before_after.png` | T0 (Q2 基线) vs 终态滚动计划 二维路径双面板, "
1044     "标注新增订单 N1 位置. |")
1045 add("| `q3_algorithm_flowchart.png` | **问题3算法流程图** — 展示 Q2 基线加载、事件触发、"
1046     "冻结前缀、事件语义修复、柔性段局部搜索、合规兜底与快照输出的动态调度闭环. |")
1047 add("| `q3_event_timeline.png` | 事件甘特图 — 每行=路线/车次, 灰=冻结/已承诺, "
1048     "蓝=柔性, 虚线为事件触发时刻. |")
1049 add("| `q3_cost_trajectory.png` | 滚动计划成本沿事件累积演变 (上: 总成本折线; "
1050     "下: 五分项堆叠面积). |")
1051 add("| `q3_response.png` | 每次事件的响应耗时 (ms) 与受影响路线数, 评估实时性. |")
1052 add("| `q3_event_cost_waterfall.png` | **事件成本瀑布图** — 左面板把 "
1053     "T0 → E1 → E2 → E3 → E4 → 终态 的总成本变化拆成逐事件增量, "
1054     "右面板把每次事件的  $\Delta C$  进一步拆到启动/能源/碳/等待/迟到 五项. "
1055     "直观解释每次重优化为何使成本上升或下降. |")
1056 add("| `q3_freeze_flex_area.png` | **冻结/柔性 节点演变堆叠面积图**."
1057     "上: 绝对量 (冻结 vs 柔性); 下: 占比. 反映滚动时域中可优化空间逐步收缩, ")

```

```

1057         "解释为何后续事件响应时延上升。|")
1058     add("| `q3_rolling_routes_3d_time.png` | **终态三维时空路线图** (X/Y/到达时间). "
1059         "点边框=动力, 点色=服务频次, 标记=冻结(方块) vs 柔性(圆点); 红色透明面=事件触发时刻. "
1060         "完整呈现动态重优化后的时空计划。|")
1061     path.write_text("\n".join(lines), encoding="utf-8")
1062 def main() -> None:
1063     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1064     baseline_pkl = RESULTS_DIR / "q2_baseline.pkl"
1065     if not baseline_pkl.exists():
1066         raise FileNotFoundError(
1067             f"未找到 Q2 基线文件 {baseline_pkl}. 请先运行 `python solve_q2.py`.")
1068     print(f"[q3] 加载数据...")
1069     base_problem = load_problem()
1070     policy = PolicyConfig()
1071     print(f"[q3] 启动滚动时域调度...")
1072     t0 = time.time()
1073     sched, events, problem = run_rolling_schedule(
1074         baseline_pkl, base_problem, policy,
1075     )
1076     total_ms = (time.time() - t0) * 1000
1077     print(f"[q3] 调度完成, 总耗时 {total_ms:.1f} ms, 共 {len(sched.snapshots)} 份快照.")
1078     print(f"[q3] 输出表格...")
1079     write_tables_xlsx(sched, events, problem, RESULTS_DIR / "q3_tables.xlsx")
1080     print(f"[q3] 输出图表...")
1081     plot_routes_before_after(
1082         sched, problem, policy, RESULTS_DIR / "q3_routes_before_after.png",
1083     )
1084     plot_event_timeline(
1085         sched, problem, events, RESULTS_DIR / "q3_event_timeline.png",
1086     )
1087     plot_cost_trajectory(sched, RESULTS_DIR / "q3_cost_trajectory.png")
1088     plot_responseBars(sched, events, RESULTS_DIR / "q3_response.png")
1089     print(f"[q3] 输出论文级图表 (waterfall / area / 3D)...")
1090     plot_event_cost_waterfall(
1091         sched, events, RESULTS_DIR / "q3_event_cost_waterfall.png",
1092     )
1093     plot_freeze_flex_area(
1094         sched, problem, events,
1095         RESULTS_DIR / "q3_freeze_flex_area.png",
1096     )
1097     plot_rolling_routes_3d_time(
1098         sched, problem, policy, events,
1099         RESULTS_DIR / "q3_rolling_routes_3d_time.png",
1100     )
1101     plot_q3_algorithm_flowchart(RESULTS_DIR / "q3_algorithm_flowchart.png")
1102     print(f"[q3] 输出总结...")
1103     write_summary_md(
1104         sched, events, problem, policy, RESULTS_DIR / "q3_summary.md",
1105     )

```

```

1106     serial = {
1107         "events": [vars(ev) for ev in events],
1108         "snapshots": [
1109             {
1110                 "label": s.label,
1111                 "t_now": s.t_now,
1112                 "routes": s.routes,
1113                 "freezes": [vars(f) for f in s.freezes],
1114                 "eval_after": {
1115                     "total_cost": s.eval_after.total_cost,
1116                     "fixed_cost": s.eval_after.fixed_cost,
1117                     "energy_cost": s.eval_after.energy_cost,
1118                     "carbon_cost_money": s.eval_after.carbon_cost_money,
1119                     "wait_cost": s.eval_after.wait_cost,
1120                     "late_cost": s.eval_after.late_cost,
1121                     "n_routes": s.eval_after.n_routes,
1122                     "feasible_policy": s.eval_after.feasible_policy,
1123                     "policy_violation_count": s.eval_after.policy_violation_count,
1124                 },
1125                 "response_ms": s.response_ms,
1126                 "cancelled_wasted": s.cancelled_wasted,
1127                 "changed_route_indices": s.changed_route_indices,
1128                 "note": s.note,
1129             }
1130             for s in sched.snapshots
1131         ],
1132     }
1133     with (RESULTS_DIR / "q3_snapshots.pkl").open("wb") as f:
1134         pickle.dump(serial, f)
1135     print("\n[q3] 结果摘要:")
1136     for s in sched.snapshots:
1137         e = s.eval_after
1138         print(f"    {s.label:4s} t={hhmm(s.t_now):5s} "
1139               f"cost={e.total_cost:9.2f} routes={e.n_routes:3d} "
1140               f"viol={e.policy_violation_count} resp={s.response_ms:6.1f}ms")
1141 if __name__ == "__main__":
1142     main()

```